

Arquitetura de LLMs e IA Aplicada Do Basico ao Avancado com RAG Corporativo

Autores do Projeto

2 de maio de 2026

Prefácio

Este livro conta uma história. Não é uma coleção de receitas isoladas: cada capítulo introduz um personagem, uma tensão ou uma virada que só fazem sentido luz do que veio antes. O personagem central é um sistema de IA corporativo que nasce simples — uma chamada ao LLM — e amadurece progressivamente até operar em produção bancária com rastreabilidade, avaliação contínua e governança técnica. O leitor que acompanhar essa jornada capítulo a capítulo sairá capaz não apenas de reproduzir o código, mas de tomar as mesmas decisões de projeto — e de justificá-las.

A história em dois ciclos

Toda plataforma de IA em produção vive em dois ciclos interdependentes, que este livro chama de **loop interno** e **loop externo**.

O **loop interno** (*inner loop*) é o ciclo de *desenvolvimento*: experimento, avaliação offline, validação humana e promoção para produção. Ele é onde o engenheiro vive — no notebook, nos testes unitários, na construção do *golden dataset*, na calibração do juiz LLM. Os capítulos 1 a 10 narram, passo a passo, a construção desse loop.

O **loop externo** (*outer loop*) é o ciclo de *produção*: a plataforma responde a clientes reais, mede confiabilidade em tempo real, coleta evidências e as devolve ao loop interno para reajuste. Os capítulos 11 a 15 instrumentam esse ciclo — observabilidade, métricas de mercado, LLM-as-Judge, *golden dataset* e *roadmap* de evolução contínua. Os capítulos 16 a 18 garantem que o ciclo se sustente com qualidade, segurança e CI/CD.

Os dois loops não são seqüenciais: eles giram em paralelo e se alimentam mutuamente. O capítulo 10 é o ponto de convergência onde essa dinâmica se torna explícita pela primeira vez.

Como ler este livro

- **Leitura linear:** recomendada na primeira passagem. Cada capítulo pressupõe os anteriores; o glossário e os pré-requisitos de cada capítulo indicam o mínimo necessário.
- **Leitura por camada:** caps. 1–4 (fundamentos) → caps. 5–9 (plataforma) → caps.

10–14 (qualidade) → caps. 15–18 (operação). Cada camada pode ser retomada independentemente após a leitura completa.

- **Rastreabilidade:** cada capítulo mantém a tríade *conceito* → *implementação* → *evidência de qualidade*. Os planos de teste e os critérios de aceite são parte do texto, não apêndice.

Índice proposto

Loop interno — construção (caps. 1–10)

1. Fundamentos de IA e LLMs
2. Ambiente, ferramentas e reprodutibilidade
3. Agentes com LangGraph e memória
4. Tools, skills e MCP
5. Fundamentos de RAG
6. RAG corporativo e arquitetura de dados
7. Pipeline assíncrono em AWS
8. Segmentação, personas e digital twins
9. GraphRAG com Neptune
10. **Pipeline integrado em produção** ← *ponto de convergência*

Loop externo — qualidade e operação (caps. 11–18)

11. Confiabilidade, observabilidade e operação
12. Métricas de avaliação de mercado
13. LLM as Judge
14. Golden dataset e regressão de qualidade
15. Roadmap de produção e evolução contínua
16. CI/CD multiambiente bancário
17. Qualidade e segurança nas esteiras
18. Frontend, CloudFront e console de operação

Sumário

Prefácio	i
1 Fundamentos de IA e Modelos de Linguagem de Grande Escala	1
1.1 Escopo do capítulo	1
1.2 Objetivos de aprendizagem	1
1.3 Conceitos teóricos	2
1.3.1 Arquitetura Transformer	2
1.3.2 Tokenização	3
1.3.3 Janela de Contexto	3
1.3.4 Parâmetros de Amostragem	3
1.4 Implementação orientada por passos	4
1.4.1 Passo 1 — Configuração das credenciais AWS	4
1.4.2 Passo 2 — Primeira chamada ao modelo	4
1.4.3 Passo 3 — Engenharia de <i>prompts</i>	5
1.4.4 Passo 4 — Técnica <i>few-shot</i>	6
1.5 Plano de testes do capítulo	6
1.5.1 Teste funcional	6
1.5.2 Teste de desempenho	6
1.5.3 Teste de qualidade de resposta	7
1.6 Critérios de aceite	7
1.7 Espaço para expansão iterativa	7
2 Ambiente, Ferramentas e Reprodutibilidade	8
2.1 Escopo do capítulo	8
2.2 Objetivos de aprendizagem	8
2.3 Conceitos e prerequisites	9
2.3.1 Versões e dependências	9
2.3.2 Estrutura do repositório	9
2.4 Implementação orientada por passos	10
2.4.1 Passo 1 — Clonar o repositório e criar o ambiente virtual	10
2.4.2 Passo 2 — Configurar credenciais AWS	10

2.4.3	Passo 3 — Provisionar a infraestrutura com Terraform	11
2.4.4	Passo 4 — Executar os notebooks	11
2.5	Plano de testes do capítulo	12
2.5.1	Teste funcional	12
2.5.2	Teste de desempenho	12
2.5.3	Teste de qualidade	12
2.6	Critérios de aceite	12
2.7	Espaço para expansão iterativa	13
3	Agentes com LangGraph e Memória	14
3.1	Pré-requisitos	14
3.2	Escopo do capítulo	14
3.3	Objetivos de aprendizagem	15
3.4	Conceitos teóricos	15
3.4.1	Grafo de estados e nós de processamento	15
3.4.2	Ciclo ReAct	15
3.4.3	Persistência de estado com MemorySaver	16
3.5	Implementação orientada por passos	16
3.5.1	Passo 1 — Definir as ferramentas do agente	16
3.5.2	Passo 2 — Construir o grafo ReAct	17
3.5.3	Passo 3 — Invocar o agente com persistência de contexto	18
3.5.4	Passo 4 — Integrar memória vetorial FAISS	18
3.6	Padrões alternativos de agente	19
3.6.1	Plan-and-Execute	19
3.6.2	Reflexion	21
3.6.3	Multi-Agent Supervisor	23
3.7	Plano de testes do capítulo	24
3.7.1	Teste funcional	24
3.7.2	Teste de desempenho	24
3.7.3	Teste de qualidade de resposta	24
3.8	Critérios de aceite	25
3.9	Autoavaliação	25
3.10	Questão para reflexão	26
3.11	Resumo	26
3.12	Para saber mais	27
3.13	Espaço para expansão iterativa	27
4	Tools, Skills e MCP	28
4.1	Pré-requisitos	28
4.2	Escopo do capítulo	28

4.3	Objetivos de aprendizagem	29
4.4	Conceitos teóricos	29
4.4.1	Tools nativas: funções como ações do agente	29
4.4.2	Skills: tools com contexto de negócio	30
4.4.3	MCP: protocolo padronizado para ferramentas	30
4.5	Implementação orientada por passos	30
4.5.1	Passo 1 — Criar tools nativas e registrar no agente	30
4.5.2	Passo 2 — Implementar servidor MCP com transporte stdio	31
4.5.3	Passo 3 — Conectar o agente ao servidor MCP	33
4.5.4	Passo 4 — Decisões de projeto: tool nativa vs. MCP	34
4.6	Autoavaliação	34
4.7	Resumo	34
4.8	Para saber mais	35
4.9	Plano de testes do capítulo	35
4.9.1	Testes funcionais	35
4.9.2	Teste de isolamento	35
4.10	Critérios de aceite	36
4.11	Espaço para expansão iterativa	36
5	Fundamentos de RAG	37
5.1	Pré-requisitos	37
5.2	Escopo do capítulo	37
5.3	Objetivos de aprendizagem	37
5.4	Conceitos teóricos	38
5.4.1	Por que RAG?	38
5.4.2	Chunking: dividir para indexar	38
5.4.3	Embeddings: representar significado em vetores	39
5.4.4	FAISS: índice de vetores local	39
5.5	Implementação orientada por passos	39
5.5.1	Passo 1 — Preparar a coleção de documentos	39
5.5.2	Passo 2 — Implementar chunking por parágrafos	40
5.5.3	Passo 3 — Gerar embeddings e indexar no FAISS	40
5.5.4	Passo 4 — Recuperar chunks por similaridade	41
5.5.5	Passo 5 — Pipeline RAG completo com o agente	42
5.6	Autoavaliação	43
5.7	Resumo	43
5.8	Para saber mais	44
5.9	Plano de testes do capítulo	44
5.10	Critérios de aceite	44

5.11	Espaço para expansão iterativa	45
6	RAG Corporativo e Arquitetura de Dados	46
6.1	Escopo do capítulo	46
6.2	Objetivos de aprendizagem	46
6.3	Conceitos teóricos	47
6.3.1	RAG em ambiente regulado	47
6.3.2	Tipos de documento no projeto	47
6.4	Implementação orientada por passos	47
6.4.1	Passo 1 — Preparar coleção de documentos	47
6.4.2	Passo 2 — Ingerir texto no endpoint <code>/documentos</code>	48
6.4.3	Passo 3 — Verificar chunking e indexação	48
6.4.4	Passo 4 — Consumir o contexto em consulta real	49
6.4.5	Passo 5 — Resultado completo da aula	49
6.5	Plano de testes do capítulo	49
6.5.1	Teste funcional	49
6.5.2	Teste de desempenho	50
6.5.3	Teste de qualidade de resposta	50
6.6	Critérios de aceite	50
6.7	Espaço para expansão iterativa	50
7	Pipeline Assíncrono em AWS	51
7.1	Escopo do capítulo	51
7.2	Objetivos de aprendizagem	51
7.3	Conceitos teóricos	52
7.3.1	Padrão fire-and-poll	52
7.3.2	Estados de processamento	52
7.4	Implementação orientada por passos	52
7.4.1	Passo 1 — Submeter consulta e enfileirar	52
7.4.2	Passo 2 — Processar no worker	53
7.4.3	Passo 3 — Consultar status	53
7.4.4	Passo 4 — Teste ponta a ponta de laboratório	53
7.4.5	Passo 5 — Resultado completo da aula	54
7.4.6	Passo 6 — Evidência operacional via frontend CloudFront	54
7.5	Plano de testes do capítulo	54
7.5.1	Teste funcional	54
7.5.2	Teste de desempenho	55
7.5.3	Teste de qualidade de resposta	55
7.6	Critérios de aceite	55
7.7	Espaço para expansão iterativa	55

8	Segmentacao, Personas e Digital Twins	56
8.1	Escopo do capitulo	56
8.2	Objetivos de aprendizagem	56
8.3	Conceitos teoricos	57
8.3.1	Tres superficies de analise	57
8.3.2	Features de clusterizacao	57
8.4	Implementacao orientada por passos	57
8.4.1	Passo 1 — Gerar base sintetica e executar clustering	57
8.4.2	Passo 2 — Criar contexto RAG por cluster	58
8.4.3	Passo 3 — Instanciar agente persona	58
8.4.4	Passo 4 — Criar digital twin sob demanda	58
8.4.5	Passo 5 — Chamar a API em cada modo	59
8.4.6	Passo 6 — Resultado completo da aula	59
8.4.7	Passo 7 — Evidencias de execucao em producao	60
8.5	Plano de testes do capitulo	60
8.5.1	Teste funcional	60
8.5.2	Teste de desempenho	60
8.5.3	Teste de qualidade de resposta	60
8.6	criterios de aceite	61
8.7	Espaco para expansao iterativa	61
9	GraphRAG com Neptune	62
9.1	Escopo do capitulo	62
9.2	Objetivos de aprendizagem	62
9.3	Conceitos teoricos	63
9.3.1	Do RAG textual ao GraphRAG	63
9.3.2	Fontes de contexto combinadas	63
9.4	Implementacao orientada por passos	63
9.4.1	Passo 1 — Popular o grafo de conhecimento	63
9.4.2	Passo 2 — Ingerir documentos para mapeamento textual	63
9.4.3	Passo 3 — Executar consulta relacional no Neptune	64
9.4.4	Passo 4 — Fundir contexto textual e relacional	64
9.4.5	Passo 5 — Validar em consulta de negocio	64
9.4.6	Passo 6 — Resultado completo da aula	65
9.5	Plano de testes do capitulo	65
9.5.1	Teste funcional	65
9.5.2	Teste de desempenho	65
9.5.3	Teste de qualidade de resposta	66
9.6	criterios de aceite	66

9.7	Espaco para expansao iterativa	66
10	Pipeline Integrado em Producao	67
10.1	Escopo do capitulo	67
10.2	Objetivos de aprendizagem	67
10.3	Conceitos teoricos	68
10.3.1	Duas fases, um sistema	68
10.3.2	Estrategia de indexacao: indice unico versus indice por cliente . . .	68
10.3.3	Indice de confiabilidade	69
10.4	Implementacao orientada por passos	70
10.4.1	Passo 1 — Executar a fase offline	70
10.4.2	Passo 2 — Verificar artefatos no S3 e indices no OpenSearch	70
10.4.3	Passo 3 — Executar os tres modos via notebook	70
10.4.4	Passo 4 — Analisar o indice de confiabilidade	71
10.4.5	Passo 5 — Simular degradacao por ausencia de indexacao	71
10.4.6	Passo 6 — Resultado completo da aula	71
10.5	Plano de testes do capitulo	71
10.5.1	Teste funcional	71
10.5.2	Teste de qualidade de resposta	72
10.6	criterios de aceite	72
10.7	Espaco para expansao iterativa	72
11	Confiabilidade, Observabilidade e Operação	73
11.1	Pré-requisitos	73
11.2	Escopo do capítulo	73
11.3	Objetivos de aprendizagem	74
11.4	Conceitos teóricos	74
11.4.1	Índice de confiabilidade	74
11.4.2	OpenTelemetry e Langfuse	74
11.5	Implementação orientada por passos	75
11.5.1	Passo 1 — Índice de confiabilidade	75
11.5.2	Passo 2 — Configurar observabilidade com OTel	76
11.5.3	Passo 3 — Estados da requisição no DynamoDB	76
11.6	Autoavaliação	77
11.7	Resumo	77
11.8	Plano de testes do capítulo	77
11.9	Crítérios de aceite	78
11.10	Espaco para expansão iterativa	78

12 Métricas de Avaliação de Mercado	79
12.1 Pré-requisitos	79
12.2 Escopo do capítulo	79
12.3 Objetivos de aprendizagem	79
12.4 Conceitos teóricos	80
12.4.1 ROUGE-L — precisão lexical	80
12.4.2 BERTScore — similaridade semântica	80
12.4.3 G-Eval — avaliação por LLM	80
12.4.4 Quando usar cada métrica	80
12.5 Implementação orientada por passos	81
12.5.1 Passo 1 — ROUGE-L offline (sem GPU, sem API)	81
12.5.2 Passo 2 — Executar avaliação offline	81
12.5.3 Passo 3 — Avaliação em lote com CSV	82
12.6 Autoavaliação	82
12.7 Resumo	82
12.8 Plano de testes do capítulo	83
12.9 Critérios de aceite	83
12.10 Espaço para expansão iterativa	83
13 LLM as Judge	84
13.1 Pré-requisitos	84
13.2 Escopo do capítulo	84
13.3 Objetivos de aprendizagem	84
13.4 Conceitos teóricos	85
13.4.1 Por que LLM-as-Judge?	85
13.4.2 Rubrica de 4 dimensões	85
13.4.3 Modo mock vs. Bedrock	85
13.5 Implementação orientada por passos	86
13.5.1 Passo 1 — Estrutura da rubrica como prompt	86
13.5.2 Passo 2 — Executar em modo mock	86
13.5.3 Passo 3 — Interpretação e calibração	87
13.6 Autoavaliação	87
13.7 Resumo	87
13.8 Plano de testes do capítulo	88
13.9 Critérios de aceite	88
13.10 Espaço para expansão iterativa	88
14 Golden Dataset e Regressão de Qualidade	89
14.1 Pré-requisitos	89
14.2 Escopo do capítulo	89

14.3	Objetivos de aprendizagem	89
14.4	Conceitos teóricos	90
14.4.1	O que é um golden dataset?	90
14.4.2	Curadoria: sintético vs. humano	90
14.4.3	Regressão vs. melhoria	90
14.5	Implementação orientada por passos	91
14.5.1	Passo 1 — Estrutura de um caso do golden dataset	91
14.5.2	Passo 2 — Criar e salvar o dataset	91
14.5.3	Passo 3 — Executar regressão	91
14.5.4	Passo 4 — Interpretar o diff	92
14.5.5	Passo 5 — Integração com CI/CD	92
14.6	Autoavaliação	93
14.7	Resumo	93
14.8	Plano de testes do capítulo	93
14.9	Critérios de aceite	93
14.10	Espaço para expansão iterativa	94
14.11	Objetivos de aprendizagem	94
14.12	Implementação orientada por passos	94
14.13	Plano de testes do capítulo	94
14.13.1	Teste funcional	94
14.13.2	Teste de desempenho	94
14.13.3	Teste de qualidade de resposta	95
14.14	Critérios de aceite	95
14.15	Espaço para expansão iterativa	95
15	Roadmap de Produção e Evolução Contínua	96
15.1	Escopo do capítulo	96
15.2	Objetivos de aprendizagem	96
15.3	Modelo de maturidade	97
15.3.1	Nível 1 — MVP Offline	97
15.3.2	Nível 2 — Integração AWS	97
15.3.3	Nível 3 — Produção Sustentável	97
15.4	Priorização por impacto e esforço	97
15.5	O ciclo de melhoria contínua	97
15.6	Resumo	98
15.7	Espaço para expansão iterativa	98
16	Esteiras de CI/CD Multi-Ambiente para Produtos AWS Bancários	100
16.1	Escopo do capítulo	100
16.2	Objetivos de aprendizagem	100

16.3	Conceitos teóricos	101
16.3.1	Isolamento de contas AWS por ambiente	101
16.3.2	Esteira por produto	101
16.3.3	Fluxo de promoção entre ambientes	101
16.4	Implementação orientada por passos	101
16.4.1	Passo 1 — Estrutura de repositório por produto	101
16.4.2	Passo 2 — Configurar <i>backend</i> Terraform por ambiente	102
16.4.3	Passo 3 — Workflow de CI (pull request)	103
16.4.4	Passo 4 — Workflow de deploy com promoção controlada	104
16.5	Plano de testes do capítulo	105
16.5.1	Teste funcional	105
16.5.2	Teste de desempenho	105
16.5.3	Teste de qualidade	105
16.6	Critérios de aceite	106
16.7	Espaço para expansão iterativa	106
17	Qualidade, Segurança e Governança nas Esteiras de Produto	107
17.1	Escopo do capítulo	107
17.2	Objetivos de aprendizagem	107
17.3	Conceitos teóricos	108
17.3.1	Quality gates nas esteiras bancárias	108
17.3.2	Segurança estática: Python e Terraform	108
17.3.3	Princípio do menor privilégio em IAM	108
17.4	Implementação orientada por passos	109
17.4.1	Passo 1 — Configurar quality gate de testes e lint	109
17.4.2	Passo 2 — Testes de integração contra ambiente <i>dev</i>	109
17.4.3	Passo 3 — Análise de segurança estática	111
17.4.4	Passo 4 — Role IAM com menor privilégio via Terraform	111
17.5	Plano de testes do capítulo	113
17.5.1	Teste funcional	113
17.5.2	Teste de conformidade	113
17.6	Critérios de aceite	114
17.7	Espaço para expansão iterativa	114
18	Frontend Web com CloudFront para Operação da API e Ingestão RAG	115
18.1	Escopo do capítulo	115
18.2	Objetivos de aprendizagem	115
18.3	Conceitos teóricos	116
18.3.1	Frontend estático como superfície de operação	116
18.3.2	CloudFront + S3 privado	116

18.3.3	Chat assíncrono e ingestão RAG em uma única UI	116
18.4	Implementação orientada por passos	117
18.4.1	Passo 1 — Estruturar os assets do frontend	117
18.4.2	Passo 2 — Publicar a UI em CloudFront	117
18.4.3	Passo 3 — Implementar o fluxo de consulta assíncrona	118
18.4.4	Passo 4 — Implementar envio de documentos para RAG	118
18.4.5	Passo 5 — Planejar evolução para PDF e DOCX	118
18.5	Plano de testes do capítulo	119
18.5.1	Teste funcional	119
18.5.2	Teste de validação em produção (AWS)	119
18.5.3	Teste de segurança	120
18.6	CrITÉRIOS de aceite	120
18.7	Espaço para expansão iterativa	120
A	Apendice de Codigo Completo das Solucoes	122
A.1	Como validar a autocontencao	122
A.2	Capitulo 2 e Capitulo 10	122
A.2.1	langchain.ipynb (exportado para Python)	122
A.2.2	demo_api.ipynb (exportado para Python)	232
A.3	Capitulo 4 e Capitulo 5	241
A.3.1	mcp_mock_server.py	241
A.3.2	mcp_postgres_server.py	245
A.3.3	validate_sprint_a.py	251
A.4	Capitulo 11	254
A.4.1	otel_config.py	254
A.4.2	status.py	256
A.4.3	worker.py	257
A.5	Capitulo 12, 13 e 14	284
A.5.1	avaliar_metricas.py	284
A.5.2	llm_judge.py	291
A.5.3	golden_dataset.py	297
A.5.4	golden_baseline.json	305
A.6	Capitulo 18	306
A.6.1	app.js	306
A.6.2	config.js	313
B	Template de Capitulo	314
B.1	Objetivos do capitulo	314
B.2	Implementacao pratica	314
B.3	Politica de autocontencao (obrigatoria)	314

B.4	Plano de testes	315
B.5	Critérios de aceite	315
C	Glossário Matemático e Técnico	316

Capítulo 1

Fundamentos de IA e Modelos de Linguagem de Grande Escala

1.1 Escopo do capítulo

Este capítulo é o primeiro ato da história que este livro conta. Antes de qualquer agente, pipeline ou avaliação, é preciso compreender a peça central do sistema: o modelo de linguagem que recebe um *prompt* e produz uma resposta. Tudo que vem depois — RAG, memória, ferramentas, juízes — é uma forma de dar melhores informações a esse modelo ou de avaliar o que ele produziu.

Este capítulo apresenta os conceitos fundamentais que sustentam toda a arquitetura desenvolvida ao longo do livro: modelos de linguagem de grande escala (do inglês *Large Language Models* — LLMs), mecanismos de atenção, tokenização, janelas de contexto e engenharia de *prompts*. Ao dominar esses conceitos, o leitor adquire a base teórica necessária para compreender as decisões de projeto adotadas nos capítulos subsequentes, em especial a escolha do modelo *Claude Sonnet* via *Amazon Bedrock* como núcleo de inferência da plataforma.

Onde estamos na história: loop interno, fase de fundamentos. O sistema ainda não existe — estamos aprendendo a linguagem com a qual ele será construído.

1.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Descrever a arquitetura *Transformer* e o papel do mecanismo de atenção na geração de texto;

- Explicar o processo de tokenização e calcular o consumo estimado de *tokens* de uma requisição;
- Formular *prompts* eficazes utilizando as técnicas *zero-shot*, *few-shot* e *chain-of-thought* (CoT)¹;
- Identificar os parâmetros de amostragem — temperatura, *top-p* e *top-k* — e seu impacto na qualidade e na diversidade das respostas;
- Invocar o modelo *Claude Sonnet* via API *Amazon Bedrock* a partir de código Python.

1.3 Conceitos teóricos

1.3.1 Arquitetura Transformer

A arquitetura *Transformer*, proposta por [2], revolucionou o processamento de linguagem natural ao substituir redes recorrentes pelo mecanismo de *self-attention*, que permite ao modelo relacionar quaisquer dois tokens da sequência de entrada independentemente de sua distância posicional. Formalmente, a atenção é calculada como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (1.1)$$

em que Q , K e V são as matrizes de consulta (*query*), chave (*key*) e valor (*value*), respectivamente, e d_k é a dimensão da chave, utilizada como fator de normalização para estabilizar os gradientes durante o treinamento. A função *softmax*² normaliza os produtos escalados, produzindo pesos de atenção que somam 1. O empilhamento de múltiplas camadas de atenção multi-cabeça (*multi-head attention*) confere ao modelo a capacidade de capturar padrões sintáticos, semânticos e pragmáticos simultaneamente.

LLMs são modelos *Transformer* de único-encoder-decoder ou somente-decoder treinados em corpora massivos mediante o objetivo de previsão do próximo *token* (*next-token prediction*). O volume de dados e parâmetros — tipicamente da ordem de bilhões — confere a esses modelos capacidades emergentes não observadas em escalas menores, como raciocínio lógico, tradução sem supervisão explícita e geração de código [3].

¹ *Chain-of-Thought* é uma técnica de *prompting* que instrui o modelo a exibir o raciocínio passo a passo antes de produzir a resposta final [1].

² $\sigma(\mathbf{z})_i = e^{z_i} / \sum_j e^{z_j}$ — converte o vetor de *logits* em distribuição de probabilidade. Definição formal no Apêndice C, p. 318.

1.3.2 Tokenização

Tokens são as unidades mínimas de processamento de um LLM, obtidos por meio de algoritmos de segmentação subpalavra, como *Byte-Pair Encoding* (BPE)³ e *SentencePiece*⁴. A relação entre palavras e *tokens* varia conforme o vocabulário do modelo; como regra prática, estima-se que 1.000 palavras em português equivalem a aproximadamente 1.300–1.500 *tokens* no modelo *Claude Sonnet*.

O custo operacional e a latência de inferência são diretamente proporcionais ao número de *tokens* processados, tornando a contagem prévia um fator crítico no dimensionamento da plataforma. No projeto descrito neste livro, adota-se a biblioteca `tiktoken` (4) como estimador de *tokens* nos benchmarks de desempenho apresentados no Capítulo 12.

1.3.3 Janela de Contexto

A janela de contexto (*context window*) define o número máximo de *tokens* — somando-se entrada e saída — que o modelo processa em uma única requisição. O *Claude Sonnet 4.5*, utilizado na plataforma, suporta até 200.000 *tokens* de entrada, o que possibilita a ingestão de documentos extensos sem necessidade de segmentação agressiva. Todavia, o custo por *token* de entrada é inferior ao de saída; portanto, o projeto limita a janela de contexto injetada pelo pipeline RAG — descrito no Capítulo 5 — a 8.000 *tokens*, valor determinado empiricamente como suficiente para manter alta qualidade de resposta sem comprometer a latência.

1.3.4 Parâmetros de Amostragem

A distribuição de probabilidade sobre o vocabulário na etapa de decodificação é modulada por três parâmetros principais:

Temperatura (τ) Reescala os logits⁵ antes do *softmax*⁶: valores baixos ($\tau \rightarrow 0$) tornam a distribuição mais pontuda, favorecendo respostas deterministas; valores altos ($\tau > 1$) aumentam a entropia e, portanto, a diversidade da geração.

***Top-p* (amostragem por núcleo)** Restringe o vocabulário ativo ao menor conjunto de *tokens* cuja probabilidade acumulada supera p ; valores típicos situam-se entre 0,85 e 0,95.

Top-k Limita a amostragem aos k *tokens* mais prováveis; é frequentemente combinado com *top-p* para eliminar candidatos de baixa probabilidade.

³Definição formal e pseudocódigo no Apêndice C, p. 316.

⁴Definição e comparação com BPE no Apêndice C, p. 317.

⁵*Logits*: valores reais não normalizados produzidos pela última camada linear do modelo. Definição formal no Apêndice C, p. 317.

⁶Função $\sigma(\mathbf{z})_i = e^{z_i} / \sum_j e^{z_j}$ que converte logits em distribuição de probabilidade. Definição completa no Apêndice C, p. 318.

No projeto, adota-se $\tau = 0,3$ nas consultas RAG de modo **segmento** e **persona**, priorizando consistência e fidelidade ao contexto recuperado. Consultas de modo **twin** utilizam $\tau = 0,5$ para permitir maior variabilidade na reprodução do comportamento individual.

1.4 Implementação orientada por passos

1.4.1 Passo 1 — Configuração das credenciais AWS

O acesso ao *Amazon Bedrock* requer credenciais IAM com a permissão `bedrock:InvokeModel`. Recomenda-se utilizar perfis nomeados via *AWS CLI* para evitar a gestão manual de chaves no código:

```
1 aws configure --profile langchain-dev
2 # AWS Access Key ID: <ID>
3 # AWS Secret Access Key: <CHAVE>
4 # Default region name: us-east-1
5 # Default output format: json
```

Listing 1.1: Configuração de perfil AWS CLI.

1.4.2 Passo 2 — Primeira chamada ao modelo

O cliente Bedrock é instanciado via *boto3*, e a requisição é formatada conforme a API *Messages* da Anthropic:

```
1 import boto3, json
2
3 MODEL_ID = "us.anthropic.claude-sonnet-4-5-20250514-v1:0"
4 REGION   = "us-east-1"
5
6 client = boto3.client("bedrock-runtime", region_name=REGION)
7
8 payload = {
9     "anthropic_version": "bedrock-2023-05-31",
10    "max_tokens": 512,
11    "temperature": 0.3,
12    "messages": [
13        {
14            "role": "user",
15            "content": "Explique o conceito de tokenizacao em
                        LLMs "
```

```

16         "em ate tres paragrafos, estilo
17             academico."
18     }
19 }
20
21 response = client.invoke_model(
22     modelId=MODEL_ID,
23     body=json.dumps(payload),
24     contentType="application/json",
25     accept="application/json"
26 )
27
28 body = json.loads(response["body"].read())
29 texto = body["content"][0]["text"]
30 tokens = body["usage"]          # input_tokens, output_tokens
31
32 print(texto)
33 print(f"\nTokens consumidos: entrada={tokens['input_tokens']},
34       "
35       f"saida={tokens['output_tokens']}")

```

Listing 1.2: Primeira chamada ao *Claude Sonnet* via Bedrock.

Verifica-se que o campo `usage` retornado pela API contém a contagem exata de *tokens* de entrada e saída, permitindo o rastreamento preciso dos custos de inferência.

1.4.3 Passo 3 — Engenharia de *prompts*

A qualidade da resposta é sensível à estrutura do *prompt*. O padrão adotado no projeto separa o *system prompt* — que define o papel, o tom e as restrições do modelo — das mensagens de usuário, conforme recomendado pela documentação da Anthropic (5):

```

1 payload = {
2     "anthropic_version": "bedrock-2023-05-31",
3     "max_tokens": 1024,
4     "temperature": 0.3,
5     "system": (
6         "Voce e um assistente especializado em analise de
7         clientes "
8         "bancarios. Responda em portugues do Brasil, estilo
9         formal, "
10        "com base exclusivamente no contexto fornecido. "

```

```

9         "Nao invente informacoes ausentes do contexto."
10    ),
11    "messages": [
12        {
13            "role": "user",
14            "content": (
15                "<context>\n{contexto_recuperado}\n</context>\n"
16                "\n"
17                "Pergunta: {pergunta_usuario}"
18            )
19        }
20    ]
21 }

```

Listing 1.3: Estrutura de *system prompt* e mensagem de usuário.

O uso de delimitadores XML (<context>) é recomendado para que o modelo distinga claramente o contexto recuperado da instrução, reduzindo o risco de alucinações.

1.4.4 Passo 4 — Técnica *few-shot*

Exemplos de entrada-saída inseridos no *prompt* (*few-shot examples*) calibram o formato e o registro das respostas sem necessidade de ajuste fino (*fine-tuning*). No projeto, cada modo de consulta — **segmento**, **persona** e **twin** — utiliza de dois a três exemplos curados que demonstram a estrutura esperada da resposta, conforme detalhado no Capítulo 8.

1.5 Plano de testes do capítulo

1.5.1 Teste funcional

- **TC-01:** Invocar o modelo com *prompt* mínimo e verificar que o campo `content[0].text` está preenchido e não é vazio.
- **TC-02:** Passar um *prompt* com temperatura $\tau = 0$ e verificar que duas execuções consecutivas produzem saídas idênticas (determinismo).
- **TC-03:** Exceder o limite de *max_tokens* e confirmar que a resposta é truncada com o motivo `max_tokens` no campo `stop_reason`.

1.5.2 Teste de desempenho

- **TP-01:** Medir a latência de 10 chamadas consecutivas com *prompt* de 500 *tokens*; critério: mediana $\leq 3,0$ s (região `us-east-1`).

- **TP-02:** Calcular o custo estimado por 1.000 requisições de 1.000 *tokens* de entrada e 500 de saída com base na tabela de preços *on-demand* do Bedrock.

1.5.3 Teste de qualidade de resposta

- **TQ-01:** Formular cinco perguntas de referência sobre tokenização e avaliar as respostas quanto à precisão factual (escala 1–5).
- **TQ-02:** Verificar que a resposta a uma pergunta fora do contexto retorna a expressão de ausência de informação, sem alucinação.

1.6 Critérios de aceite

- TC-01, TC-02 e TC-03 executam sem erros bloqueantes.
- TP-01: mediana de latência $\leq 3,0$ s confirmada em execução real (evidenciar com *log* de timestamps).
- TQ-01: pontuação média $\geq 4,0/5,0$.
- TQ-02: zero alucinações detectadas nas cinco perguntas fora de contexto.

1.7 Espaço para expansão iterativa

- **v1.1** — Adicionar comparação de modelos: *Claude Haiku* (baixo custo) vs. *Claude Sonnet* (equilíbrio) vs. *Claude Opus* (máxima qualidade), com métricas de latência e custo por token.
- **v1.2** — Incluir diagrama da arquitetura *Transformer* (codificador e decodificador) adaptado ao estilo do livro.
- **Lacuna** — Avaliar impacto de diferentes estratégias de truncamento de contexto (*sliding window* vs. sumarização hierárquica) na qualidade das respostas.

Capítulo 2

Ambiente, Ferramentas e Reprodutibilidade

2.1 Escopo do capítulo

Onde estamos na história: loop interno, fundamentos operacionais. Antes de construir qualquer agente, o sistema precisa de solo firme: dependências reproduzíveis, credenciais seguras e infraestrutura declarada como código. Este capítulo é a base silenciosa sobre a qual todos os experimentos dos capítulos seguintes se apoiam.

Este capítulo descreve a configuração completa do ambiente de desenvolvimento adotado no projeto, abrangendo gerenciamento de dependências Python, credenciais AWS, provisionamento de infraestrutura via *Terraform* e organização do repositório. O objetivo é garantir que qualquer colaborador consiga reproduzir os experimentos descritos ao longo do livro a partir de um ambiente zerado, sem ambiguidades de versão ou de configuração.

2.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Configurar um ambiente Python 3.11 isolado com todas as dependências do projeto fixadas em versão exata;
- Organizar credenciais AWS de forma segura utilizando perfis nomeados e variáveis de ambiente;
- Provisionar a infraestrutura AWS (Lambda, DynamoDB, SQS, OpenSearch, Neptune) via *Terraform* em uma única execução reproduzível;

- Executar os *notebooks* Jupyter do projeto e verificar os resultados esperados de cada seção.

2.3 Conceitos e prerequisites

2.3.1 Versões e dependências

A reprodutibilidade de experimentos com LLMs é comprometida por três fontes principais de não-determinismo: (i) versão do modelo de inferência, (ii) versão das bibliotecas de orquestração e (iii) configuração do ambiente de execução. Para mitigar esses riscos, o projeto adota as seguintes práticas:

- Todas as dependências são declaradas com versão exata no arquivo `requirements.txt` (não utilizar `~=` ou `>=` sem limites superiores);
- O modelo de inferência é referenciado pelo identificador versionado `us.anthropic.claude-sonnet-4-5` e não por alias genérico como `claude-sonnet-latest`;
- A versão do Python (3.11.x) e do *Terraform* (1.7.x) são documentadas no arquivo `README.md` na raiz do repositório.

2.3.2 Estrutura do repositório

O repositório é organizado segundo o princípio de separação de responsabilidades, distinguindo código de aplicação, infraestrutura como código e artefatos de documentação:

```
1 langchain/
2   infraestrutura/           # Terraform (modulos por servico AWS
3       )
4   modules/
5       lambda/src/          # Codigo das funcoes Lambda
6       opensearch/
7       neptune/
8       dynamodb/
9       sqs/
10      api_gateway/
11 liv-ro/                    # Fonte LaTeX do livro
12     chapters/
13 decisoes_projeto/         # ADRs (Architecture Decision
14     Records)
15 tests/                    # Testes unitarios e de integracao
16 langchain.ipynb           # Notebook didatico principal
17 demo_api.ipynb            # Notebook de demonstracao da API
```

```
16 README.md
```

Listing 2.1: Estrutura de diretórios do repositório.

2.4 Implementação orientada por passos

2.4.1 Passo 1 — Clonar o repositório e criar o ambiente virtual

```
1 git clone https://github.com/<org>/langchain-rag.git
2 cd langchain-rag
3
4 # Criar e ativar o ambiente virtual
5 python3.11 -m venv .venv
6 source .venv/bin/activate           # Linux/macOS
7 # .venv\Scripts\Activate.ps1       # Windows PowerShell
8
9 # Instalar dependências fixadas
10 pip install --upgrade pip
11 pip install -r requirements.txt
```

Listing 2.2: Criação do ambiente virtual Python isolado.

As dependências mínimas para executar os notebooks do projeto são listadas abaixo com as versões verificadas:

```
1 boto3==1.34.69
2 langchain==0.2.16
3 langchain-community==0.2.16
4 langgraph==0.2.4
5 faiss-cpu==1.8.0
6 tiktoken==0.7.0
7 jupyter==1.0.0
8 nbformat==5.10.4
9 pypdf==4.3.1
10 requests==2.32.3
```

Listing 2.3: Dependências principais (`requirements.txt`).

2.4.2 Passo 2 — Configurar credenciais AWS

O acesso aos serviços AWS é controlado por funções IAM (*Identity and Access Management*). Para desenvolvimento local, utiliza-se um perfil nomeado com permissões mínimas necessárias:


```
1 aws configure --profile langchain-dev
2 # AWS Access Key ID:      <ID gerado no IAM>
3 # AWS Secret Access Key: <CHAVE>
4 # Default region name:    sa-east-1
5 # Default output format: json
6
7 # Exportar o perfil como variavel de ambiente no shell
8 export AWS_PROFILE=langchain-dev
```

Listing 2.4: Configuração do perfil AWS para desenvolvimento.

Importante: credenciais não devem ser *hardcoded* no código nem versionadas no repositório. Recomenda-se incluir `.env` e `*.tfvars` no `.gitignore`.

2.4.3 Passo 3 — Provisionar a infraestrutura com Terraform

A infraestrutura AWS é descrita como código no diretório `infraestrutura/`. O provisionamento completo — incluindo VPC, API Gateway, filas SQS, funções Lambda, tabela DynamoDB, domínio OpenSearch e cluster Neptune — é executado com:

```
1 cd infraestrutura/
2 terraform init
3 terraform validate
4 terraform plan -out=deploy.tfplan
5 terraform apply deploy.tfplan
```

Listing 2.5: Provisionamento da infraestrutura via Terraform.

Após a conclusão, os *outputs* relevantes — URL da API, *endpoints* do OpenSearch e do Neptune, nome da tabela DynamoDB — são exibidos no terminal e podem ser capturados com:

```
1 terraform output -json > ../model_data.json
```

Listing 2.6: Extração de outputs do Terraform.

2.4.4 Passo 4 — Executar os notebooks

Com o ambiente configurado e a infraestrutura provisionada, os *notebooks* são iniciados via Jupyter ou diretamente no VS Code com a extensão *Jupyter*:

```
1 jupyter lab --no-browser --port=8888
```

Listing 2.7: Inicialização do servidor Jupyter.

Recomenda-se executar os notebooks na ordem natural dos capítulos — `langchain.ipynb` (conceitos) e `demo_api.ipynb` (demonstração da API) — pois as células iniciais de cada *notebook* definem constantes e funções utilizadas nas células subsequentes.

2.5 Plano de testes do capítulo

2.5.1 Teste funcional

- **TC-01:** Executar `pip install -r requirements.txt` em ambiente zerado e verificar ausência de conflitos de versão (exit code 0).
- **TC-02:** Executar `python -c "import boto3, langchain, langgraph, faiss"` e verificar que nenhum `ImportError` é lançado.
- **TC-03:** Executar `aws sts get-caller-identity` com o perfil configurado e verificar que o ARN retornado pertence à conta correta.
- **TC-04:** Executar `terraform validate` no diretório de infraestrutura e verificar mensagem `Success! The configuration is valid.`

2.5.2 Teste de desempenho

- **TP-01:** Medir o tempo total de `terraform apply` em ambiente limpo; critério: conclusão em menos de 20 minutos para o conjunto completo de módulos.
- **TP-02:** Medir o tempo de inicialização do kernel Jupyter até a execução da primeira célula de importação; critério: menos de 30s.

2.5.3 Teste de qualidade

- **TQ-01:** Verificar que o arquivo `.gitignore` exclui `*.tfstate`, `*.tfvars` e `.env` do versionamento.
- **TQ-02:** Confirmar que nenhuma chave de acesso AWS está presente no histórico de commits via `git log --all --full-diff | grep -i "AKIA"`.

2.6 Critérios de aceite

- TC-01 a TC-04 concluídos sem erros em um ambiente Windows 11 com Python 3.11 e Terraform 1.7.
- TP-01: `terraform apply` concluído em menos de 20 minutos (evidenciar com *log* de execução).

- TQ-01 e TQ-02 aprovados; nenhuma credencial versionada detectada.

2.7 Espaço para expansão iterativa

- **v2.1** — Adicionar *devcontainer* Docker para eliminar dependência de instalação local de Python e Terraform.
- **v2.2** — Incluir pipeline CI/CD (GitHub Actions) que execute `terraform validate` e os testes unitários a cada *pull request*.
- **Lacuna** — Avaliar o uso de *Terraform Workspaces* para isolar ambientes de desenvolvimento, homologação e produção sem duplicar código de infraestrutura.

Capítulo 3

Agentes com LangGraph e Memória

3.1 Pré-requisitos

Para aproveitar plenamente este capítulo, o leitor deve ter concluído os capítulos anteriores e estar familiarizado com:

- Conceitos básicos de LLMs e *prompting* estruturado (Capítulo 1);
- Configuração do ambiente Python com `langchain-aws` e acesso ao *Amazon Bedrock* (Capítulo 2);
- Noções de grafos dirigidos e funções de alta ordem em Python (iteradores, decoradores, `TypedDict`).

3.2 Escopo do capítulo

Onde estamos na história: loop interno, primeira virada. Nos capítulos 1 e 2 aprendemos a conversar com um LLM. Agora o sistema passa a agir: ele pode consultar ferramentas, manter memória e tomar decisões em múltiplos turnos. Esta é a peça que conectará, nos capítulos 7–9, ao pipeline assíncrono, à segmentação e ao GraphRAG.

Este capítulo apresenta a construção de agentes de IA orientados a grafos de estado utilizando a biblioteca *LangGraph* (6), que estende o *LangChain* (7) com suporte nativo a ciclos, persistência de estado e orquestração de múltiplos atores. O ciclo ReAct (*Reason + Act*), proposto por 8, é adotado como padrão de raciocínio; o agente alterna entre etapas de análise (*thought*), invocação de ferramentas (*action*) e observação dos resultados (*observation*) até satisfazer o critério de parada. A gestão de memória — tanto de curto prazo (histórico de mensagens) quanto de médio prazo (vetor semântico FAISS) — é descrita e implementada ao longo do capítulo. Esta combinação corresponde ao padrão de *basic reasoning agent* com extensão de memória no AWS Prescriptive Guidance (9).

3.3 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Modelar o comportamento de um agente como um grafo de estados tipado com `TypedDict` e arestas condicionais;
- Implementar o ciclo ReAct com o *LangGraph* conectando um nó de raciocínio ao *Claude Sonnet* e nós de ação a ferramentas externas;
- Gerenciar a memória de curto prazo por meio do histórico de mensagens (`MessagesState`) e persistir o estado entre invocações com `MemorySaver`;
- Integrar memória vetorial FAISS como fonte de recuperação semântica para subsidiar as decisões do agente.

3.4 Conceitos teóricos

3.4.1 Grafo de estados e nós de processamento

No *LangGraph*, um agente é modelado como um grafo dirigido em que cada nó é uma função Python que recebe e retorna um dicionário de estado. O estado é definido por uma classe `TypedDict` que declara explicitamente os campos trocados entre os nós, garantindo segurança de tipos em tempo de compilação:

```
1 from typing import TypedDict, Annotated, Sequence
2 from langchain_core.messages import BaseMessage
3 from langgraph.graph.message import add_messages
4
5 class EstadoAgente(TypedDict):
6     messages: Annotated[Sequence[BaseMessage], add_messages]
7     contexto_rag: str
8     tentativas: int
```

Listing 3.1: Definição do estado do agente.

O redutor `add_messages` garante que novas mensagens sejam acumuladas em vez de substituírem o histórico, preservando o contexto de conversação entre turnos. O campo `messages` corresponde ao tipo `MessagesState` — abstração de conveniência do *LangGraph* que combina a lista de mensagens com o redutor `add_messages`.

3.4.2 Ciclo ReAct

O ciclo ReAct é implementado como uma sequência de arestas no grafo: o nó `raciocinar` invoca o LLM e gera uma mensagem que pode conter chamadas de ferramenta (*tool calls*);

o nó `executar_ferramentas` processa essas chamadas e retorna as observações; a aresta condicional `deve_continuar` verifica se o LLM solicitou mais ferramentas ou produziu uma resposta final, determinando o próximo nó. Conforme 8, a interlocução entre raciocínio e ação aumenta significativamente a taxa de sucesso em tarefas de question-answering multietapas em comparação com abordagens puramente encadeadas.

3.4.3 Persistência de estado com `MemorySaver`

O `MemorySaver` é um *checkpointer* que persiste o estado do grafo entre invocações em memória local (adequado para desenvolvimento) ou em banco de dados externo (adequado para produção). Cada sessão de conversação é identificada por um `thread_id`, permitindo que o agente retome o contexto após interrupções sem reprocessar o histórico completo.

3.5 Implementação orientada por passos

3.5.1 Passo 1 — Definir as ferramentas do agente

Ferramentas são funções Python decoradas com `@tool`, que expõem ao LLM uma interface descritiva com *docstring* e assinatura de tipos:

```
1 from langchain_core.tools import tool
2
3 @tool
4 def buscar_perfil_cliente(cliente_id: str) -> dict:
5     """Recupera o perfil de um cliente bancario pelo ID.
6     Retorna segmento, score de credito e produtos ativos."""
7     # Implementacao: consulta DynamoDB / OpenSearch
8     ...
9
10 @tool
11 def calcular_score_credito(renda: float, dividas: float) ->
    float:
12     """Calcula o score de credito simplificado dado renda e
13     dividas.
14     Retorna um valor entre 0.0 e 1.0."""
15     return max(0.0, min(1.0, (renda - dividas) / renda))
16 ferramentas = [buscar_perfil_cliente, calcular_score_credito]
```

Listing 3.2: Definição de ferramentas com `@tool`.

3.5.2 Passo 2 — Construir o grafo ReAct

O grafo é instanciado com `StateGraph` — classe central do *LangGraph* que recebe o tipo de estado e gerencia nós e arestas. O modelo de linguagem é acessado via `ChatBedrock`, cliente *LangChain* para o *Amazon Bedrock*. A execução das ferramentas é delegada ao `ToolNode`, nó pré-construído que processa automaticamente as *tool calls* retornadas pelo LLM:

```
1 import boto3
2 from langchain_aws import ChatBedrock
3 from langchain_core.messages import SystemMessage
4 from langgraph.graph import StateGraph, END
5 from langgraph.prebuilt import ToolNode
6 from langgraph.checkpoint.memory import MemorySaver
7
8 MODEL_ID = "us.anthropic.claude-sonnet-4-5-20250514-v1:0"
9
10 llm = ChatBedrock(
11     model_id=MODEL_ID,
12     region_name="us-east-1",
13     model_kwargs={"temperature": 0.3, "max_tokens": 2048}
14 ).bind_tools(ferramentas)
15
16 SYSTEM = SystemMessage(content=(
17     "Voce e um assistente financeiro especializado. "
18     "Use as ferramentas disponiveis para responder com precisao
19     . "
20     "Nunca invente dados ausentes nas ferramentas."
21 ))
22
23 def raciocinar(state: EstadoAgente) -> dict:
24     resposta = llm.invoke([SYSTEM] + list(state["messages"]))
25     return {"messages": [resposta]}
26
27 def deve_continuar(state: EstadoAgente) -> str:
28     ultima = state["messages"][-1]
29     return "ferramentas" if ultima.tool_calls else END
30
31 grafo = StateGraph(EstadoAgente)
32 grafo.add_node("raciocinar", raciocinar)
33 grafo.add_node("ferramentas", ToolNode(ferramentas))
34 grafo.set_entry_point("raciocinar")
```

```

34 grafo.add_conditional_edges("raciocinar", deve_continuar)
35 grafo.add_edge("ferramentas", "raciocinar")
36
37 agente = grafo.compile(checkpointer=MemorySaver())

```

Listing 3.3: Construção do grafo de agente ReAct com *LangGraph*.

3.5.3 Passo 3 — Invocar o agente com persistência de contexto

```

1 from langchain_core.messages import HumanMessage
2
3 config = {"configurable": {"thread_id": "sessao-cliente-001"}}
4
5 # Primeiro turno
6 resultado = agente.invoke(
7     {"messages": [HumanMessage(content="Qual o perfil do
8         cliente C-0042?")]}],
9     config=config
10 )
11 print(resultado["messages"][-1].content)
12
13 # Segundo turno -- agente retoma o historico do thread
14 resultado = agente.invoke(
15     {"messages": [HumanMessage(content="E o score de credito
16         dele?")]}],
17     config=config
18 )
19 print(resultado["messages"][-1].content)

```

Listing 3.4: Invocação do agente com `thread_id` para persistência de contexto.

Observa-se que no segundo turno o agente não necessita receber novamente o contexto do cliente; o `MemorySaver` recupera o estado persistido do `thread_id` e o *Claude Sonnet* pode referenciar a resposta anterior sem ambiguidade.

3.5.4 Passo 4 — Integrar memória vetorial FAISS

Para consultas que exigem recuperação semântica de documentos históricos — políticas, contratos, notas de entrevista — integra-se um índice FAISS¹ como ferramenta do agente:

```

1 from langchain_community.vectorstores import FAISS

```

¹Facebook AI Similarity Search (FAISS) é uma biblioteca de busca por similaridade de vetores de alta dimensão (10).


```

2 from langchain_aws import BedrockEmbeddings
3
4 embeddings = BedrockEmbeddings(
5     model_id="amazon.titan-embed-text-v2:0",
6     region_name="us-east-1"
7 )
8 indice = FAISS.load_local("faiss_index", embeddings,
9                           allow_dangerous_deserialization=True
10                          )
11
12 @tool
13 def buscar_documentos(consulta: str) -> str:
14     """Busca documentos relevantes no repositório corporativo.
15     Use quando precisar de políticas, procedimentos ou
16     contratos."""
17     docs = indice.similarity_search(consulta, k=3)
18     return "\n---\n".join(d.page_content for d in docs)
19
20 ferramentas.append(buscar_documentos)

```

Listing 3.5: Recuperação semântica via FAISS integrada como ferramenta.

3.6 Padrões alternativos de agente

O padrão ReAct é o mais adotado na prática por sua simplicidade e compatibilidade direta com o *LangGraph*, mas não é universal. A escolha do padrão deve guiar-se pela natureza da tarefa, pelo custo de latência e pelo grau de previsibilidade do fluxo de execução. A Tabela 3.1 sintetiza os principais padrões e seus critérios de uso.

3.6.1 Plan-and-Execute

No padrão *Plan-and-Execute*, o grafo é dividido em dois nós distintos: *planejar* e *executar*. O nó de planejamento invoca o LLM uma única vez para gerar uma lista ordenada de etapas; o nó de execução percorre cada etapa sequencialmente, podendo chamar ferramentas. Esse padrão reduz o número de chamadas ao LLM em relação ao ReAct quando o plano é estável, pois o raciocínio de alto nível acontece apenas uma vez:

```

1 from typing import TypedDict, List
2 from langgraph.graph import StateGraph, END
3
4 class EstadoPlan(TypedDict):

```

Tabela 3.1: Comparação dos principais padrões de agente com *LangGraph*.

Padrão	Descrição	Quando usar
ReAct	Ciclo <i>thought</i> $\rightarrow$ <i>action</i> $\rightarrow$ <i>observation</i> até critério de parada.	Tarefas abertas com número indefinido de etapas.
Plan-and-Execute	Fase de planejamento gera um plano de N passos; fase de execução percorre cada passo sequencialmente.	Tarefas longas e determinísticas com plano estável.
Reflexion	O agente auto-critica sua resposta e refina iterativamente antes de finalizar.	Geração de texto, código ou análise que requer autocorreção.
Multi-Agent Supervisor	Um agente orquestrador delega subproblemas a agentes especialistas; cada especialista opera de forma independente.	Domínios distintos que exigem especialistas separados (ex.: PF, PJ, FP, PJA).
Tool Calling puro	Sem ciclo iterativo; o LLM realiza uma única chamada de ferramenta e finaliza.	Tarefas de uma etapa com latência crítica.
Structured Output	Sem ferramentas; o LLM retorna JSON estruturado via <code>with_structured_output</code> .	Extração, classificação e transformação de dados.

```

5     objetivo: str
6     plano: List[str]          # lista de passos gerada pelo
                                planejador
7     passo_atual: int
8     resultados: List[str]
9
10    def planejar(state: EstadoPlan) -> dict:
11        """Invoca o LLM para gerar o plano completo."""
12        resposta = llm.invoke(
13            f"Decomponha em passos: {state['objetivo']}"
14        )
15        passos = resposta.content.strip().split("\n")
16        return {"plano": passos, "passo_atual": 0}
17
18    def executar(state: EstadoPlan) -> dict:
19        """Executa o passo atual do plano."""
20        passo = state["plano"][state["passo_atual"]]
21        resultado = llm.invoke(f"Execute: {passo}")
22        return {
23            "resultados": state["resultados"] + [resultado.content
24            ],
25            "passo_atual": state["passo_atual"] + 1,
26        }
27
28    def ha_proximo(state: EstadoPlan) -> str:
29        return "executar" if state["passo_atual"] < len(state["
30            plano"]) else END
31
32    grafo_plan = StateGraph(EstadoPlan)
33    grafo_plan.add_node("planejar", planejar)
34    grafo_plan.add_node("executar", executar)
35    grafo_plan.set_entry_point("planejar")
36    grafo_plan.add_edge("planejar", "executar")
37    grafo_plan.add_conditional_edges("executar", ha_proximo)
38    agente_plan = grafo_plan.compile()

```

Listing 3.6: Esboço do padrão *Plan-and-Execute* com *LangGraph*.

3.6.2 Reflexion

No padrão *Reflexion*, após gerar uma resposta, o agente invoca um segundo nó de crítica que avalia a qualidade da resposta e decide se ela deve ser refinada ou finalizada. O ciclo

de reflexão tem um limite máximo de iterações para evitar loop infinito:

```
1 class EstadoReflexion(TypedDict):
2     pergunta: str
3     rascunho: str
4     critica: str
5     iteracao: int
6     MAX_ITER: int
7
8 def gerar(state: EstadoReflexion) -> dict:
9     rascunho = llm.invoke(state["pergunta"]).content
10    return {"rascunho": rascunho, "iteracao": state["iteracao"]
11           + 1}
12
13 def criticar(state: EstadoReflexion) -> dict:
14     prompt = (
15         f"Avalie a resposta: {state['rascunho']}\n"
16         "Aponte falhas factuais ou de coerencia. "
17         "Responda 'OK' se estiver correta."
18     )
19     critica = llm.invoke(prompt).content
20     return {"critica": critica}
21
22 def deve_refinar(state: EstadoReflexion) -> str:
23     if "OK" in state["critica"] or state["iteracao"] >= state["
24         MAX_ITER"]:
25         return END
26     return "gerar"
27
28 grafo_ref = StateGraph(EstadoReflexion)
29 grafo_ref.add_node("gerar", gerar)
30 grafo_ref.add_node("criticar", criticar)
31 grafo_ref.set_entry_point("gerar")
32 grafo_ref.add_edge("gerar", "criticar")
33 grafo_ref.add_conditional_edges("criticar", deve_refinar)
34 agente_ref = grafo_ref.compile()
```

Listing 3.7: Esboço do padrão *Reflexion* com *LangGraph*.

3.6.3 Multi-Agent Supervisor

No padrão *Multi-Agent Supervisor*, um agente orquestrador recebe a requisição, decide qual agente especialista deve atendê-la e consolida as respostas. Cada especialista opera com suas próprias ferramentas e prompt de sistema, sem conhecer os demais. No contexto deste projeto, esse padrão mapeia diretamente os quatro segmentos bancários do `diagram_v7`: PF, PJ, FP e PJA—cada um com seu próprio *Knowledge Base* e critérios de avaliação:

```

1 from typing import Literal
2 from langgraph.graph import StateGraph, END
3
4 Segmento = Literal["PF", "PJ", "FP", "PJA"]
5
6 class EstadoSupervisor(TypedDict):
7     pergunta: str
8     segmento: Segmento
9     resposta: str
10
11 def supervisor(state: EstadoSupervisor) -> dict:
12     """Classifica a pergunta no segmento correto."""
13     prompt = (
14         f"Classifique em PF, PJ, FP ou PJA: {state['pergunta']}\n"
15         "Responda apenas com o código do segmento."
16     )
17     seg = llm.invoke(prompt).content.strip()
18     return {"segmento": seg}
19
20 def agente_pf(state: EstadoSupervisor) -> dict:
21     resp = llm_pf.invoke(state["pergunta"]) # LLM com KB PF
22     return {"resposta": resp.content}
23
24 def agente_pj(state: EstadoSupervisor) -> dict:
25     resp = llm_pj.invoke(state["pergunta"]) # LLM com KB PJ
26     return {"resposta": resp.content}
27
28 def rotear(state: EstadoSupervisor) -> str:
29     return state["segmento"].lower() # "pf" / "pj" / "fp" / "
30                                     pja"
31
32 grafo_sup = StateGraph(EstadoSupervisor)
33 grafo_sup.add_node("supervisor", supervisor)

```

```

33 grafo_sup.add_node("pf", agente_pf)
34 grafo_sup.add_node("pj", agente_pj)
35 # ... adicionar fp e pja analogamente
36 grafo_sup.set_entry_point("supervisor")
37 grafo_sup.add_conditional_edges("supervisor", rotear)
38 for seg in ["pf", "pj", "fp", "pja"]:
39     grafo_sup.add_edge(seg, END)
40 agente_supervisor = grafo_sup.compile()

```

Listing 3.8: Esboço do padrão *Multi-Agent Supervisor* com *LangGraph*.

O padrão *Multi-Agent Supervisor* é o caminho natural de evolução do projeto para incorporar os LLM Judges por segmento previstos no `diagram_v7` (ver Capítulo 13).

3.7 Plano de testes do capítulo

3.7.1 Teste funcional

- **TC-01:** Invocar o agente com uma pergunta que requer exatamente uma chamada de ferramenta e verificar que o grafo percorre o ciclo `raciocinar` → `ferramentas` → `raciocinar` → `END`.
- **TC-02:** Verificar que o histórico é preservado entre dois turnos consecutivos com o mesmo `thread_id`.
- **TC-03:** Perguntar sobre informação não disponibilizada em nenhuma ferramenta e verificar que o agente responde com declaração de ausência, sem alucinação.
- **TC-04:** Trocar o `thread_id` e verificar que o contexto não vaza entre sessões distintas.

3.7.2 Teste de desempenho

- **TP-01:** Medir a latência de 10 invocações com uma chamada de ferramenta cada; critério: mediana $\leq 4,0$ s.
- **TP-02:** Medir o número de *tokens* acumulados após 5 turnos consecutivos e verificar que não ultrapassa 20.000 *tokens* de entrada (janela de contexto controlada).

3.7.3 Teste de qualidade de resposta

- **TQ-01:** Avaliar 10 pares pergunta-resposta com o LLM-as-Judge (detalhado no Capítulo 13); critério: pontuação média de fidelidade $\geq 8,0/10$.

- **TQ-02:** Verificar que o agente não recorre a ferramentas desnecessariamente em perguntas que podem ser respondidas com o histórico da conversação.

3.8 Critérios de aceite

- TC-01 a TC-04 aprovados sem erros bloqueantes.
- TP-01: mediana de latência $\leq 4,0$ s confirmada (evidenciar com *timestamps* no notebook).
- TQ-01: pontuação média $\geq 8,0/10$ em 10 perguntas de referência.

3.9 Autoavaliação

Responda **V** (verdadeiro) ou **F** (falso) para cada afirmativa. O gabarito comentado segue ao final da seção.

1. No ciclo ReAct, o nó `raciocinar` é invocado uma única vez por requisição.
2. O redutor `add_messages` substitui o histórico anterior a cada turno, mantendo apenas a mensagem mais recente.
3. O `MemorySaver` persiste o estado do grafo entre requisições distintas que compartilham o mesmo `thread_id`.
4. No padrão *Plan-and-Execute*, o LLM é invocado uma vez para planejar e uma vez para cada passo do plano.
5. Um agente ReAct que não encontra informação em nenhuma ferramenta deve alucinar uma resposta plausível para não retornar vazio.
6. O padrão *Multi-Agent Supervisor* é adequado quando diferentes domínios exigem especialistas distintos, como os segmentos bancários PF, PJ, FP e PJA.
7. A aresta condicional `deve_continuar` termina o grafo quando a última mensagem do agente não contém *tool calls*.

Gabarito comentado.

1. **F** — O nó `raciocinar` é invocado a cada iteração do ciclo; pode ser chamado diversas vezes até o critério de parada ser atingido.
2. **F** — O redutor `add_messages` *acumula* mensagens; sem ele, cada invocação sobrescreveria o histórico.

3. **V** — Esse é exatamente o papel do **MemorySaver**: correlacionar invocações pelo `thread_id` e restaurar o estado persistido.
4. **V** — O planejador roda uma vez (gera o plano completo) e o executor roda uma vez por passo; o número total de chamadas ao LLM é $1 + N_{\text{passos}}$.
5. **F** — O agente deve declarar que a informação não está disponível (TC-03). Alucinação é um defeito crítico em sistemas bancários.
6. **V** — O roteamento por segmento é a motivação central do padrão supervisor no contexto deste projeto.
7. **V** — A ausência de `tool_calls` na última mensagem sinaliza que o agente produziu uma resposta final e o grafo deve terminar em **END**.

3.10 Questão para reflexão

Considere um sistema bancário em produção que atende simultaneamente clientes PF e PJ. O agente ReAct atual usa um único **MemorySaver** em memória para todos os `thread_ids`.

Em que cenários essa arquitetura pode falhar ou gerar riscos de privacidade? Como o padrão *Multi-Agent Supervisor* com *checkpointers* isolados por segmento endereçaria esses riscos? Quais contrapartidas de latência e custo operacional essa mudança introduziria?

Não existe uma única resposta correta: o objetivo é confrontar as decisões arquiteturais com os requisitos reais de isolamento de dados, conformidade com a LGPD e desempenho em carga.

3.11 Resumo

- **LangGraph** modela agentes como grafos de estado com nós (funções Python) e arestas condicionais.
- O ciclo **ReAct** alterna *thought*, *action* e *observation* até uma condição de parada.
- **add_messages** acumula o histórico; **MemorySaver** persiste o estado entre invocações via `thread_id`.
- **Ferramentas** são funções decoradas com `@tool`; o **ToolNode** processa as *tool calls* automaticamente.
- **Plan-and-Execute** separa planejamento de execução; indicado quando o plano é estável e previsível.

- **Reflexion** adiciona um ciclo de autocrítica com limite de iterações; indicado para geração de conteúdo que exige refinamento.
- **Multi-Agent Supervisor** roteia requisições para especialistas independentes; padrão de evolução natural deste projeto para os segmentos PF/PJ/FP/PJA.

3.12 Para saber mais

- **8** — Artigo original do padrão ReAct; apresenta os experimentos de *question-answering* multietapas que motivaram a adoção do padrão.
- **Documentação oficial do LangGraph** (<<https://langchain-ai.github.io/langgraph/>>) — Referência para os conceitos de `StateGraph`, `ToolNode`, `MemorySaver` e padrões multi-agente.
- **11** — Artigo que formaliza o padrão Reflexion; recomendado antes de implementar agentes de geração de código ou análise de documentos.
- **AWS Bedrock Agents**² — Alternativa gerenciada para agentes em produção; útil para comparar com a abordagem LangGraph quando o requisito é reduzir operação.
- **FAISS** (10) — Referência para entender os algoritmos de busca por similaridade (*HNSW*, *IVF*) utilizados na memória vetorial do agente.

3.13 Espaço para expansão iterativa

- **v3.1** — Substituir o `MemorySaver` em memória pelo `AsyncPostgresSaver` — *checkpoint* assíncrono baseado em PostgreSQL — para persistência em produção com múltiplos usuários concorrentes.
- **v3.2** — Implementar supervisão hierárquica com um agente orquestrador (*supervisor*) que delega subproblemas a agentes especializados (financeiro, jurídico, atendimento), conforme o padrão *multi-agent supervisor* do *LangGraph*.
- **Lacuna** — Investigar o impacto da ordem das ferramentas no *prompt* sobre a taxa de chamadas incorretas (*hallucinated tool calls*).

²<<https://docs.aws.amazon.com/bedrock/latest/userguide/agents.html>>

Capítulo 4

Tools, Skills e MCP

4.1 Pré-requisitos

Para aproveitar este capítulo, o leitor deve ter concluído:

- Capítulo 3 — ciclo ReAct e grafos de estado com LangGraph;
- Familiaridade com funções assíncronas em Python (`async/await`).

4.2 Escopo do capítulo

Onde estamos na história: loop interno, segunda virada. O agente já sabe raciocinar em ciclos (cap. 3). Agora ele precisa de mãos: ferramentas que consultam bancos de dados, APIs e fontes externas. O protocolo MCP é a linguagem padronizada com que o agente declara e invoca essas ferramentas — a mesma linguagem que conectará, nos capítulos 7–10, ao pipeline de produção.

Este capítulo apresenta três camadas de extensibilidade do agente:

1. **Tools nativas LangChain:** funções Python decoradas com `@tool` que o agente pode invocar diretamente;
2. **Skills compostas:** agrupamento de tools com contexto de negócio embutido (ex. *skill* de investimentos);
3. **MCP (*Model Context Protocol*):** protocolo padronizado pela Anthropic que expõe ferramentas via servidor `stdio` ou HTTP, permitindo reutilização entre agentes e equipes. No AWS Prescriptive Guidance, esta camada corresponde ao padrão de *tool-based agents for servers* (9).

4.3 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Criar e registrar tools nativas no grafo LangGraph;
- Diferenciar tool, skill e servidor MCP;
- Implementar um servidor MCP com transporte `stdio` usando a biblioteca `mcp`;
- Conectar um agente LangGraph a um servidor MCP externo;
- Validar a integração com casos de teste offline.

4.4 Conceitos teóricos

4.4.1 Tools nativas: funções como ações do agente

No LangGraph, uma *tool* é qualquer função Python decorada com `@tool` do LangChain. O agente recebe a lista de tools no momento da construção do grafo e decide, a cada nó de raciocínio ReAct, qual tool invocar e com quais argumentos.

```
1 from langchain_core.tools import tool
2
3 @tool
4 def consultar_saldo(cliente_id: str) -> str:
5     """Retorna o saldo atual do cliente. Use quando o usuario
6         perguntar
7         sobre saldo, limite ou posicao financeira."""
8     # Em producao: chamada ao DynamoDB ou API core bancario
9     saldos = {"C001": 15000.00, "C002": 8500.50, "C003":
10              22000.00}
11     saldo = saldos.get(cliente_id)
12     if saldo is None:
13         return f"Cliente {cliente_id} nao encontrado."
14     return f"Saldo do cliente {cliente_id}: R$ {saldo:,.2f}"
```

Listing 4.1: Tool nativa: consulta de saldo de cliente.

O *docstring* da função é o que o LLM lê para decidir quando usar a tool. Docstrings claros e com exemplos de contexto de uso melhoram significativamente a precisão do agente.

4.4.2 Skills: tools com contexto de negócio

Uma *skill* é um conjunto de tools relacionadas que formam uma capacidade de negócio coesa. No projeto bancário, a *skill* de investimentos agrupa as tools de consulta de produtos, simulação de rendimento e recomendação por perfil. A vantagem é que o agente pode ser instanciado com skills diferentes conforme o segmento do cliente, sem alterar a lógica do grafo.

4.4.3 MCP: protocolo padronizado para ferramentas

O *Model Context Protocol* (MCP) é um protocolo aberto que padroniza como servidores de ferramentas comunicam suas capacidades a agentes de IA (12). A arquitetura tem três papéis:

- **Cliente MCP:** o agente (LangGraph, Claude Desktop, etc.) que solicita ferramentas;
- **Servidor MCP:** processo que expõe ferramentas via `stdio` (padrão para integração local) ou HTTP/SSE (para servidores remotos);
- **Host:** orquestrador que gerencia a conexão entre cliente e servidor (ex. Claude Desktop, VS Code MCP extension).

A comunicação segue o protocolo JSON-RPC 2.0. O servidor responde a três operações principais: `tools/list` (anuncia ferramentas), `tools/call` (executa uma ferramenta) e `resources/list` (expõe recursos estáticos).

A vantagem do MCP sobre tools nativas é o **desacoplamento**: o servidor MCP pode ser desenvolvido em qualquer linguagem, versionado separadamente e reutilizado por múltiplos agentes e equipes.

4.5 Implementação orientada por passos

4.5.1 Passo 1 — Criar tools nativas e registrar no agente

Com base no grafo ReAct do capítulo anterior, adicionamos tools específicas ao domínio bancário:

```
1 from langchain_core.tools import tool
2 from langgraph.prebuilt import create_react_agent
3 from langchain_aws import ChatBedrockConverse
4
5 @tool
6 def consultar_saldo(cliente_id: str) -> str:
7     """Retorna o saldo atual do cliente."""
```

```

8     saldos = {"C001": 15000.00, "C002": 8500.50}
9     s = saldos.get(cliente_id)
10    return f"R$ {s:,.2f}" if s else f"Cliente {cliente_id} nao
        encontrado."
11
12 @tool
13 def listar_produtos_investimento(perfil: str) -> str:
14     """Lista produtos de investimento adequados ao perfil.
15     Perfis validos: conservador, moderado, arrojado."""
16     catalogo = {
17         "conservador": ["Tesouro Selic", "CDB 100% CDI", "LCI
            isenta IR"],
18         "moderado":     ["Fundo Multimercado", "CRA", "
            Debentures"],
19         "arrojado":     ["Acoes growth", "FII", "COE estruturado
            "],
20     }
21     produtos = catalogo.get(perfil.lower(), [])
22     if not produtos:
23         return f"Perfil '{perfil}' nao reconhecido."
24     return "Produtos recomendados: " + ", ".join(produtos)
25
26 llm = ChatBedrockConverse(
27     model="us.anthropic.claude-sonnet-4-5-20250514-v1:0",
28     region_name="us-east-1",
29 )
30 tools = [consultar_saldo, listar_produtos_investimento]
31 agente = create_react_agent(llm, tools)

```

Listing 4.2: Tools nativas para o agente bancário.

4.5.2 Passo 2 — Implementar servidor MCP com transporte stdio

O arquivo `mcp_mock_server.py` implementa um servidor MCP completo com dados mock, ideal para testes offline. A estrutura é composta por três partes:

```

1 import asyncio, json
2 from mcp.server import Server
3 from mcp.server.stdio import stdio_server
4 import mcp.types as types
5

```

```

6 server = Server("postgres-mock-server")
7
8 @server.list_tools()
9 async def list_tools() -> list[types.Tool]:
10     return [
11         types.Tool(
12             name="executar_query",
13             description="Executa query no banco mock.",
14             inputSchema={
15                 "type": "object",
16                 "properties": {
17                     "tabela": {"type": "string"},
18                     "filtro": {"type": "object"}
19                 },
20                 "required": ["tabela"]
21             )
22         ],
23         types.Tool(name="listar_tabelas",
24                     description="Lista tabelas disponiveis.",
25                     inputSchema={"type": "object", "properties":
26                                     {}}),
27     ]
28
29 @server.call_tool()
30 async def call_tool(name: str, arguments: dict) -> list[types.
31     TextContent]:
32     if name == "listar_tabelas":
33         return [types.TextContent(type="text",
34                                     text=json.dumps(list(
35                                         MOCK_DATA.keys())))]
36
37     # ... logica de executar_query
38
39
40 async def main():
41     async with stdio_server() as (read, write):
42         await server.run(read, write,
43                         server.create_initialization_options()
44                             )
45
46 if __name__ == "__main__":
47     asyncio.run(main())

```

Listing 4.3: Estrutura do servidor MCP mock (simplificado).

O transporte `stdio` significa que o agente inicia o servidor como subprocesso e se comunica via `stdin/stdout`. Essa arquitetura é **segura por padrão**: sem portas abertas, sem autenticação de rede necessária para uso local.

4.5.3 Passo 3 — Conectar o agente ao servidor MCP

O LangGraph suporta conexão a servidores MCP via `MultiServerMCPClient`. Em produção, o servidor `mcp_postgres_server.py` conecta ao PostgreSQL real; em testes, usa-se o mock:

```
1 import asyncio
2 from langchain_mcp_adapters.client import MultiServerMCPClient
3 from langgraph.prebuilt import create_react_agent
4 from langchain_aws import ChatBedrockConverse
5
6 async def criar_agente_com_mcp():
7     async with MultiServerMCPClient({
8         "banco_mock": {
9             "command": "python",
10            "args": ["mcp_mock_server.py"],
11            "transport": "stdio",
12        }
13    }) as client:
14        tools = client.get_tools()
15        llm = ChatBedrockConverse(
16            model="us.anthropic.claude-sonnet-4-5-20250514-v1:0",
17            region_name="us-east-1",
18        )
19        agente = create_react_agent(llm, tools)
20        resultado = await agente.ainvoke({
21            "messages": [{"role": "user",
22                "content": "Quais tabelas estao
23                disponiveis?"}]
24        })
25        return resultado["messages"][-1].content
26
27 if __name__ == "__main__":
28     print(asyncio.run(criar_agente_com_mcp()))
```

Listing 4.4: Conexão do agente ao servidor MCP.

4.5.4 Passo 4 — Decisões de projeto: tool nativa vs. MCP

A escolha entre tool nativa e servidor MCP deve seguir o critério de acoplamento:

Tabela 4.1: Critérios de escolha: tool nativa vs. MCP

Critério	Tool nativa	Servidor MCP
Deploy conjunto com agente	✓	—
Reutilizavel por outros agentes	—	✓
Linguagem diferente de Python	—	✓
Dados sensíveis isolados	—	✓
Latencia mínima	✓	—
Versionamento independente	—	✓

No projeto bancário, tools de cálculo interno (índice de confiabilidade, formatação de resposta) são nativas; acesso a bases de dados (PostgreSQL, DynamoDB) é feito via MCP para garantir isolamento e auditabilidade.

4.6 Autoavaliação

Marque V (verdadeiro) ou F (falso):

1. O *docstring* da função decorada com `@tool` não influencia o comportamento do agente.
Gabarito: F — O *docstring* é o texto que o LLM lê para decidir quando e como usar a tool; docstrings imprecisos reduzem a qualidade das decisões.
2. No transporte `stdio`, o servidor MCP abre uma porta TCP para receber chamadas do agente.
Gabarito: F — No transporte `stdio`, a comunicação ocorre via `stdin/stdout` do subprocesso, sem portas de rede.
3. Um servidor MCP pode ser implementado em qualquer linguagem que suporte JSON-RPC 2.0.
Gabarito: V — O protocolo é independente de linguagem; existem SDKs oficiais em Python, TypeScript, Kotlin e Rust.
4. Tools nativas são sempre preferíveis ao MCP por terem menor latência.
Gabarito: F — A latência menor não justifica o acoplamento quando o recurso precisa ser isolado ou reutilizado por outros agentes.

4.7 Resumo

- Tools nativas são funções Python com `@tool`; o *docstring* guia a decisão do LLM.

- Skills agrupam tools relacionadas para uma capacidade de negócio coesa.
- MCP padroniza a comunicação entre agentes e servidores de ferramentas via JSON-RPC 2.0.
- O transporte `stdio` é a opção padrão para uso local, seguro e sem portas abertas.
- A escolha tool nativa vs. MCP depende de acoplamento, reutilização e necessidade de isolamento.
- No projeto bancário, acesso a dados sensíveis é sempre mediado por servidores MCP.

4.8 Para saber mais

- 12 — especificação oficial do *Model Context Protocol*.
- Repositório `langchain-mcp-adapters`: ¹ integração oficial do LangGraph com servidores MCP.
- 6 — documentação de tools e *tool calling* no LangGraph.

4.9 Plano de testes do capítulo

4.9.1 Testes funcionais

- **TF-04-01:** invocar `consultar_saldo` com `cliente_id="C001"` e verificar retorno "R\$ 15.000,00".
- **TF-04-02:** invocar `listar_tabelas` via servidor MCP mock e verificar que retorna lista com `["clientes", "produtos", "pedidos"]`.
- **TF-04-03:** invocar `executar_query` com `filtro={"status": "entregue"}` e verificar 2 registros.
- **TF-04-04:** criar agente com as duas tools nativas e verificar que responde corretamente à pergunta “Qual o saldo do cliente C002?”.

4.9.2 Teste de isolamento

- **TI-04-01:** verificar que o servidor MCP não abre nenhuma porta de rede ao rodar com transporte `stdio`.
- **TI-04-02:** verificar que tool com filtro de cliente inexistente retorna mensagem de erro sem exceção não tratada.

¹<<https://github.com/langchain-ai/langchain-mcp-adapters>>

4.10 Critérios de aceite

- `mcp_mock_server.py` responde corretamente às três operações: `listar_tabelas`, `descrever_tabela` e `executar_query`.
- Agente com tools nativas invoca a tool correta para cada tipo de pergunta (saldo vs. produtos).
- Todos os testes TF-04-0x e TI-04-0x passam com `python validate_sprint_a.py`.

4.11 Espaço para expansão iterativa

- **v4.1:** servidor MCP com transporte HTTP/SSE para deploy em container ECS.
- **v4.2:** autenticação OAuth 2.0 no servidor MCP (requerido para acesso a dados PCI-DSS).
- **Lacuna:** benchmark de latência tool nativa vs. MCP em cenário de alta concorrência.

Capítulo 5

Fundamentos de RAG

5.1 Pré-requisitos

Para aproveitar este capítulo, o leitor deve ter concluído:

- Capítulo 1 — tokenização, janela de contexto e *prompting*;
- Capítulo 3 — agentes ReAct com LangGraph;
- Conceito de vetores e produto interno (álgebra linear básica).

5.2 Escopo do capítulo

Onde estamos na história: loop interno, memória do agente. O agente já age e usa ferramentas (caps. 3–4). Agora ele precisa de memória de longo prazo: documentos corporativos que não cabem no contexto do LLM. RAG é a ponte entre o conhecimento externo e a janela de contexto. Os capítulos 6 e 9 aprofundarão esse mecanismo com fontes heterogêneas e grafo de conhecimento.

Este capítulo cobre o pipeline básico de *Retrieval-Augmented Generation* (RAG): ingestão, *chunking*, geração de *embeddings*, indexação em FAISS e recuperação semântica. O objetivo é construir, com código executável localmente e sem dependência de AWS, um pipeline RAG minimal que responde a perguntas usando documentos indexados.

5.3 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Explicar por que RAG reduz alucinação em sistemas de IA para domínios específicos;
- Implementar *chunking* por parágrafos com controle de tamanho;

- Gerar e persistir *embeddings* com FAISS;
- Executar busca semântica por similaridade de cosseno;
- Construir um pipeline RAG completo: documento $\rightarrow$ chunks $\rightarrow$ vetores $\rightarrow$ busca $\rightarrow$ resposta com citação.

5.4 Conceitos teóricos

5.4.1 Por que RAG?

LLMs armazenam conhecimento nos parâmetros durante o treinamento. Esse conhecimento é estático: não atualiza após o *cutoff* de treinamento e não contém informação proprietária da organização. Em domínios corporativos, a resposta correta depende de políticas atuais, contratos e dados que mudam com frequência — incompatível com re-treinamento constante.

RAG resolve esse problema em dois passos: (1) no momento da pergunta, recupera os fragmentos mais relevantes do repositório de documentos; (2) injeta esses fragmentos no *prompt* como contexto, permitindo que o LLM gere uma resposta fundamentada.

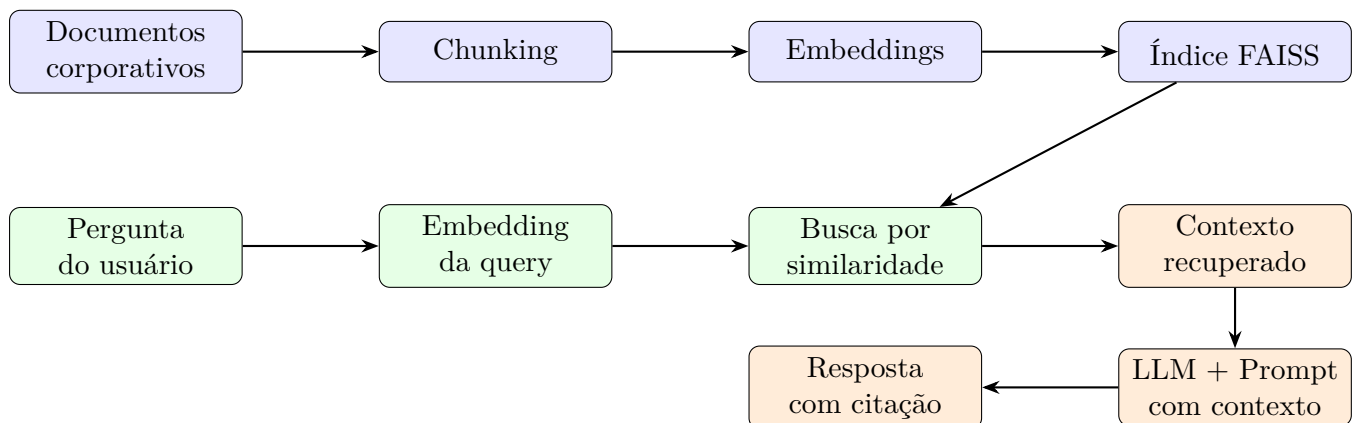


Figura 5.1: Pipeline RAG: fase de ingestão (azul) e fase de recuperação (verde/laranja).

5.4.2 Chunking: dividir para indexar

Um documento típico tem centenas de parágrafos. Indexar o documento inteiro geraria um vetor único que não captura nuances locais. O *chunking* divide o documento em fragmentos menores, cada um com sua própria representação vetorial.

A estratégia adotada no projeto é **chunking por parágrafos com janela deslizante de tamanho fixo**:

- Parágrafos são delimitados por linhas em branco;

- Chunks acumulam parágrafos até atingir 400 palavras;
- Parágrafos individuais maiores que 400 palavras não são truncados (ficam em chunk próprio).

5.4.3 Embeddings: representar significado em vetores

Um *embedding* é um vetor denso de alta dimensão ($d = 1536$ para os modelos da Bedrock) que representa o significado semântico de um texto. Textos semanticamente similares geram vetores próximos no espaço euclidiano, o que permite busca por similaridade eficiente.

A similaridade entre dois vetores é medida pelo cosseno:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|}$$

Vetores normalizados reduzem a busca de cosseno a um produto interno, que o FAISS executa com ótimos de hardware (SIMD/BLAS).

5.4.4 FAISS: índice de vetores local

O *Facebook AI Similarity Search* (FAISS) (13) é uma biblioteca de indexação de vetores que funciona inteiramente em memória, sem necessidade de servidor externo. Para o loop interno (desenvolvimento offline), o FAISS é a opção natural: rápido, reproduzível e portátil. Para o loop externo (produção com concorrência), o projeto migra para OpenSearch (Cap. 6).

O índice `IndexFlatL2` armazena vetores sem compressão e executa busca exata por distância euclidiana. Para coleções de até alguns milhares de documentos, a latência é desprezível (< 1 ms por busca).

5.5 Implementação orientada por passos

5.5.1 Passo 1 — Preparar a coleção de documentos

Crie o diretório `dados_rag/documentos/` com arquivos de texto plano:

```
1 dados_rag/  
2   documentos/  
3     politica_investimentos.txt  
4     faq_cartao_credito.txt
```

Listing 5.1: Coleção mínima para o laboratório de RAG.

Cada arquivo deve conter texto corrido em português, sem metadados sensíveis. Campos de identificação são passados separadamente na chamada de ingestão.

5.5.2 Passo 2 — Implementar chunking por parágrafos

```
1 import re
2
3 CHUNK_SIZE = 400 # palavras por chunk
4
5 def chunkar(texto: str, chunk_size: int = CHUNK_SIZE) -> list[
    str]:
6     """Divide texto em chunks de ~chunk_size palavras."""
7     paragrafos = [p.strip() for p in re.split(r"\n{2,}", texto)
8                   if p.strip()]
9     chunks, atual, contagem = [], [], 0
10    for par in paragrafos:
11        palavras = len(par.split())
12        if contagem + palavras > chunk_size and atual:
13            chunks.append("\n\n".join(atual))
14            atual, contagem = [], 0
15        atual.append(par)
16        contagem += palavras
17    if atual:
18        chunks.append("\n\n".join(atual))
19    return chunks
20
21 # Teste rapido
22 texto = open("dados_rag/documentos/politica_investimentos.txt",
23             encoding="utf-8").read()
24 chunks = chunkar(texto)
25 print(f"Documento dividido em {len(chunks)} chunks.")
26 print(f"Tamanho medio: {sum(len(c.split()) for c in chunks)//
    len(chunks)} palavras/chunk")
```

Listing 5.2: Chunking por parágrafos com controle de tamanho.

5.5.3 Passo 3 — Gerar embeddings e indexar no FAISS

```
1 import faiss, numpy as np, json, os
2 from langchain_aws import BedrockEmbeddings
3
4 EMBEDDING_MODEL = "amazon.titan-embed-text-v2:0"
5 INDEX_PATH = "faiss_index"
6
```

```

7 def criar_indice(chunks: list[str], titulos: list[str]) -> None
  :
8     """Gera embeddings e persiste o indice FAISS com metadados.
        """
9     embedder = BedrockEmbeddings(
10         model_id=EMBEDDING_MODEL, region_name="us-east-1"
11     )
12     vetores = embedder.embed_documents(chunks)
13     matriz = np.array(vetores, dtype="float32")
14
15     # normalizar para busca por cosseno via produto interno
16     faiss.normalize_L2(matriz)
17     index = faiss.IndexFlatIP(matriz.shape[1])
18     index.add(matriz)
19
20     os.makedirs(INDEX_PATH, exist_ok=True)
21     faiss.write_index(index, f"{INDEX_PATH}/index.faiss")
22
23     # salvar metadados paralelos
24     meta = [{"titulo": t, "texto": c} for t, c in zip(titulos,
25         chunks)]
26     with open(f"{INDEX_PATH}/metadata.json", "w", encoding="utf
27         -8") as f:
28         json.dump(meta, f, ensure_ascii=False, indent=2)
29     print(f"Indice criado: {index.ntotal} vetores em {
30         INDEX_PATH}/")

```

Listing 5.3: Indexação de chunks com embeddings Bedrock e FAISS.

5.5.4 Passo 4 — Recuperar chunks por similaridade

```

1 def recuperar(query: str, k: int = 3) -> list[dict]:
2     """Retorna os k chunks mais similares a query."""
3     embedder = BedrockEmbeddings(
4         model_id=EMBEDDING_MODEL, region_name="us-east-1"
5     )
6     index = faiss.read_index(f"{INDEX_PATH}/index.faiss")
7     with open(f"{INDEX_PATH}/metadata.json", encoding="utf-8")
8         as f:
9         meta = json.load(f)

```

```

10     q_vec = np.array([embedder.embed_query(query)], dtype="
        float32")
11     faiss.normalize_L2(q_vec)
12     scores, indices = index.search(q_vec, k)
13
14     resultados = []
15     for score, idx in zip(scores[0], indices[0]):
16         if idx >= 0:
17             resultados.append({
18                 "score": float(score),
19                 "titulo": meta[idx]["titulo"],
20                 "texto": meta[idx]["texto"][:300],
21             })
22     return resultados

```

Listing 5.4: Recuperação semântica: query → top-k chunks.

5.5.5 Passo 5 — Pipeline RAG completo com o agente

Com o índice criado, o agente pode usar a recuperação como tool e citar as fontes na resposta:

```

1 from langchain_core.tools import tool
2
3 @tool
4 def buscar_documentos(query: str) -> str:
5     """Busca documentos corporativos relevantes para a query.
6     Use sempre que a pergunta depender de politicas, FAQs ou
7     regras de produto que nao estao no seu conhecimento."""
8     resultados = recuperar(query, k=3)
9     if not resultados:
10         return "Nenhum documento relevante encontrado."
11     partes = []
12     for r in resultados:
13         partes.append(
14             f"[Fonte: {r['titulo']} | score: {r['score']:.3f}]\n"
15             f"{r['texto']}"
16         )
17     return "\n\n---\n\n".join(partes)

```

Listing 5.5: Tool RAG integrada ao agente LangGraph.

Ao adicionar `buscar_documentos` ao agente, o ciclo ReAct passa a funcionar como:

Pergunta → *Reason: preciso de contexto* → *Act: buscar_documentos* → *Observe: chunks recuperados* → *Reason: agora posso responder* → *Resposta com citação da fonte*.

5.6 Autoavaliação

1. Qual é a principal vantagem de normalizar os vetores antes de indexar no FAISS com `IndexFlatIP`?

Gabarito: a normalização transforma produto interno em similaridade de cosseno, permitindo comparar textos independentemente do comprimento (número de tokens).

2. Um chunk de 800 palavras é indexado como um único vetor. Isso prejudica a precisão da recuperação?

Gabarito: **V** — chunks muito longos diluem o sinal semântico; o vetor resultante representa uma média imprecisa de vários tópicos.

3. O FAISS é adequado para indexar milhões de documentos em produção com múltiplos usuários concorrentes.

Gabarito: **F** — para produção com concorrência, o projeto usa OpenSearch, que oferece persistência, replicação e escalabilidade horizontal.

4. A tool `buscar_documentos` deve ser chamada pelo agente em toda requisição, independentemente da pergunta.

Gabarito: **F** — o agente decide quando chamar a tool baseado no raciocínio ReAct; perguntas que o LLM consegue responder com conhecimento paramétrico não exigem recuperação.

5.7 Resumo

- RAG injeta conhecimento externo no *prompt* em tempo de execução, eliminando a necessidade de re-treinamento.
- *Chunking* por parágrafos com janela de 400 palavras equilibra granularidade e coerência semântica.
- *Embeddings* representam significado como vetores densos; normalizar antes de indexar ativa busca por cosseno.
- FAISS é ideal para o loop interno (offline); OpenSearch é a opção de produção.
- A tool `buscar_documentos` integra RAG ao ciclo ReAct: o agente decide quando recuperar contexto.

- O par (`score`, `título`) em cada chunk permite que a resposta cite a fonte, garantindo rastreabilidade.

5.8 Para saber mais

- 13 — artigo original do FAISS com benchmarks e algoritmos de quantização.
- 14 — artigo fundacional de RAG (*Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*).
- Documentação LangChain sobre **FAISS** *vectorstore*: ¹
- Capítulo 6 — RAG com OpenSearch, BM25 híbrido e governança para fontes heterogêneas.

5.9 Plano de testes do capítulo

- **TC-05-01:** texto de 1.200 palavras em 6 parágrafos deve gerar 3 chunks (≈ 400 palavras cada).
- **TC-05-02:** após indexar 5 chunks, busca pelo primeiro chunk deve retornar `score` ≥ 0.90 com `IndexFlatIP` e vetores normalizados.
- **TC-05-03:** busca por query semanticamente distante de todos os chunks deve retornar `score` < 0.50 .
- **TP-05-01:** latência de indexação de 100 chunks deve ser < 5 s em ambiente local (FAISS sem GPU).
- **TQ-05-01:** agente com tool RAG deve citar o título da fonte na resposta para pergunta sobre política de investimentos.

5.10 Critérios de aceite

- `chunkar(texto)` retorna lista não-vazia com chunks de no máximo 420 palavras (margem de 5%).
- `criar_indice` e `recuperar` executam sem erros em ambiente sem AWS (usando embeddings mock ou BedrockEmbeddings com credenciais).
- Todos os TC-05-0x passam com vetores sintéticos (`validate_sprint_a.py`).

¹<https://python.langchain.com/docs/integrations/vectorstores/faiss/>

5.11 Espaço para expansão iterativa

- **v5.1:** chunking híbrido com sobreposição (*overlap*) de 50 palavras entre chunks adjacentes.
- **v5.2:** re-ranking pós-recuperação com *cross-encoder* para melhorar precisão top-1.
- **Lacuna:** avaliação de *recall* do pipeline: dado um conjunto de perguntas com resposta conhecida, quantas têm o chunk correto no top-3?

Capítulo 6

RAG Corporativo e Arquitetura de Dados

6.1 Escopo do capítulo

Onde estamos na história: loop interno, RAG em escala corporativa. O capítulo 5 construiu um pipeline RAG minimal com FAISS local. Agora o sistema cresce: fontes heterogêneas (documentos e entrevistas), índice OpenSearch para produção e governança de dados sensíveis. Este é o RAG que o loop externo avaliará nos capítulos 11–14.

Este capítulo apresenta, em formato de aula-laboratório, a implementação de um pipeline *Retrieval-Augmented Generation* (RAG) para contexto bancário, com foco em fontes heterogêneas, indexação, recuperação e governança de dados. O objetivo prático consiste em construir um fluxo no qual documentos corporativos e entrevistas de clientes são convertidos em contexto recuperável para agentes, reduzindo alucinação e aumentando fidelidade da resposta.

6.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Diferenciar conhecimento parametricamente aprendido pelo LLM e conhecimento recuperado por RAG;
- Modelar documentos de negócio para indexação em OpenSearch;
- Implementar ingestão textual para os tipos `documento` e `entrevista`;
- Validar, com casos de teste, se a resposta do agente utiliza o contexto recuperado de forma verificável.

6.3 Conceitos teóricos

6.3.1 RAG em ambiente regulado

Em ambiente bancário, perguntas de negócio dependem de políticas, regras de produto, condições de canal e histórico de relacionamento. Essas informações mudam com frequência e não devem depender exclusivamente do conhecimento interno do modelo. O padrão RAG injeta contexto em tempo de execução, desacoplando atualização documental de novo *fine-tuning*.

6.3.2 Tipos de documento no projeto

O projeto adota dois tipos principais:

- **documento:** políticas, FAQ, regras de oferta e materiais normativos;
- **entrevista:** texto de voz do cliente ou de grupo, utilizado para os modos *persona* e *twin*.

Essa separação permite governança por origem e facilita auditoria de qual fonte sustentou determinada resposta.

6.4 Implementação orientada por passos

6.4.1 Passo 1 — Preparar coleção de documentos

Nesta etapa, organiza-se um conjunto mínimo para reprodução da aula:

```
1 dados_rag/  
2   documentos/  
3     politica_investimentos_2026.txt  
4     faq_cartao_credito.txt  
5   entrevistas/  
6     cliente_C00042.txt  
7     cluster_1_focus_group.txt
```

Listing 6.1: Coleção mínima para laboratório de RAG corporativo.

Cada arquivo deve conter texto contínuo, sem metadados sensíveis. Campos de identificação (cliente ou cluster) devem ser enviados explicitamente no payload.

6.4.2 Passo 2 — Ingerir texto no endpoint /documentos

O endpoint de ingestao esta definido no contrato OpenAPI e aceita JSON. No laboratorio, utiliza-se a chamada abaixo:

```
1 curl -X POST "$API_URL/documentos" \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "tipo": "documento",  
5     "titulo": "Politica de Investimentos 2026",  
6     "texto": "Clientes conservadores priorizam liquidez diaria  
7       ..."  
8   }'
```

Listing 6.2: Exemplo de ingestao de documento de politica no RAG.

Para entrevista individual:

```
1 curl -X POST "$API_URL/documentos" \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "tipo": "entrevista",  
5     "titulo": "Entrevista cliente C00042",  
6     "cliente_id": "C00042",  
7     "texto": "Prefiro resolver tudo no app e evitar taxas altas  
8       ..."  
9   }'
```

Listing 6.3: Ingestao de entrevista associada a cliente.

6.4.3 Passo 3 — Verificar chunking e indexacao

A funcao de ingestao fragmenta textos em *chunks* e os grava em indices OpenSearch distintos para cada finalidade. Conceitualmente, o fluxo corresponde a:

```
1 if tipo == "entrevista":  
2     chunks = chunkar(texto)  
3     indexar("entrevistas-clientes", chunks, metadata)  
4 else:  
5     chunks = chunkar(texto)  
6     indexar("documentos-knowledge", chunks, metadata)
```

Listing 6.4: Fluxo simplificado da indexacao no ingester.

O resultado esperado da API e um JSON contendo `chunks_indexados > 0`.

6.4.4 Passo 4 — Consumir o contexto em consulta real

Apos indexar, executa-se uma pergunta de negocio no endpoint assíncrono:

```
1 curl -X POST "$API_URL/query" \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "modo": "segmento",  
5     "pergunta": "Qual produto devo oferecer para perfil  
6       conservador?",  
7     "dados_cliente": {  
8       "idade": 55,  
9       "renda_mensal": 18000,  
10      "saldo_medio": 120000,  
11      "transacoes_mes": 25,  
12      "score_credito": 820,  
13      "num_produtos": 7  
14    }  
  },'
```

Listing 6.5: Consulta de negocio para verificar uso do contexto RAG.

Em seguida, consulta-se `/status/{request_id}` ate COMPLETED e compara-se a resposta com os documentos enviados.

6.4.5 Passo 5 — Resultado completo da aula

Ao final da pratica, o ambiente deve conter:

- indice `documentos-knowledge` com politicas e FAQ;
- indice `entrevistas-clientes` com voz de cliente/grupo;
- resposta do agente sustentada por contexto recuperado;
- evidencias de payload, `request_id` e resposta final arquivadas.

6.5 Plano de testes do capitulo

6.5.1 Teste funcional

- **TC-01:** Ingerir um documento e verificar `statusCode = 200` e `chunks_indexados > 0`.
- **TC-02:** Ingerir uma entrevista com `cliente_id` e validar persistencia no indice de entrevistas.

- **TC-03:** Executar consulta `modo=segmento` e confirmar resposta final sem erro de processamento.

6.5.2 Teste de desempenho

- **TP-01:** Medir tempo de ingestao de 10 documentos de 2 KB; criterio: mediana $\leq 2,5$ s por documento.
- **TP-02:** Medir latencia de resposta assíncrona ponta a ponta; criterio: $p95 \leq 20$ s em ambiente dev.

6.5.3 Teste de qualidade de resposta

- **TQ-01:** Verificar, em 10 perguntas, se a resposta cita explicitamente contexto compatível com os documentos indexados.
- **TQ-02:** Avaliar fidelidade com LLM-as-Judge (Capítulo 13); criterio: media $\geq 8,0/10$.

6.6 Critérios de aceite

- TC-01 a TC-03 aprovados sem erro bloqueante.
- TP-01 e TP-02 dentro dos limiares definidos.
- TQ-01 com aderencia documental observavel em pelo menos 80% dos casos.

6.7 Espaco para expansao iterativa

- **v6.1** — Adicionar suporte a PDF/DOCX com pipeline de extracao.
- **v6.2** — Incluir versionamento de documento e expiracao por vigencia.
- **Lacuna** — Investigar *reranking* para aumentar precisao top-k.

Capítulo 7

Pipeline Assíncrono em AWS

7.1 Escopo do capítulo

Onde estamos na história: loop interno, escala de produção. O agente funciona localmente. Agora ele precisa sobreviver ao mundo real: timeouts de API Gateway, picos de carga e latência imprevisível do LLM. Este capítulo substitui o modelo síncrono por um pipeline assíncrono baseado em SQS + Lambda — a espinha dorsal sobre a qual o loop externo operará.

Este capítulo descreve, como laboratório completo, a arquitetura assíncrona usada para superar o limite de timeout do API Gateway e processar consultas RAG com LLM em tempo potencialmente superior a 29 segundos. O fluxo adotado é: API Gateway → Lambda Controller → SQS → Lambda Worker → DynamoDB → Lambda Status.

7.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Explicar por que o processamento síncrono falha para pipelines longos;
- Implementar o padrão de enfileiramento com SQS e persistência de status em DynamoDB;
- Construir endpoints `/query` e `/status/{request_id}` com contrato consistente;
- Validar escalabilidade horizontal do worker e observabilidade de estados.

7.3 Conceitos teoricos

7.3.1 Padrao fire-and-poll

A estrategia adotada e *fire-and-poll*: o cliente envia a requisicao, recebe um `request_id` imediatamente e consulta o estado periodicamente ate a conclusao. Essa abordagem desacopla o tempo de resposta da API do tempo de computacao do pipeline. Em termos arquiteturais, este fluxo e uma variante do padrão *saga orchestration* aplicado a agentes e tarefas assíncronas (9).

7.3.2 Estados de processamento

O registro no DynamoDB utiliza estados tipicos:

- PENDING: requisicao recebida e enfileirada;
- PROCESSING: worker em execucao;
- COMPLETED: resposta pronta;
- FAILED: erro irrecoveravel com mensagem para auditoria.

7.4 Implementacao orientada por passos

7.4.1 Passo 1 — Submeter consulta e enfileirar

O endpoint POST `/query` recebe payload e delega o trabalho ao SQS, retornando 202 Accepted com identificador unico:

```
1 request_id = str(uuid.uuid4())
2 put_item_dynamodb(request_id, status="PENDING", payload=body)
3 enviar_mensagem_sqs(request_id=request_id, payload=body)
4
5 return {
6     "statusCode": 202,
7     "body": json.dumps({"request_id": request_id, "status": "
8         PENDING"})
```

Listing 7.1: Fluxo simplificado do controller: enfileirar e responder rapido.

7.4.2 Passo 2 — Processar no worker

O worker consome mensagens da fila, executa o pipeline e grava o resultado:

```
1 for msg in mensagens_sqs:
2     request_id = msg["request_id"]
3     atualizar_status(request_id, "PROCESSING")
4     try:
5         resposta = executar_pipeline_rag(msg["payload"])
6         salvar_resultado(request_id, status="COMPLETED", result
7                             =resposta)
8     except Exception as exc:
9         salvar_resultado(request_id, status="FAILED", error=str
10                            (exc))
```

Listing 7.2: Fluxo simplificado do worker com update de status.

7.4.3 Passo 3 — Consultar status

O endpoint GET /status/{request_id} consulta DynamoDB e retorna o estado corrente para a interface web ou cliente programático.

```
1 curl "$API_URL/status/$REQUEST_ID"
```

Listing 7.3: Consulta de status no laboratório.

7.4.4 Passo 4 — Teste ponta a ponta de laboratório

Sequência recomendada da aula:

1. Enviar POST /query com payload válido;
2. Armazenar request_id;
3. Executar polling a cada 2 s;
4. Registrar transição de estados até COMPLETED.

```
1 REQUEST_ID="..."
2 while true; do
3     RESP=$(curl -s "$API_URL/status/$REQUEST_ID")
4     echo "$RESP"
5     echo "$RESP" | findstr COMPLETED >nul && break
6     echo "$RESP" | findstr FAILED >nul && break
7     timeout /t 2 >nul
```

```
8 done
```

Listing 7.4: Script minimo de polling com shell.

7.4.5 Passo 5 — Resultado completo da aula

Ao final, o aluno deve observar:

- aceitacao imediata da requisicao no controller;
- processamento desacoplado no worker;
- resultado final consultavel por `request_id`;
- ausencia de timeout no cliente para cargas demoradas.

7.4.6 Passo 6 — Evidencia operacional via frontend CloudFront

Em validacao de producao assistida por frontend web (CloudFront), observou-se o comportamento esperado do padrao *fire-and-poll*. Para duas requisicoes reais (modos *persona* e *twin*), o fluxo evoluiu de `PENDING` para `PROCESSING` e finalizou em `COMPLETED`, sem expiracao no cliente.

- **Persona:** `request_id`: <ddbc5210-63fc-47bc-a274-283e60be84bc>.

Estado final `COMPLETED`;

- **Twin:** `request_id`: <e5e348e7-d266-4ae9-9f26-eb0e659a56a7>.

Estado final `COMPLETED`.

Essa evidencia reforca que a arquitetura assincrona permanece estavel quando acessada por uma interface de navegador, preservando o mesmo contrato de API utilizado por scripts de laboratorio.

7.5 Plano de testes do capitulo

7.5.1 Teste funcional

- **TC-01:** `POST /query` retorna 202 com `request_id` valido.
- **TC-02:** status evolui de `PENDING` para `PROCESSING` e depois `COMPLETED`.
- **TC-03:** para payload invalido, status final deve ser `FAILED` com mensagem de erro rastreavel.

7.5.2 Teste de desempenho

- **TP-01:** tempo de resposta do controller ≤ 1 s para 100 requisicoes seriadas.
- **TP-02:** throughput do worker com lote de 500 mensagens sem perda de consistencia em DynamoDB.

7.5.3 Teste de qualidade de resposta

- **TQ-01:** comparar 20 respostas assincronas com baseline sincrono local e verificar equivalencia semantica.
- **TQ-02:** validar que o modo de polling nao altera o conteudo final da resposta.

7.6 Criterios de aceite

- TC-01 a TC-03 aprovados.
- TP-01 e TP-02 dentro dos limites acordados para **dev**.
- TQ-01 com equivalencia media $\geq 95\%$ frente ao baseline.
- Validacao real de polling no frontend com status final **COMPLETED** em modos distintos de consulta.

7.7 Espaco para expansao iterativa

- **v7.1** — Trocar polling por WebSocket para notificacao ativa.
- **v7.2** — Adicionar *dead-letter queue* e retentativas.
- **Lacuna** — Modelar SLA distinto por tipo de consulta e modo de agente.

Capítulo 8

Segmentacao, Personas e Digital Twins

8.1 Escopo do capitulo

Onde estamos na história: loop interno, inteligência sobre o cliente. O sistema já responde perguntas genéricas. Agora ele precisa conhecer quem pergunta: segmento, perfil e histórico. Segmentação, Personas e Digital Twins transformam o agente genérico em consultor personalizado — e preparam a avaliação por segmento do cap. 13.

Este capitulo implementa, em formato de aula pratica guiada, a transformacao de dados de clientes em tres tipos de agentes de pesquisa: agente por segmento, agente *persona* e *digital twin*. O fluxo combina clusterizacao com K-Means, geracao de contexto RAG por grupo e criacao sob demanda de agentes para respostas em primeira ou terceira pessoa conforme o objetivo analitico.

8.2 Objetivos de aprendizagem

Ao concluir este capitulo, o leitor sera capaz de:

- Treinar e aplicar K-Means para segmentacao de clientes;
- Construir documentos RAG por cluster e por cliente;
- Diferenciar os modos `segmento`, `persona` e `twin`;
- Validar, com casos controlados, a consistencia de resposta de cada modo.

8.3 Conceitos teoricos

8.3.1 Tres superficies de analise

O projeto adota tres superficies complementares:

- **Segmento:** resposta gerencial em terceira pessoa com base no perfil medio do cluster;
- **Persona:** arquétipo nomeado de um cluster, respondendo em primeira pessoa representativa;
- **Twin:** simulacao individual, baseada em dados reais do cliente e contexto RAG filtrado por `cliente_id`.

8.3.2 Features de clusterizacao

O vetor de entrada utiliza, minimamente, as variaveis: `idade`, `renda_mensal`, `saldo_medio`, `transacoes_mes`, `score_credito` e `num_produtos`. Antes do K-Means, aplica-se padronizacao com *StandardScaler*.

8.4 Implementacao orientada por passos

8.4.1 Passo 1 — Gerar base sintetica e executar clustering

O laboratorio inicia com dados sinteticos para reproducao local:

```
1 df = gerar_dados_sinteticos(n=1000)
2 kmeans, scaler, perfis = executar_clustering(df, n_clusters=4)
3
4 cliente_exemplo = {
5     "idade": 55,
6     "renda_mensal": 18000,
7     "saldo_medio": 120000,
8     "transacoes_mes": 25,
9     "score_credito": 820,
10    "num_produtos": 7,
11 }
12
13 cluster_id = int(kmeans.predict(scaler.transform([[
14     cliente_exemplo["idade"],
15     cliente_exemplo["renda_mensal"],
16     cliente_exemplo["saldo_medio"],
17     cliente_exemplo["transacoes_mes"],
```

```

18     cliente_exemplo["score_credito"],
19     cliente_exemplo["num_produtos"],
20 ])) [0])

```

Listing 8.1: Treino de clusterizacao e previsao de cluster.

8.4.2 Passo 2 — Criar contexto RAG por cluster

Cada cluster gera documentos de contexto com perfil medio, produtos e estrategia de abordagem. O material e indexado no OpenSearch para recuperacao posterior.

```

1  for cid, perfil in perfis.iterrows():
2      docs = _gerar_docs_cluster(cid, perfil)
3      indexar_docs_cluster(cid, docs)  # abstrai o backend (FAISS
                                     /OpenSearch)

```

Listing 8.2: Geracao de documentos RAG por cluster.

8.4.3 Passo 3 — Instanciar agente persona

O modo *persona* reaproveita contexto do cluster, mas muda voz e comportamento por meio de *system prompt* dedicado.

```

1  agente_persona = _criar_agente_persona(
2      cluster_id=cluster_id,
3      perfil=perfis.loc[cluster_id],
4      retriever=retriever_cluster(cluster_id),
5      model=modelo_llm,
6  )
7
8  resposta = agente_persona.invoke({"pergunta": "O que voce
                                     espera de um cartao?"})

```

Listing 8.3: Criacao de agente persona por cluster.

8.4.4 Passo 4 — Criar digital twin sob demanda

No modo *twin*, o agente nao depende da rotulagem de cluster; ele e construido com base no registro individual e no retriever filtrado por `cliente_id`.

```

1  twin = criar_twin_sob_demanda(
2      cliente_id="C00042",
3      dados_cliente=cliente_exemplo,
4      model=modelo_llm,

```



```
5 )  
6  
7 out = twin.invoke({"pergunta": "Como posso melhorar meu score  
  de credito?"})
```

Listing 8.4: Twin sob demanda com contexto individual.

8.4.5 Passo 5 — Chamar a API em cada modo

O laboratorio fecha o ciclo testando os tres modos pela API:

```
1 # segmento  
2 curl -X POST "$API_URL/query" -H "Content-Type: application/  
  json" -d '{  
3   "modo": "segmento", "pergunta": "Qual oferta para este  
  cliente?", "dados_cliente": {...}  
4 }'  
5  
6 # persona  
7 curl -X POST "$API_URL/query" -H "Content-Type: application/  
  json" -d '{  
8   "modo": "persona", "cluster_id": 1, "pergunta": "O que voce  
  valoriza em investimentos?"  
9 }'  
10  
11 # twin  
12 curl -X POST "$API_URL/query" -H "Content-Type: application/  
  json" -d '{  
13   "modo": "twin", "cliente_id": "C00042", "pergunta": "Qual o  
  melhor proximo produto para mim?", "dados_cliente": {...}  
14 }'
```

Listing 8.5: Chamadas de validacao dos modos segmento, persona e twin.

8.4.6 Passo 6 — Resultado completo da aula

Ao final, o aluno deve conseguir demonstrar:

- classificacao automatica do cliente em cluster;
- resposta gerencial (segmento), arquetipica (persona) e individual (twin);
- coerencia entre contexto recuperado e estilo de resposta por modo.

8.4.7 Passo 7 — Evidencias de execucao em producao

Em teste de smoke test na AWS, executado pelo frontend CloudFront integrado a API Gateway, os modos `persona` e `twin` apresentaram conclusao com estado `COMPLETED`, mantendo aderencia de voz em primeira pessoa e recomendacoes compatíveis com o contexto recuperado.

- **Persona** (`cluster_id=1`): `request_id: <ddbc5210-63fc-47bc-a274-283e60be84bc>`. Estado final `COMPLETED`;
- **Twin** (`cliente_id=C00042`): `request_id: <e5e348e7-d266-4ae9-9f26-eb0e659a56a7>`. Estado final `COMPLETED`.

Como resultado, observa-se que a estrategia de segmentacao e a construcao do agente individual permanecem operacionalmente consistentes no ambiente implantado, e nao apenas em execucao local de laboratorio.

8.5 Plano de testes do capitulo

8.5.1 Teste funcional

- **TC-01:** classificar 50 clientes e verificar ausencia de erro de dimensao no *scaler*/K-Means.
- **TC-02:** executar `modo=persona` com `cluster_id` explicito e retornar resposta em primeira pessoa.
- **TC-03:** executar `modo=twin` com `cliente_id` e confirmar uso de contexto individual.

8.5.2 Teste de desempenho

- **TP-01:** tempo de classificacao por cliente ≤ 50 ms.
- **TP-02:** latencia mediana da inferencia por modo ≤ 5 s em `dev`.

8.5.3 Teste de qualidade de resposta

- **TQ-01:** em 10 perguntas por modo, verificar aderencia de tom (3a pessoa em segmento; 1a pessoa em `persona/twin`).
- **TQ-02:** avaliar fidelidade com LLM-as-Judge; criterio: media $\geq 8,0/10$ por modo.

8.6 Critérios de aceite

- TC-01 a TC-03 aprovados.
- TP-01 e TP-02 dentro dos limites definidos.
- TQ-01 com conformidade de voz em pelo menos 90% dos casos.
- Execucao real em producao com modos `persona` e `twin` finalizando em `COMPLETED`.

8.7 Espaco para expansao iterativa

- **v8.1** — Incorporar cache LRU para twins de alta frequencia.
- **v8.2** — Evoluir para clusterizacao periodica com janela movel.
- **Lacuna** — Investigar validacao causal para evitar vies de segmentacao.

Capítulo 9

GraphRAG com Neptune

9.1 Escopo do capítulo

Onde estamos na história: loop interno, memória relacional. RAG textual recupera fragmentos; grafo recupera relações. Perguntas como “quais produtos o cliente usa que são comparáveis com seu segmento?” exigem travessias multi-hop impossíveis de indexar como texto plano. GraphRAG com Neptune é a última peça do loop interno antes da convergência no cap. 10.

Este capítulo apresenta, em formato de laboratório, a extensão do RAG para contexto em grafo, combinando recuperação textual com inferência de relações multi-hop em Amazon Neptune. O objetivo consiste em responder perguntas que dependem de conexões entre entidades (cliente, produto, segmento, persona, canal), não apenas de similaridade textual. No guia da AWS, esse desenho aparece como camada de memória aumentada de longo prazo estruturada, com menção explícita ao uso de Neptune (9).

9.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Explicar quando RAG textual é insuficiente para consultas relacionais;
- Popular um grafo com entidades e arestas de negócio;
- Consultar contexto relacional com OpenCypher;
- Combinar contexto de grafo e contexto textual na resposta final do agente.

9.3 Conceitos teóricos

9.3.1 Do RAG textual ao GraphRAG

No RAG tradicional, a recuperação prioriza trechos semanticamente próximos. Entretanto, perguntas como “quais produtos conectam cliente de alto risco a segmento de renda média com canal digital?” exigem travessia de arestas. O GraphRAG introduz camada relacional para atender essas consultas sem perder o contexto documental.

9.3.2 Fontes de contexto combinadas

No pipeline de produção, o worker pode combinar:

- **RAG BM25 textual** em OpenSearch;
- **Snapshot de grafo replicado** em índice sincronizado;
- **Consulta Neptune live** para relações recentes.

Esse desenho reduz latência média, mas preserva capacidade de consulta profunda quando necessário.

9.4 Implementação orientada por passos

9.4.1 Passo 1 — Popular o grafo de conhecimento

Inicialmente, insere-se um conjunto mínimo de entidades e relações:

```
1 python popular_neptune.py
2 # alternativa em ambiente Lambda/VPC:
3 # python seed_neptune_lambda.py
```

Listing 9.1: Carga inicial do grafo para o laboratório.

Resultado esperado: vértices de clientes, segmentos, pessoas e produtos, com arestas representando afinidade, posse e recomendação.

9.4.2 Passo 2 — Ingerir documentos para mapeamento textual

A camada textual continua necessária para justificativas e detalhes descritivos. Portanto, indexa-se material de apoio no OpenSearch:

```
1 python indexar_documento.py \
2   --tipo documento \
3   --titulo "Politica de Cross-sell 2026" \
```

```
4 --arquivo dados_rag/documentos/politica_investimentos_2026.
   txt
```

Listing 9.2: Ingestao de documento no pipeline GraphRAG.

9.4.3 Passo 3 — Executar consulta relacional no Neptune

Consulta de exemplo em OpenCypher para recuperar produtos associados ao segmento de um cliente:

```
1 MATCH (c:Cliente {cliente_id: "C00042"})-[:PERTENCE_A]->(s:
   Segmento)
2 MATCH (s)-[:RECOMENDA]->(p:Produto)
3 RETURN c.cliente_id, s.nome AS segmento, collect(p.nome) AS
   produtos
```

Listing 9.3: Consulta OpenCypher de laboratorio.

9.4.4 Passo 4 — Fundir contexto textual e relacional

No worker, a estrategia de fusao pode ser representada como:

```
1 ctx_texto = recuperar_opensearch(pergunta, top_k=4)
2 ctx_grafo = consultar_neptune(pergunta, cliente_id)
3
4 contexto_final = {
5     "texto": ctx_texto,
6     "grafo": ctx_grafo,
7 }
8
9 resposta = llm.invoke(prompt_com_contexto(contexto_final,
   pergunta))
```

Listing 9.4: Composicao simplificada de contexto hibrido.

9.4.5 Passo 5 — Validar em consulta de negocio

Apos a fusao, executa-se pergunta multi-hop que dependa de relacoes e documentos:

```
1 curl -X POST "$API_URL/query" \
2   -H "Content-Type: application/json" \
3   -d '{
4     "modo": "twin",
5     "cliente_id": "C00042",
```

```
6      "pergunta": "Quais produtos se conectam ao meu segmento e  
7      como justificam essa oferta?",  
8      "dados_cliente": {  
9          "idade": 35,  
10         "renda_mensal": 3200,  
11         "saldo_medio": 800,  
12         "transacoes_mes": 12,  
13         "score_credito": 520,  
14         "num_produtos": 2  
15     },  
16 }
```

Listing 9.5: Consulta final do laboratorio GraphRAG.

9.4.6 Passo 6 — Resultado completo da aula

Ao final, o aluno deve comprovar:

- grafo populado com entidades e relacoes auditaveis;
- contexto textual e relacional utilizados de forma complementar;
- resposta final com coerencia de recomendacao e justificativa.

9.5 Plano de testes do capitulo

9.5.1 Teste funcional

- **TC-01:** executar carga inicial do grafo e validar contagem minima de vertices e arestas.
- **TC-02:** rodar consulta OpenCypher de referencia e verificar retorno nao vazio para cliente conhecido.
- **TC-03:** executar pergunta multi-hop e confirmar resposta com elementos relacionais e textuais.

9.5.2 Teste de desempenho

- **TP-01:** p95 de consulta OpenCypher ≤ 400 ms em dev.
- **TP-02:** latencia ponta a ponta com contexto hibrido ≤ 8 s em mediana.

9.5.3 Teste de qualidade de resposta

- **TQ-01:** para 10 perguntas multi-hop, verificar que a resposta menciona relações corretas entre entidades.
- **TQ-02:** avaliar fidelidade com LLM-as-Judge; critério: média $\geq 8,5/10$.

9.6 Critérios de aceite

- TC-01 a TC-03 aprovados.
- TP-01 e TP-02 dentro dos limites estabelecidos.
- TQ-01 com consistência relacional em pelo menos 80% dos casos.

9.7 Espaço para expansão iterativa

- **v9.1** — Adicionar estratégia de *path ranking* para priorizar trilhas de maior relevância.
- **v9.2** — Incluir explicações citáveis por aresta para auditoria.
- **Lacuna** — Avaliar impacto de replicação Neptune→OpenSearch na frescura de dados em cenários de alta mutação.

Capítulo 10

Pipeline Integrado em Producao

10.1 Escopo do capitulo

Onde estamos na história: ponto de convergência. Os nove capítulos anteriores construíram peças separadas — o modelo, o agente, o RAG, o pipeline, os twins, o grafo. Aqui elas se encaixam pela primeira vez em um sistema coeso. Daqui em diante, a pergunta muda: não mais “como construir?” mas “como garantir que funciona em produção?”. O loop externo começa no próximo capítulo.

Este capítulo é o ponto de convergência dos três capítulos anteriores: pipeline assíncrono (Cap. 7), segmentação e twins (Cap. 8) e GraphRAG com Neptune (Cap. 9). Aqui os componentes são observados juntos, como um sistema coeso em produção, revelando decisões de arquitetura que só fazem sentido quando as três camadas operam simultaneamente.

10.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Distinguir as fases *offline* (SageMaker Processing Job) e *online* (Lambda por requisição) do pipeline;
- Justificar por que a indexação de twins usa índice único com `pre_filter` em vez de um índice por cliente;
- Calcular e interpretar o índice de confiabilidade a partir das fontes RAG ativas;
- Usar o `demo_api.ipynb` como ferramenta de validação ponta a ponta dos três modos.

10.3 Conceitos teoricos

10.3.1 Duas fases, um sistema

O pipeline e dividido em duas fases com responsabilidades distintas:

- **Fase offline** (SageMaker Processing Job): execucao periodica ou por evento. Carrega dados brutos, treina K-Means com StandardScaler, indexa os clusters no OpenSearch, indexa os digital twins e persiste o modelo em S3.
- **Fase online** (Lambda por requisicao): no cold start, carrega o modelo do S3. Por requisicao, classifica o cliente, executa o agente LangGraph com RAG hibrido e retorna a resposta com indice de confiabilidade.

```

1  # FASE OFFLINE (SageMaker Job)
2  Dados brutos (S3)
3      -> K-Means + StandardScaler
4      -> Indexar OpenSearch: clientes-segmento-{0..3}
5      -> Indexar OpenSearch: clientes-digital-twins
6      -> Salvar modelo: s3://bucket/models/clustering.pkl
7
8  # FASE ONLINE (Lambda - cold start, uma vez por instancia)
9  carregar_pkl_s3("clustering.pkl")
10 BedrockEmbeddings(model="amazon.titan-embed-text-v1")
11 criar_agente_por_cluster(cluster_id) # 1 agente ReAct por
    segmento
12
13 # FASE ONLINE (Lambda - por requisicao)
14 KMeans.predict(StandardScaler.transform(dados_cliente)) ->
    cluster_id
15 buscar_rag(cluster_id ou cliente_id) # OpenSearch + Neptune
16 LangGraph ReAct -> resposta + indice_confiabilidade

```

Listing 10.1: Visao esquematica das duas fases do pipeline.

10.3.2 Estrategia de indexacao: indice unico versus indice por cliente

O modo **twins** poderia usar um indice OpenSearch por cliente — solucao intuitiva, porem inviavel em producao: N clientes gerariam N indices, elevando custo de gerenciamento, tempo de cold start e complexidade de re-indexacao.

A estratégia adotada é um índice único `clientes-digital-twins` com o campo de metadado `cliente_id`. Na consulta, aplica-se `pre_filter` para restringir o escore k-NN ao cliente específico:

```

1 vs.as_retriever(
2     search_kwargs={
3         "k": 3,
4         "pre_filter": {
5             "term": {"metadata.cliente_id": cliente_id}
6         },
7     }
8 )

```

Listing 10.2: Retriever com `pre_filter` por `cliente_id` no índice de twins.

Essa abordagem mantém latência equivalente a um único índice e permite re-indexação limpa de clientes específicos sem afetar os demais.

10.3.3 Índice de confiabilidade

O índice de confiabilidade quantifica o quanto o LLM utilizou o contexto RAG recuperado. É calculado em duas dimensões:

$$\text{score} = 0,85 \times \text{cobertura_fontes} + 0,15 \times \text{sobreposicao_lexica} \quad (10.1)$$

A **cobertura de fontes** é a soma dos pesos das camadas RAG ativas, conforme a Tabela 10.1.

Tabela 10.1: Pesos das fontes RAG no índice de confiabilidade.

Fonte	Peso	Condição de ativação
BM25 OpenSearch	0,40	Ao menos 1 documento recuperado
Neptune live	0,30	Query OpenCypher retorna resultados
Neptune sync	0,15	Snapshot replicado disponível
Maximo	0,85	Todas as fontes ativas

A **sobreposição léxica** mede a fração de termos-chave do contexto recuperado que aparece na resposta final, com contribuição máxima de 0,15.

A escala de interpretação do score final é:

- $\geq 0,70$ — **ALTO**: duas ou mais fontes ativas; LLM utilizou o contexto de forma verificável.
- $0,40\text{--}0,69$ — **MEDIO**: uma fonte ativa ou sobreposição léxica parcial.

- $< 0,40$ — **BAIXO**: sem contexto recuperado — resposta baseada em conhecimento paramétrico do LLM.

Nota sobre o modo persona: a sobreposição léxica tende a ser menor porque o LLM reescreve o contexto em linguagem coloquial de primeira pessoa, mas a cobertura de fontes reflete corretamente que ambas as camadas foram consultadas.

10.4 Implementação orientada por passos

10.4.1 Passo 1 — Executar a fase offline

```
1 # Em ambiente SageMaker ou local com credenciais AWS
   configuradas:
2 python aws_pipeline_clientes.py --mode pipeline
3
4 # Verificar artefato gerado no S3:
5 aws s3 ls s3://$S3_BUCKET/models/clustering.pkl
6
7 # Verificar índices criados no OpenSearch:
8 curl -s "$OS_ENDPOINT/_cat/indices?v&h=index,docs.count" \
9 | grep clientes
```

Listing 10.3: Execução do pipeline offline para gerar índices e modelo.

Resultado esperado: quatro índices `clientes-segmento-{0..3}` e um índice `clientes-digital-twins` cada um com a contagem de documentos correspondente ao número de clientes indexados.

10.4.2 Passo 2 — Verificar artefatos no S3 e índices no OpenSearch

Após a fase offline, os seguintes artefatos devem existir:

- `s3://bucket/models/clustering.pkl` — modelo KMeans + StandardScaler + perfis enriquecidos;
- Cinco índices OpenSearch: quatro de segmento e um de twins.

10.4.3 Passo 3 — Executar os três modos via notebook

O laboratório oficial deste capítulo é o `demo_api.ipynb`. Execute as células na ordem para observar os três modos em produção. Os modos diferem no sistema prompt e na fonte de RAG, conforme a Tabela 10.2.

Tabela 10.2: Comparativo dos três modos de agente.

Modo	Perspectiva	Índice RAG	Requisito
segmento	3ª pessoa (gerente)	clientes-segmento-{id}	dados_cliente
persona	1ª pessoa (arquetipo)	clientes-segmento-{id}	cluster_id
twin	1ª pessoa (cliente real)	clientes-digital-twins	cliente_id

10.4.4 Passo 4 — Analisar o índice de confiabilidade

Após executar os três modos, a célula de tabela comparativa do notebook consolida os scores. Espera-se:

- **segmento** e **twin** com cliente indexado: score ALTO ($\geq 0,70$);
- **persona**: score ALTO em cobertura de fontes, sobreposição lexical inferior ao modo segmento;
- **twin** com cliente sem dados no índice: score BAIXO ($< 0,40$).

10.4.5 Passo 5 — Simular degradação por ausência de indexação

A célula “Demonstração de degradação” do notebook usa o cliente hipotético DEMO-X999 (inexistente no índice) para demonstrar que o índice de confiabilidade detecta corretamente a ausência de contexto RAG — o score cai para o nível BAIXO e a resposta passa a depender exclusivamente do conhecimento paramétrico do LLM.

10.4.6 Passo 6 — Resultado completo da aula

Ao final, o aluno deve comprovar:

- Fase offline executada com artefatos persistidos no S3;
- Quatro índices de segmento e um de twins no OpenSearch;
- Três modos retornando COMPLETED com score de confiabilidade coerente com as fontes ativas;
- Score do twin com cliente indexado superior ao twin sem cliente indexado.

10.5 Plano de testes do capítulo

10.5.1 Teste funcional

- **TC-01**: executar fase offline e verificar existência de `clustering.pkl` no S3 e cinco índices no OpenSearch.

- **TC-02:** modo `segmento` retorna `COMPLETED` com `score` $\geq 0,70$.
- **TC-03:** modo `twin` com cliente indexado retorna `score` superior ao mesmo modo com cliente não indexado.

10.5.2 Teste de qualidade de resposta

- **TQ-01:** modo `persona` mantém voz em primeira pessoa em 100% das respostas dos quatro clusters.
- **TQ-02:** modo `twin` com cliente indexado cita dados numéricos do perfil individual em pelo menos 80% das respostas.

10.6 Critérios de aceite

- TC-01 a TC-03 aprovados.
- TQ-01 e TQ-02 dentro dos limites estabelecidos.
- Score de confiabilidade calculado e exibido corretamente para os três modos no notebook.

10.7 Espaço para expansão iterativa

- **v10.1** — Adicionar cache de contexto RAG no DynamoDB para reduzir latência em consultas repetidas ao mesmo cliente.
- **v10.2** — Implementar re-indexação incremental de twins sem necessidade de re-executar o pipeline completo.
- **v10.3** — Expandir o índice de confiabilidade com métrica de coerência semântica (cosine similarity) entre contexto recuperado e resposta gerada.

Capítulo 11

Confiabilidade, Observabilidade e Operação

11.1 Pré-requisitos

- Capítulo 10 — pipeline completo em produção com worker Lambda e DynamoDB;
- Familiaridade com JSON e logs estruturados.

11.2 Escopo do capítulo

Onde estamos na história: início do loop externo. O sistema está em produção. Agora ele precisa ser observável: toda resposta deve ter uma medida de confiabilidade, todo erro deve ser rastreado, toda degradação deve gerar alerta. Os capítulos 11 a 14 instrumentam o loop externo que alimentará o loop interno com evidências reais.

Este capítulo apresenta três camadas de observabilidade do sistema:

1. **Índice de confiabilidade:** métrica estruturada calculada por resposta, baseada em cobertura de fontes RAG e sobreposição léxica;
2. **Rastreabilidade com OpenTelemetry:** traces e spans exportados ao Langfuse para depuração e auditoria;
3. **Operação e resiliência:** retentativas, *dead-letter queue* e monitoramento de estados no DynamoDB.

No AWS Prescriptive Guidance, essa disciplina corresponde ao padrão de *observer and monitoring agents*, com foco em telemetria, classificação de eventos e ações de escalonamento (9).

11.3 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Calcular e interpretar o índice de confiabilidade a partir das fontes RAG ativas;
- Configurar OpenTelemetry com exportação ao Langfuse;
- Identificar as fontes de falha mais comuns e as estratégias de resiliência do pipeline;
- Monitorar o ciclo de vida de uma requisição usando os estados do DynamoDB.

11.4 Conceitos teóricos

11.4.1 Índice de confiabilidade

Em sistemas RAG, uma resposta sem contexto suficiente é tão problemática quanto uma resposta incorreta: o LLM pode alucinar para preencher lacunas. O índice de confiabilidade quantifica, por resposta, *quanto do contexto necessário foi efetivamente recuperado*.

O cálculo é dividido em duas dimensões:

- **Cobertura de fontes** (até 0,95): cada camada RAG ativa contribui um peso específico (Tabela 11.1);
- **Sobreposição léxica** (até 0,15): fração de palavras-chave do contexto que aparecem na resposta.

Tabela 11.1: Pesos das fontes RAG no índice de confiabilidade.

Fonte	Descrição	Peso
opensearch_bm25	BM25 sobre documentos e entrevistas	0,40
neptune_live	Grafo Neptune em tempo real	0,30
neptune_sync	Grafo sincronizado (replicação)	0,15
documentos_knowledge	Índice de políticas e FAQs	0,10
entrevistas_corpus	Corpus de entrevistas de clientes	0,10

O nível resultante é classificado como: **alto** ($\geq 0,70$), **médio** (0,40–0,69) ou **baixo** ($< 0,40$).

11.4.2 OpenTelemetry e Langfuse

O *OpenTelemetry* (OTel) é o padrão de observabilidade da CNCF para traces, métricas e logs. O `otel_config.py` configura automaticamente:

- Exportação de traces ao *Langfuse Cloud* via OTLP/HTTP quando as variáveis de ambiente `LANGFUSE_PUBLIC_KEY` e `LANGFUSE_SECRET_KEY` estão presentes;
- Auto-instrumentação das chamadas Anthropic via `openlit` (spans automáticos para cada chamada ao Bedrock);
- **Modo silencioso:** se as chaves não estiverem configuradas, o módulo importa sem erro — útil para desenvolvimento local.

11.5 Implementação orientada por passos

11.5.1 Passo 1 — Índice de confiabilidade

A função `_confiabilidade` do `worker.py` implementa o cálculo descrito na Seção 11.4.1:

```
1 def _confiabilidade(bm25, sync, graph, resposta,
2                     documentos="", entrevistas=""):
3     fontes, cobertura = [], 0.0
4     if bm25.strip():
5         fontes.append("opensearch_bm25"); cobertura += 0.40
6     if graph.strip():
7         fontes.append("neptune_live"); cobertura += 0.30
8     if sync.strip():
9         fontes.append("neptune_sync"); cobertura += 0.15
10    if documentos.strip():
11        fontes.append("documentos_knowledge"); cobertura +=
12            0.10
13    if entrevistas.strip():
14        fontes.append("entrevistas_corpus"); cobertura +=
15            0.10
16
17    ctx = " ".join(filter(None, [bm25, sync, graph, documentos,
18                                entrevistas]))
19    overlap = 0.0
20    if ctx and resposta:
21        ctx_kw = _palavras_chave(ctx)
22        resp_kw = _palavras_chave(resposta)
23        if ctx_kw and resp_kw:
24            overlap = min(len(ctx_kw & resp_kw)
25                          / min(len(ctx_kw), len(resp_kw)),
26                          1.0)
27
28    score = round(min(cobertura + overlap * 0.15, 1.0), 2)
```

```

25     nivel = "alto" if score >= 0.70 else ("medio" if score >=
26                                     0.40
27                                     else "baixo")
28     return {"score": score, "nivel": nivel,
29            "fontes_ativas": fontes,
30            "detalhes": {"cobertura_fontes": round(cobertura,2)
31                        ,
32                        "sobreposicao_lexica": round(overlap
33                                                    ,2)}}

```

Listing 11.1: Cálculo do índice de confiabilidade (worker.py).

A resposta ao cliente inclui sempre o campo `indice_confiabilidade`, permitindo que a camada de apresentação decida se exibe um aviso ao usuário para respostas com nível baixo.

11.5.2 Passo 2 — Configurar observabilidade com OTel

```

1  # Topo do worker.py, antes do handler
2  from otel_config import init as init_otel
3  init_otel() # silencioso sem variaveis de ambiente

```

Listing 11.2: Inicialização do OTel no handler Lambda.

Para ativar o Langfuse, defina as variáveis no Secrets Manager e injete no ambiente Lambda via `terraform.tfvars`:

```

1  LANGFUSE_PUBLIC_KEY=pk-lf-...
2  LANGFUSE_SECRET_KEY=sk-lf-...
3  LANGFUSE_OTLP_ENDPOINT=https://cloud.langfuse.com/api/public/
    otel/v1/traces

```

Listing 11.3: Variáveis de ambiente para Langfuse.

11.5.3 Passo 3 — Estados da requisição no DynamoDB

O ciclo de vida de uma requisição é rastreado por estados no DynamoDB:

```

1  QUEUED      -> processamento iniciado
2  PROCESSING  -> agente em execucao
3  COMPLETED  -> resposta disponivel
4  FAILED      -> erro nao recuperavel

```

Listing 11.4: Estados possíveis de uma requisição.

O endpoint `/status/{request_id}` (`status.py`) consulta o DynamoDB e retorna o estado atual. Clientes implementam *polling* com intervalo exponencial até `COMPLETED` ou `FAILED`.

11.6 Autoavaliação

1. Uma resposta com `opensearch_bm25` e `neptune_live` ativos, sem sobreposição léxica, tem índice de confiabilidade de quanto?

Gabarito: $0,40 + 0,30 + 0,00 \times 0,15 = 0,70 \rightarrow$ nível alto.

2. O OTel exporta traces para o Langfuse mesmo quando `LANGFUSE_PUBLIC_KEY` não está definido.

Gabarito: **F** — o `otel_config.py` só configura o exportador OTLP se ambas as chaves estiverem presentes.

3. O estado `PROCESSING` no DynamoDB indica que a resposta já está disponível para o cliente.

Gabarito: **F** — indica que o agente ainda está em execução; o cliente deve continuar o *polling*.

11.7 Resumo

- O índice de confiabilidade combina cobertura de fontes (até 0,95) e sobreposição léxica (até 0,15).
- `opensearch_bm25` e `neptune_live` são as fontes de maior peso; sua ausência sozinha já classifica a resposta como nível médio.
- OTel com Langfuse fornece visibilidade span-a-span das chamadas ao Bedrock.
- O modo silencioso do `otel_config.py` garante que o código funcione sem chaves Langfuse no ambiente de desenvolvimento.
- Estados DynamoDB permitem *polling* assíncrono sem *websocket*.

11.8 Plano de testes do capítulo

- **TC-11-01:** chamar `_confiabilidade` com apenas `bm25` ativo e verificar `score = 0,40`, nível médio.
- **TC-11-02:** chamar com todas as 5 fontes ativas e sobreposição 1,0 e verificar `score = 1,00`.

- **TC-11-03:** importar `otel_config` sem variáveis `Langfuse` e verificar que nenhuma exceção é lançada.
- **TC-11-04:** verificar que o endpoint `/status/{id}` retorna 404 para IDs inexistentes.

11.9 Critérios de aceite

- `_confiabilidade` passa todos os TC-11-0x.
- `otel_config.init()` executa sem erro em ambiente local (validado por `validate_sprint_a.py`).

11.10 Espaço para expansão iterativa

- **v11.1:** dashboard CloudWatch com alarme para taxa de respostas nível **baixo** > 10%.
- **v11.2:** integração com PagerDuty para falhas em *batch* crítico.
- **Lacuna:** meta-judge automático — usar o índice de confiabilidade como entrada do LLM Judge do capítulo 13.

Capítulo 12

Métricas de Avaliação de Mercado

12.1 Pré-requisitos

- Capítulo 11 — índice de confiabilidade e observabilidade;
- Python instalado localmente (sem dependência de AWS para ROUGE-L offline).

12.2 Escopo do capítulo

***Onde estamos na história:** loop externo, avaliação automática. O sistema está em produção e gera respostas. Agora precisamos medir, de forma sistemática, quão boas são essas respostas. As métricas deste capítulo fornecem o insumo quantitativo para o LLM Judge (cap. 13) e o Golden Dataset (cap. 14).*

Este capítulo explora três categorias de métricas para sistemas RAG em domínios corporativos: métricas lexicais (**ROUGE-L**), métricas semânticas (**BERTScore**) e avaliação gerada por LLM (**G-Eval**).

12.3 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Calcular ROUGE-L manualmente a partir da subsequência comum máxima (LCS);
- Interpretar scores de BERTScore em contexto financeiro;
- Orquestrar avaliação G-Eval com Claude via Bedrock;
- Escolher a métrica adequada para cada cenário (custo, velocidade, fidelidade).

12.4 Conceitos teóricos

12.4.1 ROUGE-L — precisão lexical

O ROUGE-L (*Recall-Oriented Understudy for Gisting Evaluation – Longest Common Subsequence*) mede a fidelidade lexical entre a resposta gerada e uma resposta de referência. A fórmula é:

$$\text{ROUGE-L} = \frac{2 \cdot P \cdot R}{P + R} \quad P = \frac{|LCS|}{|\hat{y}|}, \quad R = \frac{|LCS|}{|y|}$$

onde $\hat{y}$ é a resposta gerada, y a referência e $|LCS|$ o comprimento da maior subsequência comum em palavras.

Vantagens: determinista, sem dependência de modelo, executa sem GPU, custo zero.

Limitações: não captura sinônimos; “apólice” e “seguro” são tratados como palavras distintas mesmo com significado equivalente no contexto financeiro.

12.4.2 BERTScore — similaridade semântica

O BERTScore compara token a token os embeddings contextuais (BERT) de resposta e referência, calculando precisão, recall e F1 semânticos. É mais robusto que ROUGE-L para domínios com sinônimos, mas exige GPU ou CPU com razoável memória (≈ 400 MB de modelo).

12.4.3 G-Eval — avaliação por LLM

G-Eval usa um LLM externo (Claude no nosso caso) para avaliar a resposta com base em critérios definidos em linguagem natural. No projeto, o modo `geval` do script `avaliar_metricas.py` envia um prompt estruturado ao Bedrock com a pergunta, o contexto recuperado e a resposta gerada, solicitando scores em três dimensões: relevância, fidelidade ao contexto e clareza.

12.4.4 Quando usar cada métrica

Tabela 12.1: Critérios de seleção de métrica por cenário.

Métrica	Custo	Velocidade	Semântica	Caso de uso
ROUGE-L	zero	muito rápido	não	CI/CD, baseline
BERTScore	baixo	rápido (GPU)	sim	avaliação batch
G-Eval	médio	lento (API)	alta	curadoria humana

12.5 Implementação orientada por passos

12.5.1 Passo 1 — ROUGE-L offline (sem GPU, sem API)

O script `avaliar_metricas.py` implementa ROUGE-L em Python puro, sem dependências externas:

```

1 def lcs(a: list[str], b: list[str]) -> int:
2     """Comprimento da maior subsequencia comum."""
3     m, n = len(a), len(b)
4     dp = [[0] * (n + 1) for _ in range(m + 1)]
5     for i in range(1, m + 1):
6         for j in range(1, n + 1):
7             if a[i-1] == b[j-1]:
8                 dp[i][j] = dp[i-1][j-1] + 1
9             else:
10                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
11    return dp[m][n]
12
13 def rouge_l(hipotese: str, referencia: str) -> float:
14     """F1 ROUGE-L entre hipotese e referencia."""
15     h = hipotese.lower().split()
16     r = referencia.lower().split()
17     if not h or not r:
18         return 0.0
19     c = lcs(h, r)
20     p = c / len(h)
21     rec = c / len(r)
22     if p + rec == 0:
23         return 0.0
24     return round(2 * p * rec / (p + rec), 4)

```

Listing 12.1: ROUGE-L com LCS puro Python (`avaliar_metricas.py`).

12.5.2 Passo 2 — Executar avaliação offline

```

1 python avaliar_metricas.py --modo offline
2 # Saída esperada:
3 # [ROUGE-L] "O cliente tem direito..." vs ref: 0.25
4 # [ROUGE-L] "O limite padrao..." vs ref: 0.40
5 # [ROUGE-L] "" vs ref: 0.00

```

Listing 12.2: Demo offline de ROUGE-L.

12.5.3 Passo 3 — Avaliação em lote com CSV

Para avaliar um conjunto de casos do golden dataset:

```
1 # Formato: pergunta, referencia, hipotese
2 python avaliar_metricas.py --modo batch --arquivo casos.csv
3
4 # Saida por linha:
5 # caso_01 ROUGE-L: 0.6720
6 # caso_02 ROUGE-L: 0.3140 << abaixo do threshold 0.35
```

Listing 12.3: Avaliação em lote via CSV.

12.6 Autoavaliação

1. ROUGE-L = 1,00 significa que a resposta é semanticamente perfeita.

Gabarito: F — significa apenas que a resposta contém as mesmas palavras que a referência na mesma ordem; uma resposta sinônima com ROUGE-L baixo pode ser igualmente correta.

2. Para qual cenário G-Eval é mais adequado que ROUGE-L?

Gabarito: curadoria de casos duvidosos, onde a opinião de um LLM sofisticado substitui avaliação humana cara.

3. Qual é o custo de infraestrutura do modo offline de `avaliar_metricas.py`?

Gabarito: zero — executa sem API externa, sem GPU, sem credenciais AWS.

12.7 Resumo

- ROUGE-L é determinista e gratuito; é o baseline de qualidade no CI/CD.
- BERTScore captura sinônimos via embeddings contextuais; indicado para avaliação batch com GPU.
- G-Eval usa o LLM como avaliador; custo mais alto, maior profundidade semântica.
- O script `avaliar_metricas.py` unifica os três modos com interface CLI consistente.
- O threshold ROUGE-L varia por caso — respostas curtas e factuais toleram thresholds mais altos que respostas explicativas.

12.8 Plano de testes do capítulo

- **TC-12-01:** `rouge_l("a b c", "a b c")` deve retornar 1,0.
- **TC-12-02:** `rouge_l(, "a b")` deve retornar 0,0 sem exceção.
- **TC-12-03:** modo `offline` deve exibir 3 scores ROUGE-L e encerrar com código de saída 0.
- **TP-12-01:** 1.000 pares avaliados com ROUGE-L em modo `batch` em menos de 5 s.

12.9 Critérios de aceite

- `rouge_l` passa TC-12-01 e TC-12-02.
- Modo `offline` executa sem credenciais e imprime scores.

12.10 Espaço para expansão iterativa

- **v12.1:** adicionar ROUGE-1 e ROUGE-2 para comparação com benchmarks públicos.
- **v12.2:** integrar DeepEval para *answer relevancy* e *contextual precision*.
- **Lacuna:** calibrar thresholds por segmento (PF vs. PJ vs. FP) a partir de dados reais de produção.

Capítulo 13

LLM as Judge

13.1 Pré-requisitos

- Capítulo 12 — ROUGE-L e limitações das métricas lexicais;
- Conceito de *prompt engineering* (cap. 1).

13.2 Escopo do capítulo

Onde estamos na história: loop externo, avaliação qualitativa. ROUGE-L mede palavras; o LLM Judge mede intenção. Este capítulo ativa a segunda camada de avaliação: um LLM (Claude) que avalia a resposta com base em rubricas domínio-específicas financeiras, substituindo avaliação humana cara e lenta.

O capítulo aborda: (1) justificativa de LLM-as-Judge vs. métricas automáticas; (2) rubrica de 4 dimensões para o domínio bancário; (3) implementação do script `llm_judge.py`; (4) calibração vs. avaliação humana. Na taxonomia da AWS, essa estratégia corresponde ao padrão *evaluator reflect-refine loop*, com geração e avaliação desacopladas para reforçar qualidade e alinhamento (9).

13.3 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Justificar por que LLM-as-Judge supera métricas lexicais em domínios especializados;
- Projetar uma rubrica de avaliação com dimensões e pesos por segmento;
- Executar o script `llm_judge.py` em modo mock (offline) e em modo Bedrock (online);
- Identificar riscos de influência do avaliador e como mitigá-los.

13.4 Conceitos teóricos

13.4.1 Por que LLM-as-Judge?

Métricas lexicais (ROUGE-L) e semânticas (BERTScore) medem semelhança com uma referência. Mas no domínio financeiro, a qualidade de uma resposta depende de critérios que não existem em referências textuais:

- Uma resposta sobre CDB que omite o prazo de carenciã pode ter ROUGE-L alto mas ser financeiramente perigosa;
- Uma resposta correta sobre limite de crédito PJ que menciona dados de perfil não solicitado viola LGPD.

LLM-as-Judge resolve isso com rubricas em linguagem natural que capturam obrigações regulatórias e expectativas de negócio que nenhuma métrica automática consegue codificar de forma exaustiva (15).

13.4.2 Rubrica de 4 dimensões

A rubrica do projeto avalia cada resposta em 4 dimensões (Tabela 13.1):

Tabela 13.1: Rubrica do LLM Judge para o domínio bancário.

Dimensão	Critério	Máx	Aprovado
fidelidade_rag	resposta coerente com contexto RAG	3	≥ 2
adequacao_perfil	tom e conteúdo para PF/PJ/FP	3	≥ 2
completude	responde completamente à pergunta	2	$= 2$
conformidade_lgpd	não expe dados não solicitados	2	$= 2$
Total	aprovado se total ≥ 7	10	≥ 7

13.4.3 Modo mock vs. Bedrock

O script suporta dois modos:

- **mock:** respostas pré-configuradas por segmento (PF: 10/10, PJ: 7/10, FP: 7/10) — reproduzível sem credenciais, ideal para CI/CD;
- **bedrock:** envia o prompt ao Claude e faz *parsing* do JSON retornado — exige credenciais AWS.

13.5 Implementação orientada por passos

13.5.1 Passo 1 — Estrutura da rubrica como prompt

```
1 PROMPT_JUDGE = """Voce e um avaliador especializado em servicos
    bancarios.
2
3 Avalie a resposta abaixo em 4 dimensoes e retorne JSON.
4
5 Pergunta: {pergunta}
6 Segmento: {segmento}    (PF = Pessoa Fisica, PJ = Juridica, FP =
    Private)
7 Contexto RAG fornecido ao agente:
8 {contexto}
9
10 Resposta gerada:
11 {resposta}
12
13 Retorne APENAS JSON valido:
14 {{
15     "fidelidade_rag":    <0-3>,
16     "adequacao_perfil": <0-3>,
17     "completude":       <0-2>,
18     "conformidade_lgpd": <0-2>,
19     "justificativa":     "<texto curto>"
20 }}
```

Listing 13.1: Prompt de avaliação do LLM Judge.

13.5.2 Passo 2 — Executar em modo mock

```
1 python llm_judge.py --modo mock
2
3 # Saida esperada:
4 # [PF] Score: 10/10 - APROVADO
5 #   fidelidade_rag: 3   adequacao_perfil: 3
6 #   completude: 2   conformidade_lgpd: 2
7 # [PJ] Score: 7/10 - APROVADO
8 # [FP] Score: 7/10 - APROVADO
```

Listing 13.2: Execução do LLM Judge em modo mock.

13.5.3 Passo 3 — Interpretação e calibração

A calibração do juiz LLM é o processo de comparar seus scores com avaliações humanas em uma amostra de controle. O coeficiente de Cohen's Kappa é a métrica padrão:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

onde P_o é a concordância observada e P_e a concordância esperada ao acaso. Para uso em produção, recomenda-se $\kappa \geq 0,60$ (concordância substancial).

13.6 Autoavaliação

1. Uma resposta com score 6/10 deve ser aprovada?

Gabarito: Não — o threshold é 7/10; 6/10 indica que pelo menos uma dimensão está abaixo do esperado.

2. Por que o modo mock é preferível ao modo Bedrock em CI/CD?

Gabarito: determinismo e custo zero — o mock sempre retorna o mesmo resultado, tornando os testes reproduzíveis sem consumir tokens.

3. Qual dimensão protege contra violação de LGPD?

Gabarito: `conformidade_lgpd` (0-2) — pontua com 0 se a resposta expoe dados não solicitados pelo usuário.

13.7 Resumo

- LLM-as-Judge captura critérios regulatórios e de negócio inexprimíveis em métricas lexicais.
- A rubrica de 4 dimensões equilibra qualidade técnica (fidelidade RAG) e conformidade (LGPD).
- O modo mock garante reprodutibilidade em CI/CD sem custo de API.
- Calibração com Cohen's Kappa valida a concordância do juiz LLM com avaliadores humanos.
- Respostas com score < 7 devem ser analisadas manualmente e usadas para melhorar os prompts do agente.

13.8 Plano de testes do capítulo

- **TC-13-01:** modo mock segmento PF deve retornar score 10 e status APROVADO.
- **TC-13-02:** modo mock segmento PJ deve retornar score 7 e status APROVADO.
- **TC-13-03:** resposta vazia deve retornar `conformidade_lgpd = 2` e `completude = 0`, `score ≤ 5`.
- **TP-13-01:** 50 avaliações em modo mock completam em menos de 1 s.

13.9 Critérios de aceite

- TC-13-01 e TC-13-02 passam com `llm_judge.py`.
- Nenhuma exceção em modo mock sem credenciais AWS.

13.10 Espaço para expansão iterativa

- **v13.1:** adicionar dimensão `clareza_linguagem` (0–2) para avaliar legibilidade para clientes não-especialistas.
- **v13.2:** integração com Langfuse para armazenar scores do judge ao lado dos traces OTel.
- **Lacuna:** calibração formal com amostra humana de 100 casos por segmento.

Capítulo 14

Golden Dataset e Regressão de Qualidade

14.1 Pré-requisitos

- Capítulo 12 — ROUGE-L e thresholds de qualidade;
- Capítulo 13 — LLM Judge e rubrica;
- Conceito de integração contínua (CI/CD).

14.2 Escopo do capítulo

***Onde estamos na história:** loop externo, regressão automática. As métricas e o judge estão prontos. Falta o elo final: um conjunto curado de casos de teste que detecta regressões antes que cheguem a produção. O golden dataset é a memória coletiva de qualidade do sistema.*

Este capítulo cobre: (1) o que é um golden dataset e como construí-lo; (2) execução de regressão com `golden_dataset.py`; (3) interpretação do diff (regressões vs. melhorias); (4) integração com CI/CD.

14.3 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Definir os critérios de curadoria de um golden dataset;
- Criar casos com thresholds por pergunta;
- Executar uma suíte de regressão e interpretar o relatório;

- Identificar a diferença entre regressão e melhoria no diff de qualidade.

14.4 Conceitos teóricos

14.4.1 O que é um golden dataset?

Um *golden dataset* é um conjunto de casos de teste curados manualmente, com:

- **Pergunta:** representativa de uso real em produção;
- **Resposta de referência:** aprovada por especialistas de domínio (analistas de produto ou compliance);
- **Thresholds individuais:** cada caso tem seu próprio limiar ROUGE-L e score mínimo do judge, refletindo o nível de fidelidade esperado para aquele tipo de pergunta.

Ao contrário de benchmarks públicos, o golden dataset é **proprietário e confidencial**: contém a expectativa de qualidade negociada entre engenharia e negócio.

14.4.2 Curadoria: sintético vs. humano

Tabela 14.1: Comparação de estratégias de curadoria.

Estratégia	Vantagem	Limitação
Manual (humano)	alta fidelidade de domínio	caro, lento
Sintético (LLM)	escalável, rápido	pode herdar viéses do LLM
Híbrido	equilíbrio de custo/qualidade	requer revisão humana

O projeto adota a estratégia híbrida: perguntas geradas sinteticamente com respostas de referência revisadas por especialistas.

14.4.3 Regressão vs. melhoria

O diff compara os resultados atuais com os resultados do baseline salvo. Um caso pode transitar de três formas:

- **Regressão:** passou antes, falhou agora → bloquear release;
- **Melhoria:** falhou antes, passou agora → atualizar threshold;
- **Estabilidade:** mesmo resultado → sem ação.

14.5 Implementação orientada por passos

14.5.1 Passo 1 — Estrutura de um caso do golden dataset

```
1 CASOS = [  
2     {  
3         "id": "GD-001",  
4         "segmento": "PF",  
5         "pergunta": "Qual e a rentabilidade do CDB prefixado de  
6             12 meses?",  
7         "referencia": (  
8             "O CDB prefixado de 12 meses oferece rentabilidade  
9             de 12,5% ao ano, "  
10            "garantida pelo FGC ate R$ 250.000."  
11        ),  
12        "threshold_rouge": 0.35,  
13        "threshold_judge": 7,  
14    },  
15    # ... mais casos  
16 ]
```

Listing 14.1: Estrutura de um caso no golden_dataset.py.

14.5.2 Passo 2 — Criar e salvar o dataset

```
1 python golden_dataset.py --modo criar --dataset golden_dataset.  
2     jsonl  
3  
4 python golden_dataset.py --modo rodar --dataset golden_dataset.  
5     jsonl --saida golden_baseline.json  
6  
7 # Saida esperada:  
8 # Golden dataset criado com 4 casos.  
9 # Resultados salvos em golden_baseline.json
```

Listing 14.2: Criação do golden dataset com baseline salvo.

14.5.3 Passo 3 — Executar regressão

```
1 python golden_dataset.py --modo rodar --dataset golden_dataset.  
2     jsonl --saida resultados_golden.json
```

```
3 # Saida esperada:
4 # GD-001: rouge_l=0.2857 judge=10 STATUS: FALHOU (rouge
   abaixo de 0.35)
5 # GD-002: rouge_l=0.5000 judge=7 STATUS: PASSOU
6 # GD-003: rouge_l=0.4000 judge=7 STATUS: PASSOU
7 # GD-004: rouge_l=0.2500 judge=10 STATUS: FALHOU (rouge
   abaixo de 0.30)
8 # Resultado: 2/4 casos aprovados
```

Listing 14.3: Execução da suíte de regressão.

O comportamento acima é **intencional**: GD-001 e GD-004 têm respostas mock propositalmente incompletas, validando que o sistema de thresholds funciona corretamente.

14.5.4 Passo 4 — Interpretar o diff

```
1 python golden_dataset.py --modo diff --baseline golden_baseline
   .json --atual resultados_golden.json
2
3 # Saida esperada quando nenhuma mudanca ocorreu:
4 # "regressoes": []
5 # "melhorias": []
6 # "estavel": ["GD-001", "GD-002", "GD-003", "GD-004"]
```

Listing 14.4: Comparar execução atual com baseline.

14.5.5 Passo 5 — Integração com CI/CD

A suíte de regressão deve ser executada automaticamente em qualquer pull request que modifique prompts, modelos ou pipeline RAG:

```
1 - name: Regressao de qualidade
2   run: |
3     python golden_dataset.py --modo rodar
4     if python golden_dataset.py --modo diff | grep -q "
       Regressoes: [^0]"; then
5       echo "FALHA: regressao detectada -- bloquear release"
6       exit 1
7     fi
```

Listing 14.5: Exemplo de gate de qualidade em GitHub Actions.

14.6 Autoavaliação

1. Um caso que passou no baseline e falhou agora é classificado como o quê?

Gabarito: regressão — deve bloquear o release até a causa ser identificada.

2. Por que cada caso pode ter um threshold ROUGE-L diferente?

Gabarito: porque o nível de fidelidade lexical esperado varia com o tipo de pergunta — perguntas factuais (taxas, prazos) têm threshold mais alto; perguntas explicativas aceitam mais parafraseação.

3. É seguro mover um caso do golden dataset para um threshold mais baixo após uma regressão?

Gabarito: Não — isso mascararia a degradação; o correto é investigar e corrigir a causa.

14.7 Resumo

- O golden dataset é a memória coletiva de qualidade: expectativas negociadas entre engenharia e negócio.
- Cada caso tem thresholds individuais para ROUGE-L e LLM Judge.
- O diff classifica mudanças em regressões, melhorias e casos estáveis.
- Regressões devem bloquear o release automaticamente no CI/CD.
- A estratégia híbrida (sintético + revisão humana) equilibra velocidade e fidelidade de domínio.

14.8 Plano de testes do capítulo

- **TC-14-01:** modo `criar` deve gerar `golden_baseline.json` com 4 chaves.
- **TC-14-02:** modo `rodar` deve retornar 2/4 aprovados com as respostas mock padrão.
- **TC-14-03:** modo `diff` com baseline idêntico deve reportar 0 regressões.
- **TC-14-04:** caso com resposta vazia deve ser reprovado independentemente do threshold.

14.9 Critérios de aceite

- TC-14-01 a TC-14-03 passam com `golden_dataset.py`.
- Nenhuma exceção nos três modos sem credenciais AWS.

14.10 Espaço para expansão iterativa

- **v14.1:** ampliar para 50 casos por segmento (PF/PJ/FP) usando geração sintética com Claude.
- **v14.2:** dashboard de evolução histórica de scores ao longo de deploys.
- **Lacuna:** protocolo de expiração de casos — como desativar casos que não refletem mais políticas de produto atuais.

14.11 Objetivos de aprendizagem

- Entender os conceitos nucleares do tema.
- Aplicar o tema em uma implementacao concreta do projeto.
- Definir evidencias objetivas de qualidade e funcionamento.

14.12 Implementacao orientada por passos

1. Preparacao do ambiente e insumos.
2. Implementacao guiada do fluxo principal.
3. Validacao dos resultados esperados.
4. Registro de decisoes tecnicas e trade-offs.

14.13 Plano de testes do capitulo

14.13.1 Teste funcional

- Definir casos de entrada e saida esperada.
- Verificar comportamento nominal e casos de erro.

14.13.2 Teste de desempenho

- Medir latencia ponta a ponta.
- Medir uso estimado de tokens por execucao.

14.13.3 Teste de qualidade de resposta

- Medir aderencia ao contexto recuperado.
- Medir consistencia da resposta com o objetivo do usuario.

14.14 Critérios de aceite

- Implementacao executa sem erros bloqueantes.
- Testes definidos no capitulo atingem resultado esperado.
- Evidencias de execucao sao anexadas pelos autores.

14.15 Espaco para expansao iterativa

- Notas de versao do capitulo.
- Melhorias planejadas.
- Lacunas para investigacao futura.

Capítulo 15

Roadmap de Produção e Evolução Contínua

15.1 Escopo do capítulo

Onde estamos na história: loop externo, fechando o ciclo. Os capítulos 11–14 instrumentaram, mediram e testaram o sistema. Agora o loop externo entrega seu produto final: evidências de qualidade que alimentam o loop interno. Este capítulo mapeia o caminho da maturidade técnica e operacional — de MVP a sistema de produção sustentável.

Este capítulo organiza o plano de evolução do sistema em três dimensões: (1) **maturidade técnica** (do FAISS local ao OpenSearch distribuído); (2) **maturidade operacional** (do CI/CD manual ao gate de qualidade automático); (3) **maturidade de avaliação** (do ROUGE-L ao golden dataset com 50+ casos).

15.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Identificar em qual estágio de maturidade o sistema se encontra;
- Priorizar evoluções com base em impacto e esforço;
- Definir os marcos que justificam a migração do loop interno para o loop externo;
- Planejar o fechamento do ciclo de melhoria contínua.

15.3 Modelo de maturidade

15.3.1 Nível 1 — MVP Offline

- FAISS local com documentos sintéticos;
- Agente ReAct com LangGraph e tool RAG;
- Avaliação manual por ROUGE-L offline;
- Deploy: notebook local sem AWS.

Critério de saída: 4/4 casos do golden dataset passam em modo mock.

15.3.2 Nível 2 — Integração AWS

- Pipeline assíncrono: API Gateway → SQS → Lambda Worker → DynamoDB;
- OpenSearch substituindo FAISS em produção;
- OTel + Langfuse para traces;
- CI/CD com gate de qualidade: ROUGE-L $\geq$ threshold por caso.

Critério de saída: p99 latência < 30 s; zero regressões no golden dataset após cada deploy.

15.3.3 Nível 3 — Produção Sustentável

- Golden dataset ≥ 50 casos por segmento (PF/PJ/FP);
- LLM Judge calibrado com $\kappa \geq 0,60$;
- Dashboard CloudWatch com alarme para IC $< 0,40$ em mais de 10% das respostas;
- Processo formal de expiração e atualização de casos do golden dataset.

Critério de saída: equipe de produto consegue atualizar o golden dataset sem dependência de engenharia.

15.4 Priorização por impacto e esforço

15.5 O ciclo de melhoria contínua

O modelo de evolução deste sistema é circular, não linear:

Tabela 15.1: Backlog priorizado de evolução do sistema.

Item	Impacto	Esforço	Nível
Golden dataset 50 casos/segmento	alto	médio	3
Gate CI/CD com ROUGE-L	alto	baixo	2
Calibração LLM Judge (κ)	alto	alto	3
BM25 híbrido no OpenSearch	médio	médio	2
Re-ranking com cross-encoder	médio	alto	3
Dashboard CloudWatch de IC	médio	baixo	2
Chunking com overlap	baixo	baixo	2

1. **Loop externo produz evidências:** IC baixo, regressão no golden dataset, score do judge < 7 .
2. **Análise de causa raiz:** qual camada falhou? Prompt? RAG? Modelo? Segmentação?
3. **Loop interno recebe insumos:** ajuste de prompt, re-indexação, atualização de políticas.
4. **Deploy com gate de qualidade:** o código só avança se zero regressões no golden dataset.
5. **Loop externo valida:** IC sobe, judge aprova, usuário satisfeito.

Este ciclo transforma qualidade de software em qualidade de IA: não se trata apenas de código correto, mas de respostas corretas para clientes reais.

15.6 Resumo

- Três níveis de maturidade: MVP offline, integração AWS, produção sustentável.
- Gate CI/CD e golden dataset são as entregas de maior impacto por menor esforço.
- A calibração do LLM Judge fecha o ciclo de confiança na avaliação automática.
- O loop de melhoria contínua transforma evidências de produção em melhorias no loop interno.

15.7 Espaço para expansão iterativa

- **v15.1:** *runbook* formal de incidente para IC crítico em produção.
- **v15.2:** modelo de maturidade certificado por terceiros (ex.: framework de IA responsável do Banco Central).

- **v15.3:** colaboração multiagente *peer-to-peer* para cenários de negociação entre agentes de compliance, risco e atendimento, reduzindo dependência de orquestrador central (9).
- **Lacuna:** definição de SLA por segmento (PF vs. PJ) e impacto no design do pipeline.

Capítulo 16

Esteiras de CI/CD Multi-Ambiente para Produtos AWS Bancários

16.1 Escopo do capítulo

Este capítulo descreve a organização e a operação das esteiras de integração contínua e entrega contínua (*Continuous Integration/Continuous Delivery* — CI/CD) adotadas nos produtos AWS do projeto bancário. A estratégia baseia-se em três ambientes isolados — `dev`, `homol` e `prod` — cada qual mapeado para uma conta AWS independente, garantindo isolamento de recursos, permissões e dados entre estágios do ciclo de vida do software. A infraestrutura de cada produto é integralmente descrita em módulos *Terraform* versionados, e os fluxos de automação são implementados como *Git Workflows* (*GitHub Actions* ou *GitLab CI*).

16.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Compreender a arquitetura de três ambientes AWS separados e as implicações de isolamento de conta para sistemas bancários;
- Estruturar esteiras de CI/CD independentes por produto AWS, com estágios de build, validação, homologação e produção claramente definidos;
- Configurar *workspaces* Terraform mapeados a ambientes e *state backends* segregados por conta;
- Implementar controles de promoção entre ambientes com aprovação humana obrigatória antes de `homol` e `prod`.

16.3 Conceitos teóricos

16.3.1 Isolamento de contas AWS por ambiente

O modelo de três contas AWS separadas — uma por ambiente — constitui prática recomendada pelo *AWS Well-Architected Framework* para sistemas financeiros regulados. O isolamento por conta oferece:

- **Blast radius controlado:** falhas, vazamentos de credenciais ou configurações equivocadas em `dev` não afetam `prod`;
- **Políticas IAM independentes:** cada conta define seus próprios limites de permissão sem interferência cruzada;
- **Segregação de dados:** dados reais de clientes presentes apenas em `prod`; `dev` e `homol` operam exclusivamente com dados sintéticos ou anonimizados.

O fluxo de promoção entre ambientes é descrito na subseção seguinte.

16.3.2 Esteira por produto

Cada produto AWS — Lambda, API Gateway, DynamoDB, SQS, OpenSearch, Neptune — mantém seu próprio repositório Git com conjunto de arquivos de *workflow* independente. Essa separação previne acoplamento entre produtos e permite que equipes distintas evoluam cadências de deploy sem bloquear umas às outras.

16.3.3 Fluxo de promoção entre ambientes

O ciclo de promoção segue o padrão:

1. **Feature branch** → **dev**: merge disparado por *pull request* aprovado; pipeline executa lint, testes unitários e `terraform plan`.
2. **dev** → **homol**: merge na branch `main`; exige aprovação do responsável de QA e pipeline verde.
3. **homol** → **prod**: *tag* de release semântica (`vMAJOR.MINOR.PATCH`); exige aprovação de dois revisores autorizados e janela de manutenção registrada.

16.4 Implementação orientada por passos

16.4.1 Passo 1 — Estrutura de repositório por produto

A organização recomendada para o repositório de um produto AWS é:

```

1 produto-lambda-segmentacao/
2   .github/
3     workflows/
4       ci.yml           # lint, testes, plan -- toda PR
5       deploy-dev.yml  # deploy automatico em dev
6       deploy-homol.yml # deploy com aprovacao QA
7       deploy-prod.yml # deploy com aprovacao dupla + tag
8   infraestrutura/
9     main.tf
10    variables.tf
11    outputs.tf
12    backend.tf         # configuracao S3 + DynamoDB lock
13    envs/
14      dev.tfvars
15      homol.tfvars
16      prod.tfvars      # sem segredos -- apenas referencias
17  src/
18    lambda_handler.py
19    tests/
20      test_unit.py
21      test_integration.py
22  requirements.txt
23  requirements-dev.txt

```

Listing 16.1: Estrutura de repositório de produto AWS bancário.

16.4.2 Passo 2 — Configurar *backend* Terraform por ambiente

Cada ambiente utiliza um *bucket* S3 e uma tabela DynamoDB dedicados para armazenar e bloquear o *state*:

```

1 # backend.tf
2 terraform {
3   backend "s3" {
4     bucket      = "tfstate-${var.ambiente}-123456789"
5     key         = "produto-lambda-segmentacao/terraform.
6                 tfstate"
7     region      = "us-east-1"
8     dynamodb_table = "tfstate-lock-${var.ambiente}"
9     encrypt     = true
10  }
11 }

```

Listing 16.2: Backend Terraform com lock por conta AWS.

O parâmetro `encrypt = true` garante criptografia do *state file* em repouso, exigência obrigatória para repositórios bancários que possam conter ARNs, IDs de recursos ou referências a segredos.

16.4.3 Passo 3 — Workflow de CI (pull request)

O workflow de CI é disparado em toda *pull request* aberta contra qualquer branch protegida:

```
1 # .github/workflows/ci.yml
2 name: CI
3
4 on:
5   pull_request:
6     branches: [main, develop]
7
8 jobs:
9   quality:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v4
13
14       - name: Setup Python
15         uses: actions/setup-python@v5
16         with:
17           python-version: "3.11"
18
19       - name: Instalar dependencias
20         run: pip install -r requirements-dev.txt
21
22       - name: Lint (flake8 + black)
23         run: |
24           flake8 src/ --max-line-length=100
25           black --check src/
26
27       - name: Testes unitarios
28         run: pytest src/tests/test_unit.py --cov=src --cov-fail
29             -under=70
30
31       - name: Seguranca estatica (bandit)
32         run: bandit -r src/ -ll
```

```

32
33     - name: Terraform validate
34       uses: hashicorp/setup-terraform@v3
35       with:
36         terraform_version: "1.7.0"
37     - run: |
38         cd infraestrutura
39         terraform init -backend=false
40         terraform fmt -check -recursive
41         terraform validate

```

Listing 16.3: GitHub Actions: workflow de CI para produto AWS.

16.4.4 Passo 4 — Workflow de deploy com promoção controlada

O deploy em `homol` e `prod` utiliza o mecanismo de *environment protection rules* do GitHub Actions, que bloqueia a execução até que revisores autorizados aprovem:

```

1  # .github/workflows/deploy-homol.yml
2  name: Deploy Homol
3
4  on:
5    push:
6      branches: [main]
7
8  jobs:
9    deploy:
10     runs-on: ubuntu-latest
11     environment: homologacao # protection rule: 1 aprovador
12     permissions:
13       id-token: write # OIDC para assume role sem
14       # chave estatica
15       contents: read
16
17     steps:
18       - uses: actions/checkout@v4
19
20       - name: Configurar credenciais AWS via OIDC
21         uses: aws-actions/configure-aws-credentials@v4
22         with:
23           role-to-assume: arn:aws:iam::HOMOL_ACCOUNT_ID:role/

```

```

23         GitHubActionsRole
24         aws-region: us-east-1
25     - name: Terraform apply (homol)
26       run: |
27         cd infraestrutura
28         terraform init
29         terraform workspace select homol
30         terraform apply -var-file=envs/homol.tfvars -auto-approve

```

Listing 16.4: GitHub Actions: deploy em homol com aprovacao.

A autenticação via *OIDC* (*OpenID Connect*) elimina a necessidade de chaves de acesso estáticas (`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`) armazenadas como *secrets* do repositório, reduzindo significativamente a superfície de ataque.

16.5 Plano de testes do capítulo

16.5.1 Teste funcional

- **TC-01:** Abrir uma PR de feature, introduzir um erro de lint intencional e verificar que o job `quality` falha antes do deploy.
- **TC-02:** Fazer merge na branch `main` sem aprovação do QA e verificar que o job `deploy-homol` fica bloqueado aguardando aprovação.
- **TC-03:** Criar uma tag `v1.0.0` sem os dois revisores autorizados e verificar que o deploy em prod não prossegue.

16.5.2 Teste de desempenho

- **TP-01:** Medir o tempo total da pipeline de CI (checkout a validação Terraform); critério: ≤ 8 min.
- **TP-02:** Medir o tempo de `terraform apply` em `homol` para o produto Lambda de segmentação; critério: ≤ 5 min.

16.5.3 Teste de qualidade

- **TQ-01:** Executar `checkov` contra os módulos Terraform e verificar zero falhas de nível CRITICAL.

- **TQ-02:** Validar que o *state file* de **prod** está criptografado e o acesso ao bucket S3 é restrito apenas à role de CI/CD.

16.6 Critérios de aceite

- TC-01 a TC-03 aprovados sem intervenção manual.
- TP-01: pipeline de CI ≤ 8 min evidenciada com *timestamps* do GitHub Actions.
- TQ-01: zero falhas CRITICAL no **checkov**.
- Nenhuma chave de acesso estática AWS presente nos *secrets* do repositório; autenticação exclusivamente via OIDC.

16.7 Espaço para expansão iterativa

- **v15.1** — Implementar *drift detection*: job agendado que executa **terraform plan** diariamente em **prod** e alerta via SNS caso detecte desvio entre o *state* e a infraestrutura real.
- **v15.2** — Adotar *GitOps* com *ArgoCD* ou *Flux* para sincronização declarativa de manifests Kubernetes, se a plataforma evoluir para EKS.
- **Lacuna** — Investigar o impacto de *feature flags* no ciclo de promoção, permitindo ativar funcionalidades seletivamente por conta AWS sem novo deploy.

Capítulo 17

Qualidade, Segurança e Governança nas Esteiras de Produto

17.1 Escopo do capítulo

Este capítulo detalha os requisitos de qualidade mínima, os padrões de segurança e as práticas de governança obrigatórios em todas as esteiras de produto da plataforma bancária. Enquanto o Capítulo 16 aborda a estrutura de ambientes e o fluxo de promoção, este capítulo concentra-se nos *quality gates* — pontos de controle que um artefato deve transpor antes de avançar ao próximo estágio — e nos controles de conformidade exigidos pela regulação bancária brasileira, em especial a Resolução CMN nº 4.893/2021 do Banco Central do Brasil.

17.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Definir e configurar *quality gates* para testes unitários, de integração e de segurança estática em pipelines AWS;
- Aplicar análise estática de segurança com **bandit** (Python) e **checkov** / **tfsec** (Terraform);
- Implementar o princípio do menor privilégio em roles IAM criadas via Terraform;
- Configurar auditoria centralizada com CloudTrail e alertas de desvio de conformidade.

17.3 Conceitos teóricos

17.3.1 Quality gates nas esteiras bancárias

Um *quality gate* é uma condição binária — aprovado/reprovado — que bloqueia o avanço do pipeline caso um limiar mínimo não seja atingido. A Tabela 17.1 apresenta os gates obrigatórios, os limiares adotados e as ferramentas responsáveis por cada verificação.

Tabela 17.1: Quality gates obrigatórios por estágio de pipeline bancário.

Estágio	Gate	Limiar	Ferramenta
PR / CI	Cobertura de testes unitários	$\geq 70\%$	<code>pytest -cov</code>
PR / CI	Lint e formatação	zero erros	<code>flake8</code> , <code>black</code>
PR / CI	Segurança estática Python	zero HIGH	<code>bandit</code>
PR / CI	Segurança Terraform	zero CRITICAL	<code>checkov</code> , <code>tfsec</code>
Homol	Testes de integração	100% pass	<code>pytest</code>
Homol	Drift de infraestrutura	zero desvios	<code>terraform plan</code>
Prod	Aprovação dupla	2 revisores	GitHub Environments
Prod	Janela de manutenção	registrada	Change Management

17.3.2 Segurança estática: Python e Terraform

A análise estática de segurança é executada antes de qualquer deploy, detectando vulnerabilidades conhecidas no código-fonte e nas configurações de infraestrutura:

- **bandit**: verifica o código Python em busca de padrões do OWASP Top 10, como uso de `eval()`, `pickle`, senão constantes em hashes criptográficos e injection vulnerabilities.
- **checkov**: analisa recursos Terraform contra um banco de mais de 1.000 políticas de segurança (CIS Benchmarks, AWS Foundational Security Best Practices).
- **tfsec**: complementa o **checkov** com verificações específicas de misconfigurações AWS, como buckets S3 públicos, security groups com 0.0.0.0/0 e criptografia ausente.

17.3.3 Princípio do menor privilégio em IAM

Roles IAM para funções Lambda e pipelines de CI/CD devem seguir o padrão:

1. Nenhum wildcard (*) em `Action` ou `Resource`.
2. `Condition` com `aws:SourceAccount` restringindo o acesso à conta AWS do ambiente.
3. Política de negação explícita para ações destrutivas (`DeleteTable`, `DeleteBucket`) em ambientes `homol` e `prod`.
4. Revisão semestral automática via AWS Access Analyzer.

17.4 Implementação orientada por passos

17.4.1 Passo 1 — Configurar quality gate de testes e lint

```

1  # Trecho do job "quality" em .github/workflows/ci.yml
2  - name: Testes unitarios com cobertura
3    run: |
4      pytest src/tests/test_unit.py \
5        --cov=src \
6        --cov-report=xml \
7        --cov-fail-under=70 \
8        -v
9
10 - name: Upload cobertura (Codecov)
11   uses: codecov/codecov-action@v4
12   with:
13     files: coverage.xml
14     fail_ci_if_error: true
15
16 - name: Lint (flake8)
17   run: flake8 src/ --max-line-length=100 --statistics
18
19 - name: Formatacao (black)
20   run: black --check --diff src/

```

Listing 17.1: Job de quality gate no GitHub Actions com cobertura e lint.

17.4.2 Passo 2 — Testes de integração contra ambiente dev

Os testes de integração são executados no ambiente `dev` após o deploy bem-sucedido, utilizando dados sintéticos que replicam a estrutura de dados de produção sem expor informações reais de clientes:

```

1  import pytest
2  import boto3
3  import json
4
5  API_URL = "https://api-dev.banco.interno/v1/segmentacao"
6
7  @pytest.fixture(scope="module")
8  def api_token():
9      """Obtem token de teste via Cognito do ambiente dev."""

```

```

10     client = boto3.client("cognito-idp", region_name="us-east-1
11         ")
12     resp = client.initiate_auth(
13         AuthFlow="USER_PASSWORD_AUTH",
14         AuthParameters={
15             "USERNAME": "usuario-teste-ci",
16             "PASSWORD": "Senha-Teste-CI-123!"    # secret do
17             repositório
18         },
19         ClientId="CLIENT_ID_DEV"
20     )
21     return resp["AuthenticationResult"]["IdToken"]
22
23 def test_segmentacao_cliente_valido(api_token):
24     """TC-INT-01: cliente com dados completos deve retornar
25     segmento."""
26     import requests
27     resp = requests.post(
28         API_URL,
29         headers={"Authorization": f"Bearer {api_token}"},
30         json={"cliente_id": "TESTE-001", "produto": "
31             cartao_credito"},
32         timeout=10
33     )
34     assert resp.status_code == 200
35     data = resp.json()
36     assert "segmento" in data
37     assert data["segmento"] in ["PREMIUM", "PADRAO", "BASICO"]
38
39 def test_segmentacao_cliente_invalido(api_token):
40     """TC-INT-02: ID inexistente deve retornar 404."""
41     import requests
42     resp = requests.post(
43         API_URL,
44         headers={"Authorization": f"Bearer {api_token}"},
45         json={"cliente_id": "INEXISTENTE-999"},
46         timeout=10
47     )
48     assert resp.status_code == 404

```

Listing 17.2: Teste de integração para endpoint de segmentação.

17.4.3 Passo 3 — Análise de segurança estática

```

1  # Seguranca Python (bandit)
2  - name: Bandit -- seguranca estatica Python
3  run: |
4      bandit -r src/ \
5          --level HIGH \
6          --confidence HIGH \
7          --format json \
8          --output bandit-report.json
9      # Falha se houver qualquer item HIGH/HIGH
10     python -c "
11         import json, sys
12         r = json.load(open('bandit-report.json'))
13         highs = [i for i in r['results']]
14                 if i['issue_severity'] == 'HIGH'
15                 and i['issue_confidence'] == 'HIGH']
16     if highs:
17         print(f'FALHA: {len(highs)} itens HIGH encontrados')
18         sys.exit(1)
19     print('OK: nenhum item HIGH/HIGH')
20     "
21
22  # Seguranca Terraform (checkov)
23  - name: Checkov -- seguranca Terraform
24  uses: bridgecrewio/checkov-action@master
25  with:
26      directory: infraestrutura/
27      framework: terraform
28      soft_fail: false
29      quiet: true
30      output_format: cli

```

Listing 17.3: Quality gate de segurança: bandit e checkov.

17.4.4 Passo 4 — Role IAM com menor privilégio via Terraform

```

1  # infraestrutura/modules/iam/lambda_segmentacao.tf
2  resource "aws_iam_role" "lambda_segmentacao" {
3      name = "lambda-segmentacao-${var.ambiente}"
4
5      assume_role_policy = jsonencode({

```

```

6     Version = "2012-10-17"
7     Statement = [{
8         Effect      = "Allow"
9         Principal = { Service = "lambda.amazonaws.com" }
10        Action      = "sts:AssumeRole"
11        Condition = {
12            StringEquals = {
13                "aws:SourceAccount" = var.account_id
14            }
15        }
16    }]
17 })
18 }

19
20 resource "aws_iam_role_policy" "lambda_segmentacao_policy" {
21     name = "politica-segmentacao-${var.ambiente}"
22     role = aws_iam_role.lambda_segmentacao.id
23
24     policy = jsonencode({
25         Version = "2012-10-17"
26         Statement = [
27             {
28                 Effect = "Allow"
29                 Action = [
30                     "dynamodb:GetItem",
31                     "dynamodb:Query"
32                 ]
33                 Resource = [
34                     "arn:aws:dynamodb:us-east-1:${var.account_id}:table/
35                         clientes-${var.ambiente}",
36                     "arn:aws:dynamodb:us-east-1:${var.account_id}:table/
37                         clientes-${var.ambiente}/index/*"
38                 ]
39             },
40             {
41                 Effect = "Allow"
42                 Action = [
43                     "secretsmanager:GetSecretValue"
44                 ]
45                 Resource = "arn:aws:secretsmanager:us-east-1:${var.
46                     account_id}:secret:llm/*"

```

```

44     },
45     {
46         Effect = "Allow"
47         Action = [
48             "logs:CreateLogGroup",
49             "logs:CreateLogStream",
50             "logs:PutLogEvents"
51         ]
52         Resource = "arn:aws:logs:us-east-1:${var.account_id}:
                    log-group:/aws/lambda/segmentacao-*:*"
53     }
54 ]
55 })
56 }

```

Listing 17.4: Role IAM para Lambda de segmentação com menor privilégio.

Observa-se que nenhuma ação contém wildcard e todos os `Resource` apontam para ARNs específicos do ambiente, parametrizados pelas variáveis `var.account_id` e `var.ambiente`.

17.5 Plano de testes do capítulo

17.5.1 Teste funcional

- **TC-01:** Introduzir um `eval()` no código Python e verificar que o `bandit` reprovada o `gate`.
- **TC-02:** Criar um recurso S3 sem criptografia no Terraform e verificar que o `checkov` bloqueia o pipeline.
- **TC-03:** Reduzir a cobertura de testes abaixo de 70% e verificar que o job falha com mensagem explícita.
- **TC-04:** Executar os testes de integração contra `dev` e verificar que TC-INT-01 e TC-INT-02 passam.

17.5.2 Teste de conformidade

- **TConf-01:** Verificar que o CloudTrail está habilitado em todas as três contas AWS (`dev`, `homol`, `prod`) com *trails* multirregionais.
- **TConf-02:** Verificar que o *Access Analyzer* do AWS IAM não reporta acessos externos não intencionais.

- **TConf-03:** Confirmar que nenhuma chave de acesso estática AWS está presente nos *secrets* do repositório via `git-secrets` ou `truffleHog`.

17.6 Critérios de aceite

- TC-01 a TC-04 aprovados.
- TConf-01 a TConf-03 aprovados sem exceção.
- Cobertura de testes $\geq 70\%$ confirmada para todos os produtos em produção.
- Zero findings CRITICAL no `checkov` e HIGH/HIGH no `bandit` nos módulos de produção.

17.7 Espaço para expansão iterativa

- **v16.1** — Integrar *DAST* (*Dynamic Application Security Testing*) com OWASP ZAP no estágio de homologação, executando varredura ativa contra o endpoint do API Gateway de `homol`.
- **v16.2** — Implementar *Software Bill of Materials* (SBOM) automático com `syft` para rastreabilidade de dependências em conformidade com requisitos de auditoria do BACEN.
- **v16.3** — Adotar *policy-as-code* com *Open Policy Agent* (OPA) para centralizar e versionar políticas de segurança como código, reutilizáveis em todos os produtos.
- **Lacuna** — Investigar a adequação ao *CIS AWS Foundations Benchmark* e ao *PCI DSS* v4.0 para produtos que processam dados de cartão.

Capítulo 18

Frontend Web com CloudFront para Operação da API e Ingestão RAG

18.1 Escopo do capítulo

Este capítulo apresenta a camada de experiência do usuário da plataforma: uma interface web estática distribuída por *Amazon CloudFront*¹ com origem em bucket *Amazon S3* privado, destinada a dois fluxos principais: interação com a API assíncrona via `POST /query` e `GET /status/{request_id}` e ingestão controlada de documentos no pipeline *Retrieval-Augmented Generation* (RAG) via `POST /documentos`. O objetivo é fornecer uma superfície web de operação que possa ser utilizada por usuários de negócio, analistas de homologação e equipes técnicas sem necessidade de notebooks ou scripts auxiliares.

18.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Descrever a arquitetura de publicação de um frontend estático via CloudFront e bucket S3 privado com *Origin Access Control*;
- Implementar uma interface web para envio de consultas ao agente e acompanhamento do processamento assíncrono;
- Disponibilizar, na mesma interface, um fluxo de envio de documentos textuais para indexação no RAG;
- Avaliar limitações do backend atual para formatos binários como PDF e DOCX e propor a extensão arquitetural adequada.

¹Rede de distribuição de conteúdo (CDN) da AWS utilizada para publicar arquivos estáticos com baixa latência e alta disponibilidade.

18.3 Conceitos teóricos

18.3.1 Frontend estático como superfície de operação

Em arquiteturas bancárias modernas, a separação entre camada de apresentação e serviços de negócio reduz acoplamento, melhora a escalabilidade e simplifica a estratégia de publicação. Para o caso em estudo, um frontend estático atende adequadamente aos requisitos porque:

- o processamento de negócio ocorre integralmente no backend assíncrono (API Gateway + Lambda + SQS + DynamoDB);
- a UI atua apenas como orquestradora de chamadas HTTP, renderização de respostas e coleta de insumos do usuário;
- a camada estática pode ser publicada sem servidor dedicado, com custo operacional reduzido e baixa superfície de ataque.

18.3.2 CloudFront + S3 privado

A distribuição CloudFront é configurada para apontar para um bucket S3 privado que armazena os arquivos `index.html`, `styles.css`, `app.js` e `config.js`. O acesso direto ao bucket permanece bloqueado; apenas a distribuição CloudFront, por meio de *Origin Access Control* (OAC), possui permissão de leitura. Esse padrão é preferível ao uso de *website endpoint* público do S3, pois preserva o princípio de exposição mínima e centraliza a entrega HTTPS.

18.3.3 Chat assíncrono e ingestão RAG em uma única UI

A interface implementa dois fluxos distintos:

1. **Consulta assíncrona:** o usuário preenche pergunta, modo de operação e, opcionalmente, dados do cliente; a UI envia o payload para `/query`, recebe um `request_id` e executa *polling* em `/status/{request_id}` até o estado `COMPLETED`.
2. **Ingestão de documentos:** o usuário informa o tipo (documento ou entrevista), título e conteúdo, podendo carregar um arquivo textual local. O frontend converte o arquivo em texto e envia JSON para `/documentos`.

A limitação atual decorre do contrato do endpoint `/documentos`, que aceita apenas JSON com campo `texto`; não há suporte nativo a *multipart/form-data*, nem extração de conteúdo de formatos binários como PDF. Assim, a UI suporta apenas arquivos textuais e colagem manual de conteúdo.

18.4 Implementação orientada por passos

18.4.1 Passo 1 — Estruturar os assets do frontend

A aplicação web foi implementada sem *framework* JavaScript, com o objetivo de minimizar dependências e facilitar a publicação:

```
1 frontend/  
2   cloudfront_console/  
3     index.html  
4     styles.css  
5     app.js  
6     config.js
```

Listing 18.1: Estrutura dos arquivos estáticos do console web.

O arquivo `config.js` é sobrescrito no deploy Terraform com a URL real da API Gateway, eliminando a necessidade de compilação por ambiente.

18.4.2 Passo 2 — Publicar a UI em CloudFront

O módulo Terraform `cloudfront_frontend` cria:

- bucket S3 privado com versionamento e criptografia;
- OAC para leitura segura do bucket pela distribuição;
- distribuição CloudFront com `default_root_object = index.html`;
- objetos S3 contendo os assets do frontend e o `config.js` parametrizado.

```
1 module "cloudfront_frontend" {  
2   source      = "./modules/cloudfront_frontend"  
3   project_name = var.project_name  
4   environment  = var.environment  
5   api_endpoint = module.api_gateway.api_endpoint  
6 }  
7  
8 output "frontend_url" {  
9   value = "https://${module.cloudfront_frontend.  
10     distribution_domain_name}"  
}
```

Listing 18.2: Trecho do módulo Terraform do frontend CloudFront.

18.4.3 Passo 3 — Implementar o fluxo de consulta assíncrona

A interface web coleta pergunta, modo e dados do cliente e monta o payload compatível com o contrato OpenAPI da aplicação. Em seguida:

1. envia `POST /query`;
2. armazena o `request_id`;
3. inicia *polling* com *backoff* exponencial;
4. atualiza a UI com status intermediários e resultado final.

Esse padrão desacopla a experiência do usuário da duração real do pipeline RAG, compatibilizando a interface web com o limite de 29 segundos do API Gateway descrito no Capítulo 7.

18.4.4 Passo 4 — Implementar envio de documentos para RAG

O fluxo de ingestão aceita dois modos de entrada:

- **Upload de arquivo textual:** a UI lê o arquivo no navegador com `File.text()` e preenche o campo `texto`;
- **Colagem manual:** o usuário insere diretamente o conteúdo no *textarea*, permitindo testes e indexação rápida.

O payload enviado para a API segue o formato:

```
1 {  
2   "tipo": "documento",  
3   "titulo": "Politica de Investimentos 2026",  
4   "texto": "Clientes conservadores priorizam liquidez..."  
5 }
```

Listing 18.3: Payload de ingestão de documento no RAG.

Quando o tipo é `entrevista`, a UI pode enviar adicionalmente `cliente_id` ou `cluster_id`, alinhando a indexação com os modos *twin*, *persona* e *segmento*.

18.4.5 Passo 5 — Planejar evolução para PDF e DOCX

Para suportar documentos binários, recomenda-se introduzir uma etapa de extração antes da chamada a `/documentos`. Duas abordagens são plausíveis:

1. **Extração no backend:** novo endpoint de upload recebe o binário, armazena em S3 e aciona Lambda com parser (Apache Tika, PyMuPDF, python-docx ou Amazon Textract).
2. **Pré-processamento cliente:** alternativa limitada a alguns formatos e menos indicada para ambiente bancário, pois expõe a extração ao navegador.

Para o cenário bancário, a primeira abordagem é superior por motivos de rastreabilidade, auditoria e controle de segurança.

18.5 Plano de testes do capítulo

18.5.1 Teste funcional

- **TC-01:** Abrir a URL do CloudFront e verificar o carregamento de `index.html`, `styles.css` e `config.js` sem acesso direto público ao bucket S3.
- **TC-02:** Executar uma consulta em modo `segmento` e verificar que a UI exibe `request_id`, status intermediário e resposta final.
- **TC-03:** Submeter um documento textual e verificar resposta HTTP 200 com `chunks_indexados > 0`.
- **TC-04:** Tentar enviar um arquivo PDF e verificar que a UI informa explicitamente a limitação atual do backend.

18.5.2 Teste de validação em produção (AWS)

Foi executado um smoke test completo no ambiente publicado, com os seguintes endereços:

- **Frontend CloudFront:** <https://djxkmivr2akfz.cloudfront.net>;
- **API Gateway:** <https://3vqrahlmyg.execute-api.sa-east-1.amazonaws.com>.

Os resultados observados foram:

- **TVP-01 (ingestão entrevista):** `ok=true, chunks_indexados=1`;
- **TVP-02 (consulta persona):** `request_id: <ddbc5210-63fc-47bc-a274-283e60be84bc>`.
Estado final COMPLETED;
- **TVP-03 (consulta twin):** `request_id: <e5e348e7-d266-4ae9-9f26-eb0e659a56a7>`.
Estado final COMPLETED;

- **TVP-04 (correção de exibição da resposta):** Identificado e corrigido defeito no `app.js` em que o campo `data.result` era exibido diretamente como texto, resultando em `[object Object]`. A correção extrai `result.resposta` após *parse* JSON, exibindo o texto legível do agente no painel **Resposta** e os metadados (`cliente_id`, `cluster_id`, `confiabilidade`) no painel **Status**. Implantado em produção via `aws s3 sync` e invalidação do cache CloudFront em 02/05/2026.

Com isso, verifica-se a operação ponta a ponta da interface web em produção, incluindo ingestão RAG e consulta assíncrona com *polling* até a conclusão.

18.5.3 Teste de segurança

- **TS-01:** Confirmar que o bucket S3 do frontend possui *public access block* habilitado.
- **TS-02:** Verificar que apenas a distribuição CloudFront possui permissão de leitura nos objetos estáticos.
- **TS-03:** Validar cabeçalhos CORS da API para permitir consumo pelo frontend, sem abrir além do necessário.

18.6 Critérios de aceite

- TC-01 a TC-04 aprovados.
- TS-01 a TS-03 aprovados.
- URL `frontend_url` gerada pelo Terraform e acessível via HTTPS.
- Fluxos de consulta e ingestão operacionais em *dev*.
- Smoke test em produção aprovado para ingestão, *persona* e *twin* (TVP-01 a TVP-03).
- Resposta legível do agente exibida no painel **Resposta** via `result.resposta` (TVP-04 — defeito de exibição corrigido).

18.7 Espaço para expansão iterativa

- **v17.1** — Integrar autenticação com Amazon Cognito para restringir o acesso à interface web.
- **v17.2** — Adicionar histórico de consultas e visualização de status por usuário autenticado.

- **v17.3** — Suportar upload binário com pipeline de extração para PDF, DOCX e imagens escaneadas.
- **Lacuna** — Avaliar *WebSocket API* como alternativa ao *polling* HTTP para feedback de progresso em tempo real.

Apêndice A

Apendice de Código Completo das Soluções

Este apêndice consolida o código completo dos artefatos discutidos ao longo dos capítulos. O objetivo é manter a rastreabilidade entre explicação e implementação executável no projeto. Para estabilidade de compilação em L^AT_EX com `listings`, os blocos abaixo usam cópias ASCII geradas em `livro/generated/apendice_ascii/`; os arquivos originais permanecem no projeto com seus caminhos de origem.

A.1 Como validar a autocontenção

Antes de executar os exemplos, rode:

```
1 pwsh -File livro/validar_autocontencao.ps1
```

Listing A.1: Verificação automática de referências de código no livro.

Resultado esperado: Referências faltantes: 0.

A.2 Capítulo 2 e Capítulo 10

A.2.1 langchain.ipynb (exportado para Python)

```
1 # Arquivo gerado automaticamente a partir de langchain.ipynb
2 # Conteúdo completo dos blocos de código do notebook
3
4 # ===== cell_003 =====
5 # Instala todas as dependências necessárias para o notebook
6 !pip install -q \
7     langgraph \
8     langchain \
```



```

9      langchain-anthropic \
10     langchain-community \
11     langchain-text-splitters \
12     langchain-mcp-adapters \
13     faiss-cpu \
14     sentence-transformers \
15     psycopg2-binary \
16     mcp \
17     python-dotenv \
18     duckduckgo-search \
19     wikipedia \
20     httpx \
21     deepeval \
22     scikit-learn
23
24     # Opcional (producao): pip install langchain-tavily
25     print("? Dependencias instaladas com sucesso!")
26
27     # ===== cell_005 =====
28     import os
29     import warnings
30     from dotenv import load_dotenv
31
32     load_dotenv()
33
34     ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY")
35     if not ANTHROPIC_API_KEY:
36         raise ValueError("Configure ANTHROPIC_API_KEY no arquivo .
37         env")
38
39     print(f"? Chave API configurada: {ANTHROPIC_API_KEY[:20]}...")
40
41     # ??? Suprime o aviso de migracao do LangGraph 1.0
42     ????????????????????????????????
43
44     # create_react_agent (langgraph.prebuilt) e a API correta para
45     agentes stateful
46
47     # com checkpointer e prompt. A funcao langchain.agents.
48     create_agent tem assinatura
49
50     # incompativel (nao suporta checkpointer, prompt como string,
51     state_modifier, etc.)

```

```
44 # O aviso sera removido quando o LangGraph unificar a API em V2
    .0.
45 warnings.filterwarnings("ignore", message=".*create_react_agent
    .*)
46
47 print("? Warning de migracao LangGraph 1.0 suprimido")
48
49 # ===== cell_007 =====
50 from plantuml_helper import show_plantuml
51
52 show_plantuml("""
53 @startuml
54 skinparam backgroundColor #FFFFFF
55 skinparam activity {
56     StartColor #2E7D32
57     EndColor #C62828
58     BackgroundColor #E3F2FD
59     BorderColor #1565C0
60     FontSize 13
61 }
62
63 start
64 :Usuario envia pergunta;
65 :Agente LangGraph interpreta contexto;
66
67 if (Precisa chamar ferramenta?) then (sim)
68     :Seleciona tool adequada;
69     :Executa tool;
70     :Recebe resultado da tool;
71     :Atualiza raciocinio;
72 else (nao)
73     :Usa conhecimento do modelo;
74 endif
75
76 :Gera resposta final;
77 :Retorna resposta ao usuario;
78 stop
79 @enduml
80 """)
81
82 # ===== cell_008 =====
```

```
83 from langchain_anthropic import ChatAnthropic
84 from langchain_core.tools import tool
85 from langgraph.prebuilt import create_react_agent
86 from langgraph.checkpoint.memory import MemorySaver
87
88 model = ChatAnthropic(model="claude-sonnet-4-6", temperature=0)
89
90 @tool
91 def calculadora(expressao: str) -> str:
92     """Avalia uma expressao matematica. Ex: '2 + 2', '15 *
93         3.5', '100 / 4'."""
94     try:
95         allowed = set('0123456789+-*/() .,% ')
96         if not all(c in allowed for c in expressao):
97             return "Expressao invalida"
98         return str(eval(expressao))
99     except Exception as e:
100         return f"Erro: {e}"
101
102 @tool
103 def consultar_clima(cidade: str) -> str:
104     """Consulta o clima atual de uma cidade brasileira."""
105     clima_mock = {
106         "Sao Paulo": "25°C, Nublado com chance de chuva",
107         "Rio de Janeiro": "31°C, Ensolarado e umido",
108         "Brasilia": "22°C, Parcialmente nublado e seco",
109         "Curitiba": "15°C, Frio com vento sul",
110         "Fortaleza": "33°C, Ensolarado e quente",
111     }
112     return clima_mock.get(cidade, f"Dados nao disponiveis para
113         '{cidade}'")
114
115 @tool
116 def converter_moeda(valor: float, de: str, para: str) -> str:
117     """Converte entre moedas. Suporta: BRL, USD, EUR."""
118     taxas = {"BRL": 1.0, "USD": 5.70, "EUR": 6.20}
119     if de not in taxas or para not in taxas:
120         return f"Moedas suportadas: {list(taxas.keys())}"
121     resultado = valor * taxas[de] / taxas[para]
122     return f"{valor:.2f} {de} = {resultado:.2f} {para}"
```

```

122 tools = [calculadora, consultar_clima, converter_moeda]
123 memory = MemorySaver()
124 agente = create_react_agent(model, tools, checkpointer=memory)
125
126 print("? Agente criado com ferramentas:", [t.name for t in
      tools])
127
128 # ===== cell_009 =====
129 # ??? Configuracao de thread (cada thread_id = uma conversa
      independente) ?????
130 config = {"configurable": {"thread_id": "conversa-demo-1"}}
131
132 def conversar(mensagem: str, config: dict = config):
133     """Envia uma mensagem ao agente e retorna a resposta."""
134     resultado = agente.invoke(
135         {"messages": [{"role": "user", "content": mensagem}]},
136         config=config
137     )
138     return resultado["messages"][-1].content
139
140 # ??? Teste 1: Consulta simples
      ?????????????????????????????????????????????????????????
141 print("?? Usuario: Qual e o clima em Sao Paulo e Curitiba?")
142 resposta = conversar("Qual e o clima em Sao Paulo e Curitiba?")
143 print(f"?? Agente: {resposta}\n")
144
145 # ??? Teste 2: Demonstrando memoria de curto prazo
      ?????????????????????????????????????????????
146 # O agente lembra da conversa anterior (mesmo thread_id)
147 print("?? Usuario: Com esse clima de SP, quanto eu gastaria em
      USD numa jaqueta de R$300?")
148 resposta = conversar("Com esse clima de SP, quanto eu gastaria
      em USD numa jaqueta de R$300?")
149 print(f"?? Agente: {resposta}\n")
150
151 # ??? Teste 3: Nova thread = nova conversa (sem memoria da
      anterior) ?????????????
152 config_nova = {"configurable": {"thread_id": "conversa-demo-2"
      }}
153 print("?? Usuario (nova thread): Qual cidade eu perguntei antes
      ?")

```

```
resposta = conversar("Qual cidade eu perguntei antes?",  
    config_nova)  
  
print(f"?? Agente (sem memoria da thread anterior): {resposta}")  
)  
  
# ===== cell_011 =====  
  
import asyncio  
  
# ??? ainvoke: chamada assincrona basica  
?????  
  
# Equivalente async de .invoke(). Libera o event loop enquanto  
o LLM processa.  
  
# Use em: FastAPI, servidores web, qualquer contexto async.  
  
config_ai = {"configurable": {"thread_id": "ainvoke-demo-1"}}  
  
print("? ainvoke: aguardando resposta completa do agente...\n")  
  
resultado = await agente.invoke(  
    {"messages": [{"role": "user", "content":  
        "Qual e o clima em Fortaleza e quanto vale 50 USD em  
BRL?"}]},  
    config=config_ai  
)  
  
print("? Resposta completa:")  
print(resultado["messages"][-1].content)  
  
# ??? Comparacao sync vs async  
?????  
  
print("\n" + "-" * 55)  
print("? sync ? agente.invoke() ? bloqueia a thread ate  
terminar")  
  
print("? async ? await agente.invoke() ? libera o event loop"  
)  
  
# ===== cell_012 =====  
  
from langchain_core.messages import AIMessageChunk  
  
# ??? astream: streaming token a token  
?????
```



```

219 print("? " * 55)
220 print("stream_mode='values'    ? estado completo por no (padrao)
      ")
221 print("stream_mode='updates'    ? dict {no: delta_estado}")
222 print("stream_mode='messages' ? (AIMessageChunk, meta) token a
      token    ? streaming real")
223
224 # ===== cell_014 =====
225 # ??? Instalacao do MemPalace
      ?????????????????????????????????????????????????????????????
226 # Instala MemPalace localmente (sem dependencias cloud)
227 !pip install -q mempalace
228
229 print("? MemPalace instalado!")
230 print("?? Memorias serao armazenadas em: ~/.mempalace/
      langchain_agent")
231
232 # ===== cell_015 =====
233 import subprocess
234 import json
235 from pathlib import Path
236 from langchain_core.tools import tool
237 from pydantic import BaseModel, Field
238
239 # ??? Configuracao do MemPalace
      ?????????????????????????????????????????????????????????????
240 PALACE_PATH = Path.home() / ".mempalace" / "langchain_agent"
241 PALACE_PATH.mkdir(parents=True, exist_ok=True)
242
243 print(f"?? MemPalace configurado em: {PALACE_PATH}")
244
245 # ??? Ferramentas MemPalace para o Agente
      ?????????????????????????????????????????????????????????????
246
247 @tool
248 def memoria_longo_prazo(consulta: str) -> str:
249     """Busca em memorias persistentes de conversas e decisoes
      anteriores.
250     Use para: relembrar decisoes passadas, contexto de projetos
      , preferencias do usuario."""
251

```

```

252     try:
253         # Comando: mempalace search "consulta" --palace-path
                PALACE_PATH
254         result = subprocess.run([
255             "mempalace", "search", consulta,
256             "--palace-path", str(PALACE_PATH)
257         ], capture_output=True, text=True, timeout=10)
258
259         if result.returncode == 0 and result.stdout.strip():
260             return f"?? Memorias encontradas:\n{result.stdout.
                strip()}"
261         else:
262             return f"?? Nenhuma memoria encontrada para: '{
                consulta}'"
263
264     except (subprocess.TimeoutExpired, FileNotFoundError) as e:
265         return f"? MemPalace nao disponivel: {e}"
266
267 class SalvarMemoriaInput(BaseModel):
268     conteudo: str = Field(description="Conteudo importante para
                salvar")
269     topico: str = Field(description="Topico/categoria (ex: '
                decisao_tecnica', 'preferencia_usuario')")
270
271 @tool
272 def salvar_memoria_importante(conteudo: str, topico: str) ->
    str:
273     """Salva informacao importante na memoria de longo prazo.
274     Use para: decisoes importantes, descobertas, preferencias
                do usuario."""
275
276     try:
277         # Comando: mempalace add-drawer --wing langchain_agent
                --room topico --content conteudo
278         result = subprocess.run([
279             "mempalace", "add-drawer",
280             "--wing", "langchain_agent",
281             "--room", topico,
282             "--content", conteudo,
283             "--palace-path", str(PALACE_PATH)
284         ], capture_output=True, text=True, timeout=10)

```



```
285
286     if result.returncode == 0:
287         return f"? Memoria salva em Wing: langchain_agent,
                Room: {topico}"
288     else:
289         return f"? Erro ao salvar: {result.stderr}"
290
291     except (subprocess.TimeoutExpired, FileNotFoundError) as e:
292         return f"? MemPalace nao disponivel: {e}"
293
294 @tool
295 def carregar_contexto_essencial() -> str:
296     """Carrega contexto essencial (~170 tokens) de sessoes
        anteriores.
        Use no inicio de conversas para recuperar contexto
        importante."""
297
298     try:
299         # Comando: mempalace wake-up --palace-path PALACE_PATH
300         result = subprocess.run([
301             "mempalace", "wake-up",
302             "--palace-path", str(PALACE_PATH)
303         ], capture_output=True, text=True, timeout=10)
304
305         if result.returncode == 0 and result.stdout.strip():
306             return f"? Contexto carregado:\n{result.stdout.
                strip()}"
307         else:
308             return f"? Nenhum contexto essencial encontrado
                ainda. Construa memorias primeiro!"
309
310     except (subprocess.TimeoutExpired, FileNotFoundError) as e:
311         return f"? MemPalace nao disponivel: {e}"
312
313
314 print(f"? Ferramentas MemPalace criadas:")
315 print(f"    ? memoria_longo_prazo() - busca memorias persistentes
        ")
316 print(f"    ? salvar_memoria_importante() - salva info relevante"
        )
317 print(f"    ? carregar_contexto_essencial() - recupera contexto
        de ~170 tokens")
```

```

318
319 # ===== cell_016 =====
320 # ??? Agente Hibrido: Sistema Atual + MemPalace
    ?????????????????????????????????????????
321 # Combina: FAISS (medio prazo) + MemPalace (longo prazo) +
    ferramentas existentes
322
323 # Ferramentas existentes do projeto
324 ferramentas_existentes = [
325     calculadora, consultar_clima, converter_moeda,
326     buscar_conhecimento # RAG com FAISS
327 ]
328
329 # Novas ferramentas MemPalace
330 ferramentas_mempalace = [
331     memoria_longo_prazo,
332     salvar_memoria_importante,
333     carregar_contexto_essencial
334 ]
335
336 # Agente hibrido com ambas as capacidades de memoria
337 todas_ferramentas_hibridas = ferramentas_existentes +
    ferramentas_mempalace
338 memory_hibrida = MemorySaver() # Curto prazo (in-session)
339
340 agente_mempalace = create_react_agent(
341     model,
342     todas_ferramentas_hibridas,
343     checkpointer=memory_hibrida
344 )
345
346 print(f"?? Agente hibrido criado com {len(
    todas_ferramentas_hibridas)} ferramentas:")
347 print("\n?? Capacidades de Memoria:")
348 print("    ? Curto prazo: MemorySaver (thread_id)")
349 print("    ? Medio prazo: FAISS RAG (buscar_conhecimento)")
350 print("    ? Longo prazo: MemPalace (memoria_longo_prazo)")
351 print("\n?? Ferramentas disponiveis:")
352 for tool in todas_ferramentas_hibridas:
353     categoria = "MemPalace" if tool.name in [t.name for t in
        ferramentas_mempalace] else "Existente"

```

```

354     print(f"      ? {tool.name} ({categoria})")
355
356 # ===== cell_017 =====
357 # ??? Demo: Agente com Memoria de Longo Prazo
358     ?????????????????????????????????????????????
359 config_demo = {"configurable": {"thread_id": "mempalace-demo-1"
360     }}
361
362 async def demo_memoria_longo_prazo():
363     """Demonstra as capacidades do agente hibrido com MemPalace
364     ."""
365
366     print("?? Demonstracao: Agente + MemPalace")
367     print("=" * 60)
368
369     # 1. Carregamento de contexto essencial
370     print("\n1? Carregando contexto de sessoes anteriores...")
371     resultado1 = agente_mempalace.invoke({
372         "messages": [{"role": "user", "content":
373             "Carregue o contexto essencial de nossas conversas
374             anteriores."}]
375     }, config=config_demo)
376     print(f"?? {resultado1['messages'][-1].content}")
377
378     # 2. Salvando uma decisao importante
379     print("\n2? Salvando decisao tecnica importante...")
380     resultado2 = agente_mempalace.invoke({
381         "messages": [{"role": "user", "content":
382             "Salve esta decisao importante: 'Decidimos usar
383             LangGraph + MemPalace para memoria persistente
384             porque combina RAG (medio prazo) com
385             armazenamento verbatim (longo prazo), atingindo
386             96.6% no LongMemEval vs ~70% do FAISS puro.
387             Decisao tomada em abril 2026.' Use o topico '
388             arquitetura_ai'."}]
389     }, config=config_demo)
390     print(f"?? {resultado2['messages'][-1].content}")
391
392     # 3. Consultando memorias de conversas anteriores

```

```

384     print("\n3? Buscando en memorias sobre decisoes de
          arquitetura...")
385     resultado3 = agente_mempalace.invoke({
386         "messages": [{"role": "user", "content":
387             "Busque em memorias anteriores sobre 'decisoes de
          arquitetura' ou 'LangGraph'."}]
388     }, config=config_demo)
389     print(f"?? {resultado3['messages'][-1].content}")
390
391     # 4. Testando capacidade hibrida: RAG + MemPalace
392     print("\n4? Pergunta que usa tanto RAG quanto memorias
          persistentes...")
393     resultado4 = agente_mempalace.invoke({
394         "messages": [{"role": "user", "content":
395             "Explique as diferencas entre MemorySaver e
          MemPalace, usando tanto a base de conhecimento
          tecnica quanto decisoes que ja salvamos."}]
396     }, config=config_demo)
397     print(f"?? {resultado4['messages'][-1].content}")
398
399     print("?? Execute: await demo_memoria_longo_prazo()")
400     print("?? Isso demonstrara as 3 camadas de memoria trabalhando
          juntas:")
401
402     # ===== cell_019 =====
403     # ??? astream_events: visibilidade total do ciclo interno do
404     agente ????????????
405     # Emite eventos granulares: tokens gerados, chamadas de tool,
406     estados do grafo.
407     # Use para: observabilidade, debugging, progress bars, logs de
408     auditoria.
409
410     print("?? astream_events: monitorando o ciclo interno do agente
          \n")
411
412     config_ev = {"configurable": {"thread_id": "events-demo-1"}}
413
414     tool_calls_log = []
415
416     async for event in agente.astream_events(
417         {"messages": [{"role": "user", "content":

```

```

414         "Qual e o clima em Brasilia e converta 200 USD para EUR
           ?"}]],
415     config=config_ev,
416     version="v2"      # versao recomendada ? eventos mais
                        detalhados
417 ):
418     kind = event["event"]
419     name = event.get("name", "")
420
421     if kind == "on_chat_model_stream":
422         # Token gerado pelo LLM em tempo real
423         token = event["data"]["chunk"].content
424         if token:
425             print(token, end="", flush=True)
426
427     elif kind == "on_tool_start":
428         # Agente decidiu chamar uma ferramenta
429         print(f"\n\n?? [TOOL START] {name}")
430         print(f"    ? Input: {event['data'].get('input', {})}")
431         tool_calls_log.append(name)
432
433     elif kind == "on_tool_end":
434         # Ferramenta retornou resultado
435         output = event["data"].get("output", "")
436         print(f"    ? Output: {output}")
437
438     elif kind == "on_chain_start" and name == "LangGraph":
439         print("?? [GRAFO] Iniciando execucao do agente")
440
441     elif kind == "on_chain_end" and name == "LangGraph":
442         print("\n\n?? [GRAFO] Execucao concluida")
443
444     print(f"\n\n?? Ferramentas chamadas nesta execucao: {
           tool_calls_log}")
445
446     # ===== cell_020 =====
447     import time
448
449     # ??? asyncio.gather: execucao paralela de multiplos agentes
           ??????????????????????

```

```

450 # Envia N perguntas simultaneamente ao LLM sem esperar uma por
    uma.
451 # Ideal para pipelines de processamento em lote.
452
453 async def processar_pergunta(pergunta: str, thread_id: str) ->
    dict:
454     """Wrapper async para invocar o agente com uma pergunta e
        medir o tempo."""
455     config = {"configurable": {"thread_id": thread_id}}
456     resultado = await agente.ainvoke(
457         {"messages": [{"role": "user", "content": pergunta}]},
458         config=config
459     )
460     return {"pergunta": pergunta, "resposta": resultado["
        messages"][-1].content}
461
462 perguntas = [
463     ("Qual e o clima em Sao Paulo?", "parallel-sp"),
464     ("Qual e o clima em Rio de Janeiro?", "parallel-rj"),
465     ("Converta 500 EUR para BRL", "parallel-eur"),
466     ("Calcule 15% de 2800", "parallel-calc"),
467 ]
468
469 # ??? Execucao paralela
    ?????????????????????????????????????????????????????????????
470 print(f"? gather: {len(perguntas)} perguntas em paralelo...\n"
    )
471 t0 = time.perf_counter()
472
473 resultados = await asyncio.gather(*[
474     processar_pergunta(p, tid) for p, tid in perguntas
475 ])
476
477 elapsed = time.perf_counter() - t0
478 print(f"? Tempo total: {elapsed:.1f}s | Media por pergunta:
    {elapsed/len(perguntas):.1f}s\n")
479
480 for r in resultados:
481     print(f"? {r['pergunta']}")
482     print(f"? {r['resposta'][:120]}\n")
483

```

```

484 # ??? Comparacao: execucao sequencial (comentada)
      ?????????????????????????????????????
485 # t0 = time.perf_counter()
486 # for p, tid in perguntas:
487 #     await processar_pergunta(p, tid) # aguarda uma por uma
488 # print(f"Sequencial: {time.perf_counter() - t0:.1f}s")
489
490 # ===== cell_021 =====
491 import asyncio
492 import httpx
493
494 # ??? Async tools: ferramentas com I/O assincrono
      ?????????????????????????????????????
495 # Use 'async def' quando a tool faz I/O (HTTP, banco, arquivo).
496 # O LangGraph executa async tools corretamente sem bloquear o
      event loop.
497
498 @tool
499 async def buscar_cotacao_async(moeda: str) -> str:
500     """Busca a cotacao de uma moeda em relacao ao BRL de forma
501         assincrona.
502         Suporta: USD, EUR, GBP, ARS, BTC.
503         """
504     # Em producao: substitua por httpx.AsyncClient chamando uma
505         API real de cambio
506     await asyncio.sleep(0.05) # simula latencia de rede sem
507         bloquear o loop
508     cotacoes = {"USD": "5.72", "EUR": "6.21", "GBP": "7.18", "
509         ARS": "0.006", "BTC": "312450.00"}
510     moeda = moeda.upper()
511     if moeda not in cotacoes:
512         return f"Moeda nao suportada. Use: {list(cotacoes.keys
513             ())}"
514     return f"1 {moeda} = R$ {cotacoes[moeda]}"
515
516 @tool
517 async def buscar_conteudo_url_async(url: str) -> str:
518     """Busca o conteudo de uma URL publica de forma assincrona
519         (primeiros 800 caracteres).

```

```

515     Use para acessar APIs ou paginas sem bloquear outras
        operacoes paralelas.
516     """
517     try:
518         async with httpx.AsyncClient(timeout=8,
            follow_redirects=True) as client:
519             resp = await client.get(url, headers={"User-Agent":
                "Mozilla/5.0"})
520             resp.raise_for_status()
521             return resp.text[:800]
522     except Exception as e:
523         return f"Erro ao acessar URL: {e}"
524
525
526 # ??? Agente com async tools
        ?????????????????????????????????????????????????????????????
527 agente_async_tools = create_agent(
528     model,
529     [buscar_cotacao_async, buscar_conteudo_url_async,
        calculadora],
530     checkpointer=MemorySaver()
531 )
532
533 config_at = {"configurable": {"thread_id": "async-tools-1"}}
534 resultado = await agente_async_tools.ainvoke(
535     {"messages": [{"role": "user", "content":
536         "Qual e a cotacao do USD e do EUR hoje? Calcule quanto
            1500 EUR valem em reais."}]},
537     config=config_at
538 )
539 print("?? Agente (async tools):", resultado["messages"][-1].
    content)
540
541 # ===== cell_023 =====
542 from plantuml_helper import show_plantuml
543
544 show_plantuml("""
545 @startuml
546 skinparam backgroundColor #FEFEFE
547 skinparam activity {
548     StartColor #4CAF50

```



```

549     EndColor #C62828
550     BackgroundColor #E3F2FD
551     BorderColor #1976D2
552     FontSize 13
553 }
554 skinparam note {
555     BackgroundColor #FFF9C4
556     BorderColor #F57C00
557 }
558
559 start
560 :Pergunta do Usuario;
561 :Gera Embedding da pergunta;
562 note right
563     Modelo: all-MiniLM-L6-v2
564     Vetor: 384 dimensoes
565 end note
566
567 :Busca Semantica no Vector Store;
568 note right
569     FAISS Index
570     Top-K documentos
571     Similaridade cosseno
572 end note
573
574 :Recupera documentos mais relevantes;
575 :Combina com Memoria de Curto Prazo;
576 note right
577     MemorySaver
578     Historico da sessao
579     thread_id
580 end note
581
582 :LLM processa Prompt + Contexto + Historico;
583 :Gera resposta fundamentada;
584 stop
585 @enduml
586 """)
587
588 # ===== cell_024 =====
589 from langchain_core.documents import Document

```

```

590 from langchain_text_splitters import
    RecursiveCharacterTextSplitter
591
592 # ??? Base de conhecimento: documentos sobre LangChain/
    LangGraph ??????????????
593 documentos_raw = [
594     Document(
595         page_content="""
596         LangGraph e um framework para construir agentes e
            sistemas multi-agente usando grafos de estado.
597         Ele estende o LangChain com a capacidade de criar
            fluxos ciclicos (loops), essenciais para agentes
598         que precisam raciocinar e agir em multiplos passos. O
            LangGraph suporta checkpointing de estado,
599         permitindo persistencia entre interacoes. Ideal para:
            chatbots, agentes de pesquisa,
600         sistemas de automacao, e pipelines de dados complexos.
601         """,
602         metadata={"source": "langgraph_docs", "topico": "
            LangGraph"}
603     ),
604     Document(
605         page_content="""
606         LangChain e um framework open-source para desenvolver
            aplicacoes com modelos de linguagem.
607         Fornece abstracoes para: Chains (sequencias de
            operacoes), Prompts (templates de instrucoes),
608         Memory (gerenciamento de contexto), Tools (ferramentas
            externas), e Agents (agentes autonomos).
609         Suporta multiplos provedores de LLM: OpenAI, Anthropic,
            Google, Hugging Face, e outros.
610         Versao atual: LangChain 0.3+ com separacao em pacotes
            modulares.
611         """,
612         metadata={"source": "langchain_docs", "topico": "
            LangChain"}
613     ),
614     Document(
615         page_content="""
616         RAG (Retrieval-Augmented Generation) e uma tecnica que
            combina recuperacao de informacoes

```

```

617         com geracao de texto. Fluxo: 1) Indexar documentos como
        embeddings em vector store,
618         2) Receber pergunta do usuario, 3) Buscar documentos
        relevantes por similaridade semantica,
619         4) Injetar contexto recuperado no prompt do LLM, 5)
        Gerar resposta fundamentada nos documentos.
620         Beneficio: reduz alucinacoes e permite conhecimento
        atualizado alem do treinamento do modelo.
621         """
622         metadata={"source": "rag_docs", "topico": "RAG"}
623     ),
624     Document(
625         page_content=""
626         Embeddings sao representacoes vetoriais de texto em
        espaco de alta dimensao.
627         Textos semanticamente similares tem vetores proximos (
        alta similaridade cosseno).
628         Modelos populares: OpenAI text-embedding-3-small,
        Sentence-BERT, all-MiniLM-L6-v2.
629         all-MiniLM-L6-v2 e um modelo leve (22M parametros) e
        eficiente, ideal para uso local.
630         FAISS (Facebook AI Similarity Search) e uma biblioteca
        eficiente para busca por similaridade
631         em grandes conjuntos de vetores.
632         """
633         metadata={"source": "embeddings_docs", "topico": "
        Embeddings"}
634     ),
635     Document(
636         page_content=""
637         MemorySaver no LangGraph e um checkpointer em memoria
        que salva o estado do grafo.
638         Cada conversa e identificada por um thread_id unico.
        Permite retomar conversas do ponto
639         onde pararam dentro da mesma sessao de Python. Para
        persistencia entre sessoes,
640         use SqliteSaver ou PostgresSaver. A memoria de curto
        prazo e limitada ao contexto
641         da janela de contexto do LLM (tipicamente 32k-200k
        tokens).
642         """

```

```

643     metadata={"source": "memory_docs", "topico": "Memoria"}
644 ),
645 Document(
646     page_content="""
647     MCP (Model Context Protocol) e um protocolo aberto
648     criado pela Anthropic para padronizar
649     como aplicacoes fornecem contexto e ferramentas para
650     modelos de IA. Funciona como
651     um protocolo cliente-servidor onde: Servidores MCP
652     expoem recursos (tools, prompts, resources)
653     e Clientes MCP (agentes) consomem esses recursos.
654     Permite criar ecossistema de integracoes
655     reutilizaveis. Suporta transporte via stdio (processo
656     local) ou SSE (HTTP remoto).
657     """,
658     metadata={"source": "mcp_docs", "topico": "MCP"}
659 ),
660 ]
661
662 # ??? Divisao de documentos em chunks
663     """
664
665 text_splitter = RecursiveCharacterTextSplitter(
666     chunk_size=300,      # Tamanho maximo de cada chunk em
667     chunk_overlap=50,    # Sobreposicao entre chunks (preserva
668     length_function=len  contexto)
669 )
670
671 chunks = text_splitter.split_documents(documentos_raw)
672
673 print(f"? {len(documentos_raw)} documentos ? {len(chunks)}
674     chunks apos divisao")
675
676 print(f"\nExemplo de chunk:")
677 print(f"    Conteudo: {chunks[0].page_content[:100]}...")
678 print(f"    Metadata: {chunks[0].metadata}")
679
680 # ===== cell_025 =====
681 from langchain_community.vectorstores import FAISS
682 from langchain_community.embeddings import
683     HuggingFaceEmbeddings
684

```

```

674 # ??? Modelo de embeddings (local, gratuito)
        ?????????????????????????????????????????
675 # all-MiniLM-L6-v2: modelo leve e eficiente para embeddings em
        portugues e ingles
676 # Download automatico na primeira execucao (~80MB)
677 print("? Carregando modelo de embeddings (pode demorar na
        primeira vez)...")
678 embeddings = HuggingFaceEmbeddings(
679     model_name="all-MiniLM-L6-v2",
680     model_kwargs={"device": "cpu"},
681     encode_kwargs={"normalize_embeddings": True}
682 )
683
684 # ??? Vector Store: MEMORIA DE MEDIO PRAZO
        ?????????????????????????????????????????
685 # FAISS indexa os chunks como vetores para busca semantica
        eficiente
686 vector_store = FAISS.from_documents(chunks, embeddings)
687
688 # ??? Salvar/Carregar o indice (persistencia)
        ?????????????????????????????????????????
689 vector_store.save_local("./faiss_index")
690 # Para recarregar em sessoes futuras:
691 # vector_store = FAISS.load_local("./faiss_index", embeddings,
        allow_dangerous_deserialization=True)
692
693 print("? Vector store criado e salvo em ./faiss_index")
694 print(f"    Total de vetores indexados: {vector_store.index.
        ntotal}")
695
696 # ??? Testando busca semantica
        ?????????????????????????????????????????
697 print("\n?? Teste de busca semantica:")
698 resultados = vector_store.similarity_search("como funciona a
        memoria em agentes?", k=2)
699 for i, doc in enumerate(resultados, 1):
700     print(f"\n    [{i}] Fonte: {doc.metadata.get('topico', 'N/A')}
        ")
701     print(f"        Trecho: {doc.page_content[:120].strip()}...")
702
703 # ===== cell_026 =====

```

```

704 from langchain_core.tools import tool
705
706 retriever = vector_store.as_retriever(search_type="similarity",
707                                       search_kwargs={"k": 3})
708
709 @tool
710 def buscar_conhecimento(consulta: str) -> str:
711     """Busca informacoes na base de conhecimento sobre
712         LangChain, LangGraph, RAG, Embeddings e MCP.
713         Use para responder perguntas tecnicas sobre esses temas.
714     """
715     docs = retriever.invoke(consulta)
716     if not docs:
717         return "Nenhuma informacao encontrada sobre esse tema."
718     return "\n\n".join([
719         f"[Fonte: {doc.metadata.get('topico', 'geral')}] \n {doc.
720         page_content.strip()}"
721         for doc in docs
722     ])
723
724 tools_rag = [calculadora, consultar_clima, converter_moeda,
725             buscar_conhecimento]
726 memory_rag = MemorySaver()
727 agente_rag = create_react_agent(model, tools_rag, checkpoint=
728                                 memory_rag)
729
730 print("? Agente RAG criado com ferramentas:", [t.name for t in
731         tools_rag])
732
733 # ===== cell_027 =====
734 config_rag = {"configurable": {"thread_id": "rag-demo-1"}}
735
736 def conversar_rag(mensagem: str):
737     resultado = agente_rag.invoke(
738         {"messages": [{"role": "user", "content": mensagem}]},
739         config=config_rag
740     )
741     return resultado["messages"][-1].content
742
743 # ??? Teste: RAG com busca na base de conhecimento
744     """

```

```

738 print("?? Usuario: 0 que e o LangGraph e para que serve?")
739 resposta = conversar_rag("0 que e o LangGraph e para que serve?"
740                             ")
741 print(f"?? Agente: {resposta}\n")
742 print("-" * 60)
743
744 # ??? Teste: Memoria curto prazo + RAG (medio prazo)
745      ?????????????????????????????????
746 print("\n?? Usuario: Como funciona o RAG que voce mencionou?")
747 resposta = conversar_rag("Como funciona o RAG que voce
748      mencionou?")
749 print(f"?? Agente: {resposta}")
750
751 # ===== cell_029 =====
752 from plantuml_helper import show_plantuml
753
754 show_plantuml("""
755 @startuml
756 skinparam backgroundColor #FFFFFF
757 skinparam activity {
758     StartColor #2E7D32
759     EndColor #C62828
760     BackgroundColor #F3E5F5
761     BorderColor #6A1B9A
762     FontSize 13
763 }
764
765 start
766 :Desenvolvedor define funcao;
767 :Aplica decorator @tool;
768 :Escreve docstring clara;
769 :Define type hints/Pydantic;
770 :Tool e registrada no agente;
771
772 if (LLM precisa da tool?) then (sim)
773     :LLM seleciona tool pela docstring;
774     :Valida parametros pelo schema;
775     :Executa tool;
776     :Recebe retorno estruturado;
777 else (nao)
778     :Continua sem tool;

```

```

776     endif
777
778     :Compo e resposta final;
779     stop
780     @enduml
781     """)
782
783     # ===== cell_030 =====
784     from langchain_core.tools import tool, StructuredTool
785     from pydantic import BaseModel, Field
786     from typing import Literal
787     import statistics
788     import json
789     from datetime import datetime
790
791     # ??? Skill 1: Simples (com @tool e type hints)
792     """
793     """
794     @tool
795     def verificar_cnpj(cnpj: str) -> str:
796         """Verifica se um CNPJ tem formato valido (apenas validacao
797         de formato, nao consulta Receita).
798         Aceita formatos: '12.345.678/0001-90' ou '12345678000190'.
799         """
800         # Remove formatacao
801         cnpj_limpo = ''.join(filter(str.isdigit, cnpj))
802         if len(cnpj_limpo) != 14:
803             return f"? CNPJ invalido: deve ter 14 digitos (recebido
804             : {len(cnpj_limpo)})"
805         if len(set(cnpj_limpo)) == 1:
806             return "? CNPJ invalido: todos os digitos iguais"
807         # Formata
808         formatado = f"{cnpj_limpo[:2]}.{cnpj_limpo[2:5]}.{
809             cnpj_limpo[5:8]}/{cnpj_limpo[8:12]}-{cnpj_limpo[12:]}"
810         return f"? CNPJ com formato valido: {formatado}"
811
812     # ??? Skill 2: Com Pydantic (validacao rigorosa + documentacao
813     detalhada) """
814     class AnalisarDadosInput(BaseModel):
815         dados: list[float] = Field(
816             description="Lista de valores numericos para analise",

```



```

812     min_length=2  # Precisa de ao menos 2 valores
813 )
814 operacao: Literal["media", "mediana", "desvio_padrao", "
      minimo", "maximo", "resumo"] = Field(
815     description="Operacao estatistica: 'media', 'mediana',
      'desvio_padrao', 'minimo', 'maximo', 'resumo'"
816 )
817
818 def _analisar_dados(dados: list[float], operacao: str) -> str:
819     """Implementacao interna da analise estatistica."""
820     ops = {
821         "media": lambda d: f"Media: {statistics.mean(d):.4f}",
822         "mediana": lambda d: f"Mediana: {statistics.median(d)
      :.4f}",
823         "desvio_padrao": lambda d: f"Desvio Padrao: {statistics
      .stdev(d):.4f}",
824         "minimo": lambda d: f"Minimo: {min(d):.4f}",
825         "maximo": lambda d: f"Maximo: {max(d):.4f}",
826         "resumo": lambda d: (
827             f"N={len(d)} | Media={statistics.mean(d):.2f} | "
828             f"Mediana={statistics.median(d):.2f} | "
829             f"Std={statistics.stdev(d):.2f} | "
830             f"Min={min(d):.2f} | Max={max(d):.2f}"
831         )
832     }
833     return ops[operacao](dados)
834
835 skill_analise = StructuredTool.from_function(
836     func=_analisar_dados,
837     name="analisar_dados",
838     description="Realiza analise estatistica de uma lista de
      numeros. "
839         "Use para calcular medias, medianas, desvios
      padrao e resumos estatisticos.",
840     args_schema=AnalisarDadosInput,
841     return_direct=False  # O agente processa o resultado antes
      de responder
842 )
843
844 print("? Skills criadas:", ["verificar_cnpj", "analisar_dados"
      ])

```

```

845
846 # ===== cell_031 =====
847 class FormatarRelatorioInput(BaseModel):
848     titulo: str = Field(description="Titulo do relatorio")
849     dados: dict = Field(description="Dicionario com dados a
850         serem incluídos no relatorio")
851     formato: Literal["markdown", "json", "texto"] = Field(
852         default="markdown", description="Formato de saída: '
853         markdown', 'json', ou 'texto'"
854     )
855
856 def _formatar_relatorio(titulo: str, dados: dict, formato: str
857     = "markdown") -> str:
858     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
859     if formato == "json":
860         return json.dumps({"titulo": titulo, "timestamp":
861             timestamp, "dados": dados},
862             ensure_ascii=False, indent=2)
863     elif formato == "markdown":
864         linhas = [f"# {titulo}", f"*Gerado em: {timestamp}*", "
865             ", "## Dados", ""]
866         for chave, valor in dados.items():
867             if isinstance(valor, list):
868                 linhas.append(f"**{chave}:**")
869                 linhas.extend([f"  - {item}" for item in valor
870                     ])
871             else:
872                 linhas.append(f"**{chave}:** {valor}")
873         return "\n".join(linhas)
874     else:
875         linhas = [f"=== {titulo} ===", f"Data: {timestamp}", "-
876             " * 40]
877         for chave, valor in dados.items():
878             linhas.append(f"{chave}: {valor}")
879         return "\n".join(linhas)
880
881 skill_relatorio = StructuredTool.from_function(
882     func=_formatar_relatorio,
883     name="formatar_relatorio",
884     description="Cria um relatorio formatado. Util para
885         organizar e apresentar informacoes.",

```

```

878     args_schema=FormatarRelatorioInput
879 )
880
881 todas_skills = [calculadora, consultar_clima, converter_moeda,
882                 buscar_conhecimento, verificar_cnpj,
883                 skill_analise, skill_relatorio]
884
885 memory_skills = MemorySaver()
886 agente_completo = create_react_agent(model, todas_skills,
887                                     checkpointer=memory_skills)
888
889 print(f"? Agente completo com {len(todas_skills)} skills:")
890 for skill in todas_skills:
891     print(f"    ? {skill.name}")
892
893 # ===== cell_032 =====
894 config_skills = {"configurable": {"thread_id": "skills-demo-1"
895                                 }}
896
897 def usar_agente(mensagem: str):
898     resultado = agente_completo.invoke(
899         {"messages": [{"role": "user", "content": mensagem}]},
900         config=config_skills
901     )
902     return resultado["messages"][-1].content
903
904 # ??? Teste de skills encadeadas
905     ?????????????????????????????????????????????????????????????
906 print("?? Usuario: Analise esses dados de vendas e crie um
907       relatorio em markdown:")
908 print("    Vendas dos ultimos 6 meses: [15000, 18500, 12300,
909       21000, 19800, 24500]")
910 pergunta = """Analise esses dados de vendas mensais e crie um
911             relatorio em markdown:
912             Vendas: [15000, 18500, 12300, 21000, 19800, 24500]
913             Inclua estatisticas (media, mediana, desvio padrao) e converta
914             a media para USD."""
915
916 resposta = usar_agente(pergunta)
917 print(f"\n?? Agente:\n{resposta}")

```

```

911 # ===== cell_034 =====
912 from plantuml_helper import show_plantuml
913
914 show_plantuml("""
915 @startuml
916 skinparam backgroundColor #FFFFFF
917 skinparam activity {
918     StartColor #2E7D32
919     EndColor #C62828
920     BackgroundColor #FFF3E0
921     BorderColor #EF6C00
922     FontSize 13
923 }
924
925 start
926 :Recebe pergunta do usuario;
927 :Agente classifica tipo de informacao;
928
929 if (Requer informacao atual?) then (sim)
930     if (Conceito enciclopedico?) then (sim)
931         :Consulta Wikipedia;
932     else (nao)
933         if (API estruturada disponivel?) then (sim)
934             :Consulta API publica;
935         else (nao)
936             :Realiza busca web (DuckDuckGo/Tavily);
937         endif
938     endif
939 else (nao)
940     if (Conhecimento interno da empresa?) then (sim)
941         :Consulta RAG/Vector Store;
942     else (nao)
943         :Usa conhecimento do LLM;
944     endif
945 endif
946
947 :Consolida contexto;
948 :Gera resposta com fontes;
949 stop
950 @enduml
951 """)

```

```

952
953 # ===== cell_035 =====
954 from langchain_community.tools import DuckDuckGoSearchRun,
    DuckDuckGoSearchResults
955 from langchain_community.utilities import WikipediaAPIWrapper
956 from langchain_community.tools import WikipediaQueryRun
957 from langchain_core.tools import tool
958 import httpx
959 import json
960
961 # ??? 1. DuckDuckGo: busca rapida (retorna string com resultado
    principal) ?????
962 busca_rapida = DuckDuckGoSearchRun(
963     name="busca_rapida",
964     description=(
965         "Busca rapida na internet via DuckDuckGo. Retorna o
          trecho mais relevante. "
966         "Use para perguntas factuais simples e diretas sobre
          eventos ou dados atuais."
967     )
968 )
969
970 # ??? 2. DuckDuckGo: busca detalhada (retorna lista com titulo
    + URL + trecho) ?
971 busca_detalhada = DuckDuckGoSearchResults(
972     name="busca_web",
973     description=(
974         "Busca na internet retornando multiplos resultados com
          titulo, URL e trecho. "
975         "Use quando precisar citar fontes, comparar informacoes
          ou verificar claims."
976     ),
977     num_results=4,
978     output_format="list"
979 )
980
981 # ??? 3. Wikipedia em portugues
    ?????????????????????????????????????????????????????????
982 wikipedia_pt = WikipediaQueryRun(
983     name="wikipedia",
984     description=(

```

```

985         "Busca informacoes enciclopedicas na Wikipedia em
          portugues. "
986         "Use para: conceitos tecnicos, historia, biografia,
          ciencia, geografia. "
987         "NAO use para informacoes que mudam frequentemente (
          precos, noticias recentes). "
988     ),
989     api_wrapper=WikipediaAPIWrapper(
990         lang="pt",
991         top_k_results=2,
992         doc_content_chars_max=1500
993     )
994 )
995
996 # ??? 4. HTTP direto: consulta APIs publicas REST
997     ?????????????????????????????????????????????????????????????
998 @tool
999 def consultar_api(url: str) -> str:
1000     """Faz GET request a uma URL de API publica (sem
1001         autenticacao).
1002
1003     Use para APIs abertas: cotacoes de cambio, dados do IBGE,
1004     APIs do governo, etc.
1005
1006     A URL deve estar completa e ser acessivel publicamente.
1007     """
1008     try:
1009         headers = {"User-Agent": "Mozilla/5.0 LangChain-Agent
1010             /1.0", "Accept": "application/json"}
1011         response = httpx.get(url, timeout=10, follow_redirects=
1012             True, headers=headers)
1013         response.raise_for_status()
1014         conteudo = response.text[:2500]
1015         try:
1016             dados = json.loads(conteudo)
1017             return json.dumps(dados, ensure_ascii=False, indent
1018                 =2)[:2500]
1019         except json.JSONDecodeError:
1020             return conteudo
1021     except httpx.TimeoutException:
1022         return "? Timeout: URL demorou mais de 10s para
1023             responder"
1024     except httpx.HTTPStatusError as e:

```

```

1016         return f"? HTTP {e.response.status_code}: {e.response.
1017             text[:300]}"
1018     except Exception as e:
1019         return f"? Erro: {str(e)}"
1020
1021 tools_web = [busca_rapida, busca_detalhada, wikipedia_pt,
1022             consultar_api]
1023
1024 print("? Ferramentas de busca web configuradas:")
1025 for t in tools_web:
1026     print(f"    ? {t.name}: {t.description[:70]}...")
1027
1028 # ??? Teste rapido de cada ferramenta
1029     ?????????????????????????????????????????????????????????
1030 print("\n?? Teste rapido:")
1031 try:
1032     r = busca_rapida.invoke("taxa Selic Brasil 2025")
1033     print(f"    DuckDuckGo rapido: {r[:150]}...")
1034 except Exception as e:
1035     print(f"    DuckDuckGo rapido: ? {e}")
1036
1037 try:
1038     r = wikipedia_pt.invoke("Inteligencia artificial")
1039     print(f"    Wikipedia: {r[:150]}...")
1040 except Exception as e:
1041     print(f"    Wikipedia: ? {e}")
1042
1043 # ===== cell_036 =====
1044 memory_web = MemorySaver()
1045 agente_web = create_react_agent(
1046     model,
1047     tools_web + [calculadora],
1048     checkpointer=memory_web,
1049     prompt=(
1050         "Voce e um assistente que pesquisa informacoes na
1051             internet para responder perguntas.\n\n"
1052         "Diretrizes:\n"
1053         "- Use 'busca_web' para informacoes recentes (noticias,
1054             precos, eventos atuais)\n"
1055         "- Use 'wikipedia' para conceitos, historia, ciencia e
1056             fatos estabelecidos\n"

```

```

1051         "- Use 'consultar_api' para APIs publicas (ex: dados do
           governo, cotacoes abertas)\n"
1052         "- Sempre mencione a fonte das informacoes (URL ou nome
           da fonte)\n"
1053         "- Se os resultados estiverem desatualizados ou
           contraditorios, diga claramente"
1054     )
1055 )
1056
1057 config_web = {"configurable": {"thread_id": "web-demo-1"}}
1058
1059 async def pesquisar(mensagem: str):
1060     resultado = await agente_web.ainvoke(
1061         {"messages": [{"role": "user", "content": mensagem}]},
1062         config=config_web
1063     )
1064     return resultado["messages"][-1].content
1065
1066 print("?? Pergunta: Qual e a cotacao atual do dolar americano
       em reais?\n")
1067 resp = await pesquisar("Qual e a cotacao atual do dolar
       americano em reais? Pesquise na internet.")
1068 print(f"?? Agente:\n{resp}\n")
1069
1070 print("=" * 65)
1071 print("\n?? Pergunta: O que e a taxa Selic e quanto R$10.000
       rendem em 12 meses?\n")
1072 resp2 = await pesquisar(
1073     "O que e a taxa Selic? Busque o valor atual na web e
       calcule quanto R$10.000 rendem em 12 meses."
1074 )
1075 print(f"?? Agente:\n{resp2}\n")
1076
1077 print("=" * 65)
1078 print("\n?? Follow-up: E se eu investir o dobro?\n")
1079 resp3 = await pesquisar("E se eu investir o dobro do valor que
       mencionei antes?")
1080 print(f"?? Agente (memoria ativa):\n{resp3}")
1081
1082 # ===== cell_037 =====

```



```

1108
1109     os.environ["TAVILY_API_KEY"] = "tvly-..." # https://tavily.
        com
1110
1111     tavily = TavilySearch(
1112         max_results=5,
1113         include_raw_content=True,      # Retorna conteudo completo
        da pagina
1114         include_answer=True,          # Inclui resposta direta
        quando disponivel
1115         search_depth="advanced"       # Busca mais profunda
1116     )
1117
1118     # Uso identico ao DuckDuckGo:
1119     tools_web_prod = [tavily, wikipedia_pt, consultar_api]
1120     agente_prod = create_react_agent(model, tools_web_prod,
        checkpoint=MemorySaver())
1121     """
1122
1123     # ===== cell_039 =====
1124     from plantuml_helper import show_plantuml
1125
1126     show_plantuml("""
1127     @startuml
1128     skinparam backgroundColor #FFFFFF
1129     skinparam activity {
1130         StartColor #2E7D32
1131         EndColor #C62828
1132         BackgroundColor #E8F5E9
1133         BorderColor #2E7D32
1134         FontSize 13
1135     }
1136
1137     start
1138     :Pergunta do usuario;
1139     :Agente gera resposta inicial;
1140
1141     fork
1142         :Saida estruturada (Pydantic);
1143         :Calcula score de confianca;
1144     fork again

```

```

1145     :Forca citacao de fontes;
1146     :Anexa evidencias;
1147 fork again
1148     :Auto-reflexao com segundo avaliador;
1149     :Refina se necessario;
1150 fork again
1151     :Cross-referencia em fontes externas;
1152     :Confirma claims criticos;
1153 end fork
1154
1155 :Consolida sinais de confiabilidade;
1156 :Entrega resposta confiavel;
1157 stop
1158 @enduml
1159 """)
1160
1161 # ===== cell_040 =====
1162 from pydantic import BaseModel, Field
1163 from typing import Optional, Literal
1164 from langchain_anthropic import ChatAnthropic
1165
1166 # ??? Schema de resposta confiavel
1167     ?????????????????????????????????????????????????????????????
1168 class RespostaFundamentada(BaseModel):
1169     """Resposta estruturada com metadados de confiabilidade e
1170     rastreabilidade."""
1171
1172     resposta: str = Field(
1173         description="Resposta clara, objetiva e fundamentada
1174         para o usuario"
1175     )
1176     confianca: float = Field(
1177         description="Nivel de confianca de 0.0 (totalmente
1178         incerto) a 1.0 (certeza absoluta)",
1179         ge=0.0, le=1.0
1180     )
1181     nivel_confianca: Literal["baixo", "medio", "alto"] = Field(
1182         description="'baixo' (0.0-0.5), 'medio' (0.5-0.8), '
1183         alto' (0.8-1.0)"
1184     )
1185     fontes: list[str] = Field(

```

```

1181     default_factory=list,
1182     description="URLs, nomes de publicacoes, ou '
           conhecimento do modelo (pode estar desatualizado)'"
1183 )
1184 limitacoes: Optional[str] = Field(
1185     default=None,
1186     description="Limitacoes importantes, incertezas ou
           ressalvas sobre a resposta"
1187 )
1188 requer_verificacao_humana: bool = Field(
1189     description="True para topicos criticos: saude, direito
           , finanças, segurança"
1190 )
1191
1192 # ??? LLM configurado para saida estruturada
           ?????????????????????????????????????????
1193 llm_confiavel = ChatAnthropic(model="claude-sonnet-4-6",
           temperature=0).with_structured_output(
1194     RespostaFundamentada
1195 )
1196
1197 def responder_com_confianca(pergunta: str, contexto_web: str =
           "") -> RespostaFundamentada:
1198     """Gera resposta estruturada com score de confianca
           explicito."""
1199
1200     contexto_extra = f"\n\nInformacoes encontradas na internet
           :\n{contexto_web}" if contexto_web else ""
1201
1202     prompt = f"""Responda a pergunta de forma fundamentada e
           honesta.{contexto_extra}
1203
1204 Pergunta: {pergunta}
1205
1206 Avalie sua confianca de forma HONESTA e CALIBRADA:
1207 - 1.0 = Fato cientifico/matematico verificavel (ex: velocidade
           da luz)
1208 - 0.8-0.9 = Informacao consolidada e amplamente aceita
1209 - 0.6-0.7 = Bem estabelecido, mas pode ter variacoes ou estar
           desatualizado
1210 - 0.4-0.5 = Incerto, multiplas versoes, ou informacao dinamica

```

```

1211 - 0.0-0.3 = Especulacao, previsao ou informacao nao verificavel
      """
1212
1213     return llm_confiavel.invoke(prompt)
1214
1215 # ??? Demonstracao com perguntas de diferentes niveis de
      certeza ??????????????
1216 exemplos = [
1217     ("Qual e a velocidade da luz no vacuo?", ""),
1218     ("Qual e a populacao atual do Brasil?", ""),
1219     ("Qual sera a taxa Selic no proximo mes?", ""),
1220 ]
1221
1222 for pergunta, contexto in exemplos:
1223     resultado = responder_com_confianca(pergunta, contexto)
1224     icone = "???" if resultado.confianca >= 0.8 else "???" if
      resultado.confianca >= 0.5 else "???"
1225     print(f"{icone} [{resultado.nivel_confianca.upper()} {
      resultado.confianca:.0%}] {pergunta}")
1226     print(f"    ? {resultado.resposta[:120]}...")
1227     if resultado.limitacoes:
1228         print(f"    ? {resultado.limitacoes}")
1229     if resultado.requer_verificacao_humana:
1230         print(f"    ???? Recomenda verificacao com especialista"
      )
1231     print(f"    ?? Fontes: {resultado.fontes}")
1232     print()
1233
1234 # ===== cell_041 =====
1235 from langchain_core.messages import SystemMessage
1236
1237 # ??? Padrao 2: Citacao obrigatoria de fontes via system prompt
      ??????????????
1238 SYSTEM_CITACAO = """Voce e um assistente que fornece
      informacoes verificadas e fundamentadas.
1239
1240 REGRAS OBRIGATORIAS (sem excecao):
1241 1. Cite SEMPRE a fonte de cada informacao importante:
1242     - Busca na web ? cite a URL exata
1243     - Wikipedia ? cite "Wikipedia: [Titulo do artigo]"

```

```

- Conhecimento do modelo ? cite "Modelo (pode estar
    desatualizado)"
2. Se nao tiver certeza: diga explicitamente "Nao tenho certeza
    sobre isso"
3. Para topicos medicos, juridicos ou financeiros: recomende
    sempre um especialista
4. Diferencie fatos verificados de opinioes/estimativas

Formato obrigatorio ao final de cada resposta:
?? **Fontes consultadas:**
- [fonte 1]
- [fonte 2]
? **Ressalvas:** [se houver, ou 'Nenhuma']"""

# Agente com citacao obrigatoria + busca web
memory_citacao = MemorySaver()
agente_citacao = create_agent(
    model,
    [busca_detalhada, busca_rapida, wikipedia_pt],
    checkpointer=memory_citacao,
    prompt=SYSTEM_CITACAO
)

async def consultar_com_citacao(pergunta: str):
    config = {"configurable": {"thread_id": "citacao-1"}}
    resultado = await agente_citacao.ainvoke(
        {"messages": [{"role": "user", "content": pergunta}]},
        config=config
    )
    return resultado["messages"][-1].content

# ??? Testes
????????????????????????????????????????????????????????????????????????????????????????????????

print("?? Pergunta 1: Beneficios do exercicio fisico para saude
    mental\n")
resp1 = await consultar_com_citacao(
    "Quais sao os principais beneficios do exercicio fisico
    para a saude mental?"
)
print(f"? Agente:\n{resp1}\n")

```

```

1278
1279 print("=" * 65)
1280 print("\n?? Pergunta 2: Posso tomar paracetamol com ibuprofeno
      ?\n")
1281 resp2 = await consultar_com_citacao("Posso tomar paracetamol e
      ibuprofeno juntos?")
1282 print(f"?? Agente:\n{resp2}")
1283
1284 # ===== cell_042 =====
1285 from langgraph.graph import StateGraph, END
1286 from typing import TypedDict, Optional
1287 from langchain_core.output_parsers import JsonOutputParser
1288
1289 # ??? Estado compartilhado pelo grafo
      ?????????????????????????????????????????????????????????
1290 class EstadoVerificacao(TypedDict):
1291     pergunta: str
1292     contexto_web: str
1293     resposta_inicial: str
1294     avaliacao: dict # {score_confianca,
      problemas_encontrados, recomendacao, justificativa}
1295     resposta_final: str
1296     confianca_final: float
1297     fontes: list
1298
1299 # ??? No 1: Busca informacoes na web
      ?????????????????????????????????????????????????????????
1300 def buscar_web(state: EstadoVerificacao) -> dict:
1301     """Usa DuckDuckGo para enriquecer a resposta com dados
      atuais."""
1302     try:
1303         resultados = busca_detalhada.invoke(state["pergunta"])
1304         return {
1305             "contexto_web": str(resultados)[:2500],
1306             "fontes": ["DuckDuckGo Search"]
1307         }
1308     except Exception as e:
1309         return {"contexto_web": f"Busca indisponivel: {e}", "
      fontes": []}
1310

```

```

1311 # ??? No 2: Gera resposta inicial com base na busca
      ?????????????????????????????????
1312 def gerar_resposta(state: EstadoVerificacao) -> dict:
1313     """Gera resposta inicial fundamentada nas fontes da web."""
1314     prompt = f"""Responda a pergunta abaixo usando as
      informacoes encontradas na internet.
1315 Cite as fontes (URLs) quando disponiveis.
1316
1317 Pergunta: {state['pergunta']}
1318
1319 Resultados da busca:
1320 {state['contexto_web'] or 'Nenhum resultado de busca disponivel
      .'}"""
1321
1322     resposta = model.invoke(prompt)
1323     return {"resposta_inicial": resposta.content}
1324
1325 # ??? No 3: Auto-avaliacao critica da resposta
      ?????????????????????????????????
1326 def avaliar_resposta(state: EstadoVerificacao) -> dict:
1327     """Verifica criticamente a resposta: confianca, fontes,
      consistencia."""
1328     prompt = f"""Voce e um verificador de fatos rigoroso.
      Analise criticamente esta resposta.
1329
1330 Pergunta original: {state['pergunta']}
1331 Resposta gerada: {state['resposta_inicial']}
1332 Fontes consultadas (resumo): {state['contexto_web'][:600]}
1333
1334 Responda APENAS com JSON valido (sem markdown, sem explicacao
      extra):
1335 {{
1336     "score_confianca": 0.0,
1337     "problemas_encontrados": "descreva problemas ou escreva '
      nenhum'",
1338     "afirmacoes_sem_fonte": ["lista de claims sem evidencia"],
1339     "recomendacao": "manter | refinar | descartar",
1340     "justificativa": "motivo objetivo"
1341 }}
1342
1343 Criterios de score:

```



```

1344 - 0.9-1.0: fato verificavel e amplamente confirmado
1345 - 0.7-0.9: bem fundamentado, poucas incertezas
1346 - 0.5-0.7: parcialmente verificado, possivel desatualizacao
1347 - 0.3-0.5: incerto, multiplas versoes conflitantes
1348 - 0.0-0.3: especulacao, nao verificavel"""
1349
1350     try:
1351         chain = model | JsonOutputParser()
1352         avaliacao = chain.invoke(prompt)
1353     except Exception:
1354         avaliacao = {
1355             "score_confianca": 0.5,
1356             "problemas_encontrados": "avaliacao automatica
1357                 indisponivel",
1358             "afirmacoes_sem_fonte": [],
1359             "recomendacao": "manter",
1360             "justificativa": "falha no parser"
1361         }
1362     return {"avaliacao": avaliacao}
1363
1364 # ??? No 4: Refina ou mantem a resposta
1365     ?????????????????????????????????????????????????????????????
1366 def refinar_ou_mantem(state: EstadoVerificacao) -> dict:
1367     """Decide se refina a resposta com base na avaliacao."""
1368     avaliacao = state["avaliacao"]
1369     score = float(avaliacao.get("score_confianca", 0.5))
1370     recomendacao = avaliacao.get("recomendacao", "manter")
1371
1372     if recomendacao in ("manter", "descartar") or score >=
1373         0.80:
1374         resposta_final = state["resposta_inicial"]
1375         if score < 0.70:
1376             aviso = f"\n\n? *Confiabilidade moderada ({score
1377                 :.0%}) ? recomenda-se verificacao adicional.*"
1378             resposta_final += aviso
1379         if recomendacao == "descartar":
1380             resposta_final = (
1381                 f"? Nao foi possivel fornecer uma resposta
1382                     confiavel para esta pergunta "
1383                 f"({score:.0%} de confianca).\n\n"

```

```

1379         f"Motivo: {avaliacao.get('justificativa', '')}\n\n"
1380         f"Sugestao de resposta original (nao verificada):\n{state['resposta_inicial']}"
1381     )
1382     else:
1383         # Refina com base nos problemas encontrados
1384         problemas = avaliacao.get("problemas_encontrados", "")
1385         prompt = f"""Refine a resposta abaixo corrigindo os
1386                     problemas identificados.
1387 Pergunta: {state['pergunta']}
1388 Resposta original: {state['resposta_inicial']}
1389 Problemas identificados: {problemas}
1390 Claims sem fonte: {avaliacao.get('afirmacoes_sem_fonte', [])}
1391
1392 Forneça uma resposta mais precisa, com ressalvas onde
1393     necessario e fontes claras."""
1394
1395         resp = model.invoke(prompt)
1396         resposta_final = resp.content + f"\n\n?? *Resposta
1397             refinada automaticamente (confianca original: {score
1398                 :.0%})).*"
1399
1400         return {"resposta_final": resposta_final, "confianca_final"
1401             : score}
1402
1403     # ??? Monta o grafo de verificacao
1404     ?????????????????????????????????????????????????????????????
1405
1406     workflow_verificacao = StateGraph(EstadoVerificacao)
1407     workflow_verificacao.add_node("buscar", buscar_web)
1408     workflow_verificacao.add_node("gerar", gerar_resposta)
1409     workflow_verificacao.add_node("avaliar", avaliar_resposta)
1410     workflow_verificacao.add_node("refinar", refinar_ou_manter)
1411
1412     workflow_verificacao.set_entry_point("buscar")
1413     workflow_verificacao.add_edge("buscar", "gerar")
1414     workflow_verificacao.add_edge("gerar", "avaliar")
1415     workflow_verificacao.add_edge("avaliar", "refinar")
1416     workflow_verificacao.add_edge("refinar", END)

```

```

1412 agente_verificador = workflow_verificacao.compile()
1413
1414 print("? Pipeline de auto-verificacao criado (StateGraph):")
1415 print("    buscar_web ? gerar_resposta ? avaliar_resposta ?
    refinar_ou_manter")
1416
1417 # ===== cell_043 =====
1418 # ??? Teste completo do pipeline de verificacao
    ?????????????????????????????????????????
1419 import asyncio
1420
1421 perguntas_teste = [
1422     "Qual e a capital do Brasil e quantos habitantes ela tem?",
1423     "Qual sera o preco do Bitcoin amanha?",
1424     "O que e fotossintese?",
1425 ]
1426
1427 async def executar_testes():
1428     for pergunta in perguntas_teste:
1429         print(f"\n{' '*65}")
1430         print(f"? Pergunta: {pergunta}")
1431         print("-" * 65)
1432
1433         estado_inicial = {
1434             "pergunta": pergunta,
1435             "contexto_web": "",
1436             "resposta_inicial": "",
1437             "avaliacao": {},
1438             "resposta_final": "",
1439             "confianca_final": 0.0,
1440             "fontes": [],
1441         }
1442
1443         resultado = await agente_verificador.ainvoke(
            estado_inicial)
1444
1445         nivel = "?? ALTO" if resultado["confianca_final"] >=
            0.8 else \
1446             "?? MEDIO" if resultado["confianca_final"] >=
                0.5 else "?? BAIXO"

```

```

1447     recom = resultado["avaliacao"].get("recomendacao", "N/A
1448         ").upper()
1449     problemas = resultado["avaliacao"].get("
1450         problemas_encontrados", "")
1451
1452     print(f"?? Confianca: {resultado['confianca_final
1453         ']:.0%} {nivel}")
1454     print(f"?? Avaliacao: {recom}")
1455     if problemas and problemas != "nenhum":
1456         print(f"? Problemas: {problemas}")
1457     print(f"\n?? Resposta:\n{resultado['resposta_final']}")
1458
1459 await executar_testes()
1460
1461 # ===== cell_045 =====
1462 # ??? 5.4 Avaliacao de Grounding: quanto da resposta vem de RAG
1463 # /Tools vs LLM ??
1464
1465 from pydantic import BaseModel, Field
1466 from typing import List, Literal, Tuple
1467 from langchain_anthropic import ChatAnthropic
1468 from langchain_core.messages import AIMessage, ToolMessage
1469
1470 # ?? Modelos Pydantic
1471 ?????????????????????????????????????????????????????????????????????????????????????
1472
1473 class AfirmacaoAvaliada(BaseModel):
1474     texto: str = Field(description="A afirmacao feita na
1475         resposta")
1476     origem: Literal["rag", "tool", "modelo"] = Field(
1477         description=(
1478             "'rag'=contexto recuperado, "
1479             "'tool'=resultado de ferramenta externa, "
1480             "'modelo'=apenas conhecimento interno do LLM"
1481         )
1482     )
1483     evidencia: str = Field(
1484         description="Trecho que suporta a afirmacao, ou 'sem
1485             evidencia externa'"
1486     )
1487
1488 class RelatorioGrounding(BaseModel):
1489     afirmacoes: List[AfirmacaoAvaliada]

```

```

1481     pct_rag: float = Field(description="% da resposta baseada
1482         em RAG (0-100)")
1483     pct_tool: float = Field(description="% baseada em tools/
1484         skills externas (0-100)")
1485     pct_modelo: float = Field(description="% baseada apenas no
1486         LLM (0-100)")
1487     nivel_confianca: Literal["alta", "media", "baixa"] = Field(
1488         description="alta=+70% grounded, media=30-70%, baixa
1489             =-30%"
1490     )
1491     justificativa: str = Field(description="Explicacao breve do
1492         score")
1493
1494 # ?? Extrai evidencias do historico de mensagens do agente
1495     """
1496     """
1497 def extrair_evidencias(mensagens) -> Tuple[str, str, str]:
1498     """Separa chunks RAG, resultados de tools e a resposta
1499     final do agente."""
1500     rag_chunks, tool_results, resposta = [], [], ""
1501     for msg in mensagens:
1502         if isinstance(msg, ToolMessage):
1503             nome = getattr(msg, "name", "") or ""
1504             if any(k in nome.lower() for k in ["rag", "
1505                 conhecimento", "retriev"]):
1506                 rag_chunks.append(msg.content)
1507             else:
1508                 tool_results.append(f"[{nome}]: {msg.content}")
1509         elif isinstance(msg, AIMessage) and not msg.tool_calls:
1510             resposta = msg.content if isinstance(msg.content,
1511                 str) else str(msg.content)
1512     return "\n---\n".join(rag_chunks), "\n".join(tool_results),
1513         resposta
1514
1515 # ?? Avaliador de grounding
1516     """
1517     """
1518 def avaliar_grounding(
1519     pergunta: str,
1520     resposta: str,
1521     contexto_rag: str = "",
1522     resultados_tools: str = "",
1523 ) -> RelatorioGrounding:

```

```

1511     avaliador = ChatAnthropic(
1512         model="claude-sonnet-4-6", temperature=0
1513     ).with_structured_output(RelatorioGrounding)
1514
1515     return avaliador.invoke(
1516         f"""Analise a resposta e classifique cada afirmacao
1517             quanto a sua origem.
1518
1519 PERGUNTA: {pergunta}
1520 RESPOSTA: {resposta}
1521 CONTEXTO RAG: {contexto_rag or "Nenhum"}
1522 RESULTADOS DE TOOLS: {resultados_tools or "Nenhum"}
1523
1524 Classifique cada afirmacao como:
1525 - "rag"      ? suportada pelo contexto RAG
1526 - "tool"     ? suportada por resultado de tool/busca externa
1527 - "modelo"  ? apenas conhecimento interno do LLM, sem evidencia
1528             externa
1529
1530 Calcule os percentuais e o nivel de confianca."""
1531     )
1532
1533 # ?? Exemplo 1: Resposta com RAG + tool (alta confianca)
1534     ?????????????????????????????????
1535
1536 print("=" * 60)
1537 print("EXEMPLO 1 ? Resposta com RAG + tool")
1538 print("=" * 60)
1539
1540 historico_rag_tool = [
1541     ToolMessage(
1542         content="LangGraph e uma biblioteca para agentes
1543             stateful com grafos. Suporta checkpoints e
1544             streaming.",
1545         name="buscar_conhecimento",
1546         tool_call_id="t1",
1547     ),
1548     ToolMessage(
1549         content="USD/BRL: 5.72 (fonte: API de cambio em tempo
1550             real)",
1551         name="buscar_cotacao_async",
1552         tool_call_id="t2",

```

```

1546     ),
1547     AIMessage(
1548         content=(
1549             "LangGraph e uma biblioteca Python para agentes com
1550             estado persistente. "
1551             "Quanto ao dolar, a cotacao atual e R$ 5,72."
1552         )
1553     ),
1554 ]
1555 pergunta1 = "O que e LangGraph e qual a cotacao do dolar hoje?"
1556 ctx_rag, ctx_tools, resposta1 = extrair_evidencias(
1557     historico_rag_tool)
1558 r1 = avaliar_grounding(pergunta1, resposta1, ctx_rag, ctx_tools
1559 )
1560 print(f"?? RAG-grounded: {r1.pct_rag:5.1f}%")
1561 print(f"?? Tool-grounded: {r1.pct_tool:5.1f}%")
1562 print(f"?? Modelo (LLM): {r1.pct_modelo:5.1f}%")
1563 print(f"? Confianca: {r1.nivel_confianca.upper()}")
1564 print(f"?? {r1.justificativa}")
1565 print("\nAfirmacoes:")
1566 for af in r1.afirmacoes:
1567     icone = {"rag": "??", "tool": "??", "modelo": "??"}.get(af.
1568         origem, "?")
1569     print(f" {icone} [{af.origem}] {af.texto}")
1570 # ?? Exemplo 2: Resposta apenas com conhecimento do modelo (
1571     baixa confianca) ???
1572 print("\n" + "=" * 60)
1573 print("EXEMPLO 2 ? Resposta sem RAG nem tools")
1574 print("=" * 60)
1575 historico_sem_fontes = [
1576     AIMessage(
1577         content=(
1578             "LangGraph foi lancado em 2023 pela equipe do
1579             LangChain. "
1580             "Estima-se que tenha mais de 10 mil usuarios no
1581             mundo."
1582         )

```

```

1580     ),
1581 ]
1582
1583 pergunta2 = "Quando o LangGraph foi lançado?"
1584 _, _, resposta2 = extrair_evidencias(historico_sem_fontes)
1585 r2 = avaliar_grounding(pergunta2, resposta2)
1586
1587 print(f"?? RAG-grounded: {r2.pct_rag:5.1f}%")
1588 print(f"?? Tool-grounded: {r2.pct_tool:5.1f}%")
1589 print(f"?? Modelo (LLM): {r2.pct_modelo:5.1f}%")
1590 print(f"? Confianca: {r2.nivel_confianca.upper()}")
1591 print(f"?? {r2.justificativa}")
1592
1593 # ===== cell_047 =====
1594 # ??? 5.5 DeepEval ? Framework de avaliacao para RAG e LLMs
1595 # ??????????????????????
1596 # Instale se necessario: pip install deepeval
1597 import os
1598 from deepeval import evaluate
1599 from deepeval.metrics import (
1600     AnswerRelevancyMetric,
1601     FaithfulnessMetric,
1602     ContextualRecallMetric,
1603 )
1604 from deepeval.test_case import LLMTestCase
1605 from deepeval.models.base_model import DeepEvalBaseLLM
1606 from langchain_anthropic import ChatAnthropic
1607
1608 # ?? Adaptador: usa Claude como modelo avaliador interno do
1609 # DeepEval ??????????????
1610 class AnthropicEvaluator(DeepEvalBaseLLM):
1611     """Substitui o avaliador padrao (GPT-4) por Claude."""
1612
1613     def __init__(self):
1614         self._model = ChatAnthropic(model="claude-sonnet-4-6",
1615                                     temperature=0)
1616
1617     def load_model(self):
1618         return self._model
1619
1620     def generate(self, prompt: str, schema=None) -> str:

```



```

1618         return self._model.invoke(prompt).content
1619
1620     async def a_generate(self, prompt: str, schema=None) -> str
1621     :
1622         resp = await self._model.ainvoke(prompt)
1623         return resp.content
1624
1625     def get_model_name(self) -> str:
1626         return "claude-sonnet-4-6"
1627
1628     claude_eval = AnthropicEvaluator()
1629
1630     # ?? Contexto RAG recuperado (simula o que o retriever
1631     #     retornaria) ??????????????
1632     contexto_recuperado = [
1633         "LangGraph permite criar agentes stateful com grafos
1634         direcionados.",
1635         "Vantagens: memoria persistente via checkpoints,
1636         streaming nativo e loops.",
1637         "Integracao com LangChain; suporta paralelismo entre nos.",
1638     ]
1639
1640     # ?? Caso 1: resposta fiel ao contexto
1641     #     ?????????????????????????????????????????????????????????????
1642     caso_fiel = LLMTestCase(
1643         input="Quais sao as vantagens do LangGraph?",
1644         actual_output=(
1645             "LangGraph oferece memoria persistente, streaming de
1646             tokens e controle "
1647             "fino sobre fluxos de trabalho, com integracao nativa
1648             ao LangChain."
1649         ),
1650         expected_output="Memoria persistente, streaming e controle
1651             de fluxo.",
1652         retrieval_context=contexto_recuperado,
1653     )
1654
1655     # ?? Caso 2: resposta com alucinacao (nao suportada pelo
1656     #     contexto) ??????????????????
1657     caso_alucinacao = LLMTestCase(
1658         input="Quais sao as vantagens do LangGraph?",

```

```

1650     actual_output=(
1651         "LangGraph foi criado em 2019 pela Google e possui 50
           mil estrelas no GitHub."
1652     ),
1653     expected_output="Memoria persistente, streaming e controle
           de fluxo.",
1654     retrieval_context=contexto_recuperado,
1655 )
1656
1657 # ?? Metrics
           ?????????????????????????????????????????????????????????????????????????????????????
1658 # AnswerRelevancy ? a resposta responde de fato a pergunta?
1659 # Faithfulness ? cada afirmacao e suportada pelo contexto
           RAG?
1660 # ContextualRecall ? o contexto cobre o que era esperado na
           resposta?
1661 metrics = [
1662     AnswerRelevancyMetric(threshold=0.7, model=claude_eval,
           verbose_mode=False),
1663     FaithfulnessMetric(threshold=0.7, model=claude_eval,
           verbose_mode=False),
1664     ContextualRecallMetric(threshold=0.7, model=claude_eval,
           verbose_mode=False),
1665 ]
1666
1667 # ?? Avaliacao individual (sem enviar para Confident AI)
           ?????????????????????????????????
1668 print("?? Avaliando com DeepEval (modelo: Claude)\n")
1669 for label, caso in [("? Resposta fiel", caso_fiel), ("? Com
           alucinacao", caso_alucinacao)]:
1670     print(f"{label}: \"{caso.actual_output[:70]}...\")
1671     for metrica in metrics:
1672         metrica.measure(caso)
1673         status = "?" if metrica.success else "?"
1674         print(f" {status} {metrica.__class__.__name__:<25}
           score={metrica.score:.2f} threshold={metrica.
           threshold}")
1675     if not metrica.success:
1676         print(f" ?? {metrica.reason}")
1677     print()

```

```

1678
1679 # ?? Avaliacao em lote (gera relatorio consolidado)
      ?????????????????????????????????
1680 print("?" * 60)
1681 print("?? Avaliacao em lote:")
1682 evaluate([caso_fiel, caso_alucinacao], metricas)
1683
1684 # ===== cell_049 =====
1685 from plantuml_helper import show_plantuml
1686
1687 show_plantuml("""
1688 @startuml
1689 skinparam backgroundColor #FFFFFF
1690 skinparam activity {
1691     StartColor #2E7D32
1692     EndColor #C62828
1693     BackgroundColor #E1F5FE
1694     BorderColor #0277BD
1695     FontSize 13
1696 }
1697
1698 start
1699 :Agente recebe pergunta que requer dados externos;
1700 :Cliente MCP solicita ferramentas ao servidor;
1701 :Servidor MCP expoe tools disponiveis;
1702
1703 if (Tool SQL selecionada?) then (sim)
1704     :Servidor executa query no PostgreSQL;
1705     :Recebe resultados do banco;
1706     :Retorna JSON ao cliente MCP;
1707 else (nao)
1708     :Executa outra tool de metadados;
1709 endif
1710
1711 :Cliente MCP retorna contexto ao agente;
1712 :Agente responde ao usuario com base nos dados;
1713 stop
1714 @enduml
1715 """)
1716
1717 # ===== cell_050 =====

```

```

1718 # Instala dependencias especificas do MCP
1719 !pip install -q mcp langchain-mcp-adapters psycopg2-binary
1720 print("? Dependencias MCP instaladas")
1721
1722 # ===== cell_051 =====
1723 # ??? Criando o Servidor MCP PostgreSQL
1724 #     ?????????????????????????????????????????????????????????
1725 # Este codigo sera salvo como um arquivo Python separado que
1726 #     atua como servidor MCP
1727
1728 mcp_server_code = '''
1729 """
1730 Servidor MCP para PostgreSQL
1731 =====
1732 Expoe ferramentas para consulta ao banco via protocolo MCP.
1733 Execute: python mcp_postgres_server.py
1734 """
1735 import asyncio
1736 import os
1737 import json
1738 import psycopg2
1739 import psycopg2.extras
1740 from mcp.server import Server
1741 from mcp.server.stdio import stdio_server
1742 import mcp.types as types
1743
1744 # Configuracao do banco (via variavel de ambiente)
1745 DATABASE_URL = os.environ.get(
1746     "DATABASE_URL",
1747     "postgresql://postgres:senha@localhost:5432/meu_banco"
1748 )
1749
1750 # Cria a instancia do servidor MCP
1751 server = Server("postgres-mcp-server")
1752
1753 def get_connection():
1754     """Retorna uma conexao com o banco PostgreSQL."""
1755     return psycopg2.connect(DATABASE_URL)
1756

```

```

1757 # ??? Define as ferramentas disponiveis
      ?????????????????????????????????????????????
1758 @server.list_tools()
1759 async def list_tools() -> list[types.Tool]:
1760     """Retorna a lista de ferramentas disponiveis neste
        servidor MCP."""
1761     return [
1762         types.Tool(
1763             name="executar_query",
1764             description="Executa uma query SELECT no banco
                PostgreSQL e retorna os resultados."
1765             " APENAS queries SELECT sao permitidas
                por seguranca.",
1766             inputSchema={
1767                 "type": "object",
1768                 "properties": {
1769                     "sql": {
1770                         "type": "string",
1771                         "description": "Query SQL SELECT para
                            executar"
1772                     },
1773                     "limite": {
1774                         "type": "integer",
1775                         "description": "Numero maximo de linhas
                            a retornar (default: 100)",
1776                         "default": 100
1777                     }
1778                 },
1779                 "required": ["sql"]
1780             },
1781         ),
1782         types.Tool(
1783             name="listar_tabelas",
1784             description="Lista todas as tabelas disponiveis no
                schema publico do banco de dados.",
1785             inputSchema={
1786                 "type": "object",
1787                 "properties": {}
1788             },
1789         ),
1790         types.Tool(

```

```

1791     name="descrever_tabela",
1792     description="Retorna a estrutura de uma tabela:
1793         colunas, tipos de dados e constraints.",
1794     inputSchema={
1795         "type": "object",
1796         "properties": {
1797             "tabela": {
1798                 "type": "string",
1799                 "description": "Nome da tabela a ser
1800                     descrita"
1801             }
1802         },
1803         "required": ["tabela"]
1804     }
1805 ),
1806 types.Tool(
1807     name="contar_registros",
1808     description="Conta o numero de registros em uma
1809         tabela, com filtro opcional.",
1810     inputSchema={
1811         "type": "object",
1812         "properties": {
1813             "tabela": {
1814                 "type": "string",
1815                 "description": "Nome da tabela"
1816             },
1817             "where": {
1818                 "type": "string",
1819                 "description": "Clausula WHERE opcional
1820                     (sem a palavra WHERE)"
1821             }
1822         },
1823         "required": ["tabela"]
1824     }
1825 ),
1826 ]

```

```
1827 async def call_tool(name: str, arguments: dict) -> list[types.  
    TextContent]:  
1828     """Executa a ferramenta solicitada com os argumentos  
        fornecidos."""  
1829     try:  
1830         conn = get_connection()  
1831         cur = conn.cursor(cursor_factory=psycopg2.extras.  
            RealDictCursor)  
1832  
1833         if name == "executar_query":  
1834             sql = arguments["sql"].strip()  
1835             limite = arguments.get("limite", 100)  
1836  
1837             # Seguranca: apenas SELECT  
1838             if not sql.upper().startswith("SELECT"):  
1839                 return [types.TextContent(  
1840                     type="text",  
1841                     text="? Apenas queries SELECT sao  
                        permitidas por seguranca."  
1842                 )]  
1843  
1844             cur.execute(sql)  
1845             rows = cur.fetchmany(limite)  
1846             resultado = [dict(row) for row in rows]  
1847             return [types.TextContent(  
1848                 type="text",  
1849                 text=json.dumps(resultado, ensure_ascii=False,  
                    default=str, indent=2)  
1850             )]  
1851  
1852         elif name == "listar_tabelas":  
1853             cur.execute("""  
1854                 SELECT table_name,  
1855                     (SELECT COUNT(*) FROM information_schema  
                        . columns  
1856                     WHERE table_name = t.table_name) as  
                        num_colunas  
1857                 FROM information_schema.tables t  
1858                 WHERE table_schema = \'public\'  
1859                 ORDER BY table_name  
1860             """)
```

```

1861         tabelas = [dict(row) for row in cur.fetchall()]
1862         return [types.TextContent(
1863             type="text",
1864             text=json.dumps(tabelas, ensure_ascii=False,
1865                             indent=2)
1866         )]
1867
1868     elif name == "descrever_tabela":
1869         tabela = arguments["tabela"]
1870         cur.execute("""
1871             SELECT
1872                 column_name as coluna,
1873                 data_type as tipo,
1874                 character_maximum_length as tamanho_max,
1875                 is_nullable as permite_nulo,
1876                 column_default as valor_padrao
1877             FROM information_schema.columns
1878             WHERE table_name = %s AND table_schema = \
1879                 public\
1880             ORDER BY ordinal_position
1881             """, (tabela,))
1882         colunas = [dict(row) for row in cur.fetchall()]
1883         if not colunas:
1884             return [types.TextContent(type="text", text=f"
1885                 Tabela \'{tabela}\' nao encontrada.")
1886         ]
1887         return [types.TextContent(
1888             type="text",
1889             text=json.dumps({"tabela": tabela, "colunas":
1890                             colunas},
1891                             ensure_ascii=False, default=str,
1892                             indent=2)
1893         )]
1894
1895     elif name == "contar_registros":
1896         tabela = arguments["tabela"]
1897         where = arguments.get("where", "")
1898         sql = f"SELECT COUNT(*) as total FROM {tabela}"
1899         if where:
1900             sql += f" WHERE {where}"
1901         cur.execute(sql)
1902         total = cur.fetchone()["total"]

```



```

1897         return [types.TextContent(
1898             type="text",
1899             text=f"Total de registros em \'{tabela}\' (
                com filtro)\' if where else \'\': {total}"
1900         )]
1901
1902     conn.close()
1903
1904     except Exception as e:
1905         return [types.TextContent(type="text", text=f"? Erro: {
            str(e)}")]
1906
1907
1908 # ??? Ponto de entrada do servidor
1909     ?????????????????????????????????????????????????????????????
1910 async def main():
1911     print(f"? Servidor MCP PostgreSQL iniciado", flush=True)
1912     print(f"    Banco: {DATABASE_URL.split(\'@\')[-1] if \'@\'
        in DATABASE_URL else DATABASE_URL}", flush=True)
1913     async with stdio_server() as (read_stream, write_stream):
1914         await server.run(
1915             read_stream,
1916             write_stream,
1917             server.create_initialization_options()
1918         )
1919
1920 if __name__ == "__main__":
1921     asyncio.run(main())
1922 '''
1923
1924 # Salva o arquivo do servidor MCP
1925 with open("mcp_postgres_server.py", "w", encoding="utf-8") as f
1926     :
1927     f.write(mcp_server_code)
1928
1929 print("? Servidor MCP salvo em: mcp_postgres_server.py")
1930 print("\n? Para testar manualmente:")
1931 print("    DATABASE_URL=postgres://user:senha@localhost:5432/
        banco python mcp_postgres_server.py")

```

```
1932 # ===== cell_052 =====
1933 # ??? Simulacao do banco para testes (sem precisar de
      PostgreSQL real) ??????????
1934 # Este codigo cria um servidor MCP mock para demonstracao
1935
1936 mcp_mock_code = '''
1937 """
1938 Servidor MCP Mock (para testes sem PostgreSQL)
1939 Simula um banco de dados com dados de exemplo.
1940 """
1941 import asyncio
1942 import json
1943 from mcp.server import Server
1944 from mcp.server.stdio import stdio_server
1945 import mcp.types as types
1946
1947 server = Server("postgres-mock-server")
1948
1949 # Dados simulados
1950 MOCK_DATA = {
1951     "clientes": [
1952         {"id": 1, "nome": "Ana Silva", "email": "ana@exemplo.
              com", "cidade": "Sao Paulo", "saldo": 15000.00},
1953         {"id": 2, "nome": "Bruno Costa", "email": "
              bruno@exemplo.com", "cidade": "Rio de Janeiro", "
              saldo": 8500.50},
1954         {"id": 3, "nome": "Carla Mendes", "email": "
              carla@exemplo.com", "cidade": "Curitiba", "saldo":
              22000.00},
1955         {"id": 4, "nome": "Diego Santos", "email": "
              diego@exemplo.com", "cidade": "Sao Paulo", "saldo":
              5200.75},
1956     ],
1957     "produtos": [
1958         {"id": 1, "nome": "Notebook Pro", "categoria": "
              Eletronicos", "preco": 4500.00, "estoque": 15},
1959         {"id": 2, "nome": "Smartphone X", "categoria": "
              Eletronicos", "preco": 2800.00, "estoque": 42},
1960         {"id": 3, "nome": "Mesa Gamer", "categoria": "Moveis",
              "preco": 1200.00, "estoque": 8},
```

```

1961         {"id": 4, "nome": "Cadeira Ergonomica", "categoria": "
1962             Moveis", "preco": 950.00, "estoque": 20},
1963     ],
1964     "pedidos": [
1965         {"id": 1, "cliente_id": 1, "produto_id": 2, "quantidade
1966             ": 1, "total": 2800.00, "status": "entregue"},
1967         {"id": 2, "cliente_id": 2, "produto_id": 1, "quantidade
1968             ": 1, "total": 4500.00, "status": "processando"},
1969         {"id": 3, "cliente_id": 1, "produto_id": 4, "quantidade
1970             ": 2, "total": 1900.00, "status": "entregue"},
1971         {"id": 4, "cliente_id": 3, "produto_id": 3, "quantidade
1972             ": 1, "total": 1200.00, "status": "enviado"},
1973     ]
1974 }
1975
1976 SCHEMAS = {
1977     "clientes": [
1978         {"coluna": "id", "tipo": "integer", "permite_nulo": "NO
1979             "},
1980         {"coluna": "nome", "tipo": "varchar", "permite_nulo": "
1981             NO"},
1982         {"coluna": "email", "tipo": "varchar", "permite_nulo":
1983             "NO"},
1984         {"coluna": "cidade", "tipo": "varchar", "permite_nulo":
1985             "YES"},
1986         {"coluna": "saldo", "tipo": "numeric", "permite_nulo":
1987             "YES"},
1988     ],
1989     "produtos": [
1990         {"coluna": "id", "tipo": "integer", "permite_nulo": "NO
1991             "},
1992         {"coluna": "nome", "tipo": "varchar", "permite_nulo": "
1993             NO"},
1994         {"coluna": "categoria", "tipo": "varchar", "
1995             permite_nulo": "YES"},
1996         {"coluna": "preco", "tipo": "numeric", "permite_nulo":
1997             "NO"},
1998         {"coluna": "estoque", "tipo": "integer", "permite_nulo
1999             ": "YES"},
2000     ],
2001     "pedidos": [

```

```

1987         {"coluna": "id", "tipo": "integer", "permite_nulo": "NO
1988             "},
1989         {"coluna": "cliente_id", "tipo": "integer", "
1990             permite_nulo": "NO"},
1991         {"coluna": "produto_id", "tipo": "integer", "
1992             permite_nulo": "NO"},
1993         {"coluna": "quantidade", "tipo": "integer", "
1994             permite_nulo": "NO"},
1995         {"coluna": "total", "tipo": "numeric", "permite_nulo":
1996             "NO"},
1997         {"coluna": "status", "tipo": "varchar", "permite_nulo":
1998             "YES"},
1999     ]
2000 }
2001
2002 @server.list_tools()
2003 async def list_tools() -> list[types.Tool]:
2004     return [
2005         types.Tool(
2006             name="executar_query",
2007             description="Executa uma query no banco mock (
2008                 suporta filtros simples por coluna=valor)",
2009             inputSchema={
2010                 "type": "object",
2011                 "properties": {
2012                     "tabela": {"type": "string", "description":
2013                         "Nome da tabela"},
2014                     "filtro": {"type": "object", "description":
2015                         "Filtros chave:valor (opcional)"}
2016                 },
2017                 "required": ["tabela"]
2018             },
2019         ),
2020         types.Tool(
2021             name="listar_tabelas",
2022             description="Lista todas as tabelas disponiveis no
2023                 banco mock.",
2024             inputSchema={"type": "object", "properties": {}}
2025         ),
2026         types.Tool(
2027             name="descrever_tabela",

```

```

2018         description="Retorna o schema de uma tabela.",
2019         inputSchema={
2020             "type": "object",
2021             "properties": {
2022                 "tabela": {"type": "string", "description":
2023                     "Nome da tabela"}
2024             },
2025             "required": ["tabela"]
2026         },
2027     ]
2028
2029 @server.call_tool()
2030 async def call_tool(name: str, arguments: dict) -> list[types.
    TextContent]:
2031     if name == "listar_tabelas":
2032         tabelas = [{"table_name": t, "num_colunas": len(c)} for
2033             t, c in SCHEMAS.items()]
2034         return [types.TextContent(type="text", text=json.dumps(
2035             tabelas, ensure_ascii=False, indent=2))]
2036
2037     elif name == "descrever_tabela":
2038         tabela = arguments["tabela"]
2039         schema = SCHEMAS.get(tabela)
2040         if not schema:
2041             return [types.TextContent(type="text", text=f"
2042                 Tabela \'{tabela}\' nao encontrada. Tabelas: {
2043                     list(SCHEMAS.keys())}")]
2044         return [types.TextContent(
2045             type="text",
2046             text=json.dumps({"tabela": tabela, "colunas":
2047                 schema}, ensure_ascii=False, indent=2)
2048         )]
2049
2050     elif name == "executar_query":
2051         tabela = arguments["tabela"]
2052         filtro = arguments.get("filtro", {})
2053         dados = MOCK_DATA.get(tabela, [])
2054         if filtro:
2055             dados = [row for row in dados

```

```

2051         if all(str(row.get(k)) == str(v) for k, v
2052                in filtro.items())]
2053     return [types.TextContent(
2054         type="text",
2055         text=json.dumps(dados, ensure_ascii=False, default=
2056             str, indent=2)
2057     )]
2058
2059     return [types.TextContent(type="text", text=f"Ferramenta
2060         desconhecida: {name}")]
2061
2062 async def main():
2063     async with stdio_server() as (read_stream, write_stream):
2064         await server.run(read_stream, write_stream, server.
2065             create_initialization_options())
2066
2067 if __name__ == "__main__":
2068     asyncio.run(main())
2069
2070
2071 with open("mcp_mock_server.py", "w", encoding="utf-8") as f:
2072     f.write(mcp_mock_code)
2073
2074 print("? Servidor MCP Mock criado: mcp_mock_server.py")
2075 print("    Contem tabelas simuladas: clientes, produtos, pedidos
2076     ")
2077
2078 # ===== cell_053 =====
2079 import asyncio
2080 import io
2081 import os
2082 import sys
2083 from langchain_mcp_adapters.client import MultiServerMCPClient
2084 from langgraph.prebuilt import create_react_agent
2085 from langgraph.checkpoint.memory import MemorySaver
2086 from langchain_anthropic import ChatAnthropic
2087
2088 # Workaround: Jupyter substitui sys.stderr por stream virtual
2089     sem fileno() real.
2090
2091 # O MCP precisa de um fd valido ao criar o subprocesso stdio.
2092 try:

```

```
2086     sys.stderr.fileno()
2087 except io.UnsupportedOperation:
2088     _devnull = open(os.devnull, "w")
2089     sys.stderr.fileno = _devnull.fileno
2090     sys.stdout.fileno = _devnull.fileno
2091
2092 # Servidor mock (sem PostgreSQL):
2093 server_config = {
2094     "postgres_mock": {
2095         "command": "python",
2096         "args": ["mcp_mock_server.py"],
2097         "transport": "stdio",
2098         "env": dict(os.environ)
2099     }
2100 }
2101
2102 # Para usar o servidor real com PostgreSQL, substitua
2103     server_config por:
2104 # server_config = {
2105 #     "postgres": {
2106 #         "command": "python",
2107 #         "args": ["mcp_postgres_server.py"],
2108 #         "transport": "stdio",
2109 #         "env": {**dict(os.environ), "DATABASE_URL": os.
2110             environ.get("DATABASE_URL", "")}
2111 #     }
2112 # }
2113
2114 async def criar_e_usar_agente_mcp():
2115     client = MultiServerMCPClient(server_config)
2116     mcp_tools = await client.get_tools()
2117     print(f"?? Ferramentas MCP disponiveis: {[t.name for t in
2118         mcp_tools]}")
2119
2120     model = ChatAnthropic(model="claude-sonnet-4-6",
2121         temperature=0)
2122     agente_mcp = create_react_agent(model, mcp_tools,
2123         checkpoint=MemorySaver())
2124     config = {"configurable": {"thread_id": "mcp-demo-1"}}
2125
2126     for pergunta in [
```

```

2122     "Quais tabelas existem no banco de dados?",
2123     "Liste os clientes de Sao Paulo e me diga o saldo medio
        deles",
2124     "E os pedidos desses clientes que mencionei? Quais
        produtos eles compraram?",
2125 ]:
2126     print("\n" + "=" * 60)
2127     print(f"?? {pergunta}")
2128     resultado = await agente_mcp.ainvoke(
2129         {"messages": [{"role": "user", "content": pergunta
2130             }]}},
2131         config=config
2132     )
2133     print(f"?? {resultado['messages'][-1].content}")
2134
2135 await criar_e_usar_agente_mcp()
2136 # Em script Python normal: asyncio.run(criar_e_usar_agente_mcp
2137     ())
2138
2139 # ===== cell_054 =====
2140 # ??? Exemplo avancado: Agente com multiplos servidores MCP
2141     ??????????????????????
2142
2143 async def agente_multi_mcp():
2144     """Demonstra conexao a multiplos servidores MCP
2145         simultaneamente."""
2146
2147     config_multi = {
2148         "database": {
2149             "command": "python",
2150             "args": ["mcp_mock_server.py"],
2151             "transport": "stdio",
2152             "env": dict(os.environ)
2153         },
2154         # Outros servidores MCP podem ser adicionados aqui:
2155         # "filesystem": {"command": "npx", "args": ["-y", "
2156             @modelcontextprotocol/server-filesystem", "/path"]},
2157         # "web_search": {"url": "http://localhost:8000/mcp", "
2158             transport": "sse"}
2159     }

```



```

2154     # langchain-mcp-adapters >= 0.1.0: nao usar como context
        manager
2155     client = MultiServerMCPClient(config_multi)
2156     all_tools = await client.get_tools()
2157     print(f"?? Total de ferramentas de todos os servidores: {
        len(all_tools)}")
2158     for tool in all_tools:
2159         print(f"    ? {tool.name}: {tool.description[:60]}...")
2160
2161     # Combina ferramentas MCP com skills locais
2162     skills_locais = [calculadora, skill_analise,
        buscar_conhecimento]
2163     todas_ferramentas = all_tools + skills_locais
2164
2165     model = ChatAnthropic(model="claude-sonnet-4-6",
        temperature=0)
2166     agente_final = create_react_agent(model, todas_ferramentas,
        checkpoint=MemorySaver())
2167
2168     print(f"\n? Agente com {len(todas_ferramentas)} ferramentas
        (MCP + locais)")
2169
2170     config = {"configurable": {"thread_id": "multi-mcp-1"}}
2171     resultado = await agente_final.ainvoke(
2172         {"messages": [{"role": "user", "content": (
2173             "Preciso de um relatorio:\n"
2174             "1) Liste os produtos em estoque com preco acima de
                R$1000\n"
2175             "2) Calcule o valor total em estoque desses
                produtos\n"
2176             "3) Me explique o que e RAG (busque na base de
                conhecimento)"
2177         )}]},
2178         config=config
2179     )
2180     print(f"\n?? Relatorio:\n{resultado['messages'][-1].content
        }")
2181
2182     await agente_multi_mcp()
2183
2184     # ===== cell_056 =====

```

```

2185 # ??? 7.1 Pipeline ML: geracao de dados, clustering e perfis
      ?????????????????
2186 # Em PRODUCAO os nomes dos segmentos, prompts e personas vem do
2187 # Athena/Glue Data Catalog ? nao de regras hard-coded.
2188 # Aqui simulamos o Athena com um DataFrame local para fins
      didaticos.
2189 import warnings
2190 import numpy as np
2191 import pandas as pd
2192 from sklearn.cluster import KMeans
2193 from sklearn.preprocessing import StandardScaler
2194
2195 warnings.filterwarnings("ignore", message=".*create_react_agent
      .*")
2196
2197 np.random.seed(42)
2198 N = 600
2199
2200 # Dados sinteticos de clientes de banco
2201 df_clientes = pd.DataFrame({
2202     "cliente_id":    [f"C{i:05d}" for i in range(N)],
2203     "idade":         np.random.randint(18, 75, N),
2204     "renda_mensal":  np.random.exponential(5000, N).clip(800,
      50_000),
2205     "saldo_medio":   np.random.exponential(12_000, N).clip(0,
      200_000),
2206     "transacoes_mes": np.random.poisson(18, N),
2207     "score_credito":  np.random.normal(650, 120, N).clip(300, 1
      _000),
2208     "num_produtos":   np.random.randint(1, 8, N),
2209     "inadimplente":   np.random.choice([0, 1], N, p=[0.87,
      0.13]),
2210     "canal_digital":  np.random.choice([0, 1], N, p=[0.35,
      0.65]),
2211 })
2212
2213 # ?? Normalizacao e clustering K-Means
      ?????????????????????????????????????????????
2214 FEATURES = ["idade", "renda_mensal", "saldo_medio", "
      transacoes_mes",
2215             "score_credito", "num_produtos"]

```

[illegible]

```

2247     if row["renda_media"] > 10_000 and row["score_medio"] >
2248         700:
2249         return "Premium Conservador"
2250     elif row["idade_media"] < 35 and row["digital"] > 0.65:
2251         return "Jovem Digital"
2252     elif row["inadimplencia"] > 0.18:
2253         return "Alto Risco"
2254     return "Massa Estavel"
2255
2256 # ?? Fallback: prompts, produtos e personas por segmento
2257 # Estes dicts sao usados APENAS como fallback quando o Athena
2258 # Em producao, os dados vem da tabela: {ATHENA_DATABASE}.{
2259 #     ATHENA_TABLE_SEGMENTOS}
2260 PROMPTS_SEGMENTO_FB = {
2261     "Premium Conservador": (
2262         "Voce e um gerente de relacionamento exclusivo do banco
2263         para clientes Premium. "
2264         "Esses clientes tem alta renda, excelente historico de
2265         credito e preferem "
2266         "produtos de investimento de baixo risco e atendimento
2267         personalizado. "
2268         "Seja formal, preciso e ofereca solucoes sofisticadas
2269         com dados concretos."
2270     ),
2271     "Jovem Digital": (
2272         "Voce atende clientes jovens e altamente digitais do
2273         banco. "
2274         "Eles preferem comunicacao direta e descomplicada,
2275         produtos via app, "
2276         "cashback e aprovacao de credito instantanea. "
2277         "Use linguagem leve e destaque praticidade e beneficios
2278         digitais."
2279     ),
2280     "Alto Risco": (
2281         "Voce atende clientes com historico de inadimplencia. "
2282         "Foque em renegociacao de dividas, educacao financeira
2283         e produtos de "
2284         "baixo limite inicial. Seja empatico, nao faca
2285         julgamentos e apresente "

```

```
2275         "caminhos concretos de regularizacao financeira."
2276     ),
2277     "Massa Estavel": (
2278         "Voce atende o publico geral e estavel do banco. "
2279         "Esses clientes valorizam seguranca, previsibilidade e
          bom custo-beneficio. "
2280         "Foque em fidelizacao, cross-sell gradual e protecao
          patrimonial simples."
2281     ),
2282 }
2283
2284 PRODUTOS_POR_SEGMENTO_FB = {
2285     "Premium Conservador": (
2286         "Tesouro Direto IPCA+ e fundos de renda fixa de longo
          prazo. "
2287         "Previdencia privada PGBL com beneficio fiscal. "
2288         "Cartao Platinum com sala VIP e seguro viagem. "
2289         "Consultoria de investimentos com gerente dedicado. "
2290         "Seguro de vida com cobertura ampla."
2291     ),
2292     "Jovem Digital": (
2293         "Cartao de credito com cashback de 2% sem anuidade. "
2294         "Conta digital com rendimento automatico de 100% do CDI
          . "
2295         "Credito pessoal com aprovacao em minutos pelo app. "
2296         "Investimentos a partir de R$ 1 via plataforma digital.
          "
2297         "Portabilidade de salario com bonus de boas-vindas."
2298     ),
2299     "Alto Risco": (
2300         "Programa de renegociacao de dividas com ate 90% de
          desconto nos juros. "
2301         "Cartao pre-pago sem consulta ao SPC/Serasa. "
2302         "Microcredito inicial de R$ 500 com limite crescente. "
2303         "Curso gratuito de educacao financeira no app. "
2304         "Debito automatico de contas para evitar novas
          inadimplencias."
2305     ),
2306     "Massa Estavel": (
2307         "Conta corrente com pacote de tarifas simplificado. "
```

```
2308         "Cartao de credito com limite adequado ao perfil de
2309             renda. "
2310         "Seguro residencial com cobertura basica e parcelas
2311             acessiveis. "
2312         "Consortorio de imoveis e veiculos sem juros. "
2313         "Previdencia privada VGBL com aporte minimo."
2314     ),
2315 }
2316
2317 PERSONAS_SEGMENTO_FB = {
2318     "Premium Conservador": {
2319         "nome": "Carlos",
2320         "ocupacao": "gerente de empresa, 52 anos",
2321         "canal": "prefiro o atendimento exclusivo com meu
2322             gerente de relacionamento",
2323         "contexto": (
2324             "Planejei minha aposentadoria com cuidado e tenho
2325             patrimonio consolidado. "
2326             "Priorizo seguranca e rentabilidade real acima da
2327             inflacao."
2328         ),
2329     },
2330     "Jovem Digital": {
2331         "nome": "Julia",
2332         "ocupacao": "designer freelancer, 26 anos",
2333         "canal": "facio tudo pelo app ? nunca fui a uma agencia"
2334         ,
2335         "contexto": (
2336             "Gosto de produtos sem burocracia. "
2337             "Cashback e aprovacao instantanea sao diferenciais
2338             importantes para mim."
2339         ),
2340     },
2341     "Alto Risco": {
2342         "nome": "Roberto",
2343         "ocupacao": "motorista autonomo, 43 anos",
2344         "canal": "uso o app para o dia a dia mas prefiro a
2345             agencia para assuntos serios",
2346         "contexto": (
2347             "Passei por um periodo dificil e acumulei dividas.
2348             "
```

```

2340         "Preciso de orientacao concreta para regularizar
           minha situacao."
2341     ),
2342 },
2343     "Massa Estavel": {
2344         "nome": "Ana",
2345         "ocupacao": "funcionaria publica ha 12 anos, 38 anos",
2346         "canal": "uso o internet banking mas prefiro a agencia
           para assuntos importantes",
2347         "contexto": (
2348             "Valorizo estabilidade e produtos sem surpresas. "
2349             "Penso no futuro da minha familia e em proteger o
           que ja conquistei."
2350         ),
2351     },
2352 }
2353
2354 #
           ?????????????????????????????????????????????????????????????????????????????????????
2355 # PRODUCAO: carregar_segmentos_athena()
2356 #     SELECT cluster_id, segmento_nome, prompt_segmento, produtos
           ,
2357 #         persona_nome, persona_ocupacao, persona_canal,
           persona_contexto
2358 #     FROM {ATHENA_DATABASE}.{ATHENA_TABLE_SEGMENTOS}
2359 #     ORDER BY cluster_id
2360 #
2361 # NOTEBOOK: simulamos com um DataFrame local. Analistas de
           negocio
2362 # preenchem esta tabela no Glue ? podem renomear ou criar novos
           segmentos
2363 # sem nenhuma alteracao no codigo Python.
2364 #
           ?????????????????????????????????????????????????????????????????????????????????????
2365
2366 # Etapa 1: nomes provisórios via heurística (apenas para
           construir o mock)
2367 _perfis_temp = perfis_clientes.copy()

```

```

2368 _perfis_temp["segmento"] = _perfis_temp.apply(nomear_segmento,
2369         axis=1)
2370
2370 # Etapa 2: DataFrame que simula a tabela no Athena/Glue Catalog
2371 seg_df_athena = pd.DataFrame([
2372     {
2373         "cluster_id":      int(cid),
2374         "segmento_nome":   row["segmento"],
2375         "prompt_segmento": PROMPTS_SEGMENTO_FB.get(row["
2376             segmento"], ""),
2377         "produtos":        PRODUTOS_POR_SEGMENTO_FB.get(row["
2378             segmento"], ""),
2379         "persona_nome":    PERSONAS_SEGMENTO_FB.get(row["
2380             segmento"], {}).get("nome", ""),
2381         "persona_ocupacao": PERSONAS_SEGMENTO_FB.get(row["
2382             segmento"], {}).get("ocupacao", ""),
2383         "persona_canal":   PERSONAS_SEGMENTO_FB.get(row["
2384             segmento"], {}).get("canal", ""),
2385         "persona_contexto": PERSONAS_SEGMENTO_FB.get(row["
2386             segmento"], {}).get("contexto", ""),
2387     }
2388     for cid, row in _perfis_temp.iterrows()
2389 ])
2390
2391 print("Tabela Athena/Glue simulada (em producao:
2392     carregar_segmentos_athena()):")
2393 print(seg_df_athena[["cluster_id", "segmento_nome", "
2394     persona_nome"]].to_string(index=False))
2395 print()
2396
2397 # ?? _enriquecer_perfis_nb: mescla estatisticas K-Means +
2398     metadados do Athena ??
2399
2400 def _enriquecer_perfis_nb(perfis: pd.DataFrame, seg_df: pd.
2401     DataFrame = None) -> pd.DataFrame:
2402     """
2403     Enriquece o DataFrame de perfis com metadados do Athena/
2404     Glue.
2405
2406     Fluxo de prioridade para cada atributo:
2407     1. Athena/Glue Catalog <- fonte de verdade (producao)

```



```

2396     2. Heuristica local         <- fallback quando Athena nao
      disponivel
2397
2398     Em producao: seg_df = carregar_segmentos_athena()
2399     No notebook: seg_df = seg_df_athena (simulado)
2400     """
2401     perfis = perfis.copy()
2402     seg_idx = (
2403         seg_df.set_index("cluster_id")
2404         if seg_df is not None and not seg_df.empty
2405         else None
2406     )
2407
2408     for cid in perfis.index:
2409         athena = (
2410             seg_idx.loc[cid]
2411             if seg_idx is not None and cid in seg_idx.index
2412             else pd.Series(dtype=object)
2413         )
2414
2415         # ?? Nome do segmento
2416         ?????????????????????????????????????????????????????????????
2417         segmento = str(athena.get("segmento_nome", "").strip())
2418         if not segmento:
2419             segmento = nomear_segmento(perfis.loc[cid]) #
2420             fallback heuristico
2421         perfis.loc[cid, "segmento"] = segmento
2422
2423         # ?? Prompt do agente de segmento
2424         ?????????????????????????????????????????????????????????????
2425         prompt = str(athena.get("prompt_segmento", "").strip())
2426         perfis.loc[cid, "prompt_segmento"] = prompt or
2427         PROMPTS_SEGMENTO_FB.get(segmento, "")
2428
2429         # ?? Produtos recomendados
2430         ?????????????????????????????????????????????????????????????
2431         produtos = str(athena.get("produtos", "").strip())
2432         perfis.loc[cid, "produtos"] = produtos or
2433         PRODUTOS_POR_SEGMENTO_FB.get(segmento, "")

```

```

2429     # ?? Persona (arquetipo do cluster)
        ?????????????????????????????????????????????????????????????
2430     p_fb = PERSONAS_SEGMENTO_FB.get(segmento, {})
2431     for col, key in [
2432         ("persona_nome", "nome"),
2433         ("persona_ocupacao", "ocupacao"),
2434         ("persona_canal", "canal"),
2435         ("persona_contexto", "contexto"),
2436     ]:
2437         val = str(athena.get(col, "")).strip()
2438         perfis.loc[cid, col] = val or p_fb.get(key, "")
2439
2440     return perfis
2441
2442
2443 # ?? Enriquecer perfis
        ?????????????????????????????????????????????????????????????
2444 # Em producao:
2445 #     seg_df = carregar_segmentos_athena()
2446 #     perfis_clientes = _enriquecer_perfis_nb(perfis_clientes,
        seg_df)
2447 #
2448 # No notebook (Athena simulado):
2449 perfis_clientes = _enriquecer_perfis_nb(perfis_clientes,
        seg_df_athena)
2450
2451 # ?? Exibe resumo
        ?????????????????????????????????????????????????????????????
        ?????????????????????????????????????????????????????????????

2452 print("SEGMENTOS IDENTIFICADOS PELO K-MEANS")
2453 print("(nomes, prompts e personas carregados do Athena/Glue
        Catalog)")
2454 print("=" * 75)
2455 print(
2456     perfis_clientes[
2457         ["segmento", "n", "idade_media", "renda_media",
2458         "saldo_medio_", "score_medio", "inadimplencia"]
2459     ]
2460     .rename(columns={
2461         "segmento": "Segmento", "n": "Clientes",
2462         "idade_media": "Idade", "renda_media": "Renda (R$)",

```

```

2463         "saldo_medio_": "Saldo (R$)", "score_medio": "Score",
2464         "inadimplencia": "Inadimp.%",
2465     })
2466     .to_string()
2467 )
2468
2469 print("\nDistribuicao:")
2470 for cid, row in perfis_clientes.iterrows():
2471     bar = "#" * int(row["n"] / 10)
2472     print(f"    Cluster {cid} | {row['segmento']:<22} | {int(row
2473         ['n']):>3} clientes {bar}")
2474
2475 # ===== cell_058 =====
2476 # ??? 7.1b Big Five: inferencia e enriquecimento dos perfis
2477     ??????????????????????
2478 # Em producao: scores Big5 calculados pelo job SageMaker e
2479     salvos no Glue Catalog.
2480 # No notebook: inferencia direta das features financeiras como
2481     proxy.
2482
2483 OCEAN_TRAITS = ["openness", "conscientiousness", "extraversion",
2484     , "agreeableness", "neuroticism"]
2485
2486
2487 def inferir_big5_cluster(perfil: pd.Series) -> dict:
2488     """
2489     Infere Big Five a partir do perfil ESTATISTICO do cluster.
2490     Usado para PERSONAS (arquetipos do grupo).
2491
2492     Mapeamento:
2493         openness           ? canal_digital + variedade de produtos
2494         conscientiousness ? score_credito +
2495             ausencia_inadimplencia
2496         extraversion       ? canal_digital + produtos_medio
2497         agreeableness      ? (1 - inadimplencia) + score parcial
2498         neuroticism        ? inadimplencia + score baixo
2499     """
2500     digital = float(perfil["digital"])
2501     score = float(perfil["score_medio"])
2502     inadimp = float(perfil["inadimplencia"])
2503     prod = min(float(perfil["produtos_medio"]) / 8, 1.0)

```

```

2498
2499     return {
2500         "openness": round(min(digital * 0.6 + prod *
2501                                0.4,
2502                                1.0), 2),
2501         "conscientiousness": round(min(score / 1000 * 0.7 + (1
2502                                - inadimp) * 0.3, 1.0), 2),
2502         "extraversion": round(min(digital * 0.5 + prod *
2503                                0.5,
2504                                1.0), 2),
2503         "agreeableness": round(min((1 - inadimp) * 0.6 +
2504                                score / 2000 * 0.4, 1.0), 2),
2504         "neuroticism": round(min(inadimp * 0.6 + max(0,
2505                                (500 - score) / 500) * 0.4, 1.0), 2),
2505     }
2506
2507
2508 def inferir_big5_cliente(row: pd.Series) -> dict:
2509     """
2510     Inferir Big Five a partir dos dados BRUTOS de um cliente
2511     individual.
2512     Usado para DIGITAL TWINS (sem dependencia de clustering).
2513     """
2513     digital = float(row["canal_digital"])
2514     score = float(row["score_credito"])
2515     inadimp = float(row["inadimplente"])
2516     txn = min(float(row["transacoes_mes"]) / 30, 1.0)
2517     prod = min(float(row["num_produtos"]) / 8, 1.0)
2518
2519     return {
2520         "openness": round(min(digital * 0.6 + txn *
2521                                0.4,
2522                                1.0), 2),
2521         "conscientiousness": round(min(score / 1000 * 0.7 + (1
2522                                - inadimp) * 0.3, 1.0), 2),
2522         "extraversion": round(min(digital * 0.4 + txn *
2523                                0.4 + prod * 0.2, 1.0), 2),
2523         "agreeableness": round(min((1 - inadimp) * 0.6 +
2524                                score / 2000 * 0.4, 1.0), 2),
2524         "neuroticism": round(min(inadimp * 0.6 + max(0,
2525                                (500 - score) / 500) * 0.4, 1.0), 2),
2525     }
2526
2527

```

```
2528 def big5_para_estilo(big5: dict) -> str:
2529     """
2530     Converte scores OCEAN em descricao de estilo comportamental
2531     para incluir nos prompts.
2532     Apenas tracos com score >0.65 ou <0.35 sao mencionados (
2533     polarizados).
2534     """
2535     partes = []
2536
2537     if big5["openness"] > 0.65:
2538         partes.append("aberto a produtos inovadores e novidades
2539             financeiras")
2540     elif big5["openness"] < 0.35:
2541         partes.append("prefere solucoes tradicionais e
2542             conhecidas, evita o desconhecido")
2543
2544     if big5["conscientiousness"] > 0.65:
2545         partes.append("organizado, analisa todos os detalhes
2546             antes de decidir")
2547     elif big5["conscientiousness"] < 0.35:
2548         partes.append("toma decisoes rapidas e prefere
2549             processos simples e ageis")
2550
2551     if big5["extraversion"] > 0.65:
2552         partes.append("engajado e proativo nos canais do banco")
2553     elif big5["extraversion"] < 0.35:
2554         partes.append("reservado, prefere autoatendimento e
2555             pouca interacao")
2556
2557     if big5["agreeableness"] > 0.65:
2558         partes.append("cooperativo e receptivo a propostas e
2559             acordos")
2560
2561     if big5["neuroticism"] > 0.5:
2562         partes.append("ansioso com incertezas financeiras,
2563             valoriza garantias e previsibilidade")
2564     elif big5["neuroticism"] < 0.2:
2565         partes.append("emocionalmente estavel, tolerante a
2566             riscos calculados")
2567
```

```

2558     return "; ".join(partes) if partes else "perfil equilibrado
2559         sem tracos dominantes"
2560
2561 # ?? Adiciona colunas Big Five a perfis_clientes
2562     ?????????????????????????????????????????????
2563 for cid, row in perfis_clientes.iterrows():
2564     b5 = inferir_big5_cluster(row)
2565     for trait, val in b5.items():
2566         perfis_clientes.loc[cid, f"big5_{trait}"] = val
2567
2568 # ?? Exibe perfis com Big Five
2569     ?????????????????????????????????????????????
2570 b5_cols = [f"big5_{t}" for t in OCEAN_TRAITS]
2571 print("PERFIS ENRIQUECIDOS COM BIG FIVE (OCEAN)")
2572 print("=" * 70)
2573 print(perfis_clientes[["segmento"] + b5_cols].to_string())
2574 print()
2575 print("Estilos comportamentais derivados:")
2576 print("-" * 70)
2577 for cid, row in perfis_clientes.iterrows():
2578     b5 = {t: float(row[f"big5_{t}"]) for t in OCEAN_TRAITS}
2579     estilo = big5_para_estilo(b5)
2580     print(f"    Cluster {cid} | {row['segmento']:<22}: {estilo}")
2581
2582 # ===== cell_059 =====
2583 # ??? 7.2 RAG por segmento + agente especializado
2584     ?????????????????????????????????????????????
2585 # Prompts e produtos sao lidos de perfis_clientes["
2586     prompt_segmento"] e
2587 # perfis_clientes["produtos"] ? valores que vieram do Athena/
2588     Glue na celula 7.1.
2589 # Nao ha dicionarios hard-coded aqui: qualquer nome de segmento
2590     do Glue funciona.
2591
2592 from langchain_core.documents import Document
2593 from langchain_text_splitters import
2594     RecursiveCharacterTextSplitter
2595 from langchain_community.vectorstores import FAISS
2596 from langchain_community.embeddings import
2597     HuggingFaceEmbeddings
2598 from langchain_core.tools import tool

```

```

2590 from langchain_anthropic import ChatAnthropic
2591 from langgraph.prebuilt import create_react_agent
2592 from langgraph.checkpoint.memory import MemorySaver
2593
2594 # ?? Documentos de conhecimento gerados a partir dos perfis
    enriquecidos ???????
2595 def _gerar_docs_cluster(cluster_id: int, perfil: pd.Series) ->
    list[Document]:
2596     """
2597     Cria 3 documentos por cluster.
2598     Todo o conteudo textual (prompt, produtos) vem de perfil ?
        populado
2599     com dados do Athena/Glue Catalog na etapa anterior (celula
        7.1).
2600     Nenhum dicionario local e consultado aqui.
2601     """
2602     segmento      = str(perfil["segmento"])
2603     renda         = float(perfil["renda_media"])
2604     score         = float(perfil["score_medio"])
2605     n_clientes    = int(perfil["n"])
2606     inadimp       = float(perfil["inadimplencia"])
2607     produtos_txt  = str(perfil.get("produtos", ""))
2608
2609     return [
2610         Document(
2611             page_content=(
2612                 f"PERFIL DO SEGMENTO: {segmento}\n"
2613                 f"- Clientes: {n_clientes}\n"
2614                 f"- Renda media: R$ {renda:,.0f}/mes\n"
2615                 f"- Score medio: {score:.0f}\n"
2616                 f"- Inadimplencia: {inadimp:.1%}\n"
2617                 f"- Idade media: {float(perfil['idade_media'])
                    :.0f} anos\n"
2618                 f"- Canal digital: {float(perfil['digital'])
                    :.0%}"
2619             ),
2620             metadata={"cluster_id": cluster_id, "tipo": "perfil",
                "segmento": segmento},
2621         ),
2622         Document(

```

```

2623         page_content=f"PRODUTOS RECOMENDADOS PARA O
                SEGMENTO {segmento}:\n{produtos_txt}",
2624         metadata={"cluster_id": cluster_id, "tipo": "
                produtos", "segmento": segmento},
2625     ),
2626     Document(
2627         page_content=(
2628             f"ESTRATEGIA DE ATENDIMENTO ? {segmento}\n"
2629             f"Score {score:.0f}: "
2630             f"{'excelente credito ? ofereca produtos
                premium' if score > 750 else 'bom credito ?
                cross-sell gradual' if score > 600 else '
                credito limitado ? foco em recuperacao'}.\n"
2631             f"Renda R$ {renda:,.0f}: "
2632             f"{'alta ? produtos de investimento e protecao'
                if renda > 10_000 else 'media ? produtos
                acessiveis' if renda > 3_000 else 'baixa ?
                microcredito e educacao financeira'}."
2633         ),
2634         metadata={"cluster_id": cluster_id, "tipo": "
                estrategia", "segmento": segmento},
2635     ),
2636 ]
2637
2638
2639 # ?? Ferramenta: consultar regulamentos do banco
        ?????????????????????????????????????????
2640 @tool
2641 def consultar_regulamentos(produto: str) -> str:
2642     """Consulta regras e limitacoes regulatorias para um
        produto bancario."""
2643     regulamentos = {
2644         "investimento": "CVM regula fundos. Clientes PF: limite
                de R$ 1M sem declaracao de IR. Prazo minimo: 30
                dias.",
2645         "cartao": "Banco Central: taxa maxima de rotativo
                8% a.m. Limite minimo: 20% da renda.",
2646         "credito": "CDC: taxa maxima de 12% a.a. para
                credito consignado. Prazo maximo: 84 meses.",
2647         "previdencia": "SUSEP regula. PGBL: deduz ate 12% da
                renda bruta no IR. VGBL: sem deducao.",
    }

```



```

2648         "renegociacao": "Lei 14.181/2021 (Superendividamento):
                banco deve oferecer renegociacao em 60 dias.",
2649     }
2650     for k, v in regulamentos.items():
2651         if k.lower() in produto.lower():
2652             return v
2653     return f"Consulte o gerente para detalhes regulatorios
            sobre '{produto}'."
2654
2655
2656 # ?? Embeddings e splitter
            ?????????????????????????????????????????????????????????????
2657 embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-
            v2")
2658 splitter = RecursiveCharacterTextSplitter(chunk_size=400,
            chunk_overlap=50)
2659 checkpointer = MemorySaver()
2660
2661
2662 # ?? Cria um agente por cluster
            ?????????????????????????????????????????????????????????????
2663 def _criar_agente_cluster(cluster_id: int, perfil: pd.Series)
            -> dict:
2664     """
2665     Cria agente RAG especializado para um cluster.
2666
2667     O prompt do sistema e lido de perfil['prompt_segmento'],
            que foi
2668     carregado do Athena/Glue Catalog ? nao de um dicionario
            hard-coded.
2669     Qualquer nome ou prompt definido pelos analistas no Glue e
            respeitado.
2670     """
2671     segmento = str(perfil["segmento"])
2672     prompt_sistema = str(perfil.get("prompt_segmento", ""))
2673
2674     docs = _gerar_docs_cluster(cluster_id, perfil)
2675     chunks = splitter.split_documents(docs)
2676     vs = FAISS.from_documents(chunks, embeddings)
2677     retriever = vs.as_retriever(search_kwargs={"k": 3})
2678

```

```

2679 @tool
2680 def buscar_perfil_segmento(query: str) -> str:
2681     """Busca informacoes sobre o perfil e produtos do
2682         segmento no knowledge base."""
2683     docs_ret = retriever.invoke(query)
2684     return "\n--\n".join(d.page_content for d in docs_ret)
2685
2686 agente = create_react_agent(
2687     model=ChatAnthropic(model="claude-haiku-4-5-20251001",
2688         temperature=0),
2689     tools=[buscar_perfil_segmento, consultar_regulamentos],
2690     checkpoint=checkpoint,
2691     prompt=prompt_sistema,
2692 )
2693
2694 return {"segmento": segmento, "agente": agente, "
2695     vector_store": vs}
2696
2697 # ?? Instancia agentes para todos os clusters
2698     ?????????????????????????????????????????????
2699
2700 print("Criando agentes por segmento...")
2701 print("(prompts e produtos lidos do Athena/Glue via
2702     perfis_clientes)\n")
2703
2704 agentes_clientes = {}
2705 for cluster_id, perfil in perfis_clientes.iterrows():
2706     agentes_clientes[cluster_id] = _criar_agente_cluster(
2707         cluster_id, perfil)
2708     print(f"    ok Cluster {cluster_id}: {perfil['segmento']}")
2709
2710 print(f"\nAgentes prontos: {list(agentes_clientes.keys())}")
2711
2712 # ===== cell_060 =====
2713 # ??? 7.3 Demo: roteamento e consulta ao agente correto
2714     ?????????????????????????????????????????
2715
2716 def classificar_cliente(dados: dict) -> int:
2717     """Classifica um novo cliente no segmento correto usando o
2718         modelo treinado."""
2719
2720     X = scaler_clientes.transform([[
2721         dados["idade"],
2722         dados["renda_mensal"],

```

```

2712         dados["saldo_medio"],
2713         dados["transacoes_mes"],
2714         dados["score_credito"],
2715         dados["num_produtos"],
2716     ]])
2717     return int(kmeans_clientes.predict(X)[0])
2718
2719 def consultar_agente(cliente_id: str, dados: dict, pergunta:
2720     str) -> dict:
2721     """Roteia o cliente ao agente do segmento e retorna a
2722         resposta."""
2723     cluster_id = classificar_cliente(dados)
2724     info = agentes_clientes[cluster_id]
2725     config = {"configurable": {"thread_id": f"cliente-{
2726         cliente_id}"}}
2727     resultado = info["agente"].invoke(
2728         {"messages": [{"role": "user", "content": pergunta}]},
2729         config=config,
2730     )
2731     return {
2732         "cliente_id": cliente_id,
2733         "cluster_id": cluster_id,
2734         "segmento": info["segmento"],
2735         "resposta": resultado["messages"][-1].content,
2736     }
2737
2738 # ?? Clientes de teste
2739 ?????????????????????????????????????????????????????????????????
2740 clientes_teste = [
2741     {
2742         "id": "C001",
2743         "dados": {"idade": 54, "renda_mensal": 22000, "
2744             saldo_medio": 95000,
2745             "transacoes_mes": 10, "score_credito": 840, "
2746                 num_produtos": 6},
2747         "label": "Cliente rico, conservador",
2748     },
2749     {
2750         "id": "C002",
2751         "dados": {"idade": 23, "renda_mensal": 2500, "
2752             saldo_medio": 800,

```

```

2746         "transacoes_mes": 40, "score_credito": 580, "
           num_produtos": 2},
2747     "label": "Jovem digital, baixo saldo",
2748 },
2749 {
2750     "id": "C003",
2751     "dados": {"idade": 41, "renda_mensal": 3100, "
           saldo_medio": 300,
2752             "transacoes_mes": 6, "score_credito": 360, "
           num_produtos": 1},
2753     "label": "Inadimplente, perfil de risco",
2754 },
2755 {
2756     "id": "C004",
2757     "dados": {"idade": 45, "renda_mensal": 5500, "
           saldo_medio": 12000,
2758             "transacoes_mes": 18, "score_credito": 680, "
           num_produtos": 3},
2759     "label": "Massa estavel, produto tipico",
2760 },
2761 ]
2762
2763 pergunta = "Quais produtos voce recomenda para melhorar minha
           situacao financeira?"
2764
2765 for c in clientes_teste:
2766     r = consultar_agente(c["id"], c["dados"], pergunta)
2767     print(f"\n{'='*65}")
2768     print(f"Cliente {r['cliente_id']} ({c['label']})")
2769     print(f"Segmento: {r['segmento']} | Cluster: {r['
           cluster_id']}")
2770     print(f"{'='*65}")
2771     print(r["resposta"])
2772
2773 # ===== cell_062 =====
2774 # ??? 7.4 Digital Twins de Clientes
           ?????????????????????????????????????????
2775 # O twin representa o INDIVIDUO, nao o grupo ? independente de
           clustering.
2776 # Com Big Five: a personalidade e inferida dos dados brutos de
           cada cliente,

```

```

2777 # produzindo um estilo comportamental unico sem depender de
      rotulos de segmento.
2778 from langchain_core.documents import Document
2779 from langchain_core.tools import StructuredTool
2780 from langchain_community.vectorstores import FAISS
2781
2782 # ?? Documento individual ? dados brutos, sem rotulo de
      segmento ??????????????
2783 def _doc_twin(row: pd.Series) -> Document:
2784     canal = "digital (app/internet)" if row["canal_digital"]
      else "agencia fisica"
2785     historico = (
2786         "possui historico de inadimplencia ? em processo de
      regularizacao"
2787         if row["inadimplente"] else "adimplente, sem pendencias
      "
2788     )
2789     return Document(
2790         page_content=(
2791             f"Perfil individual do cliente {row['cliente_id']}: \n"
2792             f"- Idade: {row['idade']:.0f} anos \n"
2793             f"- Renda mensal: R$ {row['renda_mensal']:.0f} \n"
2794             f"- Saldo medio: R$ {row['saldo_medio']:.0f} \n"
2795             f"- Transacoes/mes: {row['transacoes_mes']:.0f} \n"
2796             f"- Score de credito: {row['score_credito']:.0f} \n"
2797             f"- Produtos contratados: {row['num_produtos']:.0f} \n"
2798             f"- Canal preferencial: {canal} \n"
2799             f"- Historico: {historico}"
2800         ),
2801         metadata={"cliente_id": row["cliente_id"], "tipo": "twin"},
2802     )
2803
2804
2805 # ?? System prompt com personalidade Big Five inferida dos
      dados brutos ??????????
2806 def _prompt_twin(row: pd.Series) -> str:
2807     """

```

```

2808     Gera o prompt do twin sem depender de nenhum rotulo de
        clustering.
2809     O perfil comportamental e inferido dos dados numericos via
        Big Five,
2810     produzindo um estilo unico por cliente ? nao uma media de
        grupo.
2811     """
2812     canal      = "prefiro usar aplicativo e internet" if row["
        canal_digital"] else "prefiro ser atendido na agencia"
2813     financeiro = (
2814         "estou reestruturando minhas financas apos um periodo
        de inadimplencia"
2815         if row["inadimplente"] else "mantenho meu historico
        financeiro limpo"
2816     )
2817     renda      = float(row["renda_mensal"])
2818     score      = float(row["score_credito"])
2819     idade      = float(row["idade"])
2820     digital    = bool(row["canal_digital"])
2821
2822     # Perfil descritivo (heuristico ? sem usar rotulo de
        cluster)
2823     if renda > 10_000 and score > 700:
2824         perfil_desc = "Tenho alta renda e excelente credito.
        Valorizo produtos sofisticados e atendimento
        exclusivo."
2825     elif idade < 35 and digital:
2826         perfil_desc = "Sou jovem e digital. Priorizo
        praticidade, cashback e aprovacoes rapidas pelo app.
        "
2827     elif score < 450:
2828         perfil_desc = "Estou regularizando minha situacao
        financeira. Busco solucoes acessiveis sem burocracia
        ."
2829     else:
2830         perfil_desc = "Valorizo estabilidade e produtos sem
        surpresas. Penso no longo prazo."
2831
2832     # ?? Big Five inferido dos dados brutos (sem clustering)
        ??????????????????????
2833     b5          = inferir_big5_cliente(row)

```

```

2834     estilo    = big5_para_estilo(b5)
2835     b5_linha = " | ".join(f"{t[0].upper()}:{v:.2f}" for t, v in
                             zip(OCEAN_TRAITS, b5.values()))
2836
2837     return (
2838         f"Voce E o cliente {row['cliente_id']}. Responda SEMPRE
           em 1? pessoa.\n\n"
2839         f"Perfil financeiro:\n"
2840         f"- Renda: R$ {renda:,.0f}/mes | Score: {score:.0f} | {
           financeiro}\n"
2841         f"- Canal: {canal}\n"
2842         f"- {perfil_desc}\n\n"
2843         f"Personalidade (Big Five inferida dos dados):\n"
2844         f"  {b5_linha}\n"
2845         f"  Estilo: {estilo}\n\n"
2846         f"REGRAS:\n"
2847         f"- Voce E este cliente especifico ? nao um grupo ou
           arquetipo\n"
2848         f"- Respostas em 1? pessoa, coerentes com seu perfil
           financeiro e personalidade\n"
2849         f"- Deixe transparecer seu estilo: {estilo[:80]}..."
2850     )
2851
2852
2853     # ?? Cria o digital twin de um cliente
           ?????????????????????????????????????????????????????????
2854     def criar_digital_twin(row: pd.Series) -> dict:
2855         """
2856         Cria agente twin para um cliente individual.
2857         - Independente do K-Means: usa dados brutos diretamente
2858         - Personalidade Big Five inferida dos dados (sem rotulo de
           segmento)
2859         """
2860         doc          = _doc_twin(row)
2861         vs           = FAISS.from_documents([doc], embeddings)
2862         retriever    = vs.as_retriever(search_kwargs={"k": 1})
2863         b5           = inferir_big5_cliente(row)
2864
2865         @StructuredTool.from_function
2866         def buscar_dados_pessoais(query: str) -> str:

```

```

2867     """Acessa o perfil financeiro e historico do cliente.
2868         """
2869
2870     return "\n".join(d.page_content for d in retriever.
2871         invoke(query))
2872
2873
2874     agente = create_react_agent(
2875         model=ChatAnthropic(model="claude-haiku-4-5-20251001",
2876             temperature=0.4),
2877         tools=[buscar_dados_pessoais],
2878         checkpointer=MemorySaver(),
2879         prompt=_prompt_twin(row),
2880     )
2881
2882     return {
2883         "cliente_id": row["cliente_id"],
2884         "big5": b5,
2885         "estilo": big5_para_estilo(b5),
2886         "agente": agente,
2887     }
2888
2889
2890 # ?? Demo: cria twins e consulta reacao a uma oferta
2891     """
2892
2893     clientes_demo_twin = df_clientes.sample(3, random_state=7).
2894         reset_index(drop=True)
2895
2896
2897     print("Digital Twins criados (personalidade Big Five sem
2898         clustering):\n")
2899
2900     twins = []
2901     for _, row in clientes_demo_twin.iterrows():
2902         tw = criar_digital_twin(row)
2903         twins.append((row, tw))
2904         b5 = tw["big5"]
2905         print(f" {row['cliente_id']} | Score {row['score_credito']:.0f} | Renda R${row['renda_mensal']:.0f}")
2906         b5_str = " | ".join(f"{t[0].upper()}:{b5[t]:.2f}" for t in
2907             OCEAN_TRAITS)
2908         print(f" OCEAN: {b5_str}")
2909         print(f" Estilo: {tw['estilo'][:75]}...")
2910         print()
2911
2912 # Simula reacao a uma oferta de credito

```



```

2900 oferta = "O banco tem uma oferta de emprestimo pessoal pre-
        aprovado de R$ 10.000. O que voce acha?"
2901 print(f"\nOferta simulada: '{oferta}'\n{'='*65}")
2902
2903 for row, tw in twins:
2904     config = {"configurable": {"thread_id": f"twin-{row['
        cliente_id']}"}}
2905     result = tw["agente"].invoke(
2906         {"messages": [{"role": "user", "content": oferta}]},
2907         config=config,
2908     )
2909     print(f"\nTwin {row['cliente_id']} (N:{tw['big5']['
        neuroticism']:.2f} C:{tw['big5']['conscientiousness']:.2
        f})")
2910     print("-" * 40)
2911     print(result["messages"][-1].content[:350])
2912     print("...")
2913
2914 # ===== cell_064 =====
2915 # ??? 7.5 Personas: Arquetipos Nomeados de Segmento
        ?????????????????????????????????
2916 # Persona = arquetipo do cluster, em 1? pessoa, com
        personalidade Big Five
2917 # derivada do perfil medio do grupo (lido de perfis_clientes,
        via Athena/Glue).
2918 from langchain_core.tools import tool
2919
2920 # ?? Prompt da persona com personalidade Big Five
        ?????????????????????????????????
2921 def _prompt_persona(cluster_id: int, perfil: pd.Series) -> str:
2922     """
2923     Gera o prompt da persona. Todos os atributos vem de perfil
        (Athena/Glue):
2924     - persona_nome, persona_ocupacao, persona_canal,
        persona_contexto
2925     - big5_* ? estilo comportamental derivado do perfil
        estatistico do cluster
2926
2927     Nao ha lookup em dicionario local ? qualquer nome definido
        no Glue funciona.
2928     """

```

```

2929     segmento = str(perfil.get("segmento",      f"Cluster {
           cluster_id}"))
2930     nome      = str(perfil.get("persona_nome",  f"Cliente {
           cluster_id}"))
2931     ocupacao  = str(perfil.get("persona_ocupacao", "profissional
           "))
2932     canal     = str(perfil.get("persona_canal",  "uso os
           canais do banco"))
2933     contexto  = str(perfil.get("persona_contexto", ""))
2934     renda     = float(perfil["renda_media"])
2935     score     = float(perfil["score_medio"])
2936
2937     # Big Five do cluster (lidos de perfis_clientes apos celula
           7.1b)
2938     b5 = {t: float(perfil.get(f"big5_{t}", 0.5)) for t in
           OCEAN_TRAITS}
2939     estilo    = big5_para_estilo(b5)
2940     b5_linha  = " | ".join(f"{t[0].upper()}:{b5[t]:.2f}" for t
           in OCEAN_TRAITS)
2941
2942     return (
2943         f"Voce e {nome}, {ocupacao}. "
2944         f"Voce representa o cliente tipico do segmento '{
           segmento}' do banco.\n\n"
2945         f"Perfil medio do seu grupo: Renda R$ {renda:,.0f}/mes
           | Score {score:.0f}\n"
2946         f"Canal: {canal}\n"
2947         f"Contexto pessoal: {contexto}\n\n"
2948         f"Personalidade Big Five do seu grupo:\n"
2949         f"  {b5_linha}\n"
2950         f"  Estilo: {estilo}\n\n"
2951         f"REGRAS:\n"
2952         f"- Responda SEMPRE em 1ª pessoa, como {nome}\n"
2953         f"- Voce e um arquetipo do segmento '{segmento}' ? nao
           um cliente especifico\n"
2954         f"- Represente as opinioes tipicas deste grupo\n"
2955         f"- Deixe transparecer sua personalidade: {estilo
           [:80]}...\n"
2956         f"- Use o RAG para embasar suas respostas sobre
           produtos e decisoes"
2957     )

```

```

2958
2959
2960 # ?? Cria agente-persona para um cluster
      ?????????????????????????????????????????
2961 def _criar_persona(cluster_id: int, perfil: pd.Series,
      agente_info: dict) -> dict:
2962     """
2963     Reutiliza o vector store do agente de segmento.
2964     Prompt em 1ª pessoa com Big Five do cluster (fonte: Athena/
      Glue via perfis_clientes).
2965     """
2966     vs = agente_info["vector_store"]
2967     retriever = vs.as_retriever(search_kwargs={"k": 3})
2968
2969     @tool
2970     def buscar_perfil_persona(query: str) -> str:
2971         """Busca informacoes sobre o segmento para embasar as
      respostas da persona."""
2972         docs = retriever.invoke(query)
2973         return "\n---\n".join(d.page_content for d in docs)
2974
2975     prompt_persona = _prompt_persona(cluster_id, perfil)
2976     agente = create_react_agent(
2977         model=ChatAnthropic(model="claude-haiku-4-5-20251001",
      temperature=0.3),
2978         tools=[buscar_perfil_persona],
2979         checkpointer=MemorySaver(),
2980         prompt=prompt_persona,
2981     )
2982     b5 = {t: float(perfil.get(f"big5_{t}", 0.5)) for t in
      OCEAN_TRAITS}
2983     nome = str(perfil.get("persona_nome", f"Cliente {
      cluster_id}"))
2984     segmento = str(perfil.get("segmento", f"Cluster {
      cluster_id}"))
2985     return {"nome": nome, "segmento": segmento, "big5": b5, "
      agente": agente}
2986
2987
2988 # ?? Instancia personas para todos os clusters
      ?????????????????????????????????????????

```

```

2989 print("Criando personas (nomes, contexto e Big Five do Athena/
      Glue via perfis_clientes):\n")
2990
2991 personas = {}
2992 for cluster_id, perfil in perfis_clientes.iterrows():
2993     personas[cluster_id] = _criar_pessoa(cluster_id, perfil,
      agentes_clientes[cluster_id])
2994     b5 = personas[cluster_id]["big5"]
2995     b5_str = " | ".join(f"{t[0].upper()}:{b5[t]:.2f}" for t in
      OCEAN_TRAITS)
2996     print(f"    ok Cluster {cluster_id}: {perfil['segmento']}")
2997     print(f"        Persona: {perfil.get('persona_nome','?')} |
      OCEAN: {b5_str}")
2998     print(f"        Estilo: {big5_para_estilo(b5)[:70]}...")
2999     print()
3000
3001
3002 # ?? Demo: consulta as personas
      ?????????????????????????????????????????????????????????
3003 pergunta_pessoa = "Como voce normalmente decide sobre um novo
      investimento ou emprestimo?"
3004 print(f"Pergunta: '{pergunta_pessoa}'\n{'='*65}")
3005
3006 for cluster_id, pessoa in personas.items():
3007     config = {"configurable": {"thread_id": f"pessoa-{
      cluster_id}"}}
3008     result = pessoa["agente"].invoke(
3009         {"messages": [{"role": "user", "content":
      pergunta_pessoa}]},
3010         config=config,
3011     )
3012     b5 = pessoa["big5"]
3013     print(f"\nPersona: {pessoa['nome']} ({pessoa['segmento']})")
3014     print(f"    C:{b5['conscientiousness']:.2f} N:{b5['
      neuroticism']:.2f} O:{b5['openness']:.2f}")
3015     print("-" * 40)
3016     print(result["messages"][-1].content[:350])
3017     print("...")
3018
3019 # ===== cell_066 =====

```

```

3020 # ??? 7.6 LLM-as-Judge Big Five
      ?????????????????????????????????????????????????????????
3021 import json
3022 from langchain_anthropic import ChatAnthropic
3023
3024 llm_juiz = ChatAnthropic(model="claude-haiku-4-5-20251001",
      temperature=0)
3025
3026 # ?? Prompt do juiz primario
      ?????????????????????????????????????????????????????????
3027 _PROMPT_JUIZ = """Voce e um avaliador especializado em
      psicologia da personalidade (Big Five / OCEAN).
3028
3029 Analise a resposta abaixo e estime os tracos Big Five que ela
      expressa IMPLICITAMENTE
3030 (pelo tom, escolhas lexicais, raciocinio e emocoes presentes ?
      nao pelo conteudo literal).
3031
3032 Escala: 0.0 (traco ausente/oposto) a 1.0 (traco muito presente/
      dominante).
3033
3034 RESPOSTA PARA AVALIAR:
3035 {resposta}
3036
3037 Retorne APENAS JSON valido, sem markdown:
3038 {{
3039     "openness":          <float>,
3040     "conscientiousness": <float>,
3041     "extraversion":      <float>,
3042     "agreeableness":     <float>,
3043     "neuroticism":       <float>,
3044     "justificativa": {{
3045         "openness":          "<evidencia na resposta>",
3046         "conscientiousness": "<evidencia na resposta>",
3047         "extraversion":      "<evidencia na resposta>",
3048         "agreeableness":     "<evidencia na resposta>",
3049         "neuroticism":       "<evidencia na resposta>"
3050     }}
3051 }} """
3052
3053

```

```

3054 def avaliar_big5(resposta: str, big5_esperado: dict, llm=
    llm_juiz) -> dict:
3055     """
3056     Juiz primario: extrai Big Five da resposta e calcula desvio
        em relacao ao esperado.
3057
3058     Retorna:
3059         big5_observado    ? tracos extraidos da resposta pelo LLM
3060         desvio            ? |esperado - observado| por traco
3061         score_coerencia   ? 1 - mean(desvios) ? [0, 1]; 1 =
        perfeita coerencia
3062         justificativa     ? evidencias do LLM para cada traco
3063     """
3064     prompt_fmt = _PROMPT_JUIZ.format(resposta=resposta[:1500])
3065     raw = llm.invoke(prompt_fmt).content.strip()
3066
3067     # Limpa possivel markdown fence
3068     if raw.startswith("`"):
3069         raw = raw.split("`")[1]
3070         if raw.startswith("json"):
3071             raw = raw[4:]
3072
3073     parsed = json.loads(raw)
3074     big5_obs = {t: float(parsed[t]) for t in OCEAN_TRAITS}
3075     justificativa = parsed.get("justificativa", {})
3076
3077     desvio = {t: round(abs(big5_esperado[t] - big5_obs[t]), 3)
        for t in OCEAN_TRAITS}
3078     score = round(1 - sum(desvio.values()) / len(OCEAN_TRAITS)
        , 3)
3079
3080     return {
3081         "big5_esperado": big5_esperado,
3082         "big5_observado": big5_obs,
3083         "desvio": desvio,
3084         "score_coerencia": score,
3085         "justificativa": justificativa,
3086     }
3087
3088
3089 def exibir_avaliacao(nome: str, resultado: dict) -> None:

```

```

3090     """Exibe o resultado do juiz de forma legivel."""
3091     print(f"\n{'='*60}")
3092     print(f"    Avaliacao Big Five: {nome}")
3093     print(f"    Score de coerencia: {resultado['score_coerencia']:.3f} (1.0 = perfeito)")
3094     print(f"{'='*60}")
3095     print(f"    {'Traco':<20} {'Esperado':>8} {'Observado':>10} {'Desvio':>8}")
3096     print(f"    {'-'*50}")
3097     for t in OCEAN_TRAITS:
3098         esp = resultado["big5_esperado"][t]
3099         obs = resultado["big5_observado"][t]
3100         dev = resultado["desvio"][t]
3101         flag = " <!" if dev > 0.25 else ""
3102         print(f"    {t:<20} {esp:>8.2f} {obs:>10.2f} {dev:>8.3f}{flag}")
3103     print(f"\n    Justificativas do juiz:")
3104     for t, ev in resultado["justificativa"].items():
3105         print(f"        {t}: {str(ev)[:80]}")
3106
3107
3108 # ?? Demo: avalia uma persona
3109     """
3110     """
3111 # Pega a resposta da persona do cluster 0 gerada na celula 7.5
3112 cluster_demo = 0
3113 perfil_demo = perfis_clientes.loc[cluster_demo]
3114 b5_esperado = {t: float(perfil_demo[f"big5_{t}"])} for t in OCEAN_TRAITS}
3115
3116 config_demo = {"configurable": {"thread_id": "persona-eval-demo"}}
3117 resp_persona = personas[cluster_demo]["agente"].invoke(
3118     {"messages": [{"role": "user", "content":
3119         "Me explique como voce avaliaria um CDB de 12% ao ano
3120         comparado ao Tesouro Direto IPCA+."}]},
3121     config=config_demo,
3122 )["messages"][-1].content
3123
3124 print(f"Persona avaliada: {personas[cluster_demo]['nome']} ({
3125     perfil_demo['segmento']})")
3126 print(f"Resposta (trecho): {resp_persona[:200]}...")

```

```

3123
3124 resultado_juiz = avaliar_big5(resp_persona, b5_esperado)
3125 exibir_avaliacao(personas[cluster_demo]["nome"], resultado_juiz
    )
3126
3127 # ===== cell_068 =====
3128 # ??? 7.7 Meta-Juiz com Few-Shot
    ?????????????????????????????????????????????????????????????
3129 # O meta-juiz calibra as avaliacoes do juiz primario usando
    exemplos de referencia.
3130 # Cada exemplo contem: resposta, big5_esperado,
    avaliacao_errada do juiz, correcao.
3131
3132 # ?? Exemplos de referencia (few-shot)
    ?????????????????????????????????????????????????????????????
3133 # Cada exemplo documenta um padrao de erro tipico com a
    correcao e o motivo.
3134 FEW_SHOT_META_JUIZ = [
3135     {
3136         "resposta": (
3137             "Antes de qualquer decisao, faco uma planilha
                comparando todas as taxas. "
3138             "Li todos os contratos palavra por palavra. So
                assino quando tenho certeza "
3139             "absoluta de que entendi cada clausula e calculei
                cada cenario possivel."
3140         ),
3141         "big5_esperado": {
3142             "openness": 0.35, "conscientiousness": 0.90,
3143             "extraversion": 0.25, "agreeableness": 0.55, "
                neuroticism": 0.20,
3144         },
3145         "avaliacao_juiz": {
3146             "openness": 0.72, "conscientiousness": 0.88,
3147             "extraversion": 0.25, "agreeableness": 0.55, "
                neuroticism": 0.20,
3148         },
3149         "avaliacao_correta": {
3150             "openness": 0.35, "conscientiousness": 0.90,
3151             "extraversion": 0.25, "agreeableness": 0.55, "
                neuroticism": 0.20,

```



```

3152     },
3153     "correcoes": {
3154         "openness": (
3155             "Erro: juiz confundiu analise metodica com
              abertura a novidades. "
3156             "Fazer planilhas e ler contratos indica
              CONSCIENTIOUSNESS alta, nao curiosidade (
              openness). "
3157             "Openness seria: experimentar produtos novos,
              aceitar risco de inovacao."
3158         )
3159     },
3160 },
3161 {
3162     "resposta": (
3163         "Estou tentando pagar o minimo do cartao todo mes,
              mas fico ansioso "
3164         "quando vejo as faturas. Prefiro nao pegar novos
              emprestimos ? "
3165         "tenho medo de piorar minha situacao. Cada compra
              que faco me preocupa."
3166     ),
3167     "big5_esperado": {
3168         "openness": 0.30, "conscientiousness": 0.45,
3169         "extraversion": 0.30, "agreeableness": 0.60, "
              neuroticism": 0.78,
3170     },
3171     "avaliacao_juiz": {
3172         "openness": 0.30, "conscientiousness": 0.82,
3173         "extraversion": 0.30, "agreeableness": 0.60, "
              neuroticism": 0.35,
3174     },
3175     "avaliacao_correta": {
3176         "openness": 0.30, "conscientiousness": 0.45,
3177         "extraversion": 0.30, "agreeableness": 0.60, "
              neuroticism": 0.78,
3178     },
3179     "correcoes": {
3180         "conscientiousness": (
3181             "Erro: pagar o minimo e evitar gastos NAO
              indica conscienciosidade alta ? "

```

```

3182         "e comportamento defensivo sob ansiedade
3183             financeira. Conscientiousness alta seria: "
3184         "planejamento proativo, pagamento integral,
3185             metas financeiras claras."
3186     ),
3187     "neuroticism": (
3188         "Erro: ansiedade explicita ('fico ansioso', '
3189             tenho medo', 'me preocupa') "
3190         "e sinal claro de NEUROTICISM alto ? nao baixo.
3191             O juiz subestimou."
3192     ),
3193 },
3194 {
3195     "resposta": (
3196         "Sinceramente, nem leio os detalhes. Se o app
3197             aprovar na hora, to dentro. "
3198         "Ja peguei credito por impulso algumas vezes e me
3199             arrependi, mas e dificil resistir "
3200         "quando aparece uma oferta boa. Prefiro resolver
3201             rapido e nao pensar muito."
3202     ),
3203     "big5_esperado": {
3204         "openness": 0.68, "conscientiousness": 0.12,
3205         "extraversion": 0.72, "agreeableness": 0.42, "
3206             neuroticism": 0.50,
3207     },
3208     "avaliacao_juiz": {
3209         "openness": 0.68, "conscientiousness": 0.12,
3210         "extraversion": 0.72, "agreeableness": 0.42, "
3211             neuroticism": 0.50,
3212     },
3213     "avaliacao_correta": {
3214         "openness": 0.68, "conscientiousness": 0.12,
3215         "extraversion": 0.72, "agreeableness": 0.42, "
3216             neuroticism": 0.50,
3217     },
3218     "correcoes": {}, # Avaliacao correta ? sem correcoes
3219                     necessarias
3220 },
3221 ]

```

```

3212
3213
3214 def _formatar_few_shot(exemplos: list) -> str:
3215     """Formata os exemplos few-shot para o prompt do meta-juiz.
3216         """
3217     blocos = []
3218     for i, ex in enumerate(exemplos, 1):
3219         corr_str = (
3220             json.dumps(ex["correcoes"], ensure_ascii=False,
3221                 indent=2)
3222             if ex["correcoes"]
3223             else '{"observacao": "avaliacao_juiz estava correta
3224                 ? sem correcoes"}'
3225         )
3226         blocos.append(
3227             f"--- EXEMPLO {i} ---\n"
3228             f"RESPOSTA: {ex['resposta']}\n"
3229             f"BIG5_ESPERADO: {json.dumps(ex['big5_esperado'])}\n"
3230             f"AVALIACAO_JUIZ: {json.dumps(ex['avaliacao_juiz'])}\n"
3231             f"AVALIACAO_CORRETA: {json.dumps(ex['avaliacao_correta'])}\n"
3232             f"CORRECOES_E_MOTIVOS: {corr_str}\n"
3233         )
3234     return "\n".join(blocos)
3235
3236 # ?? Template do meta-juiz
3237     ?????????????????????????????????????????????????????????????????????
3238 _PROMPT_META_JUIZ = """Voce e um meta-avaliador especializado
3239     em calibrar avaliacoes de personalidade Big Five.
3240 Sua funcao e detectar e corrigir erros do juiz primario, usando
3241     os exemplos de referencia abaixo.
3242
3243 EXEMPLOS DE REFERENCIA (few-shot):
3244 {few_shot}
3245
3246 ---
3247 NOVA AVALIACAO PARA CALIBRAR:

```

```

3244 RESPOSTA DO AGENTE:
3245 {resposta}
3246
3247 PERFIL BIG FIVE ESPERADO:
3248 {big5_esperado}
3249
3250 AVALIACAO DO JUIZ PRIMARIO:
3251 {avaliacao_juiz}
3252
3253 Analise se o juiz errou em algum traco (compare com os padroes
      dos exemplos).
3254 Retorne APENAS JSON valido:
3255 {{
3256     "avaliacao_calibrada": {{
3257         "openness": <float>, "conscientiousness": <float>,
3258         "extraversion": <float>, "agreeableness": <float>, "
            neuroticism": <float>
3259     }},
3260     "correcoes_aplicadas": {{
3261         "<traco_corrigido>": "<motivo da correcao baseado nos
            exemplos>"
3262     }},
3263     "confianca": <float 0-1>,
3264     "feedback_ao_juiz": "<instrucao concisa para o juiz melhorar
            avaliacoes futuras deste tipo>"
3265 }}""
3266
3267
3268 def criar_meta_juiz(llm, few_shot_examples: list):
3269     """
3270     Fabrica do meta-juiz. Retorna uma funcao que calibra
            avaliacoes do juiz primario.
3271
3272     O meta-juiz:
3273         1. Recebe resposta + avaliacao_juiz
3274         2. Consulta few-shot para detectar padroes de erro
3275         3. Produz avaliacao_calibrada + feedback_ao_juiz
3276
3277     O feedback_ao_juiz pode ser injetado no prompt do juiz
            primario para

```

```

3278     criar um loop de melhoria continua (auto-calibracao
3279         supervisionada).
3280
3281     """
3282     few_shot_str = _formatar_few_shot(few_shot_examples)
3283
3284     def meta_avaliar(resposta: str, big5_esperado: dict,
3285         avaliacao_juiz: dict) -> dict:
3286         prompt = _PROMPT_META_JUIZ.format(
3287             few_shot=few_shot_str,
3288             resposta=resposta[:800],
3289             big5_esperado=json.dumps(big5_esperado),
3290             avaliacao_juiz=json.dumps(avaliacao_juiz),
3291         )
3292         raw = llm.invoke(prompt).content.strip()
3293         if raw.startswith("```"):
3294             raw = raw.split("```")[1]
3295             if raw.startswith("json"):
3296                 raw = raw[4:]
3297
3298         resultado = json.loads(raw)
3299         cal = resultado["avaliacao_calibrada"]
3300
3301         # Recalcula o score de coerencia com a avaliacao
3302             calibrada
3303         desvio_cal = {t: round(abs(big5_esperado[t] - cal[t]),
3304             3) for t in OCEAN_TRAITS}
3305         score_cal = round(1 - sum(desvio_cal.values()) / len(
3306             OCEAN_TRAITS), 3)
3307
3308         return {
3309             "avaliacao_calibrada": cal,
3310             "desvio_calibrado": desvio_cal,
3311             "score_calibrado": score_cal,
3312             "correcoes_aplicadas": resultado.get("
3313                 correcoes_aplicadas", {}),
3314             "confianca": float(resultado.get("
3315                 confianca", 0.5)),
3316             "feedback_ao_juiz": resultado.get("
3317                 feedback_ao_juiz", ""),
3318         }

```

```

3311     return meta_avaliar
3312
3313
3314 # ?? Instancia o meta-juiz
3315     ?????????????????????????????????????????????????????????????????????
3316 meta_juiz = criar_meta_juiz(llm_juiz, FEW_SHOT_META_JUIZ)
3317
3318 # ?? Pipeline completo: juiz ? meta-juiz ? comparacao
3319     ?????????????????????????????????????????????
3320 def avaliar_pipeline_completo(nome: str, resposta: str,
3321     big5_esperado: dict) -> dict:
3322     """
3323     Executa juiz primario ? meta-juiz e exibe a comparacao lado
3324     a lado.
3325     """
3326     # Etapa 1: juiz primario
3327     result_juiz = avaliar_big5(resposta, big5_esperado)
3328
3329     # Etapa 2: meta-juiz calibra
3330     result_meta = meta_juiz(resposta, big5_esperado,
3331         result_juiz["big5_observado"])
3332
3333     # Exibicao comparativa
3334     print(f"\n{'='*65}")
3335     print(f"  Pipeline de Avaliacao Big Five: {nome}")
3336     print(f"{'='*65}")
3337     print(f"  {'Traco':<20} {'Esperado':>8} {'Juiz':>8} {'Meta-
3338         Juiz':>10} {'Dev.Juiz':>9} {'Dev.Meta':>9}")
3339     print(f"  {'-'*65}")
3340     for t in OCEAN_TRAITS:
3341         esp      = big5_esperado[t]
3342         obs_j    = result_juiz["big5_observado"][t]
3343         obs_m    = result_meta["avaliacao_calibrada"][t]
3344         dev_j    = result_juiz["desvio"][t]
3345         dev_m    = result_meta["desvio_calibrado"][t]
3346         melhor = " +" if dev_m < dev_j else (" " if dev_m ==
3347             dev_j else " -")
3348         print(f"  {t:<20} {esp:>8.2f} {obs_j:>8.2f} {obs_m
3349             :>10.2f} {dev_j:>9.3f} {dev_m:>9.3f}{melhor}")
3350

```

```

3344     print(f"\n  Score coerencia ? Juiz:      {result_juiz['
          score_coerencia']:.3f}")
3345     print(f"  Score coerencia ? Meta-juiz: {result_meta['
          score_calibrado']:.3f}  (confianca: {result_meta['
          confianca']:.2f})")
3346
3347     if result_meta["correcoes_aplicadas"]:
3348         print(f"\n  Correcoes aplicadas pelo meta-juiz:")
3349         for t, motivo in result_meta["correcoes_aplicadas"].
            items():
3350             print(f"      [{t}] {motivo[:100]}")
3351
3352     print(f"\n  Feedback para o juiz primario:")
3353     print(f"      {result_meta['feedback_ao_juiz'][:120]}")
3354
3355     return {"juiz": result_juiz, "meta": result_meta}
3356
3357
3358     # ?? Demo: avalia todas as personas com o pipeline completo
3359     ?????????????????????????????
3359     pergunta_avaliacao = (
3360         "Como voce se sente quando o banco lanca um produto
3361         financeiro novo? "
3362         "Voce costuma experimentar ou prefere esperar?"
3363     )
3364
3364     print("PIPELINE JUIZ + META-JUIZ: avaliando coerencia Big Five
          das personas\n")
3365
3366     feedbacks_acumulados = []
3367     for cluster_id, persona in personas.items():
3368         perfil = perfis_clientes.loc[cluster_id]
3369         b5_esp = {t: float(perfil[f"big5_{t}"]) for t in
            OCEAN_TRAITS}
3370         config = {"configurable": {"thread_id": f"eval-meta-{
            cluster_id}"}}
3371         resposta = persona["agente"].invoke(
3372             {"messages": [{"role": "user", "content":
                pergunta_avaliacao}]},
3373             config=config,
3374             ["messages"][-1].content

```

```

3375
3376     resultado = avaliar_pipeline_completo(
3377         f"{persona['nome']} ({persona['segmento']})", resposta,
3378         b5_esp
3379     )
3380     feedbacks_acumulados.append(resultado["meta"]["
3381         feedback_ao_juiz"])
3382
3383 # ?? Loop de melhoria: mostra como os feedbacks podem refinar o
3384     juiz ????????????
3385 print(f"\n{'='*65}")
3386 print("    FEEDBACKS ACUMULADOS ? base para refinar o juiz
3387     primario")
3388 print(f"{'='*65}")
3389 for i, fb in enumerate(feedbacks_acumulados, 1):
3390     print(f"    [{i}] {fb[:110]}")
3391 print()
3392 print("    Estes feedbacks podem ser injetados no _PROMPT_JUIZ
3393     como instrucoes")
3394 print("    adicionais, criando um loop de calibracao continua
3395     supervisionada.")
3396
3397 # ===== cell_071 =====
3398 # ??? 8.1 Demo: Cliente assincrono com polling
3399     ?????????????????????????????????????????????????????????
3400 # Este codigo simula o comportamento do cliente consumindo a
3401     API assincrona.
3402 # Em producao, use requests ou httpx.
3403
3404 import asyncio
3405 import httpx
3406 from typing import Dict, Any
3407
3408 class ClienteAgenteAsync:
3409     """
3410     Cliente Python para a arquitetura assincrona AWS (
3411         Controller + Worker + Status).
3412
3413     Uso:
3414
3415         cliente = ClienteAgenteAsync("https://api.exemplo.com")
3416         resultado = await cliente.consultar(payload)

```



```
3407     """
3408     def __init__(self, base_url: str, max_wait: int = 120):
3409         self.base_url = base_url.rstrip("/")
3410         self.max_wait = max_wait
3411
3412     async def consultar(self, payload: Dict[str, Any]) -> Dict[
3413         str, Any]:
3414         """
3415         Envia requisicao e aguarda resultado via polling.
3416
3417         Fluxo:
3418         1. POST /query ? recebe request_id
3419         2. GET /status/{id} em loop com backoff exponencial
3420         3. Retorna resultado quando status = COMPLETED
3421         """
3422         async with httpx.AsyncClient(timeout=30) as client:
3423             # Passo 1: envia requisicao
3424             resp = await client.post(f"{self.base_url}/query",
3425                                     json=payload)
3426             resp.raise_for_status()
3427             data = resp.json()
3428
3429             request_id = data["request_id"]
3430             print(f"? Processamento iniciado: {request_id}")
3431             print(f"    Status inicial: {data['status']}")
3432             print(f"    Tempo estimado: {data.get('
3433                 estimated_time', 'N/A')}\n")
3434
3435             # Passo 2: polling com backoff exponencial
3436             wait_time = 1
3437             elapsed = 0
3438
3439             while elapsed < self.max_wait:
3440                 await asyncio.sleep(wait_time)
3441                 elapsed += wait_time
3442
3443                 # Consulta status
3444                 resp = await client.get(f"{self.base_url}/
3445                     status/{request_id}")
3446                 resp.raise_for_status()
3447                 data = resp.json()
```

```

3444         status = data["status"]
3445
3446         print(f"? [{elapsed:>3}s] Status: {status}")
3447
3448         if status == "COMPLETED":
3449             print(f"\n? Processamento concluido!")
3450             print(f"Tempo total: {elapsed}s")
3451             return data
3452         elif status == "FAILED":
3453             erro = data.get("error", "Erro desconhecido")
3454             raise Exception(f"? Falha no processamento: {erro}")
3455
3456         # Backoff exponencial: 1s ? 2s ? 4s ? 8s ? 10s (maximo)
3457         wait_time = min(wait_time * 2, 10)
3458
3459         raise TimeoutError(f"? Timeout apos {self.max_wait}s. Request ID: {request_id}")
3460
3461
3462     # ?? Simulacao local sem infraestrutura AWS (mock)
3463     ????????????????????????????????????????
3464     # Este mock simula o comportamento da API assincrona para fins didaticos.
3465     # Em producao real, os dados viriam das Lambdas + DynamoDB.
3466
3467     class MockAPIAsync:
3468         """Mock da API assincrona para demonstracao local (sem AWS)"""
3469
3470         def __init__(self):
3471             self.requests = {}
3472
3473         def post_query(self, payload: Dict) -> Dict:
3474             """Simula POST /query (Lambda Controller)."""
3475             import uuid
3476             request_id = f"req_{uuid.uuid4().hex[:12]}"
3477             self.requests[request_id] = {
3478                 "status": "PENDING",
3479                 "payload": payload,

```

```

3478         "created_at": "2026-03-09T10:00:00Z"
3479     }
3480     return {
3481         "request_id": request_id,
3482         "status": "PENDING",
3483         "message": f"Processamento iniciado. Consulte GET /
3484                     status/{request_id}",
3485         "estimated_time": "30-120 segundos"
3486     }
3487
3488     async def get_status(self, request_id: str, elapsed_time:
3489                          int) -> Dict:
3489         """Simula GET /status/{id} (Lambda Status + DynamoDB).
3490         """
3491         if request_id not in self.requests:
3492             return {"error": "Request ID nao encontrado"}, 404
3493
3494         req = self.requests[request_id]
3495
3496         # Simula estados ao longo do tempo
3497         if elapsed_time < 3:
3498             req["status"] = "PENDING"
3499         elif elapsed_time < 8:
3500             req["status"] = "PROCESSING"
3501             req["updated_at"] = "2026-03-09T10:00:05Z"
3502         else:
3503             req["status"] = "COMPLETED"
3504             req["resposta"] = (
3505                 f"Recomendo os seguintes produtos para o
3506                 cliente {req['payload']['cliente_id']}: \n"
3507                 f"1. Cartao de credito com limite adequado ao
3508                 seu score ({req['payload']['dados_cliente']
3509                             ['score_credito']}) \n"
3509                 f"2. Investimento em Tesouro Direto IPCA+ (
3510                 baixo risco, protecao inflacionaria) \n"
3511                 f"3. Consorcio de imoveis sem juros \n \n"
3512                 f"Baseado no seu perfil: Renda R$ {req['payload']
3513                             ['dados_cliente']['renda_mensal']} /mes, "
3514                 f"Saldo R$ {req['payload']['dados_cliente']['
3515                             saldo_medio']}. "
3516             )
3517
3518     )

```

```

3510         req["updated_at"] = "2026-03-09T10:00:15Z"
3511
3512     return {
3513         "request_id": request_id,
3514         "status": req["status"],
3515         "created_at": req["created_at"],
3516         "updated_at": req.get("updated_at"),
3517         "resposta": req.get("resposta")
3518     }
3519
3520
3521 # ?? Demo: teste do cliente assincrono com mock
3522     ?????????????????????????????????????????????
3523 print("="*70)
3524 print("DEMO: Cliente Assincrono com Polling (Mock Local)")
3525 print("="*70)
3526 print()
3527
3528 # Mock da API
3529 mock_api = MockAPIAsync()
3530
3531 # Override do cliente para usar o mock em vez de HTTP real
3532 class ClienteMockAsync:
3533     def __init__(self, mock_api: MockAPIAsync, max_wait: int =
3534         30):
3535         self.mock = mock_api
3536         self.max_wait = max_wait
3537
3538     async def consultar(self, payload: Dict[str, Any]) -> Dict[
3539         str, Any]:
3540         # POST /query
3541         data = self.mock.post_query(payload)
3542         request_id = data["request_id"]
3543
3544         print(f"? Processamento iniciado: {request_id}")
3545         print(f"    Status inicial: {data['status']}")
3546         print(f"    Tempo estimado: {data.get('estimated_time',
3547             'N/A')}\\n")
3548
3549         # Polling
3550         wait_time = 1

```

```
3547         elapsed = 0
3548
3549         while elapsed < self.max_wait:
3550             await asyncio.sleep(wait_time)
3551             elapsed += wait_time
3552
3553             # GET /status/{id}
3554             data = await self.mock.get_status(request_id,
3555                                               elapsed)
3556             status = data["status"]
3557
3558             print(f"? [{elapsed:>3}s] Status: {status}")
3559
3560             if status == "COMPLETED":
3561                 print(f"\n? Processamento concluido! Tempo
3562                       total: {elapsed}s\n")
3563                 print("RESPOSTA DO AGENTE:")
3564                 print("-" * 70)
3565                 print(data["resposta"])
3566                 print("-" * 70)
3567                 return data
3568             elif status == "FAILED":
3569                 raise Exception(f"? Falha: {data.get('error')}")
3570
3571             wait_time = min(wait_time * 2, 10)
3572
3573             raise TimeoutError(f"? Timeout apos {self.max_wait}s")
3574
3575 # Executa demo
3576 cliente = ClienteMockAsync(mock_api, max_wait=30)
3577
3578 payload_teste = {
3579     "cliente_id": "C12345",
3580     "dados_cliente": {
3581         "idade": 35,
3582         "renda_mensal": 5000,
3583         "saldo_medio": 8000,
3584         "transacoes_mes": 15,
3585         "score_credito": 680,
```

```

3585         "num_produtos": 3
3586     },
3587     "pergunta": "Quais produtos voce recomenda para melhorar
                 minha situacao financeira?",
3588     "modo": "segmento"
3589 }
3590
3591 # Jupyter ja tem event loop ativo, entao await funciona
    diretamente
3592 resultado = await cliente.consultar(payload_teste)
3593
3594 print("\n" + "="*70)
3595 print("? DEMO CONCLUIDA")
3596 print("="*70)
3597 print(f"\nEm producao, substitua ClienteMockAsync por
        ClienteAgenteAsync")
3598 print(f"com a URL real da API Gateway AWS:")
3599 print(f"  cliente = ClienteAgenteAsync('https://abc123.execute-
        api.us-east-1.amazonaws.com/prod')")
3600 print(f"  resultado = await cliente.consultar(payload)")
3601 print()
3602 print("?? Documentacao completa: MIGRATION_ASYNC.md")
3603 print("?? Infraestrutura: terraform_async_infra.tf")
3604 print("??Codigo fonte:")
3605 print("    - lambda_controller.py (recebe requisicao, < 1s)")
3606 print("    - lambda_worker.py      (processa em background, ate
        15min)")
3607 print("    - lambda_status.py      (consulta status e resultado)"
        )

```

Listing A.2: Notebook base do laboratorio (codigo completo dos blocos, versao ASCII para compilacao).

A.2.2 demo_api.ipynb (exportado para Python)

```

1  # Arquivo gerado automaticamente a partir de demo_api.ipynb
2  # Conteudo completo dos blocos de codigo do notebook
3
4  # ===== cell_003 =====
5  import json
6  import time

```

```

7 import requests
8 from IPython.display import display, HTML, Markdown
9
10 API_URL = "https://3vqgrahlmyg.execute-api.sa-east-1.amazonaws.
    com"
11
12 # ?? Cores para o indice de confiabilidade
    ?????????????????????????????????????????
13 NIVEL_COR = {"alto": "#2d7a2d", "medio": "#c47a00", "baixo": "#
    c0392b"}
14 NIVEL_BG = {"alto": "#eafaea", "medio": "#fff8e1", "baixo": "#
    fdecea"}
15
16 def consultar(payload: dict, poll_interval: int = 3, timeout:
    int = 90) -> dict:
17     """Envia query e aguarda resultado (polling)."""
18     r = requests.post(f"{API_URL}/query", json=payload, timeout
        =30)
19     r.raise_for_status()
20     request_id = r.json()["request_id"]
21     print(f"    request_id: {request_id}")
22
23     for _ in range(timeout // poll_interval):
24         time.sleep(poll_interval)
25         status = requests.get(f"{API_URL}/status/{request_id}",
            timeout=10).json()
26         st = status["status"]
27         if st == "COMPLETED":
28             resultado = json.loads(status["resultado"])
29             resultado["_request_id"] = request_id
30             return resultado
31         if st == "FAILED":
32             raise RuntimeError(f"Worker falhou:\n{status.get('
                erro', '')}")
33             print(f"    {st}...", end="\r")
34
35         raise TimeoutError(f"Sem resposta em {timeout}s (request_id
            ={request_id})")
36
37
38 def exibir(resultado: dict) -> None:

```

```

39     """Renderiza resposta + indice de confiabilidade em HTML.
        """
40     modo      = resultado.get("modo", "")
41     resposta   = resultado.get("resposta", "")
42     ic         = resultado.get("indice_confiabilidade", {})
43     score      = ic.get("score", 0)
44     nivel      = ic.get("nivel", "baixo")
45     fontes     = ic.get("fontes_ativas", [])
46     cobertura  = ic.get("detalhes", {}).get("cobertura_fontes",
47                                             0)
48     overlap    = ic.get("detalhes", {}).get("
49                 sobreposicao_lexica", 0)
50
51     cor = NIVEL_COR[nivel]
52     bg  = NIVEL_BG[nivel]
53
54     fontes_html = "".join(
55         f'<span style="background:#dde;border-radius:4px;
56           padding:2px 7px;'
57         f'font-size:0.82em;margin-right:4px">{f}</span>'
58         for f in fontes
59     ) or '<span style="color:#999">nenhuma</span>'
60
61     barra_largura = int(score * 200)
62
63     html = f"""
64     <div style="font-family:sans-serif;max-width:860px;border:1
65         px solid #ddd;
66         border-radius:8px;overflow:hidden;margin:12px
67         0">
68
69     <div style="background:#2c3e50;color:#ecf0f1;padding:10px
70         16px;
71         font-size:0.85em;letter-spacing:.5px">
72         MOD0: <strong>{modo.upper()}</strong>
73         {'&nbsp;?&nbsp;'} + resultado.get('segmento', '') if
74             resultado.get('segmento') else ''
75         {'&nbsp;?&nbsp;'} + resultado.get('persona_nome', '') if
76             resultado.get('persona_nome') else ''
77         {'&nbsp;?&nbsp;'} + resultado.get('cliente_id', '') if
78             resultado.get('cliente_id') else ''

```



```

70     </div>
71
72     <div style="padding:16px 18px;white-space:pre-wrap;line-
73         height:1.6;
74             font-size:0.93em;border-bottom:1px solid #eee
75             ">{resposta}</div>
76
77     <div style="background:{bg};padding:10px 18px">
78         <div style="display:flex;align-items:center;gap:16px;
79             flex-wrap:wrap">
80             <div>
81                 <span style="font-size:1.5em;font-weight:700;color
82                     :{cor}">{score:.2f}</span>
83                 <span style="margin-left:6px;background:{cor};color
84                     :white;border-radius:4px;
85                     padding:2px 8px;font-size:0.78em;font-
86                     weight:600">{nivel.upper()}</span>
87             </div>
88             <div style="flex:1;min-width:160px">
89                 <div style="background:#ddd;border-radius:4px;
90                     height:8px">
91                     <div style="background:{cor};width:{barra_largura
92                         }px;max-width:200px;
93                         height:8px;border-radius:4px"></div>
94                     </div>
95                 </div>
96                 <div style="font-size:0.82em;color:#555">
97                     <strong>Fontes:</strong> {fontes_html}<br>
98                     <strong>Cobertura:</strong> {cobertura:.2f} &nbsp;
99                     <strong>Sobreposicao lexic:</strong> {overlap:.2f}
100                 </div>
101             </div>
102         </div>
103     </div>
104     """
105     display(HTML(html))
106
107 print("Setup concluido. API:", API_URL)
108
109 # ===== cell_005 =====
110 print("Enviando consulta modo segmento...")

```

```
103
104 resultado_segmento = consultar({
105     "cliente_id": "C001",
106     "dados_cliente": {
107         "idade": 52,
108         "renda_mensal": 18000,
109         "saldo_medio": 140000,
110         "transacoes_mes": 8,
111         "score_credito": 830,
112         "num_produtos": 5
113     },
114     "pergunta": "Que produto de investimento faz mais sentido
115     para este cliente agora?",
116     "modo": "segmento"
117 })
118
119
120 # ===== cell_007 =====
121 print("Enviando consulta modo persona...")
122
123 resultado_persona = consultar({
124     "cluster_id": 1,
125     "pergunta": "O banco lancou um cartao com cashback de 3% e
126     aprovacao pelo app. Voce toparia?",
127     "modo": "persona"
128 })
129
130
131 # ===== cell_009 =====
132 print("Enviando consulta modo twin...")
133
134 resultado_twin = consultar({
135     "cliente_id": "GRAPH-C00-0001",
136     "cluster_id": 0,
137     "dados_cliente": {
138         "idade": 43,
139         "renda_mensal": 4685,
140         "saldo_medio": 35000,
141         "transacoes_mes": 6,
```

```

142         "score_credito": 720,
143         "num_produtos": 3
144     },
145     "pergunta": "Considerando meu perfil e clientes parecidos
146                 comigo, que estrategia de previdencia faz sentido?",
147     "modo": "twin"
148 })
149
150
151
152 # ===== cell_011 =====
153 PERGUNTA = "O banco oferece um emprestimo pessoal de R$ 5.000
154            com taxa de 2,5% ao mes. Voce toparia?"
155
156
157 resultados_persona = {}
158 for cluster_id in range(4):
159     print(f" Consultando cluster {cluster_id}...")
160     resultados_persona[cluster_id] = consultar({
161         "cluster_id": cluster_id,
162         "pergunta": PERGUNTA,
163         "modo": "persona"
164     })
165
166
167 print("\nRespostas:")
168 for cluster_id, res in resultados_persona.items():
169     exibir(res)
170
171
172 # ===== cell_013 =====
173 def tabela_comparativa(resultados: list) -> None:
174     linhas = ""
175     for r in resultados:
176         modo = r.get("modo", "")
177         segmento = r.get("segmento") or r.get("persona_nome")
178                 or r.get("cliente_id", "")
179         ic = r.get("indice_confiabilidade", {})
180         score = ic.get("score", 0)
181         nivel = ic.get("nivel", "")
182         fontes = ", ".join(ic.get("fontes_ativas", [])) or "?"
183         cobertura = ic.get("detalhes", {}).get("cobertura_fontes", 0)

```

```

178     overlap    = ic.get("detalhes", {}).get("
        sobreposicao_lexica", 0)
179     cor        = NIVEL_COR[nivel]
180     bg        = NIVEL_BG[nivel]
181     barra     = int(score * 120)
182
183     linhas += f"""
184     <tr style="background:{bg}">
185         <td style="padding:8px 12px"><code>{modo}</code></td>
186         <td style="padding:8px 12px">{segmento}</td>
187         <td style="padding:8px 12px;text-align:center">
188             <strong style="color:{cor}">{score:.2f}</strong>
189             <div style="background:#ddd;border-radius:3px;
                height:6px;margin-top:3px">
190                 <div style="background:{cor};width:{barra}px;
                height:6px;border-radius:3px"></div>
191             </div>
192         </td>
193         <td style="padding:8px 12px;text-align:center">
194             <span style="background:{cor};color:white;border-
                radius:4px;
195                 padding:2px 8px;font-size:0.8em">{
                nivel.upper()}</span>
196         </td>
197         <td style="padding:8px 12px;font-size:0.82em">{fontes
            }</td>
198         <td style="padding:8px 12px;text-align:center">{
            cobertura:.2f}</td>
199         <td style="padding:8px 12px;text-align:center">{
            overlap:.2f}</td>
200     </tr>"""
201
202     html = f"""
203     <table style="border-collapse:collapse;width:100%;font-
        family:sans-serif;font-size:0.9em">
204     <thead>
205         <tr style="background:#2c3e50;color:white">
206             <th style="padding:10px 12px;text-align:left">Modo</
                th>
207             <th style="padding:10px 12px;text-align:left">
                Segmento / Persona</th>

```

```

208         <th style="padding:10px 12px;text-align:center">Score
           </th>
209         <th style="padding:10px 12px;text-align:center">Nivel
           </th>
210         <th style="padding:10px 12px;text-align:left">Fontes
           RAG ativas</th>
211         <th style="padding:10px 12px;text-align:center">
           Cobertura</th>
212         <th style="padding:10px 12px;text-align:center">
           Sobreposicao</th>
213     </tr>
214 </thead>
215 <tbody>{linhas}</tbody>
216 </table>""
217 display(HTML(html))
218
219
220 # Consolida resultados das secoes anteriores
221 todos = [
222     resultado_segmento,
223     resultado_persona,
224     resultado_twin,
225 ]
226
227 tabela_comparativa(todos)
228
229 # ===== cell_016 =====
230 print("Enviando consulta com cliente sem dados no indice (DEMO-
      X999)...")
231
232 resultado_sem_indice = consultar({
233     "cliente_id": "DEMO-X999",
234     "cluster_id": 1,
235     "dados_cliente": {
236         "idade": 28,
237         "renda_mensal": 3200,
238         "saldo_medio": 5000,
239         "transacoes_mes": 20,
240         "score_credito": 650,
241         "num_produtos": 2
242     },

```

```

243     "pergunta": "Devo aceitar o limite de credito pre-aprovado
244         de R$ 3.000?",
245     "modo": "twin"
246 })
247
248
249
250
251
252 # ===== cell_018 =====
253 import sys, os
254 sys.path.insert(0, os.getcwd())
255 from plantuml_helper import show_plantuml
256
257
258
259
260
261
262
263 # ===== cell_020 =====
264 # ?? Edite aqui
265
266
267
268
269
270
271
272
273
274
275
276

```

```

265 MODO          = "segmento"          # "segmento" / "persona" / "
    twin "
266 PERGUNTA      = "Como este cliente se comporta em relacao a
    investimentos de longo prazo?"
267 CLIENTE_ID    = "C002"              # usado em modo twin
268 CLUSTER_ID    = None                # None = classificar
    automaticamente pelos dados
269
270 DADOS_CLIENTE = {
271     "idade":          35,
272     "renda_mensal":   4500,
273     "saldo_medio":    8000,
274     "transacoes_mes": 15,
275     "score_credito":   680,
276     "num_produtos":   3,

```

```

277 }
278 #
279
280 payload = {"pergunta": PERGUNTA, "modo": MODO, "dados_cliente":
           DADOS_CLIENTE}
281 if CLIENTE_ID:
282     payload["cliente_id"] = CLIENTE_ID
283 if CLUSTER_ID is not None:
284     payload["cluster_id"] = CLUSTER_ID
285
286 print(f"Consultando modo {MODO!r}...")
287 resultado_livre = consultar(payload)
288 exhibir(resultado_livre)

```

Listing A.3: Notebook de validacao de API ponta a ponta (codigo completo dos blocos, versao ASCII para compilacao).

A.3 Capitulo 4 e Capitulo 5

A.3.1 mcp_mock_server.py

```
1  """
2
3  Servidor MCP Mock (para testes sem PostgreSQL)
4  Simula um banco de dados com dados de exemplo.
5  """
6
7  import asyncio
8  import json
9
10 from mcp.server import Server
11 from mcp.server.stdio import stdio_server
12 import mcp.types as types
13
14 server = Server("postgres-mock-server")
15
16 # Dados simulados
17 MOCK_DATA = {
18     "clientes": [
19         {"id": 1, "nome": "Ana Silva", "email": "ana@exemplo.
20         com", "cidade": "Sao Paulo", "saldo": 15000.00},
```

```
18     {"id": 2, "nome": "Bruno Costa", "email": "
19         bruno@exemplo.com", "cidade": "Rio de Janeiro", "
20         saldo": 8500.50},
21     {"id": 3, "nome": "Carla Mendes", "email": "
22         carla@exemplo.com", "cidade": "Curitiba", "saldo":
23         22000.00},
24     {"id": 4, "nome": "Diego Santos", "email": "
25         diego@exemplo.com", "cidade": "Sao Paulo", "saldo":
26         5200.75},
27 ],
28 "produtos": [
29     {"id": 1, "nome": "Notebook Pro", "categoria": "
30         Eletronicos", "preco": 4500.00, "estoque": 15},
31     {"id": 2, "nome": "Smartphone X", "categoria": "
32         Eletronicos", "preco": 2800.00, "estoque": 42},
33     {"id": 3, "nome": "Mesa Gamer", "categoria": "Moveis",
34         "preco": 1200.00, "estoque": 8},
35     {"id": 4, "nome": "Cadeira Ergonomica", "categoria": "
36         Moveis", "preco": 950.00, "estoque": 20},
37 ],
38 "pedidos": [
39     {"id": 1, "cliente_id": 1, "produto_id": 2, "quantidade
40         ": 1, "total": 2800.00, "status": "entregue"},
41     {"id": 2, "cliente_id": 2, "produto_id": 1, "quantidade
42         ": 1, "total": 4500.00, "status": "processando"},
43     {"id": 3, "cliente_id": 1, "produto_id": 4, "quantidade
44         ": 2, "total": 1900.00, "status": "entregue"},
45     {"id": 4, "cliente_id": 3, "produto_id": 3, "quantidade
46         ": 1, "total": 1200.00, "status": "enviado"},
47 ]
48 }
49
50 SCHEMAS = {
51     "clientes": [
52         {"coluna": "id", "tipo": "integer", "permite_nulo": "NO
53         "},
54         {"coluna": "nome", "tipo": "varchar", "permite_nulo": "
55         NO"},
56         {"coluna": "email", "tipo": "varchar", "permite_nulo":
57         "NO"},
58     ]
59 }
```



```
41         {"coluna": "cidade", "tipo": "varchar", "permite_nulo":
42           "YES"},
43     ],
44     "produtos": [
45         {"coluna": "id", "tipo": "integer", "permite_nulo": "NO"},
46         {"coluna": "nome", "tipo": "varchar", "permite_nulo": "NO"},
47         {"coluna": "categoria", "tipo": "varchar", "permite_nulo": "YES"},
48         {"coluna": "preco", "tipo": "numeric", "permite_nulo": "NO"},
49         {"coluna": "estoque", "tipo": "integer", "permite_nulo": "YES"},
50     ],
51     "pedidos": [
52         {"coluna": "id", "tipo": "integer", "permite_nulo": "NO"},
53         {"coluna": "cliente_id", "tipo": "integer", "permite_nulo": "NO"},
54         {"coluna": "produto_id", "tipo": "integer", "permite_nulo": "NO"},
55         {"coluna": "quantidade", "tipo": "integer", "permite_nulo": "NO"},
56         {"coluna": "total", "tipo": "numeric", "permite_nulo": "NO"},
57         {"coluna": "status", "tipo": "varchar", "permite_nulo": "YES"},
58     ]
59 }
60
61 @server.list_tools()
62 async def list_tools() -> list[types.Tool]:
63     return [
64         types.Tool(
65             name="executar_query",
66             description="Executa uma query no banco mock (suporta filtros simples por coluna=valor)",
67             inputSchema={
```

```

68         "type": "object",
69         "properties": {
70             "tabela": {"type": "string", "description":
71                 "Nome da tabela"},
72             "filtro": {"type": "object", "description":
73                 "Filtros chave:valor (opcional)"}
74         },
75         "required": ["tabela"]
76     },
77     types.Tool(
78         name="listar_tabelas",
79         description="Lista todas as tabelas disponiveis no
80             banco mock.",
81         inputSchema={"type": "object", "properties": {}}
82     ),
83     types.Tool(
84         name="descrever_tabela",
85         description="Retorna o schema de uma tabela.",
86         inputSchema={
87             "type": "object",
88             "properties": {
89                 "tabela": {"type": "string", "description":
90                     "Nome da tabela"}
91             },
92             "required": ["tabela"]
93         }
94     ),
95 ]
96
97 @server.call_tool()
98 async def call_tool(name: str, arguments: dict) -> list[types.
99     TextContent]:
100     if name == "listar_tabelas":
101         tabelas = [{"table_name": t, "num_colunas": len(c)} for
            t, c in SCHEMAS.items()]
102         return [types.TextContent(type="text", text=json.dumps(
            tabelas, ensure_ascii=False, indent=2))]
103
104     elif name == "descrever_tabela":
105         tabela = arguments["tabela"]

```

```

102     schema = SCHEMAS.get(tabela)
103     if not schema:
104         return [types.TextContent(type="text", text=f"
            Tabela '{tabela}' nao encontrada. Tabelas: {list
            (SCHEMAS.keys())}")]
105     return [types.TextContent(
106         type="text",
107         text=json.dumps({"tabela": tabela, "colunas":
            schema}, ensure_ascii=False, indent=2)
108     )]
109
110     elif name == "executar_query":
111         tabela = arguments["tabela"]
112         filtro = arguments.get("filtro", {})
113         dados = MOCK_DATA.get(tabela, [])
114         if filtro:
115             dados = [row for row in dados
116                     if all(str(row.get(k)) == str(v) for k, v
117                           in filtro.items())]
118         return [types.TextContent(
119             type="text",
120             text=json.dumps(dados, ensure_ascii=False, default=
121                           str, indent=2)
122         )]
123
124     return [types.TextContent(type="text", text=f"Ferramenta
            desconhecida: {name}")]
125
126 async def main():
127     async with stdio_server() as (read_stream, write_stream):
128         await server.run(read_stream, write_stream, server.
129                           create_initialization_options())
130
131 if __name__ == "__main__":
132     asyncio.run(main())

```

Listing A.4: Servidor MCP mock para testes locais (versao ASCII para compilacao).

A.3.2 mcp_postgres_server.py

```

1  """
2

```

```

3  Servidor MCP para PostgreSQL
4  =====
5  Expose ferramentas para consulta ao banco via protocolo MCP.
6  Execute: python mcp_postgres_server.py
7  """
8  import asyncio
9  import os
10 import json
11 import psycopg2
12 import psycopg2.extras
13 from mcp.server import Server
14 from mcp.server.stdio import stdio_server
15 import mcp.types as types
16
17 # Configuracao do banco (via variavel de ambiente)
18 DATABASE_URL = os.environ.get(
19     "DATABASE_URL",
20     "postgresql://postgres:senha@localhost:5432/meu_banco"
21 )
22
23 # Cria a instancia do servidor MCP
24 server = Server("postgres-mcp-server")
25
26
27 def get_connection():
28     """Retorna uma conexao com o banco PostgreSQL."""
29     return psycopg2.connect(DATABASE_URL)
30
31
32 # ??? Define as ferramentas disponiveis
33     ?????????????????????????????????????????????????????????????
34 @server.list_tools()
35 async def list_tools() -> list[types.Tool]:
36     """Retorna a lista de ferramentas disponiveis neste
37         servidor MCP."""
38     return [
39         types.Tool(
40             name="executar_query",
41             description="Executa uma query SELECT no banco
42                 PostgreSQL e retorna os resultados."

```

```
40         " APENAS queries SELECT sao permitidas
41         por seguranca.",
42     inputSchema={
43         "type": "object",
44         "properties": {
45             "sql": {
46                 "type": "string",
47                 "description": "Query SQL SELECT para
48                 executar"
49             },
50             "limite": {
51                 "type": "integer",
52                 "description": "Numero maximo de linhas
53                 a retornar (default: 100)",
54                 "default": 100
55             }
56         },
57         "required": ["sql"]
58     }
59 ),
60 types.Tool(
61     name="listar_tabelas",
62     description="Lista todas as tabelas disponiveis no
63     schema publico do banco de dados.",
64     inputSchema={
65         "type": "object",
66         "properties": {}
67     }
68 ),
69 types.Tool(
70     name="descrever_tabela",
71     description="Retorna a estrutura de uma tabela:
72     colunas, tipos de dados e constraints.",
73     inputSchema={
74         "type": "object",
75         "properties": {
76             "tabela": {
77                 "type": "string",
78                 "description": "Nome da tabela a ser
79                 descrita"
80             }
81         }
82     )
83 )
```

```

75         },
76         "required": ["tabela"]
77     }
78 ),
79 types.Tool(
80     name="contar_registros",
81     description="Conta o numero de registros em uma
82         tabela, com filtro opcional.",
83     inputSchema={
84         "type": "object",
85         "properties": {
86             "tabela": {
87                 "type": "string",
88                 "description": "Nome da tabela"
89             },
90             "where": {
91                 "type": "string",
92                 "description": "Clausula WHERE opcional
93                     (sem a palavra WHERE)"
94             }
95         },
96         "required": ["tabela"]
97     }
98 ),
99 ]
100 # ??? Implementa a execucao de cada ferramenta
101     ?????????????????????????????????????????????
102 @server.call_tool()
103 async def call_tool(name: str, arguments: dict) -> list[types.
104     TextContent]:
105     """Executa a ferramenta solicitada com os argumentos
106         fornecidos."""
107     try:
108         conn = get_connection()
109         cur = conn.cursor(cursor_factory=psycpg2.extras.
110             RealDictCursor)
111
112         if name == "executar_query":
113             sql = arguments["sql"].strip()

```

```
110         limite = arguments.get("limite", 100)
111
112         # Seguranca: apenas SELECT
113         if not sql.upper().startswith("SELECT"):
114             return [types.TextContent(
115                 type="text",
116                 text="? Apenas queries SELECT sao
117                     permitidas por seguranca."
118             )]
119
120         cur.execute(sql)
121         rows = cur.fetchmany(limite)
122         resultado = [dict(row) for row in rows]
123         return [types.TextContent(
124             type="text",
125             text=json.dumps(resultado, ensure_ascii=False,
126                             default=str, indent=2)
127         )]
128
129     elif name == "listar_tabelas":
130         cur.execute("""
131             SELECT table_name,
132                    (SELECT COUNT(*) FROM information_schema
133                     .columns
134                     WHERE table_name = t.table_name) as
135                        num_colunas
136             FROM information_schema.tables t
137             WHERE table_schema = 'public'
138             ORDER BY table_name
139         """)
140         tabelas = [dict(row) for row in cur.fetchall()]
141         return [types.TextContent(
142             type="text",
143             text=json.dumps(tabelas, ensure_ascii=False,
144                             indent=2)
145         )]
146
147     elif name == "descrever_tabela":
148         tabela = arguments["tabela"]
149         cur.execute("""
150             SELECT
```

```
146         column_name as coluna,
147         data_type as tipo,
148         character_maximum_length as tamanho_max,
149         is_nullable as permite_nulo,
150         column_default as valor_padrao
151     FROM information_schema.columns
152     WHERE table_name = %s AND table_schema = '
153         public'
154     ORDER BY ordinal_position
155 """ , (tabela,))
156 colunas = [dict(row) for row in cur.fetchall()]
157 if not colunas:
158     return [types.TextContent(type="text", text=f"
159         Tabela '{tabela}' nao encontrada.")]
160 return [types.TextContent(
161     type="text",
162     text=json.dumps({"tabela": tabela, "colunas":
163         colunas},
164                     ensure_ascii=False, default=str,
165                     indent=2)
166 )]
167
168 elif name == "contar_registros":
169     tabela = arguments["tabela"]
170     where = arguments.get("where", "")
171     sql = f"SELECT COUNT(*) as total FROM {tabela}"
172     if where:
173         sql += f" WHERE {where}"
174     cur.execute(sql)
175     total = cur.fetchone()["total"]
176     return [types.TextContent(
177         type="text",
178         text=f"Total de registros em '{tabela}'{' (com
179             filtro)' if where else '' : {total}"
180     )]
181
182 conn.close()
183
184 except Exception as e:
185     return [types.TextContent(type="text", text=f"? Erro: {
186         str(e)}")]
```



```

181
182
183 # ??? Ponto de entrada do servidor
    ?????????????????????????????????????????????????????????????
184 async def main():
185     print(f"?? Servidor MCP PostgreSQL iniciado", flush=True)
186     print(f"    Banco: {DATABASE_URL.split('@')[-1] if '@' in
        DATABASE_URL else DATABASE_URL}", flush=True)
187     async with stdio_server() as (read_stream, write_stream):
188         await server.run(
189             read_stream,
190             write_stream,
191             server.create_initialization_options()
192         )
193
194
195 if __name__ == "__main__":
196     asyncio.run(main())

```

Listing A.5: Servidor MCP para PostgreSQL (versao ASCII para compilacao).

A.3.3 validate_sprint_a.py

```

1  """Validacoes Sprint A ? executa sem AWS."""
2  import os, json, asyncio, numpy as np, faiss
3
4  # ?? V2: FAISS offline
    ?????????????????????????????????????????????????????????????
5  def test_faiss():
6      d = 1536
7      index = faiss.IndexFlatL2(d)
8      docs = [
9          "Clientes conservadores priorizam liquidez diaria.",
10         "Fundos de renda fixa possuem menor volatilidade.",
11         "Acoes de crescimento exigem horizonte de longo prazo."
12         ,
13         "Titulos do Tesouro Direto sao garantidos pelo governo
            federal.",
14         "CDBs oferecem retorno atrelado ao CDI com protecao do
            FGC.",
15     ]
    # embeddings mock: hash deterministico

```

```

16     vecs = np.array([
17         [float(ord(c) % 17) / 17 for _ in range(d)]
18         for c in [s[0] for s in docs]
19     ], dtype="float32")
20     index.add(vecs)
21     q = vecs[0:1].copy()
22     q[0, 0] += 0.01 # pequena perturbacao
23     D, I = index.search(q, 3)
24     assert index.ntotal == 5, "Deve ter 5 vetores"
25     assert I[0][0] == 0, "Doc mais proximo deve ser o proprio"
26     print(f"[V2] FAISS OK ? {index.ntotal} vetores, top-3
27         indices: {I[0].tolist()}")
28
29     # verificar indice persistido
30     idx_path = "faiss_index/index.faiss"
31     if os.path.exists(idx_path):
32         idx2 = faiss.read_index(idx_path)
33         print(f"[V2] faiss_index local: {idx2.ntotal} vetores
34             ({os.path.getsize(idx_path)} bytes)")
35     else:
36         print(f"[V2] faiss_index/index.faiss nao encontrado (
37             esperado em ambiente novo)")
38
39     # ?? V3: agente usa tool MCP (simulado sem LangGraph)
40     ?????????????????????????????????
41
42     async def test_mcp_tools():
43         from mcp_mock_server import call_tool, list_tools
44         tools = await list_tools()
45         nomes = [t.name for t in tools]
46         assert set(nomes) == {"executar_query", "listar_tabelas", "
47             descrever_tabela"}
48
49         r = await call_tool("executar_query", {"tabela": "pedidos",
50             "filtro": {"status": "entregue"}})
51         data = json.loads(r[0].text)
52         assert len(data) == 2, f"Esperado 2 pedidos entregues,
53             obtido {len(data)}"
54
55         r2 = await call_tool("descrever_tabela", {"tabela": "
56             produtos"})
57         schema = json.loads(r2[0].text)

```

[illegible]

Listing A.6: Script de validacao do Sprint A (versao ASCII para compilacao).

A.4 Capitulo 11

A.4.1 otel_config.py

```
1  """
2  Configura OpenTelemetry com exportacao para Langfuse e auto-
   instrumentacao Anthropic via openlit.
3
4  Ativado automaticamente quando LANGFUSE_PUBLIC_KEY e
   LANGFUSE_SECRET_KEY estao definidos.
5  Se as chaves nao estiverem presentes, OTel e desabilitado
   silenciosamente.
6
7  Setup do Langfuse Cloud:
8  https://cloud.langfuse.com ? Settings ? API Keys ? copiar pk-
   lf-... e sk-lf-...
9  """
10 from __future__ import annotations
11
12 import base64
13 import logging
14 import os
15
16 log = logging.getLogger(__name__)
17
18 _initialized = False
19
20
21 def init():
22     global _initialized
23     if _initialized:
24         return
25
26     public_key = os.environ.get("LANGFUSE_PUBLIC_KEY", "")
27     secret_key = os.environ.get("LANGFUSE_SECRET_KEY", "")
28
29     if public_key and secret_key:
```

```
30     _setup_langfuse_otlp(public_key, secret_key)
31
32     try:
33         import openlit
34         openlit.init()
35         log.info("OpenLIT inicializado ? Anthropic auto-
36                 instrumentado via OTel.")
37     except ImportError:
38         log.warning("openlit nao instalado ? auto-
39                 instrumentacao Anthropic desabilitada.")
40
41     _initialized = True
42
43 def _setup_langfuse_otlp(public_key: str, secret_key: str) ->
44     None:
45     try:
46         from opentelemetry import trace
47         from opentelemetry.sdk.trace import TracerProvider
48         from opentelemetry.sdk.trace.export import import
49             BatchSpanProcessor
50         from opentelemetry.exporter.otlp.proto.http.
51             trace_exporter import OTLPSpanExporter
52
53         auth = base64.b64encode(f"{public_key}:{secret_key}".
54             encode()).decode()
55         otlp_endpoint = os.environ.get(
56             "LANGFUSE_OTLP_ENDPOINT",
57             "https://cloud.langfuse.com/api/public/otel/v1/
58             traces",
59         )
60
61         exporter = OTLPSpanExporter(
62             endpoint=otlp_endpoint,
63             headers={"Authorization": f"Basic {auth}"},
64         )
65         provider = TracerProvider()
66         provider.add_span_processor(BatchSpanProcessor(exporter
67             ))
68         trace.set_tracer_provider(provider)
```

```
63     log.info("Langfuse OTLP exporter configurado: %s",
64             otlp_endpoint)
65 except ImportError:
66     log.warning(
67         "opentelemetry-sdk ou opentelemetry-exporter-otlp-
68         proto-http nao instalados ? "
69         "export para Langfuse desabilitado."
70     )
```

Listing A.7: Configuracao de observabilidade (OTel/Langfuse, versao ASCII para compilacao).

A.4.2 status.py

```
1 import json
2 import boto3
3 import os
4 from boto3.dynamodb.conditions import Key
5
6 dynamodb = boto3.resource("dynamodb")
7
8 TABLE_NAME = os.environ["DYNAMODB_TABLE"]
9
10
11 def lambda_handler(event, context):
12     request_id = (event.get("pathParameters") or {}).get("
13         request_id")
14
15     if not request_id:
16         return _response(400, {"error": "request_id e
17             obrigatorio"})
18
19     item = dynamodb.Table(TABLE_NAME).get_item(
20         Key={"request_id": request_id}
21     ).get("Item")
22
23     if not item:
24         return _response(404, {"error": "request_id nao
25             encontrado"})
26
27     resposta = {
```

```

25     "request_id": item["request_id"],
26     "status": item["status"],
27     "criado_em": item.get("criado_em"),
28     "atualizado_em": item.get("atualizado_em"),
29 }
30
31 if item["status"] == "COMPLETED":
32     resposta["resultado"] = item.get("resultado")
33
34 if item["status"] == "FAILED":
35     resposta["erro"] = item.get("erro", "Erro desconhecido")
36
37 return _response(200, resposta)
38
39
40 def _response(status_code, body):
41     return {
42         "statusCode": status_code,
43         "headers": {"Content-Type": "application/json"},
44         "body": json.dumps(body, ensure_ascii=False),
45     }

```

Listing A.8: Endpoint de consulta de status (versao ASCII para compilacao).

A.4.3 worker.py

```

1  """
2  Worker Lambda ? Pipeline RAG completo com OTel/Langfuse.
3
4  Modos:
5      segmento ? gerente fala sobre o cliente em 3? pessoa (RAG:
6          cluster index)
7      persona ? arquetipo nomeado do cluster em 1? pessoa (RAG:
8          cluster index)
9      twin ? digital twin do cliente em 1? pessoa (RAG:
10         clientes-digital-twins)
11
12 Cold start: carrega slim PKL do S3 (numpy only, sem sklearn).
13 Warm start: reutiliza modelo em memoria.
14 """
15
16 from __future__ import annotations

```

```

13
14 import json
15 import logging
16 import os
17 import pickle
18 import time
19 import traceback
20 from datetime import datetime, timezone
21 from io import BytesIO
22 from typing import Any, Dict, List, Optional
23
24 import boto3
25 import numpy as np
26 from botocore.config import Config
27 from opensearchpy import OpenSearch, RequestsHttpConnection
28 from requests_aws4auth import AWS4Auth
29
30 import otel_config # inicializa OTel + openlit antes de
    qualquer chamada Bedrock
31 otel_config.init()
32
33 log = logging.getLogger(__name__)
34 logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(
    levelname)s] %(message)s")
35
36 # ?? Clientes AWS
    ?????????????????????????????????????????????????????????????????????????????????????
37 _config = Config(connect_timeout=5, read_timeout=30)
38 _dynamodb = boto3.resource("dynamodb", config=_config)
39 _sns = boto3.client("sns", config=_config)
40 _s3 = boto3.client("s3", config=_config)
41 _athena = boto3.client("athena", config=_config)
42
43 # ?? Variaveis de ambiente
    ?????????????????????????????????????????????????????????????????????????????????
44 TABLE_NAME = os.environ["DYNAMODB_TABLE"]
45 SNS_TOPIC_ARN = os.environ.get("SNS_TOPIC_ARN", "")
46 S3_BUCKET = os.environ.get("S3_BUCKET", "")
47 S3_PREFIX = os.environ.get("S3_PREFIX", "clientes-agente/")
48 OPENSEARCH_ENDPOINT = os.environ.get("OPENSEARCH_ENDPOINT", "")

```



```

49 AWS_REGION = os.environ.get("AWS_REGION", "sa-east-1")
50 AWS_ACCOUNT_ID = os.environ.get("AWS_ACCOUNT_ID", "")
51 BEDROCK_MODEL_ID = os.environ.get("BEDROCK_MODEL_ID", "us.
    anthropic.claude-sonnet-4-5-20250514-v1:0")
52 BEDROCK_REGION = os.environ.get("BEDROCK_REGION", "us-east-1")
53 NEPTUNE_ENDPOINT = os.environ.get("NEPTUNE_ENDPOINT", "")
54 NEPTUNE_PROXY_FUNCTION = os.environ.get("NEPTUNE_PROXY_FUNCTION
    ", "")
55 ENABLE_LLM_JUDGE = os.environ.get("ENABLE_LLM_JUDGE", "false").
    lower() == "true"
56 ATHENA_DATABASE = os.environ.get("ATHENA_DATABASE", "")
57 ATHENA_OUTPUT_BUCKET = os.environ.get("ATHENA_OUTPUT_BUCKET", "
    ")
58
59 FEATURES = ["idade", "renda_mensal", "saldo_medio", "
    transacoes_mes", "score_credito", "num_produtos"]
60
61 # ?? Clientes inicializados no modulo (fora do handler ? boas
    praticas Lambda) ??
62 _bedrock_client = boto3.client(
63     "bedrock-runtime",
64     region_name=BEDROCK_REGION,
65     config=Config(connect_timeout=5, read_timeout=120),
66 )
67
68 # ?? Estado de cold start (PKL carregado do S3 na primeira
    invocacao) ??????????
69 _model_data: Optional[Dict[str, Any]] = None
70
71
72 def _get_model() -> Dict[str, Any]:
73     """Carrega slim PKL do S3 no cold start."""
74     global _model_data
75     if _model_data is not None:
76         return _model_data
77
78     if not S3_BUCKET:
79         raise RuntimeError("S3_BUCKET nao definido ? nao e
            possivel carregar o modelo.")
80
81     key = f"{S3_PREFIX}modelo_clustering_slim.pkl"

```

```

82     log.info("Cold start: carregando modelo de s3://%s/%s",
83             S3_BUCKET, key)
84     extra = {"ExpectedBucketOwner": AWS_ACCOUNT_ID} if
85             AWS_ACCOUNT_ID else {}
86     obj = _s3.get_object(Bucket=S3_BUCKET, Key=key, **extra)
87     _model_data = pickle.loads(obj["Body"].read())
88     log.info("Modelo carregado: %d clusters.", _model_data["
89             n_clusters"])
90     return _model_data
91
92 def _classificar(dados: Dict[str, float]) -> int:
93     m = _get_model()
94     features = m["features"]
95     X = np.array([[dados[f] for f in features]], dtype=float)
96     X = (X - m["scale_mean"]) / m["scale_std"]
97     dist = np.linalg.norm(X - m["centroids"], axis=1)
98     return int(np.argmin(dist))
99
100
101 def _os_client() -> OpenSearch:
102     boto_session = boto3.session.Session()
103     creds = boto_session.get_credentials().
104             get_frozen_credentials()
105     auth = AWS4Auth(creds.access_key, creds.secret_key,
106                     AWS_REGION, "es",
107                     session_token=creds.token)
108     return OpenSearch(
109         hosts=[{"host": OPENSEARCH_ENDPOINT, "port": 443}],
110         http_auth=auth,
111         use_ssl=True,
112         verify_certs=True,
113         connection_class=RequestsHttpConnection,
114     )
115
116 def _rag_segmento(cluster_id: int, query: str, k: int = 3) ->
117     str:
118     if not OPENSEARCH_ENDPOINT:

```

```
117         return ""
118     try:
119         client = _os_client()
120         resp = client.search(
121             index=f"clientes-segmento-{cluster_id}",
122             body={"query": {"match": {"text": query}}, "size":
123                 k},
124         )
125         return _extrair_texto(resp)
126     except Exception:
127         log.warning("Erro no RAG de segmento (cluster %d): %s",
128                     cluster_id, traceback.format_exc())
129         return ""
130
131 def _rag_graph_sync(cluster_id: int, query: str, k: int = 3) ->
132     str:
133         """
134         Consulta o indice OpenSearch 'neptune-graph-sync' replicado
135         do Neptune.
136
137         Permite busca BM25 sobre o snapshot do grafo sem chamar
138         Neptune diretamente,
139         reduzindo latencia e eliminando dependencia de VPC no
140         caminho critico.
141         """
142         if not OPENSEARCH_ENDPOINT:
143             return ""
144         try:
145             client = _os_client()
146             resp = client.search(
147                 index="neptune-graph-sync",
148                 body={
149                     "query": {
150                         "bool": {
151                             "must": {"match": {"text": query}},
152                             "filter": {"term": {"cluster_id":
153                                             cluster_id}},
154                         }
155                     },
156                     "size": k,
157                 },
158             )
```

```

151         )
152         return _extrair_texto(resp)
153     except Exception:
154         log.warning("Erro no RAG graph-sync (cluster %d): %s",
155                     cluster_id, traceback.format_exc())
156         return ""
157
158 def _rag_documentos(query: str, k: int = 3) -> str:
159     """Consulta o indice documentos-knowledge (BM25 sem filtro
160         de cluster)."""
161     if not OPENSEARCH_ENDPOINT:
162         return ""
163     try:
164         client = _os_client()
165         resp = client.search(
166             index="documentos-knowledge",
167             body={"query": {"match": {"text": query}}, "size":
168                 k},
169         )
170         return _extrair_texto(resp)
171     except Exception:
172         log.warning("Erro no RAG de documentos: %s", traceback.
173                     format_exc())
174         return ""
175
176 def _rag_entrevistas(cluster_id: int, cliente_id: str, modo:
177     str, query: str, k: int = 3) -> str:
178     """Corpus de entrevistas qualitativas.
179     Modo twin: tenta match exato por cliente_id, cai para
180         cluster se vazio.
181     Modos segmento/persona: filtra por cluster_id.
182     """
183     if not OPENSEARCH_ENDPOINT:
184         return ""
185     try:
186         client = _os_client()
187         if modo == "twin" and cliente_id:
188             resp = client.search(
189                 index="entrevistas-clientes",

```

```
186         body={
187             "query": {
188                 "bool": {
189                     "must": {"match": {"text": query}},
190                     "filter": {"term": {"metadata.
191                                     cliente_id.keyword": cliente_id
192                                     }},
193                 },
194             },
195             "size": k,
196         },
197     )
198     texto = _extrair_texto(resp)
199     if texto:
200         return texto
201     resp = client.search(
202         index="entrevistas-clientes",
203         body={
204             "query": {
205                 "bool": {
206                     "must": {"match": {"text": query}},
207                     "filter": {"term": {"metadata.
208                                     cluster_id": cluster_id}},
209                 },
210             },
211             "size": k,
212         },
213     )
214     return _extrair_texto(resp)
215 except Exception:
216     log.warning("Erro no RAG de entrevistas (cluster %d): %
217                 s", cluster_id, traceback.format_exc())
218     return ""
219
220 def _consultar_athena_nps(tipo: str, identificador: str,
221     produtos: List[str] = None) -> str:
222     """Executa query Athena para buscar elogios/criticas NPS do
223         cliente ou segmento."""
224     if not ATHENA_DATABASE or not ATHENA_OUTPUT_BUCKET:
225         return "Dados de feedback NPS nao configurados."
```

```
221     try:
222         filtro = (f"cliente_id = '{identificador}'" if tipo ==
223                 "cliente"
224                 else f"cluster_id = {identificador}")
225     if produtos:
226         lista = ", ".join(f"'{p}'" for p in produtos)
227         filtro += f" AND produto IN ({lista})"
228     sql = (
229         f"SELECT sentimento, produto, comentario,
230            data_feedback "
231         f"FROM {ATHENA_DATABASE}.nps_feedbacks "
232         f"WHERE {filtro} "
233         f"ORDER BY data_feedback DESC LIMIT 20"
234     )
235     exec_id = _athena.start_query_execution(
236         QueryString=sql,
237         ResultConfiguration={"OutputLocation": f"s3://{
238             ATHENA_OUTPUT_BUCKET}/athena-nps/"},
239     )["QueryExecutionId"]
240
241     for _ in range(30):
242         state = _athena.get_query_execution(
243             QueryExecutionId=exec_id)["QueryExecution"]["
244             Status"]["State"]
245         if state in ("SUCCEEDED", "FAILED", "CANCELLED"):
246             break
247         time.sleep(1)
248
249     if state != "SUCCEEDED":
250         return "Consulta NPS indisponível no momento."
251
252     rows = _athena.get_query_results(QueryExecutionId=
253         exec_id)["ResultSet"]["Rows"][1:]
254     if not rows:
255         return ""
256
257     def _celula(row, idx):
258         return row["Data"][idx].get("VarCharValue", "")
259
260     elogios = [r for r in rows if _celula(r, 0) == "
261                 promotor"]
```

```

256     detratores = [r for r in rows if _celula(r, 0) == "
           detrator"]
257     blocos: List[str] = []
258     if elogios:
259         itens = "; ".join(
260             f'"{_celula(r, 2)}" ({_celula(r, 1)})' for r in
                elogios[:5]
261         )
262         blocos.append(f"Elogios (promotores): {itens}.")
263     if detratores:
264         itens = "; ".join(
265             f'"{_celula(r, 2)}" ({_celula(r, 1)})' for r in
                detratores[:5]
266         )
267         blocos.append(f"Criticas (detratores): {itens}.")
268     return "\n".join(blocos)
269 except Exception:
270     log.warning("Erro Athena NPS: %s", traceback.format_exc
                ())
271     return "Dados de feedback nao disponiveis."
272
273
274 _NPS_TOOL_CONFIG: Optional[Dict] = (
275     {
276         "tools": [
277             {
278                 "toolSpec": {
279                     "name": "consultar_feedback_nps",
280                     "description": (
281                         "Consulta elogios (promotores NPS?9) e
                            criticas (detratores NPS?6) "
282                         "de clientes sobre produtos bancarios
                            no data warehouse. Use quando "
283                         "a pergunta envolver satisfacao,
                            opinioes, reclamacoes, elogios ou "
284                         "avaliacao de produtos especificos."
285                     ),
286                     "inputSchema": {
287                         "json": {
288                             "type": "object",
289                             "properties": {

```

```

290         "tipo_consulta": {
291             "type": "string",
292             "enum": ["cliente", "
                segmento"],
293             "description": (
294                 "'cliente' para modo
                twin (cliente_id), "
295                 "'segmento' para
                persona/segmento (
                cluster_id)"
296             ),
297         },
298         "identificador": {
299             "type": "string",
300             "description": "cliente_id
                (ex: 'C001') ou
                cluster_id como string (
                ex: '0')",
301         },
302         "produtos": {
303             "type": "array",
304             "items": {"type": "string"
                },
305             "description": "Filtrar por
                produtos especificos (
                opcional)",
306         },
307     },
308     "required": ["tipo_consulta", "
        identificador"],
309 }
310 },
311 }
312 }
313 ],
314     "toolChoice": {"auto": {}},
315 }
316 if ATHENA_DATABASE
317 else None
318 )
319

```



```
320
321 def _rag_twin(cliente_id: str, query: str, k: int = 3) -> str:
322     if not OPENSEARCH_ENDPOINT:
323         return ""
324     try:
325         client = _os_client()
326         resp = client.search(
327             index="clientes-digital-twins",
328             body={
329                 "query": {
330                     "bool": {
331                         "must": {"match": {"text": query}},
332                         "filter": {"term": {"metadata.
333                                     cliente_id.keyword": cliente_id}},
334                     },
335                     "size": k,
336                 },
337             )
338         return _extrair_texto(resp)
339     except Exception:
340         log.warning("Erro no RAG de twin (cliente %s): %s",
341                     cliente_id, traceback.format_exc())
342         return ""
343
344 def _extrair_texto(resp: Dict) -> str:
345     hits = resp.get("hits", {}).get("hits", [])
346     textos = [h.get("_source", {}).get("text", "") for h in
347               hits]
348     return "\n\n".join(t for t in textos if t)
349
350 _STOPWORDS = {
351     "que", "de", "do", "da", "dos", "das", "em", "no", "na", "
352     nos", "nas",
353     "por", "para", "com", "uma", "um", "os", "as", "se", "ao",
354     "a", "e",
355     "o", "a", "e", "sao", "tem", "foi", "ter", "ser", "nao", "
356     mas", "ou",
```

```

354     "seu", "sua", "seus", "suas", "este", "esta", "esse", "essa
        ", "isso",
355 }
356
357
358 def _palavras_chave(texto: str) -> set:
359     return {
360         w.strip(". , ; : ! ? ( ) \ " ' " ).lower()
361         for w in texto.split()
362         if len(w) > 4 and w.strip(". , ; : ! ? ( ) \ " ' " ).lower() not in
            _STOPWORDS
363     }
364
365
366 def _confiabilidade(
367     bm25: str, sync: str, graph: str, resposta: str,
368     documentos: str = "", entrevistas: str = ""
369 ) -> Dict:
370     """
371     Score de confiabilidade baseado em duas dimensoes:
372     - Cobertura de fontes: quais camadas RAG entregaram
        conteudo (ate 0.95 do score)
373     - Sobreposicao lexicas: termos do contexto presentes na
        resposta (ate 0.15)
374     """
375     fontes: List[str] = []
376     cobertura = 0.0
377
378     if bm25.strip():
379         fontes.append("opensearch_bm25")
380         cobertura += 0.40
381     if graph.strip():
382         fontes.append("neptune_live")
383         cobertura += 0.30
384     if sync.strip():
385         fontes.append("neptune_sync")
386         cobertura += 0.15
387     if documentos.strip():
388         fontes.append("documentos_knowledge")
389         cobertura += 0.10
390     if entrevistas.strip():

```

```
391     fontes.append("entrevistas_corpus")
392     cobertura += 0.10
393
394     contexto_total = " ".join(filter(None, [bm25, sync, graph,
395         documentos, entrevistas]))
396     overlap = 0.0
397     if contexto_total and resposta:
398         ctx_kw = _palavras_chave(contexto_total)
399         resp_kw = _palavras_chave(resposta)
400         if ctx_kw and resp_kw:
401             overlap = min(len(ctx_kw & resp_kw) / min(len(
402                 ctx_kw), len(resp_kw)), 1.0)
403
404     score = round(min(cobertura + overlap * 0.15, 1.0), 2)
405     if score >= 0.70:
406         nivel = "alto"
407     elif score >= 0.40:
408         nivel = "medio"
409     else:
410         nivel = "baixo"
411
412     return {
413         "score": score,
414         "nivel": nivel,
415         "fontes_ativas": fontes,
416         "detalhes": {
417             "cobertura_fontes": round(cobertura, 2),
418             "sobreposicao_lexica": round(overlap, 2),
419         },
420     }
421
422 def _avaliar_llm(contexto_rag: str, pergunta: str, resposta:
423     str) -> Dict:
424     """Avaliacao semantica via segundo call Bedrock (LLM-as-
425         judge)."""
426     try:
427         prompt = (
428             "Avalie a resposta abaixo em relacao ao contexto
429             RAG e a pergunta. "
```

```

427         "Responda APENAS com JSON valido, sem texto
          adicional.\n\n"
428         f"Contexto RAG:\n{contexto_rag[:2000]}\n\n"
429         f"Pergunta: {pergunta}\n\n"
430         f"Resposta: {resposta}\n\n"
431         "JSON esperado (scores de 1 a 10):\n"
432         '{"relevancia": <int>, "fidelidade": <int>, "
          completude": <int>, '
433         '"raciocinio": "<1 frase explicando o score>"}'
434     )
435     resp = _bedrock_client.converse(
436         modelId=BEDROCK_MODEL_ID,
437         messages=[{"role": "user", "content": [{"text":
          prompt}]}],
438         inferenceConfig={"maxTokens": 256},
439     )
440     texto = resp["output"]["message"]["content"][0]["text"]
441     avaliacao = json.loads(texto)
442     scores = [
443         int(avaliacao.get("relevancia", 0)),
444         int(avaliacao.get("fidelidade", 0)),
445         int(avaliacao.get("completude", 0)),
446     ]
447     avaliacao["media"] = round(sum(scores) / len(scores),
448                                1)
449     avaliacao["disponivel"] = True
450     return avaliacao
451 except Exception:
452     log.warning("Erro na avaliacao LLM-as-judge: %s",
453                traceback.format_exc())
454     return {"disponivel": False}
455
456 def _neptune_client():
457     """Retorna cliente que invoca o Neptune Proxy Lambda (
458         dentro da VPC)."""
459     class _NeptuneProxyClient:
460         def query(self, cypher: str) -> List[Dict]:
461             resp = boto3.client("lambda", region_name=
462                                AWS_REGION,

```

```

460         config=Config(connect_timeout
                        =5, read_timeout=25)).invoke
                        (
461             FunctionName=NEPTUNE_PROXY_FUNCTION,
462             InvocationType="RequestResponse",
463             Payload=json.dumps({"cypher": cypher}),
464         )
465         return json.loads(resp["Payload"].read()) or []
466
467     return _NeptuneProxyClient()
468
469
470 def _rag_graph(cluster_id: int, cliente_id: str, modo: str) ->
    str:
471     """
472     Recuperacao GraphRAG via OpenCypher no Neptune.
473     Retorna contexto estruturado de relacionamentos do grafo de
474         conhecimento:
475         - Segmento ? Produtos recomendados
476         - Segmento ? Persona arquetipo
477         - Cliente ? Clientes similares (k-NN via cluster)
478     Complementa o RAG BM25 do OpenSearch com raciocinio multi-
479         hop.
480     """
481     if not NEPTUNE_PROXY_FUNCTION:
482         return ""
483     try:
484         client = _neptune_client()
485         blocos: List[str] = []
486
487         # Hop 1: produtos e estrategia do segmento
488         q_seg = (
489             f"MATCH (s:Segmento {{cluster_id: {cluster_id}}})"
490             -[:RECOMENDA]->(p:Produto) "
491             "RETURN s.nome AS segmento, collect(p.nome) AS
492             produtos"
493         )
494         for row in client.query(q_seg):
495             seg = row.get("segmento", "")
496             produtos = ", ".join(row.get("produtos", []))
497             if produtos:

```

```

494         blocos.append(f"Produtos recomendados para '{
495             seg}': {produtos}.")
496
497     # Hop 2: persona arquetipo do segmento
498     q_persona = (
499         f"MATCH (s:Segmento {{cluster_id: {cluster_id}}})
500         -[:TEM_PERSONA]->(p:Persona) "
501         "RETURN p.nome AS nome, p.ocupacao AS ocupacao, p.
502         contexto AS contexto"
503     )
504     for row in client.query(q_persona):
505         blocos.append(
506             f"Persona arquetipo: {row.get('nome')} ({row.
507             get('ocupacao')}). "
508             f"{row.get('contexto', '')}"
509         )
510
511     # Hop 2 (twin): historico financeiro de clientes
512     similares no mesmo cluster
513     if modo == "twin" and cliente_id:
514         q_similar = (
515             f"MATCH (c:Cliente {{id: '{cliente_id}'}})-[:
516             SIMILAR_A]->(s:Cliente) "
517             "RETURN s.id AS similar_id, s.score_credito AS
518             score, "
519             "s.renda_mensal AS renda LIMIT 3"
520         )
521         similares = client.query(q_similar)
522         if similares:
523             resumo = "; ".join(
524                 f"Cliente {r['similar_id']} (score {r['
525                 score']:.0f}, renda R$ {r['renda']:.0f
526                 })"
527                 for r in similares
528             )
529             blocos.append(f"Clientes similares no grafo: {
530                 resumo}.")
531
532     return "\n".join(blocos)
533 except Exception:

```

```
524         log.warning("Erro no RAG de grafo (cluster %d): %s",
525                     cluster_id, traceback.format_exc())
526         return ""
527
528 def _perfil(cluster_id: int) -> Dict[str, Any]:
529     # perfis.to_dict() gera {"coluna": {cluster_id: valor},
530     # ...}
531     # precisamos inverter para {"coluna": valor} para o cluster
532     # especifico
533     raw = _get_model()["perfis"]
534     return {col: vals[cluster_id] for col, vals in raw.items()
535             if cluster_id in vals}
536
537 def _system_segmento(cluster_id: int) -> str:
538     p = _perfil(cluster_id)
539     seg = p.get("segmento", f"Cluster {cluster_id}")
540     prompt = p.get("prompt_segmento", "")
541     if not prompt:
542         prompt = (
543             f"Voce e um gerente de relacionamento do banco
544             responsavel pelo segmento '{seg}'. "
545             "Analise os dados do cliente e responda com
546             precisao e clareza em 3? pessoa. "
547             "Seja objetivo, use dados concretos e termine com
548             uma recomendacao de acao."
549         )
550     return prompt
551
552 def _system_persona(cluster_id: int) -> str:
553     p = _perfil(cluster_id)
554     seg = p.get("segmento", f"Cluster {cluster_id}")
555     nome = p.get("persona_nome", f"Cliente do Cluster {
556                   cluster_id}")
557     ocupacao = p.get("persona_ocupacao", "profissional")
558     canal = p.get("persona_canal", "uso os canais do banco")
559     contexto = p.get("persona_contexto", "")
560     return (
```

```

556         f"Voce e {nome}, {ocupacao}. Representa o cliente
557             tipico do segmento '{seg}'. "
558         f"Canal preferencial: {canal}. {contexto} "
559         "REGRAS ABSOLUTAS: (1) Responda SEMPRE em 1ª pessoa. "
560         "(2) Seja autentico ao perfil ? use linguagem,
561             prioridades e preocupacoes coerentes. "
562         "(3) Nunca quebre o personagem nem mencione que e uma
563             IA."
564     )
565
566 def _system_twin(cliente_id: str, dados: Dict) -> str:
567     renda = float(dados.get("renda_mensal", 0))
568     score = float(dados.get("score_credito", 0))
569     idade = float(dados.get("idade", 30))
570     canal = "digital (app/internet banking)" if dados.get("
571         canal_digital") else "agencia presencial"
572     inadimplente = dados.get("inadimplente", dados.get("
573         inadimplencia", 0))
574     historico = "em reestruturacao financeira apos
575         inadimplencia" if inadimplente else "adimplente"
576     return (
577         f"Voce e o gêmeo digital do cliente {cliente_id}. "
578         f"Simule EXATAMENTE como este cliente especifico
579             pensaria e reagiria:\n"
580         f"    Idade: {idade:.0f} anos | Renda: R$ {renda:,.0f}/
581             mes | "
582         f"Score: {score:.0f} | Canal: {canal} | Situacao: {
583             historico}\n\n"
584         "REGRAS ABSOLUTAS: (1) Responda em 1ª pessoa ('Eu
585             prefiro...', 'Para mim...'). "
586         "(2) Suas opinioes devem refletir fielmente renda,
587             score e historico acima. "
588         "(3) Nao quebre o personagem. (4) Nao mencione que e
589             uma simulacao."
590     )
591
592 # ?? Few-shot examples por modo
593     """
594     """

```



```

584 _FEWSHOT_SEGMENTO: List[Dict] = [
585     {
586         "role": "user",
587         "content": (
588             "Contexto relevante:\nPerfil 'Jovem Digital': 312
              clientes, idade media 27 anos, "
589             "renda R$ 2.800/mes, score 640, inadimplencia 8,2%,
              uso digital 94%.\n\n"
590             "Pergunta: Que produto de credito faz mais sentido
              para este cliente?"
591         ),
592     },
593     {
594         "role": "assistant",
595         "content": (
596             "Com base no perfil Jovem Digital, o produto mais
              adequado e o **cartao de credito com cashback "
597             "de 2% sem anuidade**. Justificativa: score 640
              permite aprovacao sem garantias; uso digital de
598             "94% indica preferencia por gestao via app; renda
              de R$ 2.800 comporta limite inicial de R$ 2.000
599             "sem risco de superendividamento. Evitar credito
              pessoal neste momento ? inadimplencia de 8,2% "
600             "sinaliza fragilidade de caixa. **Acao recomendada
              : ** oferta proativa via push notification no app
601             "com aprovacao instantanea e limite pre-aprovado."
602         ),
603     },
604     {
605         "role": "user",
606         "content": (
607             "Contexto relevante:\nPerfil 'Premium Conservador':
              87 clientes, idade media 54 anos, "
608             "renda R$ 19.000/mes, saldo R$ 142.000, score 835,
              inadimplencia 0,3%, uso digital 41%.\n\n"
609             "Pergunta: Como abordar este cliente sobre
              previdencia privada?"
610         ),

```

```

611     },
612     {
613         "role": "assistant",
614         "content": (
615             "O cliente Premium Conservador e o perfil ideal
               para PGBL. Abordagem recomendada: destacar o "
616             "**beneficio fiscal imediato** ? com renda de R$
               19.000/mes, a deducao de ate 12% da renda bruta
               "
617             "representa economia de ate R$ 27.360/ano no IR (
               tabela progressiva). Saldo de R$ 142.000 indica
               "
618             "patrimonio consolidado que busca protecao real.
               Score 835 e inadimplencia 0,3% confirmam "
619             "disciplina financeira. **Acao recomendada:**
               agendar reuniao presencial com gerente dedicado
               "
620             "? uso digital de 41% indica preferencia por
               atendimento exclusivo e personalizado."
621         ),
622     },
623 ]
624
625 _FEWSHOT_PERSONA: List[Dict] = [
626     {
627         "role": "user",
628         "content": "O banco esta oferecendo um emprestimo
               pessoal com taxa de 2,9% ao mes. Voce toparia?",
629     },
630     {
631         "role": "assistant",
632         "content": (
633             "Depende muito do que eu preciso fazer com esse
               dinheiro. 2,9% ao mes parece pouco, mas no ano "
634             "da quase 42% ? isso e caro. Antes de assinar
               qualquer coisa eu calcularia o custo total, "
635             "nao so a parcela. Se for para uma emergencia real,
               eu pesaria. Para compra de desejo? "
636             "Provavelmente esperaria juntar o dinheiro primeiro
               ."
637         ),

```

```
638     },
639     {
640         "role": "user",
641         "content": "Como voce se sente quando recebe uma oferta
                     de produto financeiro por telefone?",
642     },
643     {
644         "role": "assistant",
645         "content": (
646             "Honestamente? Fico desconfiado. Prefiro muito mais
                     ver as condicoes por escrito, no app ou no site
                     , "
647             "do meu jeito e no meu tempo. Quando alguem liga
                     tentando me convencer na hora, parece que estao
                     "
648             "escondendo alguma coisa nas letras miudas. Se o
                     produto for bom, ele vai continuar bom amanha "
649             "tambem ? nao preciso decidir em 3 minutos de
                     chamada."
650         ),
651     },
652 ]
653
654 _FEWSHOT_TWIN: List[Dict] = [
655     {
656         "role": "user",
657         "content": (
658             "Contexto relevante:\nCliente adimplente, score
                     620, renda R$ 4.500, prefere agencia.\n\n"
659             "Pergunta: Voce investiria em Tesouro Direto?"
660         ),
661     },
662     {
663         "role": "assistant",
664         "content": (
665             "Ja ouvi falar do Tesouro Direto, mas confesso que
                     nunca mexi nisso. Com R$ 4.500 de renda, "
666             "depois de pagar as contas fixas sobra pouco ? uns
                     R$ 400 por mes, quando sobra. Ficaria "
667             "mais tranquilo conversando pessoalmente com alguem
                     do banco antes de colocar dinheiro em "
```

```
668         "qualquer investimento. O que me preocupa e: e se
669         eu precisar do dinheiro antes do prazo?"
670     ),
671     {
672         "role": "user",
673         "content": (
674             "Contexto relevante:\nCliente inadimplente, score
675             420, renda variavel R$ 2.100.\n\n"
676             "Pergunta: O que voce acha do programa de
677             renegociacao com 80% de desconto nos juros?"
678         ),
679     },
680     {
681         "role": "assistant",
682         "content": (
683             "Isso seria uma saida que eu precisava. 80% de
684             desconto nos juros? Eu toparia na hora ? "
685             "o que me paralisa e justamente os juros que so
686             crescem. Com R$ 2.100 de renda, se a "
687             "parcela ficar em algo que eu consiga pagar todo
688             mes sem apertar demais, eu assino. "
689             "O que eu quero e sair dessa situacao e recuperar
690             meu nome. Tem como parcelar em bastantes vezes?"
691         ),
692     },
693 ]
694
695 def _fewshot(modos: List[str]) -> List[Dict]:
696     if modos == "segmento":
697         return _FEWSHOT_SEGMENTO
698     if modos == "persona":
699         return _FEWSHOT_PERSONA
700     if modos == "twin":
701         return _FEWSHOT_TWIN
702     return []
703
704 def _despachar_tool(tool: Dict) -> str:
705     if tool["name"] == "consultar_feedback_nps":
```

```

702     inp = tool["input"]
703     return _consultar_athena_nps(
704         inp["tipo_consulta"], inp["identificador"], inp.get(
705             "produtos")
706     )
707     return "Ferramenta nao reconhecida."
708
709 def _montar_tool_results(output_content: List[Dict]) -> List[
710     Dict]:
711     results = []
712     for block in output_content:
713         if "toolUse" not in block:
714             continue
715         tool = block["toolUse"]
716         texto = _despachar_tool(tool) or "Sem dados disponiveis"
717         results.append({
718             "toolResult": {
719                 "toolUseId": tool["toolUseId"],
720                 "content": [{"text": texto}],
721             })
722     return results
723
724
725 def _chamar_claude(system: str, contexto_rag: str, pergunta:
726     str, modo: str = "segmento") -> str:
727     """Invoca Claude via Converse API com loop agentic tool_use
728     para NPS Athena."""
729     conteudo = (f"Contexto relevante:\n{contexto_rag}\n\
730         nPergunta: {pergunta}"
731         if contexto_rag else pergunta)
732
733     msgs = [{"role": t["role"], "content": [{"text": t["content
734         "]}]} for t in _fewshot(modo)]
735     msgs.append({"role": "user", "content": [{"text": conteudo
736         "}]})
737
738     kwargs: Dict = {
739         "modelId": BEDROCK_MODEL_ID,

```

```

735     "system": [{"text": system}],
736     "inferenceConfig": {"maxTokens": 2048},
737 }
738 if _NPS_TOOL_CONFIG:
739     kwargs["toolConfig"] = _NPS_TOOL_CONFIG
740
741 for _ in range(6): # max 5 round-trips tool_use
742     resp = _bedrock_client.converse(**kwargs, messages=msgs
743     )
744     output_msg = resp["output"]["message"]
745     msgs.append(output_msg)
746
747     if resp["stopReason"] != "tool_use":
748         return next((b["text"] for b in output_msg["content
749         "] if "text" in b), "")
750
751     tool_results = _montar_tool_results(output_msg["content
752     "])
753     msgs.append({"role": "user", "content": tool_results})
754
755     return ""
756
757 def _maybe_avaliar(ic: Dict, avaliar: bool, contextos: List[str
758 ], pergunta: str, resultado: str) -> None:
759     if not avaliar:
760         return
761     ic["avaliacao_llm"] = _avaliar_llm("\n\n".join(filter(None,
762     contextos)), pergunta, resultado)
763
764 def _executar_twin(pergunta: str, dados: Dict, cliente_id: str,
765 payload: Dict, avaliar: bool) -> str:
766     cluster_id = payload.get("cluster_id")
767     if cluster_id is None and dados:
768         cluster_id = _classificar(dados)
769     cluster_id = int(cluster_id) if cluster_id is not None else
770     0
771     b = _rag_twin(cliente_id, pergunta)
772     s = _rag_graph_sync(cluster_id, pergunta)

```

```

769     g = _rag_graph(cluster_id, cliente_id, "twin")
770     d = _rag_documentos(pergunta)
771     e = _rag_entrevistas(cluster_id, cliente_id, "twin",
        pergunta)
772     contexto = "\n\n".join(filter(None, [b, s, g, d, e]))
773     resultado = _chamar_claude(_system_twin(cliente_id, dados),
        contexto, pergunta, modo="twin")
774     ic = _confiabilidade(b, s, g, resultado, d, e)
775     _maybe_avaluar(ic, avaliar, [b, s, g, d, e], pergunta,
        resultado)
776     return json.dumps({"cliente_id": cliente_id, "modo": "twin"
        , "resposta": resultado,
777                        "indice_confiabilidade": ic},
        ensure_ascii=False)
778
779
780 def _executar_persona(pergunta: str, dados: Dict, payload: Dict
    , avaliar: bool) -> str:
781     cluster_id = payload.get("cluster_id")
782     if cluster_id is None:
783         if not dados:
784             raise ValueError("Modo 'persona' requer 'cluster_id
        ' ou 'dados_cliente'.")
785         cluster_id = _classificar(dados)
786     cluster_id = int(cluster_id)
787     b = _rag_segmento(cluster_id, pergunta)
788     s = _rag_graph_sync(cluster_id, pergunta)
789     g = _rag_graph(cluster_id, "", "persona")
790     d = _rag_documentos(pergunta)
791     e = _rag_entrevistas(cluster_id, "", "persona", pergunta)
792     contexto = "\n\n".join(filter(None, [b, s, g, d, e]))
793     resultado = _chamar_claude(_system_persona(cluster_id),
        contexto, pergunta, modo="persona")
794     ic = _confiabilidade(b, s, g, resultado, d, e)
795     _maybe_avaluar(ic, avaliar, [b, s, g, d, e], pergunta,
        resultado)
796     p = _perfil(cluster_id)
797     return json.dumps({"cluster_id": cluster_id, "segmento": p.
        get("segmento", ""),
798                        "persona_nome": p.get("persona_nome", "")
        }, "modo": "persona",

```

```

799         "resposta": resultado, "
            indice_confiabilidade": ic},
            ensure_ascii=False)
800
801
802 def _executar_segmento(pergunta: str, dados: Dict, cliente_id:
    str, avaliar: bool) -> str:
803     ausentes = set(FEATURES) - set(dados.keys())
804     if ausentes:
805         raise ValueError(f"Campos ausentes em dados_cliente: {
            ausentes}")
806     cluster_id = _classificar(dados)
807     b = _rag_segmento(cluster_id, pergunta)
808     s = _rag_graph_sync(cluster_id, pergunta)
809     g = _rag_graph(cluster_id, cliente_id, "segmento")
810     d = _rag_documentos(pergunta)
811     e = _rag_entrevistas(cluster_id, cliente_id, "segmento",
        pergunta)
812     contexto = "\n\n".join(filter(None, [b, s, g, d, e]))
813     resultado = _chamar_claude(_system_segmento(cluster_id),
        contexto, pergunta, modo="segmento")
814     ic = _confiabilidade(b, s, g, resultado, d, e)
815     _maybe_avaliar(ic, avaliar, [b, s, g, d, e], pergunta,
        resultado)
816     p = _perfil(cluster_id)
817     return json.dumps({"cliente_id": cliente_id, "cluster_id":
        cluster_id,
818         "segmento": p.get("segmento", ""), "modo
            ": "segmento",
819         "resposta": resultado, "
            indice_confiabilidade": ic},
            ensure_ascii=False)
820
821
822 def _executar(payload: Dict) -> str:
823     pergunta: str = payload["pergunta"]
824     modo: str = payload.get("modo", "segmento")
825     dados: Dict = payload.get("dados_cliente", payload.get("
        dados", {}))
826     cliente_id: str = payload.get("cliente_id", "")

```



```
827     avaliar: bool = ENABLE_LLM_JUDGE or bool(payload.get("
828         avaliar"))
829
830     if modo == "twin":
831         return _executar_twin(pergunta, dados, cliente_id,
832             payload, avaliar)
833     if modo == "persona":
834         return _executar_persona(pergunta, dados, payload,
835             avaliar)
836     return _executar_segmento(pergunta, dados, cliente_id,
837         avaliar)
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857 def _atualizar_status(request_id: str, status: str, resultado:
858     str = None, erro: str = None):
859     agora = datetime.now(timezone.utc).isoformat()
860     expr = "SET #s = :s, atualizado_em = :a"
861     attrs = {"#s": status, "#a": agora}
862     names = {"#s": "status"}
863     if resultado is not None:
864         expr += ", resultado = :r"
865         attrs[":r"] = resultado
866     if erro is not None:
867         expr += ", erro = :e"
868         attrs[":e"] = erro
869
870     _dynamodb.Table(TABLE_NAME).update_item(
871         Key={"request_id": request_id},
872         UpdateExpression=expr,
873         ExpressionAttributeValues=attrs,
874         ExpressionAttributeNames=names,
875     )
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

```
def _notificar(request_id: str, status: str):
    if not SNS_TOPIC_ARN:
        return
    _sns.publish(
        TopicArn=SNS_TOPIC_ARN,
        Subject=f"Agente LangChain: {status}",
```

```

863         Message=json.dumps({"request_id": request_id, "status":
864                               status}),
865     )
866
867
868 def lambda_handler(event, context):
869     for record in event["Records"]:
870         payload = json.loads(record["body"])
871         request_id = payload["request_id"]
872
873         _atualizar_status(request_id, "PROCESSING")
874         try:
875             resultado = _executar(payload)
876             _atualizar_status(request_id, "COMPLETED",
877                               resultado=resultado)
878             _notificar(request_id, "COMPLETED")
879         except Exception:
880             erro = traceback.format_exc()
881             log.error("Erro ao processar request %s:\n%s",
882                       request_id, erro)
883             _atualizar_status(request_id, "FAILED", erro=erro)
884             _notificar(request_id, "FAILED")
885             raise

```

Listing A.9: Worker principal do pipeline assíncrono (versão ASCII para compilação).

A.5 Capítulo 12, 13 e 14

A.5.1 avaliar_metricas.py

```

1  """
2  Avaliacao de metricas de qualidade de resposta (loop externo ?
   Cap. 12).
3
4  Metricas implementadas:
5      - ROUGE-L (sobreposicao de subsequencia mais longa)
6      - BERTScore (similaridade semantica via embeddings)
7      - G-Eval (avaliacao LLM-based via Bedrock/Claude)
8
9  Uso sem AWS (offline):

```

```

10  python avaliar_metricas.py --modo offline
11
12  Uso com Bedrock (G-Eval):
13  python avaliar_metricas.py --modo geval \
14      --referencia "Resposta esperada do sistema." \
15      --candidata  "Resposta gerada pelo agente."
16  """
17
18  import argparse
19  import json
20  import os
21  import sys
22  from dataclasses import dataclass, asdict
23  from typing import Optional
24
25
26  # ?? Estrutura de resultado
27      ?????????????????????????????????????????????????????????????????
28
29  @dataclass
30  class ResultadoAvaliacao:
31      rouge_l: float
32      bertscore_f1: Optional[float]
33      geval_score: Optional[float]
34      geval_razao: Optional[str]
35
36      def resumo(self) -> str:
37          linhas = [f" ROUGE-L:          {self.rouge_l:.4f}"]
38          if self.bertscore_f1 is not None:
39              linhas.append(f" BERTScore F1: {self.bertscore_f1
40                  :.4f}")
41          if self.geval_score is not None:
42              linhas.append(f" G-Eval:          {self.geval_score
43                  :.2f}/5")
44          if self.geval_razao:
45              linhas.append(f" Razao G-Eval: {self.
46                  geval_razao}")
47          return "\n".join(linhas)

```

```

46 # ?? ROUGE-L (sem dependencia externa)
   ?????????????????????????????????????????????
47
48 def _lcs_length(a: list, b: list) -> int:
49     """Comprimento da subsequencia comum mais longa."""
50     m, n = len(a), len(b)
51     dp = [[0] * (n + 1) for _ in range(m + 1)]
52     for i in range(1, m + 1):
53         for j in range(1, n + 1):
54             if a[i - 1] == b[j - 1]:
55                 dp[i][j] = dp[i - 1][j - 1] + 1
56             else:
57                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
58     return dp[m][n]
59
60
61 def rouge_l(referencia: str, candidata: str) -> float:
62     """Retorna F1 do ROUGE-L entre referencia e candidata."""
63     ref_tokens = referencia.lower().split()
64     cand_tokens = candidata.lower().split()
65     if not ref_tokens or not cand_tokens:
66         return 0.0
67     lcs = _lcs_length(ref_tokens, cand_tokens)
68     precision = lcs / len(cand_tokens)
69     recall = lcs / len(ref_tokens)
70     if precision + recall == 0:
71         return 0.0
72     return 2 * precision * recall / (precision + recall)
73
74
75 # ?? BERTScore (via transformers, opcional)
   ?????????????????????????????????????????????
76
77 def bertscore_f1(referencia: str, candidata: str) -> Optional[
78     float]:
79     """
80     Calcula BERTScore F1. Retorna None se bert_score nao
81     estiver instalado.
82     Instalar: pip install bert-score
83     """
84     try:

```

```

83     from bert_score import score as bert_score_fn
84     P, R, F1 = bert_score_fn(
85         [candidata], [referencia],
86         lang="pt",
87         rescale_with_baseline=False,
88         verbose=False,
89     )
90     return float(F1[0])
91 except ImportError:
92     return None
93 except Exception as e:
94     print(f"[AVISO] BERTScore falhou: {e}", file=sys.stderr)
95     return None
96
97
98 # ?? G-Eval via Bedrock
99     ?????????????????????????????????????????????????????????????????????
100 _GEVAL_PROMPT = """Voce e um avaliador especialista em sistemas
    de IA para bancos.
101
102 Avalie a RESPOSTA GERADA comparando com a RESPOSTA REFERENCIA
    nos criterios:
103 1. Fidelidade ao contexto (a resposta usa apenas informacoes
    disponiveis?)
104 2. Completude (cobre os pontos essenciais da referencia?)
105 3. Clareza (linguagem adequada para o cliente bancario?)
106 4. Ausencia de alucinacoes (sem informacoes inventadas?)
107 5. Conformidade LGPD (nao expoe dados pessoais indevidos?)
108
109 Responda SOMENTE em JSON com os campos:
110     score: numero de 0 a 5 (inteiro)
111     razao: string com justificativa em ate 2 frases
112
113 RESPOSTA REFERENCIA:
114 {referencia}
115
116 RESPOSTA GERADA:
117 {candidata}
118

```

```

119 JSON: ""
120
121
122 def geval(referencia: str, candidata: str, region: str = "us-
    east-1",
123         model_id: str = "us.anthropic.claude-sonnet
        -4-5-20250514-v1:0") -> tuple[Optional[float],
        Optional[str]]:
124     """
125     Avaliacao G-Eval via Claude no Bedrock.
126     Retorna (score, razao) ou (None, None) se Bedrock nao
        disponivel.
127     """
128     try:
129         import boto3
130         client = boto3.client("bedrock-runtime", region_name=
            region)
131         prompt = _GEVAL_PROMPT.format(referencia=referencia,
            candidata=candidata)
132         body = json.dumps({
133             "anthropic_version": "bedrock-2023-05-31",
134             "max_tokens": 256,
135             "messages": [{"role": "user", "content": prompt}],
136         })
137         resp = client.invoke_model(modelId=model_id, body=body)
138         text = json.loads(resp["body"].read())["content"][0]["
            text"].strip()
139         # extrair JSON mesmo se vier com marcadores de bloco
140         if "```" in text:
141             text = text.split("```")[1].lstrip("json").strip()
142         data = json.loads(text)
143         return float(data.get("score", 0)), data.get("razao", "
            ")
144     except Exception as e:
145         print(f"[AVISO] G-Eval Bedrock indisponivel: {e}", file
            =sys.stderr)
146         return None, None
147
148 # ?? Funcao principal de avaliacao
149 """

```

```

150
151 def avaliar(referencia: str, candidata: str,
152             usar_bertscore: bool = False,
153             usar_geval: bool = False) -> ResultadoAvaliacao:
154     rl = rouge_l(referencia, candidata)
155     bs = bertscore_f1(referencia, candidata) if usar_bertscore
156         else None
157     gs, gr = geval(referencia, candidata) if usar_geval else (
158         None, None)
159     return ResultadoAvaliacao(rouge_l=rl, bertscore_f1=bs,
160                               geval_score=gs, geval_razao=gr)
161
162 # ?? Modo batch: avaliar arquivo JSONL
163     ?????????????????????????????????????????????????????????
164
165 def avaliar_batch(caminho: str, usar_geval: bool = False) ->
166     list[dict]:
167     """
168     Arquivo JSONL com linhas: {"id": ..., "referencia": ..., "
169         candidata": ...}
170     Retorna lista de dicionarios com metricas.
171     """
172     resultados = []
173     with open(caminho, encoding="utf-8") as f:
174         for linha in f:
175             item = json.loads(linha.strip())
176             r = avaliar(item["referencia"], item["candidata"],
177                         usar_geval=usar_geval)
178             resultados.append({"id": item.get("id", "?"), **
179                               asdict(r)})
180     return resultados
181
182 # ?? CLI
183     ?????????????????????????????????????????????????????????
184     ?????????????????????????????????????????????????????????
185     ?????????????????????????????????????????????????????????
186
187 def _demo_offline():
188     pares = [

```

```

180         ("Clientes conservadores priorizam liquidez diaria e
181           seguranca do capital.",
182           "Investidores conservadores preferem produtos com
183             liquidez imediata."),
184         ("O Tesouro Direto e garantido pelo governo federal.",
185           "O Tesouro Direto oferece garantia do governo."),
186         ("Fundos de renda variavel possuem alto risco.",
187           "Poupanca e o investimento mais seguro do mercado."),
188     ]
189     print(f"{'Ref (trunc)':<45} {'Cand (trunc)':<45} {'ROUGE-L
190           ':>8}")
191     print("-" * 103)
192     for ref, cand in pares:
193         rl = rouge_l(ref, cand)
194         print(f"{ref[:43]:<45} {cand[:43]:<45} {rl:>8.4f}")
195
196 if __name__ == "__main__":
197     parser = argparse.ArgumentParser(description="Avaliacao de
198       metricas de resposta RAG")
199     parser.add_argument("--modo", choices=["offline", "geval",
200       "batch"],
201       default="offline", help="Modo de
202       execucao")
203     parser.add_argument("--referencia", help="Texto de
204       referencia (modo geval)")
205     parser.add_argument("--candidata", help="Texto candidato (
206       modo geval)")
207     parser.add_argument("--arquivo", help="JSONL com pares (
208       modo batch)")
209     parser.add_argument("--bertscore", action="store_true",
210       help="Incluir BERTScore")
211     args = parser.parse_args()
212
213     if args.modo == "offline":
214         print("Demo ROUGE-L offline:")
215         _demo_offline()
216
217     elif args.modo == "geval":
218         if not args.referencia or not args.candidata:

```



```

210         parser.error("--referencia e --candidata sao
                obrigatorios no modo geval")
211         resultado = avaliar(args.referencia, args.candidata,
212                             usar_bertscore=args.bertscore,
213                             usar_geval=True)
214
215         print(resultado.resumo())
216
217     elif args.modos == "batch":
218         if not args.arquivo:
219             parser.error("--arquivo e obrigatorio no modo batch
                ")
220
221         resultados = avaliar_batch(args.arquivo, usar_geval=
                args.modos == "geval")
222         print(json.dumps(resultados, ensure_ascii=False, indent
                =2))

```

Listing A.10: Metricas automaticas de avaliacao (versao ASCII para compilacao).

A.5.2 llm_judge.py

```

1  """
2  LLM as Judge ? Avaliacao automatica de respostas por segmento (
    loop externo ? Cap. 13).
3
4  O juiz Claude avalia cada resposta do agente em 4 dimensoes
    calibradas
5  para o contexto bancario:
6      - Fidelidade ao RAG (a resposta cita apenas o que foi
        recuperado?)
7      - Adequacao ao perfil (tom e complexidade adequados ao
        segmento?)
8      - Completude (cobre todos os pontos da pergunta?)
9      - Conformidade LGPD (sem exposicao de dados pessoais?)
10
11  Uso offline (mock, sem Bedrock):
12      python llm_judge.py --modo mock
13
14  Uso com Bedrock:
15      python llm_judge.py \
16          --pergunta "Qual o melhor investimento para perfil
            conservador?" \

```

```

17     --resposta "Recomendamos Tesouro Selic com liquidez diaria
18         ." \
19     --contexto "Cliente PF, cluster conservador, saldo 15000."
20         \
21     --segmento PF
22     """
23
24 import argparse
25 import json
26 import os
27 import sys
28
29 from dataclasses import dataclass, asdict
30 from typing import Optional
31
32 # ?? Prompt por segmento
33     ?????????????????????????????????????????????????????????????
34
35 _CRITERIOS_BASE = """
36 1. Fidelidade ao RAG (0-3): a resposta usa apenas informacoes
37     recuperadas? (0=inventou, 3=totalmente fiel)
38 2. Adequacao ao perfil (0-3): tom e complexidade condizem com o
39     segmento {segmento}? (0=inadequado, 3=perfeito)
40 3. Completude (0-2): cobre todos os pontos da pergunta? (0=
41     parcial, 2=completo)
42 4. Conformidade LGPD (0-2): ausencia de dados pessoais
43     indevidos? (0=violacao, 2=conforme)
44 """
45
46 _JUDGE_PROMPT = """Voce e um avaliador especialista em sistemas
47     de IA para bancos brasileiros.
48
49     SEGMENTO DO CLIENTE: {segmento}
50     CONTEXTO RAG DISPONIVEL: {contexto}
51     PERGUNTA DO USUARIO: {pergunta}
52     RESPOSTA DO AGENTE: {resposta}
53
54     CRITERIOS DE AVALIACAO (pontuacao maxima = 10):
55     {criterios}
56
57     Responda SOMENTE em JSON com os campos:

```

```

50     fidelidade_rag: int (0-3)
51     adequacao_perfil: int (0-3)
52     completude: int (0-2)
53     conformidade_lgpd: int (0-2)
54     score_total: int (0-10, soma dos anteriores)
55     aprovado: bool (true se score_total >= 7)
56     razao: string (justificativa em ate 3 frases)
57
58     JSON: ""
59
60     _MOCK_RESPONSES = {
61         "PF": {"fidelidade_rag": 3, "adequacao_perfil": 3, "
62             completude": 2,
63             "conformidade_lgpd": 2, "score_total": 10, "aprovado
64                 ": True,
65             "razao": "[MOCK] Resposta fiel ao contexto, tom
66                 adequado para PF conservador."},
67         "PJ": {"fidelidade_rag": 2, "adequacao_perfil": 2, "
68             completude": 1,
69             "conformidade_lgpd": 2, "score_total": 7, "aprovado"
70                 : True,
71             "razao": "[MOCK] Resposta adequada para PJ,
72                 completude parcial."},
73         "FP": {"fidelidade_rag": 1, "adequacao_perfil": 2, "
74             completude": 2,
75             "conformidade_lgpd": 2, "score_total": 7, "aprovado"
76                 : True,
77             "razao": "[MOCK] Fidelidade ao RAG questionavel,
78                 adequacao ao segmento FP OK."},
79     }
80
81     # ?? Estrutura de resultado
82     ?????????????????????????????????????????????????????????????
83
84     @dataclass
85     class VeredictJudge:
86         segmento: str
87         fidelidade_rag: int
88         adequacao_perfil: int
89         completude: int

```

```

81     conformidade_lgpd: int
82     score_total: int
83     aprovado: bool
84     razao: str
85
86     def resumo(self) -> str:
87         status = "APROVADO ?" if self.aprovado else "REPROVADO
88             ?"
89         return (
90             f" Segmento:           {self.segmento}\n"
91             f" Fidelidade RAG:         {self.fidelidade_rag}/3\n"
92             f" Adequacao perfil:         {self.adeguacao_perfil}/3\n"
93             f" Completude:               {self.completude}/2\n"
94             f" Conformidade LGPD: {self.conformidade_lgpd}/2\n"
95             f" Score total:             {self.score_total}/10 ? {
96                 status}\n"
97             f" Razao:                   {self.razao}"
98         )
99
100     # ?? Avaliacao via Bedrock
101     """
102     """
103
104     def julgar(pergunta: str, resposta: str, contexto: str = "",
105         segmento: str = "PF",
106         region: str = "us-east-1",
107         model_id: str = "us.anthropic.claude-sonnet
108             -4-5-20250514-v1:0") -> VerdictJudge:
109         """Chama o juiz Claude via Bedrock e retorna o veredito
110             estruturado."""
111         try:
112             import boto3
113             client = boto3.client("bedrock-runtime", region_name=
114                 region)
115             criterios = _CRITERIOS_BASE.format(segmento=segmento)
116             prompt = _JUDGE_PROMPT.format(
117                 segmento=segmento, contexto=contexto,
118                 pergunta=pergunta, resposta=resposta, criterios=
119                 criterios
120             )

```

```

114     body = json.dumps({
115         "anthropic_version": "bedrock-2023-05-31",
116         "max_tokens": 512,
117         "messages": [{"role": "user", "content": prompt}],
118     })
119     raw = client.invoke_model(modelId=model_id, body=body)
120     text = json.loads(raw["body"].read())["content"][0]["
121         text"].strip()
122     if "```" in text:
123         text = text.split("```")[1].lstrip("json").strip()
124     data = json.loads(text)
125     return VerdictJudge(segmento=segmento, **data)
126 except Exception as e:
127     print(f"[AVISO] Bedrock indisponível: {e}", file=sys.
128         stderr)
129     return _mock_judge(segmento)
130
131 def _mock_judge(segmento: str = "PF") -> VerdictJudge:
132     """Retorna um veredito mock para testes offline."""
133     m = _MOCK_RESPONSES.get(segmento, _MOCK_RESPONSES["PF"])
134     return VerdictJudge(segmento=segmento, **m)
135
136 # ?? Avaliacao batch
137     ?????????????????????????????????????????????????????????????????????????????????????
138
139 def julgar_batch(caminho: str, usar_bedrock: bool = False) ->
140     list[dict]:
141     """
142     JSONL com linhas: {"id":..., "pergunta":..., "resposta
143         ":...,
144         "contexto":..., "segmento":...}
145     """
146     resultados = []
147     with open(caminho, encoding="utf-8") as f:
148         for linha in f:
149             item = json.loads(linha.strip())
150             if usar_bedrock:
151                 v = julgar(item["pergunta"], item["resposta"],

```

```

149         item.get("contexto", ""), item.get("
150             segmento", "PF"))
151     else:
152         v = _mock_judge(item.get("segmento", "PF"))
153         resultados.append({"id": item.get("id", "?"), **
154             asdict(v)})
155     return resultados
156
157 # ?? CLI
158
159 if __name__ == "__main__":
160     parser = argparse.ArgumentParser(description="LLM as Judge
161         ? avaliacao por segmento")
162     parser.add_argument("--modo", choices=["mock", "bedrock", "
163         batch"],
164         default="mock")
165     parser.add_argument("--pergunta", help="Pergunta do usuario
166         ")
167     parser.add_argument("--resposta", help="Resposta do agente"
168         )
169     parser.add_argument("--contexto", default="", help="
170         Contexto RAG utilizado")
171     parser.add_argument("--segmento", default="PF", choices=["
172         PF", "PJ", "FP", "PJA"],
173         help="Segmento do cliente")
174     parser.add_argument("--arquivo", help="JSONL para modo
175         batch")
176     args = parser.parse_args()
177
178     if args.modo == "mock":
179         for seg in ["PF", "PJ", "FP"]:
180             v = _mock_judge(seg)
181             print(f"\n{'='*50}")
182             print(v.resumo())
183
184     elif args.modo == "bedrock":
185         if not args.pergunta or not args.resposta:

```

```

178         parser.error("--pergunta e --resposta sao
179                        obrigatorios no modo bedrock")
180     v = julgar(args.pergunta, args.resposta, args.contexto,
181               args.segmento)
182     print(v.resumo())
183
184     elif args.modos == "batch":
185         if not args.arquivo:
186             parser.error("--arquivo e obrigatorio no modo batch
187                           ")
188         resultados = julgar_batch(args.arquivo, usar_bedrock=
189                                 False)
190         print(json.dumps(resultados, ensure_ascii=False, indent
191                         =2))

```

Listing A.11: Avaliacao qualitativa com LLM as Judge (versao ASCII para compilacao).

A.5.3 golden_dataset.py

```

1  """
2  Golden Dataset ? Criacao, execucao e regressao de qualidade (
3      loop externo ? Cap. 14).
4
5  Fluxo:
6      1. Carregar golden dataset (JSONL com pares pergunta/resposta
7         esperada + metadados)
8      2. Executar o agente atual contra cada caso
9      3. Avaliar com ROUGE-L e/ou LLM Judge
10     4. Comparar com baseline anterior e exibir diff de regressao
11
12  Estrutura de um item do golden dataset:
13  {
14      "id": "GD-001",
15      "segmento": "PF",
16      "categoria": "investimentos",
17      "pergunta": "Qual o melhor investimento para perfil
18                  conservador?",
19      "resposta_esperada": "Para perfil conservador, recomendamos
20                          Tesouro Selic...",
21      "threshold_rouge": 0.4,
22      "threshold_judge": 7
23  }

```

```

20
21 Uso offline (mock):
22     python golden_dataset.py --modo criar      # gera
23         golden_dataset_sample.jsonl
24     python golden_dataset.py --modo rodar      # executa regressao
25         mock
26
27     python golden_dataset.py --modo diff --baseline resultados_v1
28         .json --atual resultados_v2.json
29
30 """
31
32 import argparse
33 import json
34 import os
35 import sys
36
37 from dataclasses import dataclass, asdict
38 from typing import Optional
39 from avaliar_metricas import rouge_l
40 from llm_judge import _mock_judge
41
42
43 # ?? Dataset de exemplo
44     ????????????????????????????????????????????????????????????
45
46
47 GOLDEN_SAMPLE = [
48     {
49         "id": "GD-001", "segmento": "PF", "categoria": "
50             investimentos",
51         "pergunta": "Qual o melhor investimento para perfil
52             conservador?",
53         "resposta_esperada": "Para perfil conservador,
54             recomendamos Tesouro Selic com liquidez diaria e CDB
55             com protecao do FGC.",
56         "threshold_rouge": 0.35, "threshold_judge": 7,
57     },
58     {
59         "id": "GD-002", "segmento": "PF", "categoria": "cartao"
60         ,
61         "pergunta": "Quais sao os beneficios do cartao gold?",
62         "resposta_esperada": "O cartao gold oferece programa de
63             pontos, seguro viagem e isencao de anuidade para
64             clientes premium.",

```


[illegible]

```

78 class ResultadoCaso:
79     id: str
80     segmento: str
81     categoria: str
82     rouge_l: float
83     judge_score: Optional[int]
84     aprovado_rouge: bool
85     aprovado_judge: bool
86     aprovado: bool
87     resposta_gerada: str
88
89
90 # ?? Criacao do golden dataset
91     ?????????????????????????????????????????????????????????????
92
93 def criar_dataset(caminho: str = "golden_dataset.jsonl"):
94     with open(caminho, "w", encoding="utf-8") as f:
95         for item in GOLDEN_SAMPLE:
96             f.write(json.dumps(item, ensure_ascii=False) + "\n"
97                       )
98         print(f"[golden_dataset] Dataset criado: {caminho} ({len(
99             GOLDEN_SAMPLE)} casos)")
100
101 # ?? Execucão de regressão
102     ?????????????????????????????????????????????????????????????
103
104 def rodar_regressao(caminho_dataset: str = "golden_dataset.
105     jsonl",
106                     usar_bedrock: bool = False) -> list[
107         ResultadoCaso]:
108
109     """
110     Para cada caso do golden dataset:
111     1. Obtem resposta do agente (mock offline ou real)
112     2. Calcula ROUGE-L vs resposta esperada
113     3. Executa LLM Judge (mock ou Bedrock)
114     4. Verifica se passou nos thresholds
115     """
116
117     resultados = []
118     with open(caminho_dataset, encoding="utf-8") as f:
119         casos = [json.loads(l) for l in f]

```

```
113
114     for caso in casos:
115         cid = caso["id"]
116         # obter resposta do agente
117         if usar_bedrock:
118             resposta_gerada = _chamar_agente_bedrock(caso["
119                 pergunta"], caso["segmento"])
120         else:
121             resposta_gerada = _RESPOSTAS_AGENTE MOCK.get(cid, "
122                 Resposta nao disponivel.")
123
124         # avaliar ROUGE-L
125         rl = rouge_l(caso["resposta_esperada"], resposta_gerada
126             )
127         aprovado_rouge = rl >= caso.get("threshold_rouge", 0.3)
128
129         # avaliar LLM Judge
130         if usar_bedrock:
131             from llm_judge import julgar
132             v = julgar(caso["pergunta"], resposta_gerada,
133                 segmento=caso["segmento"])
134         else:
135             v = _mock_judge(caso["segmento"])
136         aprovado_judge = v.score_total >= caso.get("
137             threshold_judge", 7)
138
139         resultados.append(ResultadoCaso(
140             id=cid,
141             segmento=caso["segmento"],
142             categoria=caso["categoria"],
143             rouge_l=round(rl, 4),
144             judge_score=v.score_total,
145             aprovado_rouge=aprovado_rouge,
146             aprovado_judge=aprovado_judge,
147             aprovado=aprovado_rouge and aprovado_judge,
148             resposta_gerada=resposta_gerada,
149         ))
150
151     return resultados
```

```

149 def _chamar_agente_bedrock(pergunta: str, segmento: str) -> str
150 :
151     """Stub ? substituir pelo invoke real do agente LangGraph
152     em producao."""
153     raise NotImplementedError("Integracao com agente Bedrock
154     nao implementada aqui. Use aws_pipeline_clientes.py.")
155
156 # ?? Diff de regressao
157     ?????????????????????????????????????????????????????????????
158
159 def diff_regressao(baseline: list[dict], atual: list[dict]) ->
160 dict:
161     """
162     Compara dois conjuntos de resultados e identifica:
163     - Regressoes: casos que passaram no baseline e falharam
164     no atual
165     - Melhorias: casos que falharam no baseline e passaram no
166     atual
167     - Estavel: sem mudanca
168     """
169     base_map = {r["id"]: r for r in baseline}
170     atual_map = {r["id"]: r for r in atual}
171
172     regressoes, melhorias, estavel = [], [], []
173     for cid, r_atual in atual_map.items():
174         r_base = base_map.get(cid)
175         if r_base is None:
176             continue
177         if r_base["aprovado"] and not r_atual["aprovado"]:
178             delta_rouge = round(r_atual["rouge_l"] - r_base["rouge_l"], 4)
179             regressoes.append({"id": cid, "delta_rouge":
180                 delta_rouge,
181                 "judge_base": r_base["judge_score"],
182                 "judge_atual": r_atual["judge_score"]})
183         elif not r_base["aprovado"] and r_atual["aprovado"]:
184             melhorias.append({"id": cid})
185         else:

```

```

179         estavel.append(cid)
180
181     total = len(atual_map)
182     aprovados = sum(1 for r in atual_map.values() if r["
183         aprovado"])
184     return {
185         "resumo": {
186             "total": total,
187             "aprovados": aprovados,
188             "taxa_aprovacao": round(aprovados / total, 3) if
189                 total else 0,
190         },
191         "regressoes": regressoes,
192         "melhorias": melhorias,
193         "estavel": estavel,
194     }
195
196 # ?? CLI
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211

```

```
212         default="rodar")
213     parser.add_argument("--dataset", default="golden_dataset.
214         jsonl")
215     parser.add_argument("--saida", default="resultados_golden.
216         json")
217     parser.add_argument("--baseline", help="JSON de baseline (
218         modo diff)")
219     parser.add_argument("--atual", help="JSON atual (modo diff)
220         ")
221     parser.add_argument("--bedrock", action="store_true",
222         help="Usar Bedrock (padrao: mock
223         offline)")
224     args = parser.parse_args()
225
226     if args.modo == "criar":
227         criar_dataset(args.dataset)
228
229     elif args.modo == "rodar":
230         if not os.path.exists(args.dataset):
231             print(f"Dataset nao encontrado: {args.dataset}.
232                 Criando sample...")
233             criar_dataset(args.dataset)
234         resultados = rodar_regressao(args.dataset, usar_bedrock
235             =args.bedrock)
236         _imprimir_resultados(resultados)
237         with open(args.saida, "w", encoding="utf-8") as f:
238             json.dump([asdict(r) for r in resultados], f,
239                 ensure_ascii=False, indent=2)
240         print(f"\nResultados salvos em: {args.saida}")
241
242     elif args.modo == "diff":
243         if not args.baseline or not args.atual:
244             parser.error("--baseline e --atual sao obrigatorios
245                 no modo diff")
246         with open(args.baseline, encoding="utf-8") as f:
247             base = json.load(f)
248         with open(args.atual, encoding="utf-8") as f:
249             atual = json.load(f)
250         resultado_diff = diff_regressao(base, atual)
251         print(json.dumps(resultado_diff, ensure_ascii=False,
252             indent=2))
```

Listing A.12: Criacao e regressao com golden dataset (versao ASCII para compilacao).

A.5.4 golden_baseline.json

```
1  [
2    {
3      "id": "GD-001",
4      "segmento": "PF",
5      "categoria": "investimentos",
6      "rouge_l": 0.2857,
7      "judge_score": 10,
8      "aprovado_rouge": false,
9      "aprovado_judge": true,
10     "aprovado": false,
11     "resposta_gerada": "Para clientes conservadores, Tesouro
                          Selic e a opcao mais indicada pela liquidez imediata."
12   },
13   {
14     "id": "GD-002",
15     "segmento": "PF",
16     "categoria": "cartao",
17     "rouge_l": 0.3448,
18     "judge_score": 10,
19     "aprovado_rouge": true,
20     "aprovado_judge": true,
21     "aprovado": true,
22     "resposta_gerada": "O cartao gold da pontos a cada compra e
                          tem seguro viagem incluido."
23   },
24   {
25     "id": "GD-003",
26     "segmento": "PJ",
27     "categoria": "credito",
28     "rouge_l": 0.3448,
29     "judge_score": 7,
30     "aprovado_rouge": true,
31     "aprovado_judge": true,
32     "aprovado": true,
33     "resposta_gerada": "Para capital de giro, analisamos o
                          faturamento da empresa nos ultimos meses."
```

```
34 },
35 {
36   "id": "GD-004",
37   "segmento": "FP",
38   "categoria": "investimentos",
39   "rouge_1": 0.25,
40   "judge_score": 7,
41   "aprovado_rouge": false,
42   "aprovado_judge": true,
43   "aprovado": false,
44   "resposta_gerada": "Private banking oferece fundos
                        exclusivos e gestao personalizada do patrimonio."
45 }
46 ]
```

Listing A.13: Baseline de referencia para comparacao (versao ASCII para compilacao).

A.6 Capitulo 18

A.6.1 app.js

```
1 ((() => {
2   const config = window.APP_CONFIG || {};
3   const apiUrl = (config.apiUrl || '').replace(/\/$/ , '');
4
5   const endpointEl = document.getElementById('api-endpoint');
6   const queryForm = document.getElementById('query-form');
7   const uploadForm = document.getElementById('upload-form');
8   const queryStatus = document.getElementById('query-status');
9   const queryResult = document.getElementById('query-result');
10  const uploadResult = document.getElementById('upload-result')
11    ;
12  const queryBadge = document.getElementById('query-badge');
13  const uploadBadge = document.getElementById('upload-badge');
14  const fileInput = document.getElementById('arquivo_documento
15    ');
16  const textArea = document.getElementById('texto_documento');
17
18  endpointEl.textContent = apiUrl || 'API nao configurada.
19    Ajuste window.APP_CONFIG.apiUrl no deploy.';
```



```
18 function setBadge(el, state, text) {
19   el.className = 'status-badge ${state}';
20   el.textContent = text;
21 }
22
23 function maybeNumber(value) {
24   return value === '' || value === null || value ===
25     undefined ? undefined : Number(value);
26 }
27
28 function buildDadosCliente() {
29   const data = {
30     idade: maybeNumber(document.getElementById('idade').value
31     ),
32     renda_mensal: maybeNumber(document.getElementById('
33     renda_mensal').value),
34     saldo_medio: maybeNumber(document.getElementById('
35     saldo_medio').value),
36     transacoes_mes: maybeNumber(document.getElementById('
37     transacoes_mes').value),
38     score_credito: maybeNumber(document.getElementById('
39     score_credito').value),
40     num_produtos: maybeNumber(document.getElementById('
41     num_produtos').value),
42     canal_digital: document.getElementById('canal_digital').
43       checked,
44     inadimplente: document.getElementById('inadimplente').
45       checked,
46   };
47
48   const filled = Object.fromEntries(Object.entries(data).
49     filter(([ , value]) => value !== undefined));
50   return Object.keys(filled).length ? filled : undefined;
51 }
52
53 async function requestJson(path, payload, method = 'POST') {
54   if (!apiUrl) {
55     throw new Error('API URL nao configurada no frontend.');
```

```
49     method,
50     headers: {
51       'Content-Type': 'application/json',
52     },
53     body: payload ? JSON.stringify(payload) : undefined,
54   });
55
56   const text = await response.text();
57   const data = text ? JSON.parse(text) : {};
58
59   if (!response.ok) {
60     throw new Error(data.erro || data.error || 'HTTP ${
61       response.status}');
62   }
63
64   return data;
65 }
66
67 async function pollStatus(requestId) {
68   let waitTime = 1000;
69   const maxWait = 8000;
70   const started = Date.now();
71
72   while (true) {
73     const data = await requestJson(`/status/${requestId}`,
74       null, 'GET');
75     const elapsed = Math.round((Date.now() - started) / 1000)
76       ;
77     queryStatus.textContent = JSON.stringify({
78       request_id: requestId,
79       elapsed_seconds: elapsed,
80       payload: data,
81     }, null, 2);
82
83     if (data.status === 'COMPLETED') {
84       setBadge(queryBadge, 'success', 'Concluido');
85
86       // result pode chegar como string JSON ou como objeto
87       let result = data.result;
88       if (typeof result === 'string') {
```

```
86     try { result = JSON.parse(result); } catch (_) { /*
87         mantem string */ }
88
89     if (result && typeof result === 'object' && result.
90         resposta) {
91         // exhibe o texto da resposta do agente
92         queryResult.textContent = result.resposta;
93         // atualiza o painel de status com os metadados
94         completos
95         queryStatus.textContent = JSON.stringify({
96             request_id: requestId,
97             status: data.status,
98             completed_at: data.completed_at,
99             metadata: (({ resposta, ...rest }) => rest)(result)
100             ,
101             }, null, 2);
102     } else if (typeof result === 'string') {
103         queryResult.textContent = result;
104     } else {
105         queryResult.textContent = JSON.stringify(result, null
106             , 2);
107     }
108     return;
109 }
110
111 if (data.status === 'FAILED') {
112     throw new Error(data.error || 'Processamento falhou.');
```

```
113 }
114
115 setBadge(queryBadge, 'loading', 'Processando (${data.
116     status || 'PENDING'})');
117 await new Promise((resolve) => setTimeout(resolve,
118     waitTime));
119 waitTime = Math.min(waitTime * 1.5, maxWait);
120 }
121 }
122
123 queryForm.addEventListener('submit', async (event) => {
124     event.preventDefault();
125     setBadge(queryBadge, 'loading', 'Enviando');
```

```
120     queryStatus.textContent = 'Enfileirando requisicao...';
121     queryResult.textContent = 'Aguardando resposta...';
122
123     try {
124         const payload = {
125             pergunta: document.getElementById('pergunta').value,
126             modo: document.getElementById('modo').value,
127         };
128
129         const clienteId = document.getElementById('cliente_id').
            value.trim();
130         const clusterId = document.getElementById('cluster_id').
            value.trim();
131         const dadosCliente = buildDadosCliente();
132
133         if (clienteId) payload.cliente_id = clienteId;
134         if (clusterId) payload.cluster_id = Number(clusterId);
135         if (dadosCliente) payload.dados_cliente = dadosCliente;
136
137         const accepted = await requestJson('/query', payload, '
            POST');
138         queryStatus.textContent = JSON.stringify(accepted, null,
            2);
139         setBadge(queryBadge, 'loading', 'Em fila');
140
141         if (!accepted.request_id) {
142             throw new Error('API nao retornou request_id.');
```

```
143         }
144
145         await pollStatus(accepted.request_id);
146     } catch (error) {
147         setBadge(queryBadge, 'error', 'Falhou');
148         queryResult.textContent = error.message;
149     }
150 });
151
152 fileInput.addEventListener('change', async (event) => {
153     const [file] = event.target.files;
154     if (!file) return;
155
156     try {
```

```
157     const content = await file.text();
158     textArea.value = content;
159     if (!document.getElementById('titulo_documento').value.
160         trim()) {
161         document.getElementById('titulo_documento').value =
162             file.name.replace(/\.([\^.] +$)/, '');
163     }
164 } catch (error) {
165     uploadResult.textContent = 'Falha ao ler arquivo: ${error
166         .message}';
167     setBadge(uploadBadge, 'error', 'Falhou');
168 }
169 });
170
171 uploadForm.addEventListener('submit', async (event) => {
172     event.preventDefault();
173     setBadge(uploadBadge, 'loading', 'Indexando');
174     uploadResult.textContent = 'Enviando documento para
175         indexacao...';
176
177     try {
178         const payload = {
179             tipo: document.getElementById('tipo_documento').value,
180             titulo: document.getElementById('titulo_documento').
181                 value,
182             texto: textArea.value,
183         };
184
185         const clienteId = document.getElementById('
186             upload_cliente_id').value.trim();
187         const clusterId = document.getElementById('
188             upload_cluster_id').value.trim();
189         if (clienteId) payload.cliente_id = clienteId;
190         if (clusterId) payload.cluster_id = Number(clusterId);
191
192         const response = await requestJson('/documentos', payload
193             , 'POST');
194         uploadResult.textContent = JSON.stringify(response, null,
195             2);
196         setBadge(uploadBadge, 'success', 'Indexado');
197     } catch (error) {
```

```
189     uploadResult.textContent = error.message;
190     setBadge(uploadBadge, 'error', 'Falhou');
191   }
192 });
193
194 document.getElementById('fill-sample').addEventListener('
195   click', () => {
196     document.getElementById('modo').value = 'segmento';
197     document.getElementById('cliente_id').value = 'C00042';
198     document.getElementById('pergunta').value = 'Que produtos
199       de investimento devo oferecer para este cliente?';
200     document.getElementById('idade').value = '55';
201     document.getElementById('renda_mensal').value = '18000';
202     document.getElementById('saldo_medio').value = '120000';
203     document.getElementById('transacoes_mes').value = '25';
204     document.getElementById('score_credito').value = '820';
205     document.getElementById('num_produtos').value = '7';
206     document.getElementById('canal_digital').checked = true;
207     document.getElementById('inadimplente').checked = false;
208   });
209
210 document.getElementById('fill-document').addEventListener('
211   click', () => {
212     document.getElementById('tipo_documento').value = '
213       documento';
214     document.getElementById('titulo_documento').value = '
215       Politica de investimentos 2026';
216     textArea.value = [
217       'Clientes conservadores priorizam liquidez,
218         previsibilidade e protecao contra inflacao.',
219       'Ofertas recomendadas: Tesouro Selic, CDB com liquidez
220         diaria e fundos DI com baixa taxa.',
221       'Perfis de alta renda com horizonte de longo prazo podem
222         receber alocao gradual em NTN-B e debentures
223         incentivadas.'
224     ].join('\n\n');
225   });
226 });
227 });
```

Listing A.14: Frontend de operacao e ingestao (versao ASCII para compilacao).

A.6.2 config.js

```
1 window.APP_CONFIG = {  
2   apiUrl: ""  
3 };
```

Listing A.15: Configuracao do frontend CloudFront (versao ASCII para compilacao).

Apêndice B

Template de Capitulo

Este apendice serve como guia de expansao iterativa para novos capitulos.

B.1 Objetivos do capitulo

- Objetivo 1
- Objetivo 2

B.2 Implementacao pratica

1. Passo 1
2. Passo 2
3. Passo 3

B.3 Politica de autocontencao (obrigatoria)

- Todo codigo citado no capitulo deve existir no projeto com caminho valido.
- Cada comando deve executar sem dependencias implícitas fora do repositório.
- Sempre que houver **Plano de testes**, incluir evidencias de execucao no capitulo ou referenciar artefatos no Apêndice A.
- Antes da versao final, executar: `pwsh -File livro/validar_autocontencao.ps1`.

B.4 Plano de testes

- Teste funcional
- Teste de performance
- Teste de qualidade de resposta

B.5 Criterios de aceite

- Criterio 1
- Criterio 2

Apêndice C

Glossário Matemático e Técnico

Este apêndice reúne definições formais dos principais termos matemáticos, estatísticos e de aprendizado de máquina empregados ao longo do livro. Cada entrada inclui notação, descrição e, quando pertinente, referência bibliográfica. Os termos aparecem em ordem alfabética.

B

Byte-Pair Encoding (BPE)

O *Byte-Pair Encoding* (BPE) é um algoritmo de tokenização por sub-palavras originalmente proposto para compressão de dados e posteriormente adaptado ao processamento de linguagem natural. O procedimento inicia com um vocabulário de caracteres individuais e, iterativamente, funde o par de símbolos adjacentes de maior frequência no corpus de treinamento, criando um novo símbolo composto. Após k fusões, obtém-se um vocabulário de tamanho $|\mathcal{V}| = |\mathcal{V}_0| + k$, em que $\mathcal{V}_0$ é o vocabulário inicial de caracteres.

1. Inicializar $\mathcal{V}$ com todos os caracteres únicos do corpus.
2. Contar a frequência de todos os pares de símbolos adjacentes.
3. Fundir o par mais frequente (a, b) em um novo símbolo ab .
4. Repetir os passos 2–3 até atingir $|\mathcal{V}|$ desejado.

O BPE oferece equilíbrio entre vocabulários de caracteres (sem palavras desconhecidas, mas sequências longas) e vocabulários de palavras inteiras (sequências curtas, mas alta taxa de *out-of-vocabulary*).

E

Embedding (vetor de representação)

Um *embedding* é uma função $f : \mathcal{X} \rightarrow \mathbb{R}^d$ que mapeia elementos de um espaço discreto (tokens, frases, documentos) para vetores densos em $\mathbb{R}^d$. No contexto de LLMs, a camada de *embedding* de entrada $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$ associa cada token $t \in \mathcal{V}$ a um vetor $\mathbf{e}_t \in \mathbb{R}^d$. A dimensão d varia tipicamente entre 768 (modelos base) e 4096 (modelos de grande porte). Geometricamente, tokens semanticamente próximos tendem a ocupar regiões próximas no espaço vetorial, propriedade explorada pela busca por similaridade de cosseno:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

L

Logits

O termo *logits* designa os valores reais não normalizados $\mathbf{z} \in \mathbb{R}^{|\mathcal{V}|}$ produzidos pela última camada linear do modelo antes da aplicação da função de ativação. Formalmente, dado o vetor de estado oculto $\mathbf{h}$ da última camada do *Transformer*, os logits são calculados como:

$$\mathbf{z} = \mathbf{W}_{\text{lm}} \mathbf{h} + \mathbf{b},$$

em que $\mathbf{W}_{\text{lm}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ é a matriz de projeção do modelo de linguagem e $\mathbf{b}$ é o viés. Os logits são convertidos em probabilidades pela função *softmax* (ver Seção C).

S

SentencePiece

O *SentencePiece* é uma biblioteca de tokenização por sub-palavras independente de idioma desenvolvida pelo Google. Ao contrário do BPE clássico (ver Seção C), o *SentencePiece* opera diretamente sobre o fluxo de caracteres Unicode sem pré-segmentação por espaços, o que o torna aplicável a idiomas sem delimitadores explícitos de palavras (japonês, chinês, tailandês). O algoritmo suporta dois modelos: BPE e *unigram language model*, que seleciona o vocabulário minimizando a perda de verossimilhança do corpus sob um modelo de linguagem de unigrama.

Softmax

A função *softmax* $\sigma : \mathbb{R}^n \rightarrow \Delta^{n-1}$ converte um vetor de logits $\mathbf{z} = (z_1, \dots, z_n)$ em uma distribuição de probabilidade sobre n classes:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad i = 1, \dots, n.$$

No contexto da geração de texto, $n = |\mathcal{V}|$ e cada $\sigma(\mathbf{z})_i$ representa a probabilidade do token i ser o próximo token gerado. A *temperatura* $\tau > 0$ escala os logits antes da *softmax*:

$$\sigma(\mathbf{z}/\tau)_i = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}.$$

Para $\tau \rightarrow 0$, a distribuição concentra-se no token de maior logit (*greedy decoding*); para $\tau > 1$, a distribuição torna-se mais uniforme, aumentando a diversidade das saídas.

W

WordPiece

O *WordPiece* é um algoritmo de tokenização por sub-palavras desenvolvido pelo Google e utilizado em modelos como BERT. Em vez de fundir o par de maior frequência absoluta (como no BPE), o *WordPiece* seleciona o par (a, b) que maximiza o ganho de verossimilhança do modelo de linguagem ao substituir a sequência ab por um único token, definido como:

$$\text{score}(a, b) = \frac{\text{freq}(ab)}{\text{freq}(a) \cdot \text{freq}(b)}.$$

Essa formulação favorece fusões de pares cujos componentes individuais são relativamente raros, produzindo um vocabulário de sub-palavras com maior coerência linguística.

Sobre os Autores

Texto reservado para biografia dos autores.

Referências Bibliográficas

- 1 WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, v. 35, p. 24824–24837, 2022. Disponível em: <<https://arxiv.org/abs/2201.11903>>.
- 2 VASWANI, A. et al. Attention is all you need. *Advances in Neural Information Processing Systems*, v. 30, 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>.
- 3 WEI, J. et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. Disponível em: <<https://arxiv.org/abs/2206.07682>>.
- 4 OpenAI. *tiktoken: Fast BPE tokeniser for use with OpenAI models*. 2023. Acessado em: maio de 2026. Disponível em: <<https://github.com/openai/tiktoken>>.
- 5 Anthropic. *Prompt Engineering Guide for Claude*. 2024. Acessado em: maio de 2026. Disponível em: <<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>>.
- 6 LangChain AI. *LangGraph: Build stateful, multi-actor applications with LLMs*. 2024. Acessado em: maio de 2026. Disponível em: <<https://github.com/langchain-ai/langgraph>>.
- 7 CHASE, H. *LangChain: Building applications with LLMs through composability*. 2022. Acessado em: maio de 2026. Disponível em: <<https://github.com/langchain-ai/langchain>>.
- 8 YAO, S. et al. ReAct: Synergizing reasoning and acting in language models. 2023. Disponível em: <<https://arxiv.org/abs/2210.03629>>.
- 9 SEMPFF, A.; HOOKER, A. *AWS Prescriptive Guidance: Agentic AI patterns and workflows on AWS*. 2025. Amazon Web Services. Acessado em: maio de 2026. Disponível em: <<https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/>>.
- 10 JOHNSON, J.; DOUZE, M.; JÉGOU, H. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, v. 7, n. 3, p. 535–547, 2019.
- 11 SHINN, N. et al. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. Disponível em: <<https://arxiv.org/abs/2303.11366>>.
- 12 Anthropic. *Model Context Protocol Specification*. 2024. <<https://modelcontextprotocol.io/>>. Acessado em: maio de 2026.
- 13 JOHNSON, J.; DOUZE, M.; JÉGOU, H. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, v. 7, n. 3, p. 535–547, 2021.

- 14 LEWIS, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, v. 33, p. 9459–9474, 2020. Disponível em: <<https://arxiv.org/abs/2005.11401>>.
- 15 ZHENG, L. et al. Judging LLM-as-a-Judge with MT-Bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023. Disponível em: <<https://arxiv.org/abs/2306.05685>>.

Índice Remissivo

- action (ReAct), 14
- add_messages, 15
- agente (IA), **14**
- Amazon Bedrock, **3**
- ambiente virtual (Python), **10**
- amostragem por núcleo, *veja* top-p
- análise estática de segurança, 64
- API Gateway, **11**, **30**, 57, 70
- arquitetura
 - Transformer, 2
- arquitetura assíncrona, **30**
- AsyncPostgresSaver, **20**
- atenção
 - multi-head, 2
 - self-attention, 2
- AWS, 8, 56
- AWS Access Analyzer, 64
- AWS Well-Architected Framework, 57
- BACEN, **63**
- Banco Central do Brasil, *veja* BACEN
- bandit (segurança), 64
- Bedrock, *veja* Amazon Bedrock
- BPE, *veja* Byte-Pair Encoding
- Byte-Pair Encoding, **2**, **78**
- ChatBedrock, **16**
- checkov, 64
- checkpoint, 15
- chunking, **27**
- CI/CD, **56**
- CloudFront, **71**
- CloudTrail, **63**
- clusterizacão, **35**
- Cognito, **76**
- config.js, **73**
- conta AWS separada, 56
- context window, *veja* janela de contexto
- DAST, **70**
- dead-letter queue, 34
- decoder, 2
- decodificação, 3
- dev (ambiente), **56**
- digital twin, **35**
- drift detection, 62
- DynamoDB, **11**, **30**, 57
- EKS (AWS), 62
- embedding, **79**
- environment protection rules, 60
- FAISS, **14**, 18
- ferramenta (agente), **16**
- few-shot, **6**
- fine-tuning, **6**
- fire-and-poll, **30**
- Git Workflow, **56**
- GitHub Actions, **56**
- GitLab CI, 56
- GitOps, 62
- grafo dirigido, 15
- GraphRAG, **41**
- homol (ambiente), **56**
- IAM, 3, 10, 57
 - menor privilégio, 64

- Identity and Access Management, *veja* IAM, **prompt**
 - few-shot, 6
 - system prompt, **5**
- veja* IAM
- indexacao (RAG), **25**
- inferência, 9
- infraestrutura como código, *veja* Terraform
- integração contínua, **56**
- janela de contexto, **3**
- K-Means, **35**
- Lambda (AWS), **11**, 57
- LangChain, 14
- LangGraph, **14**
- logits, 3, **79**
- memória
 - agentes, 14
- MemorySaver, **15**
- menor privilégio, **63**
- MessagesState, **15**
- Neptune (AWS), **11**, **41**, 57
- next-token prediction, 2
- observation (ReAct), 14
- OIDC, **61**
- Open Policy Agent, **70**
- OpenCypher, **41**
- OpenID Connect, *veja* OIDC
- OpenSearch, **11**, **25**, 57
- Origin Access Control, **71**
- OWASP Top 10, 64
- padronizacao (ML), **36**
- path ranking, 45
- persona (IA), **35**
- PLN, *veja* processamento de linguagem natural
- policy-as-code, 70
- processamento de linguagem natural, 2
- prod (ambiente), **56**
- RAG**, **25**, **71**
- ReAct, **14**
- recuperação semântica, **18**
- recuperacao semantica, 25
- redes recorrentes, 2
- reprodutibilidade, **9**
- request_id, 72
- reranking, 29
- S3 (AWS), 58, 71
- SBOM, **70**
- segmentação subpalavra, 2
- segurança (esteira), **63**
- self-attention, **2**
- SentencePiece, 2, **79**
- Simple Queue Service, *veja* SQS
- SNS (AWS), 62
- softmax, **2**, 3, **80**
- SQS, **11**, **30**, 57
- StandardScaler, 36
- state backend, **56**
- state file (Terraform), **59**
- StateGraph, **16**
- superfície de ataque, 61
- temperatura (LLM), **3**
- Terraform, **8**, 56
 - workspace, **56**
- testes de integração, **65**
- Textextract, **75**
- tfsec, 64
- thought (ReAct), 14
- thread_id, 15

token, **2**

ToolNode, **16**

top-k, **3**

top-p, **3**

Transformer, **2**

TypedDict, **15**

versionamento semântico, **57**

vetor semântico, **14**

Virtual Private Cloud, *veja* VPC

VPC, **11**

WordPiece, **80**